arXiv:1904.07272v8 [cs.LG] 3 Apr 2024

Introduction to Multi-Armed Bandits

Aleksandrs Slivkins

Microsoft Research NYC

First draft: January 2017
Published: November 2019
Latest version: April 2024

Abstract

Multi-armed bandits a simple but very powerful framework for algorithms that make decisions over time under uncertainty. An enormous body of work has accumulated over the years, covered in several books and surveys. This book provides a more introductory, textbook-like treatment of the subject. Each chapter tackles a particular line of work, providing a self-contained, teachable technical introduction and a brief review of the further developments; many of the chapters conclude with exercises.

The book is structured as follows. The first four chapters are on IID rewards, from the basic model to impossibility results to Bayesian priors to Lipschitz rewards. The next three chapters cover adversarial rewards, from the full-feedback version to adversarial bandits to extensions with linear rewards and combinatorially structured actions. Chapter 8 is on contextual bandits, a middle ground between IID and adversarial bandits in which the change in reward distributions is completely explained by observable contexts. The last three chapters cover connections to economics, from learning in repeated games to bandits with supply/budget constraints to exploration in the presence of incentives. The appendix provides sufficient background on concentration and KL-divergence.

The chapters on "bandits with similarity information", "bandits with knapsacks" and "bandits and agents" can also be consumed as standalone surveys on the respective topics.

Published with Foundations and Trends® in Machine Learning, November 2019.

This online version is a revision of the "Foundations and Trends" publication. It contains numerous edits for presentation and accuracy (based in part on readers' feedback), some new exercises, and updated and expanded literature reviews. *Further comments, suggestions and bug reports are very welcome!*

© 2017-2024: Aleksandrs Slivkins.
Author's webpage: `https://www.microsoft.com/en-us/research/people/slivkins`.
Email: `slivkins at microsoft.com`.

Preface

Multi-armed bandits is a rich, multi-disciplinary research area which receives attention from computer science, operations research, economics and statistics. It has been studied since (Thompson, 1933), with a big surge of activity in the past 15-20 years. An enormous body of work has accumulated over time, various subsets of which have been covered in several books (Berry and Fristedt, 1985; Cesa-Bianchi and Lugosi, 2006; Gittins et al., 2011; Bubeck and Cesa-Bianchi, 2012).

This book provides a more textbook-like treatment of the subject, based on the following principles. The literature on multi-armed bandits can be partitioned into a dozen or so lines of work. Each chapter tackles one line of work, providing a self-contained introduction and pointers for further reading. We favor fundamental ideas and elementary proofs over the strongest possible results. We emphasize accessibility of the material: while exposure to machine learning and probability/statistics would certainly help, a standard undergraduate course on algorithms, *e.g.,* one based on (Kleinberg and Tardos, 2005), should suffice for background. With the above principles in mind, the choice specific topics and results is based on the author's subjective understanding of what is important and "teachable", *i.e.,* presentable in a relatively simple manner. Many important results has been deemed too technical or advanced to be presented in detail.

The book is based on a graduate course at University of Maryland, College Park, taught by the author in Fall 2016. Each chapter corresponds to a week of the course. Five chapters were used in a similar course at Columbia University, co-taught by the author in Fall 2017. Some of the material has been updated since then, to improve presentation and reflect the latest developments.

To keep the book manageable, and also more accessible, we chose not to dwell on the deep connections to online convex optimization. A modern treatment of this fascinating subject can be found, *e.g.,* in Shalev-Shwartz (2012); Hazan (2015). Likewise, we do not venture into reinforcement learning, a rapidly developing research area and subject of several textbooks such as Sutton and Barto (1998); Szepesvári (2010); Agarwal et al. (2020). A course based on this book would be complementary to graduate-level courses on online convex optimization and reinforcement learning. Also, we do not discuss Markovian models of multi-armed bandits; this direction is covered in depth in Gittins et al. (2011).

The author encourages colleagues to use this book in their courses. A brief email regarding which chapters have been used, along with any feedback, would be appreciated.

A simultaneous book. An excellent recent book on bandits, Lattimore and Szepesvári (2020), has evolved over several years simultaneously and independently with mine. Their book is longer, provides deeper treatment for some topics (esp. for adversarial and linear bandits), and omits some others (*e.g.,* Lipschitz bandits, bandits with knapsacks, and connections to economics). Reflecting the authors' differing tastes and presentation styles, the two books are complementary to one another.

Acknowledgements. Most chapters originated as lecture notes from my course at UMD; the initial versions of these lectures were scribed by the students. Presentation of some of the fundamental results is influenced by (Kleinberg, 2007). I am grateful to Alekh Agarwal, Bobby Kleinberg, Akshay Krishnamurthy, Yishay Mansour, John Langford, Thodoris Lykouris, Rob Schapire, and Mark Sellke for discussions, comments, and advice. Chapters 9, 10 have benefited tremendously from numerous conversations with Karthik Abinav Sankararaman. Special thanks go to my PhD advisor Jon Kleinberg and my postdoc mentor Eli Upfal; Jon has shaped my taste in research, and Eli has introduced me to multi-armed bandits back in 2006. Finally, I wish to thank my parents and my family for love, inspiration and support.

Contents

Introduction: Scope and Motivation **1**

1 Stochastic Bandits **4**
1.1 Model and examples . . . 4
1.2 Simple algorithms: uniform exploration . . . 6
1.3 Advanced algorithms: adaptive exploration . . . 8
1.4 Forward look: bandits with initial information . . . 12
1.5 Literature review and discussion . . . 13
1.6 Exercises and hints . . . 15

2 Lower Bounds **17**
2.1 Background on KL-divergence . . . 18
2.2 A simple example: flipping one coin . . . 19
2.3 Flipping several coins: "best-arm identification" . . . 20
2.4 Proof of Lemma 2.8 for the general case . . . 21
2.5 Lower bounds for non-adaptive exploration . . . 23
2.6 Instance-dependent lower bounds (without proofs) . . . 24
2.7 Literature review and discussion . . . 25
2.8 Exercises and hints . . . 26

3 Bayesian Bandits and Thompson Sampling **27**
3.1 Bayesian update in Bayesian bandits . . . 28
3.2 Algorithm specification and implementation . . . 32
3.3 Bayesian regret analysis . . . 34
3.4 Thompson Sampling with no prior (and no proofs) . . . 36
3.5 Literature review and discussion . . . 36

4 Bandits with Similarity Information **38**
4.1 Continuum-armed bandits . . . 38
4.2 Lipschitz bandits . . . 42
4.3 Adaptive discretization: the Zooming Algorithm . . . 45
4.4 Literature review and discussion . . . 50
4.5 Exercises and hints . . . 55

5 Full Feedback and Adversarial Costs **59**
5.1 Setup: adversaries and regret . . . 60
5.2 Initial results: binary prediction with experts advice . . . 62
5.3 Hedge Algorithm . . . 64
5.4 Literature review and discussion . . . 68
5.5 Exercises and hints . . . 69

6 Adversarial Bandits **71**
6.1 Reduction from bandit feedback to full feedback . . . 72
6.2 Adversarial bandits with expert advice . . . 72
6.3 Preliminary analysis: unbiased estimates . . . 73
6.4 Algorithm `Exp4` and crude analysis . . . 74
6.5 Improved analysis of `Exp4` . . . 75
6.6 Literature review and discussion . . . 77
6.7 Exercises and hints . . . 80

7 Linear Costs and Semi-Bandits **82**
7.1 Online routing problem . . . 83
7.2 Combinatorial semi-bandits . . . 85
7.3 Online Linear Optimization: Follow The Perturbed Leader . . . 87
7.4 Literature review and discussion . . . 91

8 Contextual Bandits **93**
8.1 Warm-up: small number of contexts . . . 94
8.2 Lipshitz contextual bandits . . . 94
8.3 Linear contextual bandits (no proofs) . . . 96
8.4 Contextual bandits with a policy class . . . 97
8.5 Learning from contextual bandit data . . . 100
8.6 Contextual bandits in practice: challenges and a system design . . . 102
8.7 Literature review and discussion . . . 107
8.8 Exercises and hints . . . 109

9 Bandits and Games **110**
9.1 Basics: guaranteed minimax value . . . 111
9.2 The minimax theorem . . . 113
9.3 Regret-minimizing adversary . . . 114
9.4 Beyond zero-sum games: coarse correlated equilibrium . . . 116
9.5 Literature review and discussion . . . 117
9.6 Exercises and hints . . . 119

10 Bandits with Knapsacks **121**
10.1 Definitions, examples, and discussion . . . 121
10.2 Examples . . . 123
10.3 LagrangeBwK: a game-theoretic algorithm for `BwK` . . . 125
10.4 Optimal algorithms and regret bounds (no proofs) . . . 131
10.5 Literature review and discussion . . . 133
10.6 Exercises and hints . . . 142

11 Bandits and Agents **144**
11.1 Problem formulation: incentivized exploration . . . 145
11.2 How much information to reveal? . . . 147
11.3 Basic technique: hidden exploration . . . 149
11.4 Repeated hidden exploration . . . 151
11.5 A necessary and sufficient assumption on the prior . . . 152
11.6 Literature review and discussion: incentivized exploration . . . 153
11.7 Literature review and discussion: other work on bandits and agents . . . 158
11.8 Exercises and hints . . . 160

A Concentration inequalities **161**

B Properties of KL-divergence **162**

Bibliography **164**

Introduction: Scope and Motivation

Multi-armed bandits is a simple but very powerful framework for algorithms that make decisions over time under uncertainty. Let us outline some of the problems that fall under this framework.

We start with three running examples, concrete albeit very stylized:

News website When a new user arrives, a website site picks an article header to show, observes whether the user clicks on this header. The site's goal is maximize the total number of clicks.

Dynamic pricing A store is selling a digital good, *e.g.,* an app or a song. When a new customer arrives, the store chooses a price offered to this customer. The customer buys (or not) and leaves forever. The store's goal is to maximize the total profit.

Investment Each morning, you choose one stock to invest into, and invest \$1. In the end of the day, you observe the change in value for each stock. The goal is to maximize the total wealth.

Multi-armed bandits unifies these examples (and many others). In the basic version, an algorithm has K possible actions to choose from, a.k.a. *arms*, and T rounds. In each round, the algorithm chooses an arm and collects a reward for this arm. The reward is drawn independently from some distribution which is fixed (*i.e.,* depends only on the chosen arm), but not known to the algorithm. Going back to the running examples:

Example	Action	Reward
News website	an article to display	1 if clicked, 0 otherwise
Dynamic pricing	a price to offer	p if sale, 0 otherwise
Investment	a stock to invest into	change in value during the day

In the basic model, an algorithm observes the reward for the chosen arm after each round, but not for the other arms that could have been chosen. Therefore, the algorithm typically needs to *explore*: try out different arms to acquire new information. Indeed, if an algorithm always chooses arm 1, how would it know if arm 2 is better? Thus, we have a tradeoff between exploration and *exploitation*: making optimal near-term decisions based on the available information. This tradeoff, which arises in numerous application scenarios, is essential in multi-armed bandits. Essentially, the algorithm strives to learn which arms are best (perhaps approximately so), while not spending too much time exploring.

The term "multi-armed bandits" comes from a stylized gambling scenario in which a gambler faces several slot machines, a.k.a. one-armed bandits, that appear identical, but yield different payoffs.

Multi-dimensional problem space

Multi-armed bandits is a huge problem space, with many "dimensions" along which the models can be made more expressive and closer to reality. We discuss some of these modeling dimensions below. Each dimension gave rise to a prominent line of work, discussed later in this book.

Auxiliary feedback. What feedback is available to the algorithm after each round, other than the reward for the chosen arm? Does the algorithm observe rewards for the other arms? Let's check our examples:

Example	Auxiliary feedback	Rewards for any other arms?
News website	N/A	no (*bandit feedback*).
Dynamic pricing	sale $\Rightarrow$ sale at any lower price, no sale $\Rightarrow$ no sale at any higher price	yes, for some arms, but not for all (*partial feedback*).
Investment	change in value for all other stocks	yes, for all arms (*full feedback*).

We distinguish three types of feedback: *bandit feedback*, when the algorithm observes the reward for the chosen arm, and no other feedback; *full feedback*, when the algorithm observes the rewards for all arms that could have been chosen; and *partial feedback*, when some information is revealed, in addition to the reward of the chosen arm, but it does not always amount to full feedback.

This book mainly focuses on problems with bandit feedback. We also cover some of the fundamental results on full feedback, which are essential for developing subsequent bandit results. Partial feedback sometimes arises in extensions and special cases, and can be used to improve performance.

Rewards model. Where do the rewards come from? Several alternatives has been studied:

- *IID rewards:* the reward for each arm is drawn independently from a fixed distribution that depends on the arm but not on the round t.
- *Adversarial rewards:* rewards can be arbitrary, as if they are chosen by an "adversary" that tries to fool the algorithm. The adversary may be *oblivious* to the algorithm's choices, or *adaptive* thereto.
- *Constrained adversary:* rewards are chosen by an adversary that is subject to some constraints, *e.g.,* reward of each arm cannot change much from one round to another, or the reward of each arm can change at most a few times, or the total change in rewards is upper-bounded.
- *Random-process rewards*: an arm's state, which determines rewards, evolves over time as a random process, *e.g.,* a random walk or a Markov chain. The state transition in a particular round may also depend on whether the arm is chosen by the algorithm.

Contexts. In each round, an algorithm may observe some *context* before choosing an action. Such context often comprises the known properties of the current user, and allows for personalized actions.

Example	Context
News website	user location and demographics
Dynamic pricing	customer's device (*e.g.,* cell or laptop), location, demographics
Investment	current state of the economy.

Reward now depends on both the context and the chosen arm. Accordingly, the algorithm's goal is to find the best *policy* which maps contexts to arms.

Bayesian priors. In the *Bayesian* approach, the problem instance comes from a known distribution, called the *Bayesian prior*. One is typically interested in provable guarantees in expectation over this distribution.

Structured rewards. Rewards may have a known structure, *e.g.,* arms correspond to points in $\mathbb{R}^d$, and in each round the reward is a linear (resp., concave or Lipschitz) function of the chosen arm.

Global constraints. The algorithm can be subject to global constraints that bind across arms and across rounds. For example, in dynamic pricing there may be a limited inventory of items for sale.

Structured actions. An algorithm may need to make several decisions at once, *e.g.,* a news website may need to pick a slate of articles, and a seller may need to choose prices for the entire slate of offerings.

Application domains

Multi-armed bandit problems arise in a variety of application domains. The original application has been the design of "ethical" medical trials, so as to attain useful scientific data while minimizing harm to the patients. Prominent modern applications concern the Web: from tuning the look and feel of a website, to choosing which content to highlight, to optimizing web search results, to placing ads on webpages. Recommender systems can use exploration to improve its recommendations for movies, restaurants, hotels, and so forth. Another cluster of applications pertains to economics: a seller can optimize its prices and offerings; likewise, a frequent buyer such as a procurement agency can optimize its bids; an auctioneer can adjust its auction over time; a crowdsourcing platform can improve the assignment of tasks, workers and prices. In computer systems, one can experiment and learn, rather than rely on a rigid design, so as to optimize datacenters and networking protocols. Finally, one can teach a robot to better perform its tasks.

Application domain	Action	Reward
medical trials	which drug to prescribe	health outcome.
web design	*e.g.,* font color or page layout	#clicks.
content optimization	which items/articles to emphasize	#clicks.
web search	search results for a given query	#satisfied users.
advertisement	which ad to display	revenue from ads.
recommender systems	*e.g.,* which movie to watch	1 if follows recommendation.
sales optimization	which products to offer at which prices	revenue.
procurement	which items to buy at which prices	#items procured
auction/market design	*e.g.,* which reserve price to use	revenue
crowdsourcing	match tasks and workers, assign prices	#completed tasks
datacenter design	*e.g.,* which server to route the job to	job completion time.
Internet	*e.g.,* which TCP settings to use?	connection quality.
radio networks	which radio frequency to use?	#successful transmissions.
robot control	a "strategy" for a given task	job completion time.

(Brief) bibliographic notes

Medical trials has a major motivation for introducing multi-armed bandits and exploration-exploitation tradeoff (Thompson, 1933; Gittins, 1979). Bandit-like designs for medical trials belong to the realm of *adaptive* medical trials (Chow and Chang, 2008), which can also include other "adaptive" features such as early stopping, sample size re-estimation, and changing the dosage.

Applications to the Web trace back to (Pandey et al., 2007a,b; Langford and Zhang, 2007) for ad placement, (Li et al., 2010, 2011) for news optimization, and (Radlinski et al., 2008) for web search. A survey of the more recent literature is beyond our scope. Bandit algorithms tailored to recommendation systems are studied, *e.g.,* in (Bresler et al., 2014; Li et al., 2016; Bresler et al., 2016).

Applications to problems in economics comprise many aspects: optimizing seller's prices, a.k.a. *dynamic pricing* or *learn-and-earn*, (Boer, 2015, a survey); optimizing seller's product offerings, a.k.a. *dynamic assortment* (*e.g.,* Sauré and Zeevi, 2013; Agrawal et al., 2019); optimizing buyers prices, a.k.a. *dynamic procurement* (*e.g.,* Badanidiyuru et al., 2012, 2018); design of auctions (*e.g.,* Bergemann and Said, 2011; Cesa-Bianchi et al., 2013; Babaioff et al., 2015b); design of information structures (Kremer et al., 2014, starting from), and design of crowdsourcing platforms (Slivkins and Vaughan, 2013, a survey).

Applications of bandits to Internet routing and congestion control were started in theory, starting with (Awerbuch and Kleinberg, 2008; Awerbuch et al., 2005), and in systems (Dong et al., 2015, 2018; Jiang et al., 2016, 2017). Bandit problems directly motivated by radio networks have been studied starting from (Lai et al., 2008; Liu and Zhao, 2010; Anandkumar et al., 2011).

Chapter 1

Stochastic Bandits

This chapter covers bandits with IID rewards, the basic model of multi-arm bandits. We present several algorithms, and analyze their performance in terms of regret. The ideas introduced in this chapter extend far beyond the basic model, and will resurface throughout the book.

Prerequisites: Hoeffding inequality (Appendix A).

1.1 Model and examples

We consider the basic model with IID rewards, called *stochastic bandits*.[1] An algorithm has K possible actions to choose from, a.k.a. *arms*, and there are T rounds, for some known K and T. In each round, the algorithm chooses an arm and collects a reward for this arm. The algorithm's goal is to maximize its total reward over the T rounds. We make three essential assumptions:

- The algorithm observes only the reward for the selected action, and nothing else. In particular, it does not observe rewards for other actions that could have been selected. This is called *bandit feedback*.
- The reward for each action is IID. For each action a, there is a distribution $\mathcal{D}_a$ over reals, called the *reward distribution*. Every time this action is chosen, the reward is sampled independently from this distribution. The reward distributions are initially unknown to the algorithm.
- Per-round rewards are bounded; the restriction to the interval $[0, 1]$ is for simplicity.

Thus, an algorithm interacts with the world according to the protocol summarized below.

Problem protocol: Stochastic bandits

Parameters: K arms, T rounds (both known); reward distribution $\mathcal{D}_a$ for each arm a (unknown).
In each round $t \in [T]$:

1. Algorithm picks some arm a_t.
2. Reward $r_t \in [0, 1]$ is sampled independently from distribution $\mathcal{D}_a$, $a = a_t$.
3. Algorithm collects reward r_t, and observes nothing else.

We are primarily interested in the *mean reward vector* $\mu \in [0, 1]^K$, where $\mu(a) = \mathbb{E}[\mathcal{D}_a]$ is the mean reward of arm a. Perhaps the simplest reward distribution is the Bernoulli distribution, when the reward of each arm a can be either 1 or 0 ("success or failure", "heads or tails"). This reward distribution is fully specified by the mean reward, which in this case is simply the probability of the successful outcome. The problem instance is then fully specified by the time horizon T and the mean reward vector.

[1] Here and elsewhere, *IID* stands for "independent and identically distributed".

Our model is a simple abstraction for an essential feature of reality that is present in many application scenarios. We proceed with three motivating examples:

1. **News**: in a very stylized news application, a user visits a news site, the site presents a news header, and a user either clicks on this header or not. The goal of the website is to maximize the number of clicks. So each possible header is an arm in a bandit problem, and clicks are the rewards. Each user is drawn independently from a fixed distribution over users, so that in each round the click happens independently with a probability that depends only on the chosen header.

2. **Ad selection**: In website advertising, a user visits a webpage, and a learning algorithm selects one of many possible ads to display. If ad a is displayed, the website observes whether the user clicks on the ad, in which case the advertiser pays some amount $v_a \in [0,1]$. So each ad is an arm, and the paid amount is the reward. The v_a depends only on the displayed ad, but does not change over time. The click probability for a given ad does not change over time, either.

3. **Medical Trials:** a patient visits a doctor and the doctor can prescribe one of several possible treatments, and observes the treatment effectiveness. Then the next patient arrives, and so forth. For simplicity of this example, the effectiveness of a treatment is quantified as a number in $[0,1]$. Each treatment can be considered as an arm, and the reward is defined as the treatment effectiveness. As an idealized assumption, each patient is drawn independently from a fixed distribution over patients, so the effectiveness of a given treatment is IID.

Note that the reward of a given arm can only take two possible values in the first two examples, but could, in principle, take arbitrary values in the third example.

Remark 1.1. We use the following conventions in this chapter and throughout much of the book. We will use *arms* and *actions* interchangeably. Arms are denoted with a, rounds with t. There are K arms and T rounds. The set of all arms is $\mathcal{A}$. The mean reward of arm a is $\mu(a) := \mathbb{E}[\mathcal{D}_a]$. The best mean reward is denoted $\mu^* := \max_{a\in\mathcal{A}} \mu(a)$. The difference $\Delta(a) := \mu^* - \mu(a)$ describes how bad arm a is compared to μ^*; we call it the *gap* of arm a. An optimal arm is an arm a with $\mu(a) = \mu^*$; note that it is not necessarily unique. We take a^* to denote some optimal arm. $[n]$ denotes the set $\{1, 2, \ldots, n\}$.

Regret. How do we argue whether an algorithm is doing a good job across different problem instances? The problem is, some problem instances inherently allow higher rewards than others. One standard approach is to compare the algorithm's cumulative reward to the *best-arm benchmark* $\mu^* \cdot T$: the expected reward of always playing an optimal arm, which is the best possible total expected reward for a particular problem instance. Formally, we define the following quantity, called *regret* at round T:

$$R(T) = \mu^* \cdot T - \sum_{t=1}^{T} \mu(a_t). \tag{1.1}$$

Indeed, this is how much the algorithm "regrets" not knowing the best arm in advance. Note that a_t, the arm chosen at round t, is a random quantity, as it may depend on randomness in rewards and/or in the algorithm. So, $R(T)$ is also a random variable. We will typically talk about *expected* regret $\mathbb{E}[R(T)]$.

We mainly care about the dependence of $\mathbb{E}[R(T)]$ regret on the time horizon T. We also consider the dependence on the number of arms K and the mean rewards $\mu(\cdot)$. We are less interested in the fine-grained dependence on the reward distributions (beyond the mean rewards). We will usually use big-O notation to focus on the asymptotic dependence on the parameters of interests, rather than keep track of the constants.

Remark 1.2 (Terminology). Since our definition of regret sums over all rounds, we sometimes call it *cumulative* regret. When/if we need to highlight the distinction between $R(T)$ and $\mathbb{E}[R(T)]$, we say *realized*

regret and *expected regret*; but most of the time we just say "regret" and the meaning is clear from the context. The quantity $R(T)$ is sometimes called *pseudo-regret* in the literature.

1.2 Simple algorithms: uniform exploration

We start with a simple idea: explore arms uniformly (at the same rate), regardless of what has been observed previously, and pick an empirically best arm for exploitation. A natural incarnation of this idea, known as *Explore-first* algorithm, is to dedicate an initial segment of rounds to exploration, and the remaining rounds to exploitation.

1 Exploration phase: try each arm N times;
2 Select the arm $\hat{a}$ with the highest average reward (break ties arbitrarily);
3 Exploitation phase: play arm $\hat{a}$ in all remaining rounds.

Algorithm 1.1: Explore-First with parameter N.

The parameter N is fixed in advance; it will be chosen later as function of the time horizon T and the number of arms K, so as to minimize regret. Let us analyze regret of this algorithm.

Let the average reward for each action a after exploration phase be denoted $\bar{\mu}(a)$. We want the average reward to be a good estimate of the true expected rewards, i.e. the following quantity should be small: $|\bar{\mu}(a) - \mu(a)|$. We bound it using the Hoeffding inequality (Theorem A.1):

$$\Pr\left[\,|\bar{\mu}(a) - \mu(a)| \leq \texttt{rad}\,\right] \geq 1 - 2/T^4, \text{ where } \texttt{rad} := \sqrt{2\log(T)\,/\,N}. \tag{1.2}$$

Remark 1.3. Thus, $\mu(a)$ lies in the known interval $[\bar{\mu}(a) - \texttt{rad}, \bar{\mu}(a) + \texttt{rad}]$ with high probability. A known interval containing some scalar quantity is called the *confidence interval* for this quantity. Half of this interval's length (in our case, rad) is called the *confidence radius*.[2]

We define the *clean event* to be the event that (1.2) holds for all arms simultaneously. We will argue separately the clean event, and the "bad event" – the complement of the clean event.

Remark 1.4. With this approach, one does not need to worry about probability in the rest of the analysis. Indeed, the probability has been taken care of by defining the clean event and observing that (1.2) holds therein. We do not need to worry about the bad event, either: essentially, because its probability is so tiny. We will use this "clean event" approach in many other proofs, to help simplify the technical details. This simplicity usually comes at the cost of slightly larger constants in $O()$, compared to more careful arguments which explicitly track low-probability events throughout the proof.

For simplicity, let us start with the case of $K = 2$ arms. Consider the clean event. We will show that if we chose the worse arm, it is not so bad because the expected rewards for the two arms would be close.

Let the best arm be a^*, and suppose the algorithm chooses the other arm $a \neq a^*$. This must have been because its average reward was better than that of a^*: $\bar{\mu}(a) > \bar{\mu}(a^*)$. Since this is a clean event, we have:

$$\mu(a) + \texttt{rad} \geq \bar{\mu}(a) > \bar{\mu}(a^*) \geq \mu(a^*) - \texttt{rad}$$

Re-arranging the terms, it follows that $\mu(a^*) - \mu(a) \leq 2\,\texttt{rad}$.

Thus, each round in the exploitation phase contributes at most $2\,\texttt{rad}$ to regret. Each round in exploration trivially contributes at most 1. We derive an upper bound on the regret, which consists of two parts: for exploration, when each arm is chosen N times, and then for the remaining $T - 2N$ rounds of exploitation:

$$R(T) \leq N + 2\,\texttt{rad} \cdot (T - 2N) < N + 2\,\texttt{rad} \cdot T.$$

[2] It is called a "radius" because an interval can be seen as a "ball" on the real line.

Recall that we can select any value for N, as long as it is given to the algorithm in advance. So, we can choose N so as to (approximately) minimize the right-hand side. Since the two summands are, resp., monotonically increasing and monotonically decreasing in N, we can set N so that they are approximately equal. For $N = T^{2/3} (\log T)^{1/3}$, we obtain:

$$R(T) \leq O\left(T^{2/3} (\log T)^{1/3}\right).$$

It remains to analyze the "bad event". Since regret can be at most T (each round contributes at most 1), and the bad event happens with a very small probability, regret from this event can be neglected. Formally:

$$\begin{aligned}
\mathbb{E}[R(T)] &= \mathbb{E}[R(T) \mid \text{clean event}] \times \Pr[\text{clean event}] \;+\; \mathbb{E}[R(T) \mid \text{bad event}] \times \Pr[\text{bad event}] \\
&\leq \mathbb{E}[R(T) \mid \text{clean event}] + T \times O\left(T^{-4}\right) \\
&\leq O\left((\log T)^{1/3} \times T^{2/3}\right). \qquad (1.3)
\end{aligned}$$

This completes the proof for $K = 2$ arms.

For $K > 2$ arms, we apply the union bound for (1.2) over the K arms, and then follow the same argument as above. Note that $T \geq K$ without loss of generality, since we need to explore each arm at least once. For the final regret computation, we will need to take into account the dependence on K: specifically, regret accumulated in exploration phase is now upper-bounded by KN. Working through the proof, we obtain $R(T) \leq NK + 2\,\texttt{rad} \cdot T$. As before, we approximately minimize it by approximately minimizing the two summands. Specifically, we plug in $N = (T/K)^{2/3} \cdot O(\log T)^{1/3}$. Completing the proof same way as in (1.3), we obtain:

Theorem 1.5. *Explore-first achieves regret* $\mathbb{E}[R(T)] \leq T^{2/3} \times O(K \log T)^{1/3}$.

1.2.1 Improvement: Epsilon-greedy algorithm

One problem with Explore-first is that its performance in the exploration phase may be very bad if many/most of the arms have a large gap $\Delta(a)$. It is usually better to spread exploration more uniformly over time. This is done in the *Epsilon-greedy* algorithm:

```
for each round t = 1, 2, ... do
    Toss a coin with success probability ε_t;
    if success then
        explore: choose an arm uniformly at random
    else
        exploit: choose the arm with the highest average reward so far
end
```

Algorithm 1.2: Epsilon-Greedy with exploration probabilities $(\epsilon_1, \epsilon_2, \ldots)$.

Choosing the best option in the short term is often called the "greedy" choice in the computer science literature, hence the name "Epsilon-greedy". The exploration is uniform over arms, which is similar to the "round-robin" exploration in the explore-first algorithm. Since exploration is now spread uniformly over time, one can hope to derive meaningful regret bounds even for small t. We focus on exploration probability $\epsilon_t \sim t^{-1/3}$ (ignoring the dependence on K and $\log t$ for a moment), so that the expected number of exploration rounds up to round t is on the order of $t^{2/3}$, same as in Explore-first with time horizon $T = t$. We derive the same regret bound as in Theorem 1.5, but now it holds for all rounds t.

Theorem 1.6. *Epsilon-greedy algorithm with exploration probabilities* $\epsilon_t = t^{-1/3} \cdot (K \log t)^{1/3}$ *achieves regret bound* $\mathbb{E}[R(t)] \leq t^{2/3} \cdot O(K \log t)^{1/3}$ *for each round* t.

The proof relies on a more refined clean event, introduced in the next section; we leave it as Exercise 1.2.

1.2.2 Non-adaptive exploration

Explore-first and Epsilon-greedy do not adapt their exploration schedule to the history of the observed rewards. We refer to this property as *non-adaptive exploration*, and formalize it as follows:

Definition 1.7. A round t is an *exploration round* if the data (a_t, r_t) from this round is used by the algorithm in the future rounds. A deterministic algorithm satisfies *non-adaptive exploration* if the set of all exploration rounds and the choice of arms therein is fixed before round 1. A randomized algorithm satisfies *non-adaptive exploration* if it does so for each realization of its random seed.

Next, we obtain much better regret bounds by adapting exploration to the observations. Non-adaptive exploration is indeed the key obstacle here. Making this point formal requires information-theoretic machinery developed in Chapter 2; see Section 2.5 for the precise statements.

1.3 Advanced algorithms: adaptive exploration

We present two algorithms which achieve much better regret bounds. Both algorithms adapt exploration to the observations so that very under-performing arms are phased out sooner.

Let's start with the case of $K = 2$ arms. One natural idea is as follows:

alternate the arms until we are confident which arm is better, and play this arm thereafter. (1.4)

However, how exactly do we determine whether and when we are confident? We flesh this out next.

1.3.1 Clean event and confidence bounds

Fix round t and arm a. Let $n_t(a)$ be the number of rounds before t in which this arm is chosen, and let $\bar{\mu}_t(a)$ be the average reward in these rounds. We will use Hoeffding Inequality (Theorem A.1) to derive

$$\Pr\left[\,|\bar{\mu}_t(a) - \mu(a)| \leq r_t(a)\,\right] \geq 1 - \tfrac{2}{T^4}, \text{ where } r_t(a) = \sqrt{2\log(T)\,/\,n_t(a)}. \tag{1.5}$$

However, Eq. (1.5) does not follow immediately. This is because Hoeffding Inequality only applies to a fixed number of independent random variables, whereas here we have $n_t(a)$ random samples from reward distribution $\mathcal{D}_a$, where $n_t(a)$ is itself is a random variable. Moreover, $n_t(a)$ can depend on the past rewards from arm a, so conditional on a particular realization of $n_t(a)$, the samples from a are not necessarily independent! For a simple example, suppose an algorithm chooses arm a in the first two rounds, chooses it again in round 3 if and only if the reward was 0 in the first two rounds, and never chooses it again.

So, we need a slightly more careful argument. We present an elementary version of this argument (whereas a more standard version relies on the concept of *martingale*). For each arm a, let us imagine there is a *reward tape*: an $1 \times T$ table with each cell independently sampled from $\mathcal{D}_a$, as shown in Figure 1.1. Without loss of generality, the reward tape encodes rewards as follows: the j-th time a given arm a is chosen by the algorithm, its reward is taken from the j-th cell in this arm's tape. Let $\bar{v}_j(a)$ represent the average reward at arm a from first j times that arm a is chosen. Now one can use Hoeffding Inequality to derive that

$$\forall j \quad \Pr\left[\,|\bar{v}_j(a) - \mu(a)| \leq r_t(a)\,\right] \geq 1 - 2/T^4.$$

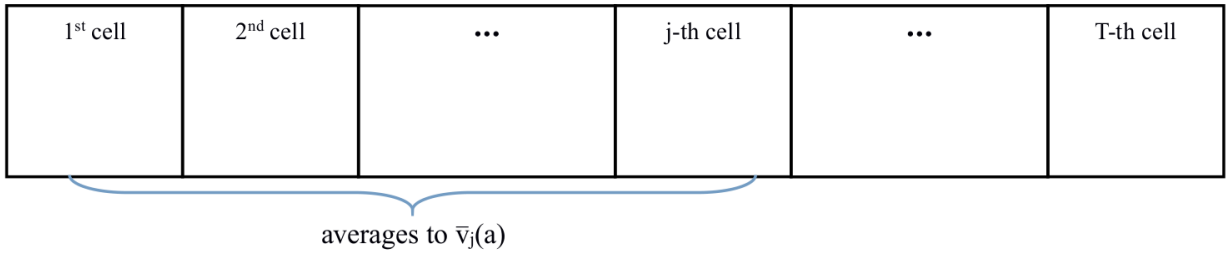


Figure 1.1: the j-th cell contains the reward of the j-th time we pull arm a, *i.e.,* reward of arm a when $n_t(a) = j$

Taking a union bound, it follows that (assuming $K = \#\text{arms} \leq T$)

$$\Pr[\mathcal{E}] \geq 1 - 2/T^2, \text{ where } \mathcal{E} := \{\forall a \forall t \quad |\bar{\mu}_t(a) - \mu(a)| \leq r_t(a)\}. \tag{1.6}$$

The event $\mathcal{E}$ in (1.6) will be the *clean event* for the subsequent analysis.

For each arm a at round t, we define *upper* and *lower confidence bounds*,

$$\texttt{UCB}_t(a) = \bar{\mu}_t(a) + r_t(a),$$
$$\texttt{LCB}_t(a) = \bar{\mu}_t(a) - r_t(a).$$

As per Remark 1.3, we have *confidence interval* $[\texttt{LCB}_t(a), \texttt{UCB}_t(a)]$ and *confidence radius* $r_t(a)$.

1.3.2 Successive Elimination algorithm

Let's come back to the case of $K = 2$ arms, and recall the idea (1.4). Now we can naturally implement this idea via the confidence bounds. The full algorithm for two arms is as follows:

> Alternate two arms until $\texttt{UCB}_t(a) < \texttt{LCB}_t(a')$ after some even round t;
> Abandon arm a, and use arm a' forever since.

Algorithm 1.3: "High-confidence elimination" algorithm for two arms

For analysis, assume the clean event. Note that the "abandoned" arm cannot be the best arm. But how much regret do we accumulate *before* disqualifying one arm?

Let t be the last round when we did *not* invoke the stopping rule, i.e., when the confidence intervals of the two arms still overlap (see Figure 1.2). Then

$$\Delta := |\mu(a) - \mu(a')| \leq 2(r_t(a) + r_t(a')).$$

Since the algorithm has been alternating the two arms before time t, we have $n_t(a) = t/2$ (up to floor and ceiling), which yields

$$\Delta \leq 2\left(r_t(a) + r_t(a')\right) \leq 4\sqrt{2\log(T) / \lfloor t/2 \rfloor} = O\left(\sqrt{\log(T)/t}\right).$$

Then the total regret accumulated till round t is

$$R(t) \leq \Delta \times t \leq O\left(t \cdot \sqrt{\tfrac{\log T}{t}}\right) = O\left(\sqrt{t \log T}\right).$$

Since we've chosen the best arm from then on, we have $R(t) \leq O\left(\sqrt{t \log T}\right)$. To complete the analysis, we need to argue that the "bad event" $\bar{\mathcal{E}}$ contributes a negligible amount to regret, like in (1.3):

$$\begin{aligned}
\mathbb{E}[R(t)] &= \mathbb{E}[R(t) \mid \text{clean event}] \times \Pr[\text{clean event}] \;+\; \mathbb{E}[R(t) \mid \text{bad event}] \times \Pr[\text{bad event}] \\
&\leq \mathbb{E}[R(t) \mid \text{clean event}] + t \times O(T^{-2}) \\
&\leq O\left(\sqrt{t \log T}\right).
\end{aligned}$$

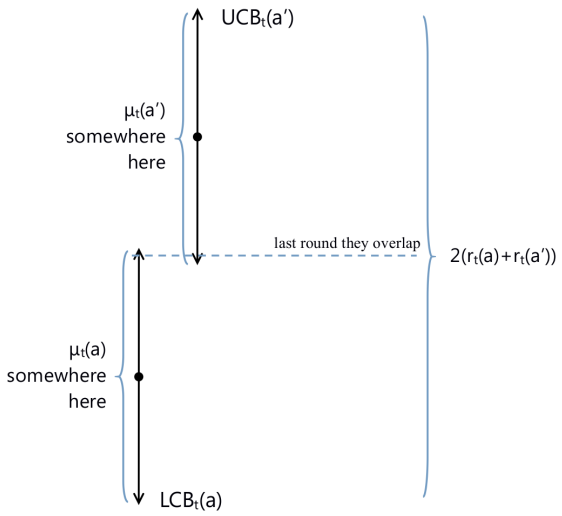


Figure 1.2: t is the last round that the two confidence intervals still overlap

We proved the following:

Lemma 1.8. *For two arms, Algorithm 1.3 achieves regret* $\mathbb{E}[\,R(t)\,] \leq O\left(\sqrt{t \log T}\right)$ *for each round* $t \leq T$.

Remark 1.9. The $\sqrt{t}$ dependence in this regret bound should be contrasted with the $T^{2/3}$ dependence for Explore-First. This improvement is possible due to adaptive exploration.

This approach extends to $K > 2$ arms as follows: alternate the arms until some arm a is *worse* than some other arm with high probability. When this happens, discard all such arms a and go to the next phase. This algorithm is called *Successive Elimination*.

All arms are initially designated as *active*
loop {new phase}
  play each active arm once
  deactivate all arms a such that, letting t be the current round,
    $\texttt{UCB}_t(a) < \texttt{LCB}_t(a')$ for some other arm a' {deactivation rule}
end loop

Algorithm 1.4: Successive Elimination

To analyze the performance of this algorithm, it suffices to focus on the clean event (1.6); as in the case of $K = 2$ arms, the contribution of the "bad event" $\bar{\mathcal{E}}$ can be neglected.

Let a^* be an optimal arm, and note that it cannot be deactivated. Fix any arm a such that $\mu(a) < \mu(a^*)$. Consider the last round $t \leq T$ when deactivation rule was invoked and arm a remained active. As in the argument for $K = 2$ arms, the confidence intervals of a and a^* must overlap at round t. Therefore,

$$\Delta(a) := \mu(a^*) - \mu(a) \leq 2\,(\,r_t(a^*) + r_t(a)\,) = 4 \cdot r_t(a).$$

The last equality is because $n_t(a) = n_t(a^*)$, since the algorithm has been alternating active arms and both a and a^* have been active before round t. By the choice of t, arm a can be played at most once afterwards: $n_T(a) \leq 1 + n_t(a)$. Thus, we have the following crucial property:

$$\Delta(a) \leq O(r_T(a)) = O\left(\sqrt{\log(T)/\,n_T(a)}\right) \quad \text{for each arm } a \text{ with } \mu(a) < \mu(a^*). \tag{1.7}$$

Informally: an arm played many times cannot be too bad. The rest of the analysis only relies on (1.7). In other words, it does not matter which algorithm achieves this property.

The contribution of arm a to regret at round t, denoted $R(t;a)$, can be expressed as $\Delta(a)$ for each round this arm is played; by (1.7) we can bound this quantity as

$$R(t;a) = n_t(a) \cdot \Delta(a) \leq n_t(a) \cdot O\left(\sqrt{\log(T) \,/\, n_t(a)}\right) = O\left(\sqrt{n_t(a) \log T}\right).$$

Summing up over all arms, we obtain that

$$R(t) = \textstyle\sum_{a \in \mathcal{A}} R(t;a) \leq O\left(\sqrt{\log T}\right) \sum_{a \in \mathcal{A}} \sqrt{n_t(a)}. \tag{1.8}$$

Since $f(x) = \sqrt{x}$ is a real concave function, and $\sum_{a \in \mathcal{A}} n_t(a) = t$, by Jensen's Inequality we have

$$\frac{1}{K} \sum_{a \in \mathcal{A}} \sqrt{n_t(a)} \leq \sqrt{\frac{1}{K} \sum_{a \in \mathcal{A}} n_t(a)} = \sqrt{\frac{t}{K}}.$$

Plugging this into (1.8), we see that $R(t) \leq O\left(\sqrt{Kt \log T}\right)$. Thus, we have proved:

Theorem 1.10. *Successive Elimination algorithm achieves regret*

$$\mathbb{E}\left[\, R(t) \,\right] = O\left(\sqrt{Kt \log T}\right) \quad \textit{for all rounds } t \leq T. \tag{1.9}$$

We can also use property (1.7) to obtain another regret bound. Rearranging the terms in (1.7), we obtain $n_T(a) \leq O\left(\log(T) \,/\, [\Delta(a)]^2\right)$. In words, a bad arm cannot be played too often. So,

$$R(T;a) = \Delta(a) \cdot n_T(a) \leq \Delta(a) \cdot O\left(\frac{\log T}{[\Delta(a)]^2}\right) = O\left(\frac{\log T}{\Delta(a)}\right). \tag{1.10}$$

Summing up over all suboptimal arms, we obtain the following theorem.

Theorem 1.11. *Successive Elimination algorithm achieves regret*

$$\mathbb{E}[R(T)] \leq O(\log T) \left[\sum_{\textit{arms } a \textit{ with } \mu(a) < \mu(a^*)} \frac{1}{\mu(a^*) - \mu(a)} \right]. \tag{1.11}$$

This regret bound is logarithmic in T, with a constant that can be arbitrarily large depending on a problem instance. In particular, this constant is at most $O(K/\Delta)$, where

$$\Delta = \min_{\text{suboptimal arms } a} \Delta(a) \qquad \textit{(minimal gap)}. \tag{1.12}$$

The distinction between regret bounds achievable with an absolute constant (as in Theorem 1.10) and regret bounds achievable with an instance-dependent constant is typical for multi-armed bandit problems. The existence of logarithmic regret bounds is another benefit of adaptive exploration.

Remark 1.12. For a more formal terminology, consider a regret bound of the form $C \cdot f(T)$, where $f(\cdot)$ does not depend on the mean reward vector μ, and the "constant" C does not depend on T. Such regret bound is called *instance-independent* if C does not depend on μ, and *instance-dependent* otherwise.

Remark 1.13. It is instructive to derive Theorem 1.10 in a different way: starting from the logarithmic regret bound in (1.10). Informally, we need to get rid of arbitrarily small gaps $\Delta(a)$ in the denominator. Let us fix some $\epsilon > 0$, then regret consists of two parts:

- all arms a with $\Delta(a) \leq \epsilon$ contribute at most ϵ per round, for a total of ϵT;
- each arm a with $\Delta(a) > \epsilon$ contributes $R(T;a) \leq O\left(\frac{1}{\epsilon}\log T\right)$, under the clean event.

Combining these two parts and assuming the clean event, we see that

$$R(T) \leq O\left(\epsilon T + \tfrac{K}{\epsilon}\log T\right).$$

Since this holds for any $\epsilon > 0$, we can choose one that minimizes the right-hand side. Ensuring that $\epsilon T = \frac{K}{\epsilon}\log T$ yields $\epsilon = \sqrt{\frac{K}{T}\log T}$, and therefore $R(T) \leq O\left(\sqrt{KT\log T}\right)$.

1.3.3 Optimism under uncertainty

Let us consider another approach for adaptive exploration, known as *optimism under uncertainty*: assume each arm is as good as it can possibly be given the observations so far, and choose the best arm based on these optimistic estimates. This intuition leads to the following simple algorithm called UCB1:

Try each arm once
for each round $t = 1, \ldots, T$ **do**
  pick arm some a which maximizes $\texttt{UCB}_t(a)$.
end for

Algorithm 1.5: Algorithm UCB1

Remark 1.14. Let's see why UCB-based selection rule makes sense. An arm a can have a large $\texttt{UCB}_t(a)$ for two reasons (or combination thereof): because the average reward $\bar{\mu}_t(a)$ is large, in which case this arm is likely to have a high reward, and/or because the confidence radius $r_t(a)$ is large, in which case this arm has not been explored much. Either reason makes this arm worth choosing. Put differently, the two summands in $\texttt{UCB}_t(a) = \bar{\mu}_t(a) + r_t(a)$ represent, resp., exploitation and exploration, and summing them up is one natural way to resolve exploration-exploitation tradeoff.

To analyze this algorithm, let us focus on the clean event (1.6), as before. Recall that a^* is an optimal arm, and a_t is the arm chosen by the algorithm in round t. According to the algorithm, $\texttt{UCB}_t(a_t) \geq \texttt{UCB}_t(a^*)$. Under the clean event, $\mu(a_t) + r_t(a_t) \geq \bar{\mu}_t(a_t)$ and $\texttt{UCB}_t(a^*) \geq \mu(a^*)$. Therefore:

$$\mu(a_t) + 2r_t(a_t) \geq \bar{\mu}_t(a_t) + r_t(a_t) = \texttt{UCB}_t(a_t) \geq \texttt{UCB}_t(a^*) \geq \mu(a^*). \tag{1.13}$$

It follows that

$$\Delta(a_t) := \mu(a^*) - \mu(a_t) \leq 2r_t(a_t) = 2\sqrt{2\log(T)\,/\,n_t(a_t)}. \tag{1.14}$$

This cute trick resurfaces in the analyses of several UCB-like algorithms for more general settings.

For each arm a consider the last round t when this arm is chosen by the algorithm. Applying (1.14) to this round gives us property (1.7). The rest of the analysis follows from that property, as in the analysis of Successive Elimination.

Theorem 1.15. *Algorithm* UCB1 *satisfies regret bounds in (1.9) and (1.11).*

1.4 Forward look: bandits with initial information

Some information about the mean reward vector μ may be known to the algorithm beforehand, and may be used to improve performance. This "initial information" is typically specified via a constraint on μ or a Bayesian prior thereon. In particular, such models may allow for non-trivial regret bounds that are independent of the number of arms, and hold even for infinitely many arms.

Constrained mean rewards. The canonical modeling approach embeds arms into $\mathbb{R}^d$, for some fixed $d \in \mathbb{N}$. Thus, arms correspond to points in $\mathbb{R}^d$, and μ is a function on (a subset of) $\mathbb{R}^d$ which maps arms to their respective mean rewards. The constraint is that μ belongs to some family $\mathcal{F}$ of "well-behaved" functions. Typical assumptions are:

- *linear functions*: $\mu(a) = w \cdot a$ for some fixed but unknown vector $w \in \mathbb{R}^d$.
- *concave functions*: the set of arms is a convex subset in $\mathbb{R}^d$, $\mu''(\cdot)$ exists and is negative.
- *Lipschitz functions*: $|\mu(a) - \mu(a')| \leq L \cdot \|a - a'\|_2$ for all arms a, a' and some fixed constant L.

Such assumptions introduce dependence between arms, so that one can infer something about the mean reward of one arm by observing the rewards of some other arms. In particular, Lipschitzness allows only "local" inferences: one can learn something about arm a only by observing other arms that are not too far from a. In contrast, linearity and concavity allow "long-range" inferences: one can learn about arm a by observing arms that lie very far from a.

One usually proves regret bounds that hold for each $\mu \in \mathcal{F}$. A typical regret bound allows for infinitely many arms, and only depends on the time horizon T and the dimension d. The drawback is that such results are only as good as the *worst case* over $\mathcal{F}$. This may be overly pessimistic if the "bad" problem instances occur very rarely, or overly optimistic if $\mu \in \mathcal{F}$ is itself a very strong assumption.

Bayesian bandits. Here, μ is drawn independently from some distribution $\mathbb{P}$, called the *Bayesian prior*. One is interested in *Bayesian regret*: regret in expectation over $\mathbb{P}$. This is special case of the Bayesian approach, which is very common in statistics and machine learning: an instance of the model is sampled from a known distribution, and performance is measured in expectation over this distribution.

The prior $\mathbb{P}$ implicitly defines the family $\mathcal{F}$ of feasible mean reward vectors (*i.e.,* the support of $\mathbb{P}$), and moreover specifies whether and to which extent some mean reward vectors in $\mathcal{F}$ are more likely than others. The main drawbacks are that the sampling assumption may be very idealized in practice, and the "true" prior may not be fully known to the algorithm.

1.5 Literature review and discussion

This chapter introduces several techniques that are broadly useful in multi-armed bandits, beyond the specific setting discussed in this chapter. These are the four algorithmic techniques (Explore-first, Epsilon-greedy, Successive Elimination, and UCB-based arm selection), the 'clean event' technique in the analysis, and the "UCB trick" from (1.13). Successive Elimination is from Even-Dar et al. (2002), and `UCB1` is from Auer et al. (2002a). Explore-first and Epsilon-greedy algorithms have been known for a long time; it is unclear what the original references are. The original version of `UCB1` has confidence radius

$$r_t(a) = \sqrt{\alpha \cdot \ln(t) \,/\, n_t(a)} \tag{1.15}$$

with $\alpha = 2$; note that $\log T$ is replaced with $\log t$ compared to the exposition in this chapter (see (1.5)). This version allows for the same regret bounds, at the cost of a somewhat more complicated analysis.

Optimality. Regret bounds in (1.9) and (1.11) are near-optimal, according to the lower bounds which we discuss in Chapter 2. The instance-dependent regret bound in (1.9) is optimal up to $O(\log T)$ factors. Audibert and Bubeck (2010) shave off the $\log T$ factor, obtaining an instance dependent regret bound $O(\sqrt{KT})$.

The logarithmic regret bound in (1.11) is optimal up to constant factors. A line of work strived to optimize the multiplicative constant in $O()$. Auer et al. (2002a); Bubeck (2010); Garivier and Cappé (2011) analyze this constant for `UCB1`, and eventually improve it to $\frac{1}{2 \ln 2}$. [3] This factor is the best possible in view

[3] More precisely, Garivier and Cappé (2011) derive the constant $\frac{\alpha}{2 \ln 2}$, using confidence radius (1.15) with any $\alpha > 1$. The original analysis in Auer et al. (2002a) obtained constant $\frac{8}{\ln 2}$ using $\alpha = 2$.

of the lower bound in Section 2.6. Further, (Audibert et al., 2009; Honda and Takemura, 2010; Garivier and Cappé, 2011; Maillard et al., 2011) refine the UCB1 algorithm and obtain regret bounds that are at least as good as those for UCB1, and get better for some reward distributions.

High-probability regret. In order to upper-bound expected regret $\mathbb{E}[R(T)]$, we actually obtained a high-probability upper bound on $R(T)$. This is common for regret bounds obtained via the "clean event" technique. However, high-probability regret bounds take substantially more work in some of the more advanced bandit scenarios, *e.g.,* for adversarial bandits (see Chapter 6).

Regret for all rounds at once. What if the time horizon T is not known in advance? Can we achieve similar regret bounds that hold for all rounds t, not just for all $t \leq T$? Recall that in Successive Elimination and UCB1, knowing T was needed only to define the confidence radius $r_t(a)$. There are several remedies:

- If an upper bound on T is known, one can use it instead of T in the algorithm. Since our regret bounds depend on T only logarithmically, rather significant over-estimates can be tolerated.
- Use UCB1 with confidence radius $r_t(a) = \sqrt{\frac{2\log t}{n_t(a)}}$, as in (Auer et al., 2002a). This version does not input T, and its regret analysis works for an arbitrary T.
- Any algorithm for known time horizon can (usually) be converted to an algorithm for an arbitrary time horizon using the *doubling trick*. Here, the new algorithm proceeds in phases of exponential duration. Each phase $i = 1, 2, \ldots$ lasts 2^i rounds, and executes a fresh run of the original algorithm. This approach achieves the "right" theoretical guarantees (see Exercise 1.5). However, forgetting everything after each phase is not very practical.

Instantaneous regret. An alternative notion of regret considers each round separately: *instantaneous regret* at round t (also called *simple regret*) is defined as $\Delta(a_t) = \mu^* - \mu(a_t)$, where a_t is the arm chosen in this round. In addition to having low cumulative regret, it may be desirable to spread the regret more "uniformly" over rounds, so as to avoid spikes in instantaneous regret. Then one would also like an upper bound on instantaneous regret that decreases monotonically over time. See Exercise 1.3.

Bandits with predictions. While the standard goal for bandit algorithms is to maximize cumulative reward, an alternative goal is to output a prediction a_t^* after each round t. The algorithm is then graded only on the quality of these predictions. In particular, it does not matter how much reward is accumulated. There are two standard ways to formalize this objective: (i) minimize instantaneous regret $\mu^* - \mu(a_t^*)$, and (ii) maximize the probability of choosing the best arm: $\Pr[a_t^* = a^*]$. The former is called *pure-exploration bandits*, and the latter is called *best-arm identification*. Essentially, good algorithms for cumulative regret, such as Successive Elimination and UCB1, are also good for this version (more on this in Exercises 1.3 and 1.4). However, improvements are possible in some regimes (*e.g.,* Mannor and Tsitsiklis, 2004; Even-Dar et al., 2006; Bubeck et al., 2011a; Audibert et al., 2010). See Exercise 1.4.

Available arms. What if arms are not always available to be selected by the algorithm? The *sleeping bandits* model in Kleinberg et al. (2008a) allows arms to "fall asleep", *i.e.,* become unavailable.[4] Accordingly, the benchmark is not the best fixed arm, which may be asleep at some rounds, but the best fixed *ranking* of arms: in each round, the benchmark selects the highest-ranked arm that is currently "awake". In the *mortal bandits* model in Chakrabarti et al. (2008), arms become permanently unavailable, according to a (possibly randomized) schedule that is known to the algorithm. Versions of UCB1 works well for both models.

[4]Earlier work (Freund et al., 1997; Blum, 1997) studied this problem under full feedback.

Partial feedback. Alternative models of partial feedback has been studied. In *dueling bandits* (Yue et al., 2012; Yue and Joachims, 2009), numeric reward is unavailable to the algorithm. Instead, one can choose *two* arms for a "duel", and find out whose reward in this round is larger. Motivation comes from web search optimization. When actions correspond to slates of search results, a numerical reward would only be a crude approximation of user satisfaction. However, one can measure which of the two slates is better in a much more reliable way using an approach called *interleaving*, see survey (Hofmann et al., 2016) for background. Further work on this model includes (Ailon et al., 2014; Zoghi et al., 2014; Dudík et al., 2015).

Another model, termed *partial monitoring* (Bartók et al., 2014; Antos et al., 2013), posits that the outcome for choosing an arm a is a (reward, feedback) pair, where the feedback can be an arbitrary message. Under the IID assumption, the outcome is chosen independently from some fixed distribution D_a over the possible outcomes. So, the feedback could be *more* than bandit feedback, *e.g.,* it can include the rewards of some other arms, or *less* than bandit feedback, *e.g.,* it might only reveal whether or not the reward is greater than 0, while the reward could take arbitrary values in $[0, 1]$. A special case is when the structure of the feedback is defined by a graph on vertices, so that the feedback for choosing arm a includes the rewards for all arms adjacent to a (Alon et al., 2013, 2015).

Relaxed benchmark. If there are too many arms (and no helpful additional structure), then perhaps competing against the best arm is perhaps too much to ask for. But suppose we relax the benchmark so that it ignores the best ϵ-fraction of arms, for some fixed $\epsilon > 0$. We are interested in regret relative to the best remaining arm, call it *ϵ-regret* for brevity. One can think of $\frac{1}{\epsilon}$ as an effective number of arms. Accordingly, we'd like a bound on ϵ-regret that depends on $\frac{1}{\epsilon}$, but not on the actual number of arms K. Kleinberg (2006) achieves ϵ-regret of the form $\frac{1}{\epsilon} \cdot T^{2/3} \cdot \text{polylog}(T)$. (The algorithm is very simple, based on a version on the "doubling trick" described above, although the analysis is somewhat subtle.) This result allows the ϵ-fraction to be defined relative to an arbitrary "base distribution" on arms, and extends to infinitely many arms. It is unclear whether better regret bounds are possible for this setting.

Bandits with initial information. Bayesian Bandits, Lipschitz bandits and Linear bandits are covered in Chapter 3, Chapter 4 and Chapter 7, respectively. Bandits with concave rewards / convex costs require fairly advanced techniques, and are not covered in this book. This direction has been initiated in Kleinberg (2004); Flaxman et al. (2005), with important recent advances (Hazan and Levy, 2014; Bubeck et al., 2015, 2017). A comprehensive treatment of this subject can be found in (Hazan, 2015).

1.6 Exercises and hints

All exercises below are fairly straightforward given the material in this chapter.

Exercise 1.1 (rewards from a small interval). Consider a version of the problem in which all the realized rewards are in the interval $[\,1/2, 1/2 + \epsilon\,]$ for some $\epsilon \in (0, 1/2)$. Define versions of `UCB1` and Successive Elimination attain improved regret bounds (both logarithmic and root-T) that depend on the ϵ.

Hint: Use a version of Hoeffding Inequality with ranges.

Exercise 1.2 (Epsilon-greedy). Prove Theorem 1.6: derive the $O(t^{2/3}) \cdot (K \log t)^{1/3}$ regret bound for the epsilon-greedy algorithm exploration probabilities $\epsilon_t = t^{-1/3} \cdot (K \log t)^{1/3}$.

Hint: Fix round t and analyze $\mathbb{E}\,[\,\Delta(a_t)\,]$ for this round separately. Set up the "clean event" for rounds $1, \ldots, t$ much like in Section 1.3.1 (treating t as the time horizon), but also include the number of exploration rounds up to time t.

Exercise 1.3 (instantaneous regret). Recall that instantaneous regret at round t is $\Delta(a_t) = \mu^* - \mu(a_t)$.

(a) Prove that Successive Elimination achieves "instance-independent" regret bound of the form

$$\mathbb{E}\left[\,\Delta(a_t)\,\right] \leq \frac{\text{polylog}(T)}{\sqrt{t/K}} \quad \text{for each round } t \in [T]. \tag{1.16}$$

(b) Derive an "instance-independent" upper bound on instantaneous regret of Explore-first.

Exercise 1.4 (pure exploration). Recall that in "pure-exploration bandits", after T rounds the algorithm outputs a prediction: a guess y_T for the best arm. We focus on the instantaneous regret $\Delta(y_T)$ for the prediction.

(a) Take any bandit algorithm with an instance-independent regret bound $E[R(T)] \leq f(T)$, and construct an algorithm for "pure-exploration bandits" such that $\mathbb{E}[\Delta(y_T)] \leq f(T)/T$.

Note: Surprisingly, taking $y_T = a_t$ does not seem to work in general – definitely not immediately. Taking y_T to be the arm with a maximal empirical reward does not seem to work, either. But there is a simple solution ...

Take-away: We can easily obtain $\mathbb{E}[\Delta(y_T)] = O(\sqrt{K\log(T)/T}$ from standard algorithms such as `UCB1` and Successive Elimination. However, as parts (bc) show, one can do much better!

(b) Consider Successive Elimination with $y_T = a_T$. Prove that (with a slightly modified definition of the confidence radius) this algorithm can achieve

$$\mathbb{E}\left[\,\Delta(y_T)\,\right] \leq T^{-\gamma} \quad \text{if } T > T_{\mu,\gamma},$$

where $T_{\mu,\gamma}$ depends only on the mean rewards $\mu(a) : a \in \mathcal{A}$ and the γ. This holds for an arbitrarily large constant γ, with only a multiplicative-constant increase in regret.

Hint: Put the γ inside the confidence radius, so as to make the "failure probability" sufficiently low.

(c) Prove that alternating the arms (and predicting the best one) achieves, for any fixed $\gamma < 1$:

$$\mathbb{E}\left[\,\Delta(y_T)\,\right] \leq e^{-\Omega(T^{\gamma})} \quad \text{if } T > T_{\mu,\gamma},$$

where $T_{\mu,\gamma}$ depends only on the mean rewards $\mu(a) : a \in \mathcal{A}$ and the γ.

Hint: Consider Hoeffding Inequality with an arbitrary constant α in the confidence radius. Pick α as a function of the time horizon T so that the failure probability is as small as needed.

Exercise 1.5 (Doubling trick). Take any bandit algorithm $\mathcal{A}$ for fixed time horizon T. Convert it to an algorithm $\mathcal{A}_\infty$ which runs forever, in phases $i = 1, 2, 3, \ldots$ of 2^i rounds each. In each phase i algorithm $\mathcal{A}$ is restarted and run with time horizon 2^i.

(a) State and prove a theorem which converts an instance-independent upper bound on regret for $\mathcal{A}$ into similar bound for $\mathcal{A}_\infty$ (so that this theorem applies to both `UCB1` and Explore-first).

(b) Do the same for $\log(T)$ instance-dependent upper bounds on regret. (Then regret increases by a $\log(T)$ factor.)

Chapter 2

Lower Bounds

This chapter is about what bandit algorithms *cannot* do. We present several fundamental results which imply that the regret rates in Chapter 1 are essentially the best possible.

Prerequisites: Chapter 1 (minimally: the model and the theorem statements).

We revisit the setting of stochastic bandits from Chapter 1 from a different perspective: we ask what bandit algorithms *cannot* do. We are interested in lower bounds on regret which apply to all bandit algorithms at once. Rather than analyze a particular bandit algorithm, we show that any bandit algorithm cannot achieve a better regret rate. We prove that any algorithm suffers regret $\Omega(\sqrt{KT})$ on some problem instance. Then we use the same technique to derive stronger lower bounds for non-adaptive exploration. Finally, we formulate and discuss the instance-dependent $\Omega(\log T)$ lower bound (albeit without a proof). These fundamental lower bounds elucidate what are the best possible *upper* bounds that one can hope to achieve.

The $\Omega(\sqrt{KT})$ lower bound is stated as follows:

Theorem 2.1. *Fix time horizon T and the number of arms K. For any bandit algorithm, there exists a problem instance such that $\mathbb{E}[R(T)] \geq \Omega(\sqrt{KT})$.*

This lower bound is "worst-case", leaving open the possibility that a particular bandit algorithm has low regret for many/most other problem instances. To prove such a lower bound, one needs to construct a family $\mathcal{F}$ of problem instances that can "fool" any algorithm. Then there are two standard ways to proceed:

(i) prove that any algorithm has high regret on some instance in $\mathcal{F}$,

(ii) define a distribution over problem instances in $\mathcal{F}$, and prove that any algorithm has high regret in expectation over this distribution.

Remark 2.2. Note that (ii) implies (i), is because if regret is high in expectation over problem instances, then there exists at least one problem instance with high regret. Conversely, (i) implies (ii) if $|\mathcal{F}|$ is a constant: indeed, if we have high regret H for some problem instance in $\mathcal{F}$, then in expectation over a uniform distribution over $\mathcal{F}$ regret is least $H/|\mathcal{F}|$. However, this argument breaks if $|\mathcal{F}|$ is large. Yet, a stronger version of (i) which says that regret is high for a *constant fraction* of the instances in $\mathcal{F}$ implies (ii), with uniform distribution over the instances, regardless of how large $|\mathcal{F}|$ is.

On a very high level, our proof proceeds as follows. We consider 0-1 rewards and the following family of problem instances, with parameter $\epsilon > 0$ to be adjusted in the analysis:

$$\mathcal{I}_j = \begin{cases} \mu_i = 1/2 + \epsilon/2 & \text{for arm } i = j \\ \mu_i = 1/2 & \text{for each arm } i \neq j. \end{cases} \tag{2.1}$$

for each $j = 1, 2, \ldots, K$, where K is the number of arms. Recall from the previous chapter that sampling each arm $\tilde{O}(1/\epsilon^2)$ times suffices for our upper bounds on regret.[1] We will prove that sampling each arm $\Omega(1/\epsilon^2)$ times is *necessary* to determine whether this arm is good or bad. This leads to regret $\Omega(K/\epsilon)$. We complete the proof by plugging in $\epsilon = \Theta(\sqrt{K/T})$.

However, the technical details are quite subtle. We present them in several relatively gentle steps.

2.1 Background on KL-divergence

The proof relies on *KL-divergence*, an important tool from information theory. We provide a brief introduction to KL-divergence for finite sample spaces, which suffices for our purposes. This material is usually covered in introductory courses on information theory.

Throughout, consider a finite sample space Ω, and let p, q be two probability distributions on Ω. Then, the Kullback-Leibler divergence or *KL-divergence* is defined as:

$$\texttt{KL}(p, q) = \sum_{x \in \Omega} p(x) \ln \frac{p(x)}{q(x)} = \mathbb{E}_p \left[\ln \frac{p(x)}{q(x)} \right].$$

This is a notion of distance between two distributions, with the properties that it is non-negative, 0 iff $p = q$, and small if the distributions p and q are close to one another. However, KL-divergence is not symmetric and does not satisfy the triangle inequality.

Remark 2.3. KL-divergence is a mathematical construct with amazingly useful properties, see Theorem 2.4. The precise definition is not essential to us: any other construct with the same properties would work just as well. The deep reasons as to why KL-divergence should be defined in this way are beyond our scope.

That said, here's see some intuition behind this definition. Suppose data points $x_1, \ldots, x_n \in \Omega$ are drawn independently from some fixed, but unknown distribution p^*. Further, suppose we know that this distribution is either p or q, and we wish to use the data to estimate which one is more likely. One standard way to quantify whether distribution p is more likely than q is the *log-likelihood ratio*,

$$\Lambda_n := \sum_{i=1}^{n} \ln \frac{p(x_i)}{q(x_i)}.$$

KL-divergence is the expectation of this quantity when $p^* = p$, and also the limit as $n \to \infty$:

$$\lim_{n \to \infty} \Lambda_n = \mathbb{E}[\Lambda_n] = \texttt{KL}(p, q) \quad \text{if } p^* = p.$$

We present fundamental properties of KL-divergence which we use in this chapter. Throughout, let $\texttt{RC}_\epsilon$, $\epsilon \geq 0$, denote a biased random coin with bias $\epsilon/2$, *i.e.,* a distribution over $\{0, 1\}$ with expectation $(1 + \epsilon)/2$.

Theorem 2.4. *KL-divergence satisfies the following properties:*

(a) **Gibbs' Inequality***:* $\texttt{KL}(p, q) \geq 0$ *for any two distributions* p, q*, with equality if and only if* $p = q$*.*

(b) **Chain rule for product distributions***: Let the sample space be a product* $\Omega = \Omega_1 \times \Omega_1 \times \cdots \times \Omega_n$*. Let* p *and* q *be two distributions on* Ω *such that* $p = p_1 \times p_2 \times \cdots \times p_n$ *and* $q = q_1 \times q_2 \times \cdots \times q_n$*, where* p_j, q_j *are distributions on* Ω_j*, for each* $j \in [n]$*. Then* $\texttt{KL}(p, q) = \sum_{j=1}^{n} \texttt{KL}(p_j, q_j)$*.*

[1] Indeed, Eq. (1.7) in Chapter 1 asserts that Successive Elimination would sample each suboptimal arm at most $\tilde{O}(1/\epsilon^2)$ times, and the remainder of the analysis applies to any algorithm which satisfies (1.7).

(c) **Pinsker's inequality***: for any event $A \subset \Omega$ we have $2\left(p(A) - q(A)\right)^2 \leq \mathtt{KL}(p,q)$.*

(d) **Random coins***: $\mathtt{KL}(\mathtt{RC}_\epsilon, \mathtt{RC}_0) \leq 2\epsilon^2$, and $\mathtt{KL}(\mathtt{RC}_0, \mathtt{RC}_\epsilon) \leq \epsilon^2$ for all $\epsilon \in (0, \frac{1}{2})$.*

The proofs of these properties are fairly simple (and teachable). We include them in Appendix B for completeness, and so as to "demystify" the KL technique.

A typical usage is as follows. Consider the setting from part (b) with n samples from two random coins: $p_j = \mathtt{RC}_\epsilon$ is a biased random coin, and $q_j = \mathtt{RC}_0$ is a fair random coin, for each $j \in [n]$. Suppose we are interested in some event $A \subset \Omega$, and we wish to prove that $p(A)$ is not too far from $q(A)$ when ϵ is small enough. Then:

$$\begin{aligned}
2(p(A) - q(A))^2 &\leq \mathtt{KL}(p,q) && \textit{(by Pinsker's inequality)}\\
&= \sum_{j=1}^{n} \mathtt{KL}(p_j, q_j) && \textit{(by Chain Rule)}\\
&\leq n \cdot \mathtt{KL}(\mathtt{RC}_\epsilon, \mathtt{RC}_0) && \textit{(by definition of } p_j, q_j\textit{)}\\
&\leq 2n\epsilon^2. && \textit{(by part (d))}
\end{aligned}$$

It follows that $|p(A) - q(A)| \leq \epsilon\sqrt{n}$. In particular, $|p(A) - q(A)| < \frac{1}{2}$ whenever $\epsilon < \frac{1}{2\sqrt{n}}$. Thus:

Lemma 2.5. *Consider sample space $\Omega = \{0,1\}^n$ and two distributions on Ω, $p = \mathtt{RC}_\epsilon^n$ and $q = \mathtt{RC}_0^n$, for some $\epsilon > 0$. Then $|p(A) - q(A)| \leq \epsilon\sqrt{n}$ for any event $A \subset \Omega$.*

Remark 2.6. The asymmetry in the definition of KL-divergence does not matter in the argument above: we could have written $\mathtt{KL}(q,p)$ instead of $\mathtt{KL}(p,q)$. Likewise, it does not matter throughout this chapter.

2.2 A simple example: flipping one coin

We start with a simple application of the KL-divergence technique, which is also interesting as a standalone result. Consider a biased random coin: a distribution on $\{0,1\}$) with an unknown mean $\mu \in [0,1]$. Assume that $\mu \in \{\mu_1, \mu_2\}$ for two known values $\mu_1 > \mu_2$. The coin is flipped T times. The goal is to identify if $\mu = \mu_1$ or $\mu = \mu_2$ with low probability of error.

Let us make our goal a little more precise. Define $\Omega := \{0,1\}^T$ to be the sample space for the outcomes of T coin tosses. Let us say that we need a decision rule

$$\mathtt{Rule} : \Omega \to \{\mathtt{High}, \mathtt{Low}\}$$

which satisfies the following two properties:

$$\Pr\left[\,\mathtt{Rule}(\text{observations}) = \mathtt{High} \mid \mu = \mu_1\,\right] \geq 0.99, \tag{2.2}$$

$$\Pr\left[\,\mathtt{Rule}(\text{observations}) = \mathtt{Low} \mid \mu = \mu_2\,\right] \geq 0.99. \tag{2.3}$$

How large should T be for for such a decision rule to exist? We know that $T \sim (\mu_1 - \mu_2)^{-2}$ is sufficient. What we prove is that it is also necessary. We focus on the special case when both μ_1 and μ_2 are close to $\frac{1}{2}$.

Lemma 2.7. *Let $\mu_1 = \frac{1+\epsilon}{2}$ and $\mu_2 = \frac{1}{2}$. Fix a decision rule which satisfies (2.2) and (2.3). Then $T > \frac{1}{4\,\epsilon^2}$.*

Proof. Let $A_0 \subset \Omega$ be the event this rule returns `High`. Then

$$\Pr[A_0 \mid \mu = \mu_1] - \Pr[A_0 \mid \mu = \mu_2] \geq 0.98. \tag{2.4}$$

Let $P_i(A) = \Pr[A \mid \mu = \mu_i]$, for each event $A \subset \Omega$ and each $i \in \{1, 2\}$. Then $P_i = P_{i,1} \times \ldots \times P_{i,T}$, where $P_{i,t}$ is the distribution of the tth coin toss if $\mu = \mu_i$. Thus, the basic KL-divergence argument summarized in Lemma 2.5 applies to distributions P_1 and P_2. It follows that $|P_1(A) - P_2(A)| \leq \epsilon\sqrt{T}$. Plugging in $A = A_0$ and $T \leq \frac{1}{4\epsilon^2}$, we obtain $|P_1(A_0) - P_2(A_0)| < \frac{1}{2}$, contradicting (2.4). □

Remarkably, the proof applies to all decision rules at once!

2.3 Flipping several coins: "best-arm identification"

Let us extend the previous example to multiple coins. We consider a bandit problem with K arms, where each arm is a biased random coin with unknown mean. More formally, the reward of each arm is drawn independently from a fixed but unknown Bernoulli distribution. After T rounds, the algorithm outputs an arm y_T: a prediction for which arm is optimal (has the highest mean reward). We call this version "best-arm identification". We are only be concerned with the quality of prediction, rather than regret.

As a matter of notation, the set of arms is $[K]$, $\mu(a)$ is the mean reward of arm a, and a problem instance is specified as a tuple $\mathcal{I} = (\mu(a) : a \in [K])$.

For concreteness, let us say that a good algorithm for "best-arm identification" should satisfy

$$\Pr\left[\text{prediction } y_T \text{ is correct } \mid \mathcal{I}\right] \geq 0.99 \tag{2.5}$$

for each problem instance $\mathcal{I}$. We will use the family (2.1) of problem instances, with parameter $\epsilon > 0$, to argue that one needs $T \geq \Omega\left(\frac{K}{\epsilon^2}\right)$ for any algorithm to "work", *i.e.,* satisfy property (2.5), on all instances in this family. This result is of independent interest, regardless of the regret bound that we've set out to prove.

In fact, we prove a stronger statement which will also be the crux in the proof of the regret bound.

Lemma 2.8. *Consider a "best-arm identification" problem with $T \leq \frac{cK}{\epsilon^2}$, for a small enough absolute constant $c > 0$. Fix any deterministic algorithm for this problem. Then there exists at least $\lceil K/3 \rceil$ arms a such that, for problem instances $\mathcal{I}_a$ defined in (2.1), we have*

$$\Pr\left[\, y_T = a \mid \mathcal{I}_a \,\right] < 3/4. \tag{2.6}$$

The proof for $K = 2$ arms is simpler, so we present it first. The general case is deferred to Section 2.4.

Proof ($K = 2$ arms). Let us set up the sample space which we will use in the proof. Let

$$\left(r_t(a) : \ a \in [K], t \in [T] \right)$$

be a tuple of mutually independent Bernoulli random variables such that $\mathbb{E}\left[\, r_t(a) \,\right] = \mu(a)$. We refer to this tuple as the *rewards table*, where we interpret $r_t(a)$ as the reward received by the algorithm for the t-th time it chooses arm a. The sample space is $\Omega = \{0, 1\}^{K \times T}$, where each outcome $\omega \in \Omega$ corresponds to a particular realization of the rewards table. Any event about the algorithm can be interpreted as a subset of Ω, *i.e.,* the set of outcomes $\omega \in \Omega$ for which this event happens; we assume this interpretation throughout without further notice. Each problem instance $\mathcal{I}_j$ defines distribution P_j on Ω:

$$P_j(A) = \Pr\left[\, A \mid \text{instance } \mathcal{I}_j \,\right] \quad \text{for each } A \subset \Omega.$$

Let $P_j^{a,t}$ be the distribution of $r_t(a)$ under instance $\mathcal{I}_j$, so that $P_j = \prod_{a\in[K],\, t\in[T]} P_j^{a,t}$.

We need to prove that (2.6) holds for at least one of the arms. For the sake of contradiction, assume it fails for both arms. Let $A = \{y_T = 1\} \subset \Omega$ be the event that the algorithm predicts arm 1. Then $P_1(A) \geq 3/4$ and $P_2(A) \leq 1/4$, so their difference is $P_1(A) - P_2(A) \geq 1/2$.

To arrive at a contradiction, we use a similar KL-divergence argument as before:

$$\begin{aligned} 2\,(\,P_1(A) - P_2(A)\,)^2 &\leq \mathtt{KL}(P_1, P_2) && \textit{(by Pinsker's inequality)} \\ &= \sum_{a=1}^{K}\sum_{t=1}^{T} \mathtt{KL}(P_1^{a,t}, P_2^{a,t}) && \textit{(by Chain Rule)} \\ &\leq 2T\cdot 2\epsilon^2 && \textit{(by Theorem 2.4(d)).} \end{aligned} \tag{2.7}$$

The last inequality holds because for each arm a and each round t, one of the distributions $P_1^{a,t}$ and $P_2^{a,t}$ is a fair coin $\mathtt{RC}_0$, and another is a biased coin $\mathtt{RC}_\epsilon$. Simplifying (2.7),

$$P_1(A) - P_2(A) \leq \epsilon\sqrt{2\,T} < \tfrac{1}{2} \quad \text{whenever } T \leq (\tfrac{1}{4\epsilon})^2. \qquad \square$$

Corollary 2.9. *Assume T is as in Lemma 2.8. Fix any algorithm for "best-arm identification". Choose an arm a uniformly at random, and run the algorithm on instance $\mathcal{I}_a$. Then $\Pr[y_T \neq a] \geq \frac{1}{12}$, where the probability is over the choice of arm a and the randomness in rewards and the algorithm.*

Proof. Lemma 2.8 immediately implies this corollary for deterministic algorithms. The general case follows because any randomized algorithm can be expressed as a distribution over deterministic algorithms. $\square$

Finally, we use Corollary 2.9 to finish our proof of the $\sqrt{KT}$ lower bound on regret.

Theorem 2.10. *Fix time horizon T and the number of arms K. Fix a bandit algorithm. Choose an arm a uniformly at random, and run the algorithm on problem instance $\mathcal{I}_a$. Then*

$$\mathbb{E}[R(T)] \geq \Omega(\sqrt{KT}), \tag{2.8}$$

where the expectation is over the choice of arm a and the randomness in rewards and the algorithm.

Proof. Fix the parameter $\epsilon > 0$ in (2.1), to be adjusted later, and assume that $T \leq \frac{cK}{\epsilon^2}$, where c is the constant from Lemma 2.8.

Fix round t. Let us interpret the algorithm as a "best-arm identification" algorithm, where the prediction is simply a_t, the arm chosen in this round. We can apply Corollary 2.9, treating t as the time horizon, to deduce that $\Pr[a_t \neq a] \geq \frac{1}{12}$. In words, the algorithm chooses a non-optimal arm with probability at least $\frac{1}{12}$. Recall that for each problem instances $\mathcal{I}_a$, the "gap" $\Delta(a_t) := \mu^* - \mu(a_t)$ is $\epsilon/2$ whenever a non-optimal arm is chosen. Therefore,

$$\mathbb{E}[\Delta(a_t)] = \Pr[a_t \neq a]\cdot\tfrac{\epsilon}{2} \geq \epsilon/24.$$

Summing up over all rounds, $\mathbb{E}[R(T)] = \sum_{t=1}^{T}\mathbb{E}[\Delta(a_t)] \geq \epsilon T/24$. We obtain (2.8) with $\epsilon = \sqrt{cK/T}$. $\square$

2.4 Proof of Lemma 2.8 for the general case

Reusing the proof for $K = 2$ arms only works for time horizon $T \leq c/\epsilon^2$, which yields the lower bound of $\Omega(\sqrt{T})$. Increasing T by the factor of K requires a more delicate version of the KL-divergence argument, which improves the right-hand side of (2.7) to $O(T\epsilon^2/K)$.

For the sake of the analysis, we will consider an additional problem instance

$$\mathcal{I}_0 = \{\mu_i = \tfrac{1}{2} \text{ for all arms } i\ \},$$

which we call the "base instance". Let $\mathbb{E}_0[\cdot]$ be the expectation given this problem instance. Also, let T_a be the total number of times arm a is played.

We consider the algorithm's performance on problem instance $\mathcal{I}_0$, and focus on arms j that are "neglected" by the algorithm, in the sense that the algorithm does not choose arm j very often *and* is not likely to pick j for the guess y_T. Formally, we observe the following:

$$\text{There are } \geq \tfrac{2K}{3} \text{ arms } j \text{ such that } \mathbb{E}_0(T_j) \leq \tfrac{3T}{K}, \tag{2.9}$$

$$\text{There are } \geq \tfrac{2K}{3} \text{ arms } j \text{ such that } P_0(y_T = j) \leq \tfrac{3}{K}. \tag{2.10}$$

(To prove (2.9), assume for contradiction that we have more than $\frac{K}{3}$ arms with $\mathbb{E}_0(T_j) > \frac{3T}{K}$. Then the expected total number of times these arms are played is strictly greater than T, which is a contradiction. (2.10) is proved similarly.) By Markov inequality,

$$\mathbb{E}_0(T_j) \leq \tfrac{3T}{K} \text{ implies that } \Pr[T_j \leq \tfrac{24T}{K}] \geq 7/8.$$

Since the sets of arms in (2.9) and (2.10) must overlap on least $\frac{K}{3}$ arms, we conclude:

$$\text{There are at least } K/3 \text{ arms } j \text{ such that } \Pr\left[\, T_j \leq \tfrac{24T}{K} \,\right] \geq 7/8 \text{ and } P_0(y_T = j) \leq 3/K. \tag{2.11}$$

We will now refine our definition of the sample space. For each arm a, define the t-round sample space $\Omega_a^t = \{0,1\}^t$, where each outcome corresponds to a particular realization of the tuple $(r_s(a) : \ s \in [t])$. (Recall that we interpret $r_t(a)$ as the reward received by the algorithm for the t-th time it chooses arm a.) Then the "full" sample space we considered before can be expressed as $\Omega = \prod_{a\in[K]} \Omega_a^T$.

Fix an arm j satisfying the two properties in (2.11). We will prove that

$$P_j\left[\, Y_T = j \,\right] \leq 1/2. \tag{2.12}$$

Since there are at least $K/3$ such arms, this suffices to imply the Lemma.

We consider a "reduced" sample space in which arm j is played only $m = \min(\, T,\ 24T/K \,)$ times:

$$\Omega^* = \Omega_j^m \times \prod_{\text{arms } a \neq j} \Omega_a^T. \tag{2.13}$$

For each problem instance $\mathcal{I}_\ell$, we define distribution P_ℓ^* on Ω^* as follows:

$$P_\ell^*(A) = \Pr[A \mid \mathcal{I}_\ell] \quad \text{for each } A \subset \Omega^*.$$

In other words, distribution P_ℓ^* is a restriction of P_ℓ to the reduced sample space Ω^*.

We apply the KL-divergence argument to distributions P_0^* and P_j^*. For each event $A \subset \Omega^*$:

$$\begin{aligned}
2\left(P_0^*(A) - P_j^*(A)\right)^2 &\leq \mathtt{KL}(P_0^*, P_j^*) && \textit{(by Pinsker's inequality)}\\
&= \sum_{\text{arms } a} \sum_{t=1}^{T} \mathtt{KL}(P_0^{a,t}, P_j^{a,t}) && \textit{(by Chain Rule)}\\
&= \sum_{\text{arms } a \neq j} \sum_{t=1}^{T} \mathtt{KL}(P_0^{a,t}, P_j^{a,t}) + \sum_{t=1}^{m} \mathtt{KL}(P_0^{j,t}, P_j^{j,t})\\
&\leq 0 + m \cdot 2\epsilon^2 && \textit{(by Theorem 2.4(d)).}
\end{aligned}$$

The last inequality holds because each arm $a \neq j$ has identical reward distributions under problem instances $\mathcal{I}_0$ and $\mathcal{I}_j$ (namely the fair coin $\texttt{RC}_0$), and for arm j we only need to sum up over m samples rather than T.

Therefore, assuming $T \leq \frac{cK}{\epsilon^2}$ with small enough constant c, we can conclude that

$$|P_0^*(A) - P_j^*(A)| \leq \epsilon\sqrt{m} < \tfrac{1}{8} \quad \text{for all events } A \subset \Omega^*. \tag{2.14}$$

To apply (2.14), we need to make sure that $A \subset \Omega^*$, *i.e.,* that whether this event holds is completely determined by the first m samples of arm j (and all samples of other arms). In particular, we cannot take $A = \{y_T = j\}$, the event that we are interested in, because this event may depend on more than m samples of arm j. Instead, we apply (2.14) twice: to events

$$A = \{y_T = j \text{ and } T_j \leq m\} \text{ and } A' = \{T_j > m\}. \tag{2.15}$$

Recall that we interpret these events as subsets of the sample space $\Omega = \{0,1\}^{K\times T}$, *i.e.,* as a subset of possible realizations of the rewards table. Note that $A, A' \subset \Omega^*$; indeed, $A' \subset \Omega^*$ because whether the algorithm samples arm j more than m times is completely determined by the first m samples of this arm (and all samples of the other arms). We are ready for the final computation:

$$\begin{aligned}
P_j(A) &\leq \tfrac{1}{8} + P_0(A) && \text{(by (2.14))}\\
&\leq \tfrac{1}{8} + P_0(y_T = j)\\
&\leq \tfrac{1}{4} && \text{(by our choice of arm } j\text{).}\\
P_j(A') &\leq \tfrac{1}{8} + P_0(A') && \text{(by (2.14))}\\
&\leq \tfrac{1}{4} && \text{(by our choice of arm } j\text{).}\\
P_j(Y_T = j) &\leq P_j^*(Y_T = j \text{ and } T_j \leq m) + P_j^*(T_j > m)\\
&= P_j(A) + P_j(A') \leq 1/2.
\end{aligned}$$

This completes the proof of Eq. (2.12), and hence that of the Lemma.

2.5 Lower bounds for non-adaptive exploration

The same information-theoretic technique implies much stronger lower bounds for non-adaptive exploration, as per Definition 1.7. First, the $T^{2/3}$ upper bounds from Section 1.2 are essentially the best possible.

Theorem 2.11. *Consider any algorithm which satisfies non-adaptive exploration. Fix time horizon T and the number of arms $K < T$. Then there exists a problem instance such that $\mathbb{E}[R(T)] \geq \Omega\left(T^{2/3} \cdot K^{1/3}\right)$.*

Second, we rule out logarithmic upper bounds such as (1.11). The statement is more nuanced, requiring the algorithm to be at least somewhat reasonable in the worst case.[2]

Theorem 2.12. *In the setup of Theorem 2.11, suppose $\mathbb{E}[R(T)] \leq C \cdot T^{\gamma}$ for all problem instances, for some numbers $\gamma \in [2/3, 1)$ and $C > 0$. Then for any problem instance a random permutation of arms yields*

$$\mathbb{E}\left[\, R(T) \,\right] \geq \Omega\left(C^{-2} \cdot T^{\lambda} \cdot \textstyle\sum_a \Delta(a)\right), \quad \text{where } \lambda = 2(1-\gamma).$$

[2]The worst-case assumption is necessary. For example, if we only focus on problem instances with minimum gap at least Δ, then Explore-first with $N = O(\Delta^{-2}\log T)$ rounds of exploration yields logarithmic regret.

In particular, if an algorithm achieves regret $\mathbb{E}[R(T)] \leq \tilde{O}\left(T^{2/3} \cdot K^{1/3}\right)$ over all problem instances, like Explore-first and Epsilon-greedy, this algorithm incurs a similar regret for *every* problem instance, if the arms therein are randomly permuted: $\mathbb{E}[R(T)] \geq \tilde{\Omega}\left(\Delta \cdot T^{2/3} \cdot K^{1/3}\right)$, where Δ is the minimal gap. This follows by taking $C = \tilde{O}(K^{1/3})$ in the theorem.

The KL-divergence technique is "imported" via Corollary 2.9. Theorem 2.12 is fairly straightforward given this corollary, and Theorem 2.11 follows by taking $C = K^{1/3}$; see Exercise 2.1 and hints therein.

2.6 Instance-dependent lower bounds (without proofs)

The other fundamental lower bound asserts $\Omega(\log T)$ regret with an instance-dependent constant and, unlike the $\sqrt{KT}$ lower bound, applies to every problem instance. This lower bound complements the $\log(T)$ *upper* bound that we proved for algorithms UCB1 and Successive Elimination. We formulate and explain this lower bound below, without presenting a proof. The formulation is quite subtle, so we present it in stages.

Let us focus on 0-1 rewards. For a particular problem instance, we are interested in how $\mathbb{E}[R(t)]$ grows with t. We start with a simpler and weaker version:

Theorem 2.13. *No algorithm can achieve regret $\mathbb{E}[R(t)] = o(c_{\mathcal{I}} \log t)$ for all problem instances $\mathcal{I}$, where the "constant" $c_{\mathcal{I}}$ can depend on the problem instance but not on the time t.*

This version guarantees at least one problem instance on which a given algorithm has "high" regret. We would like to have a stronger lower bound which guarantees "high" regret for each problem instance. However, such lower bound is impossible because of a trivial counterexample: an algorithm which always plays arm 1, as dumb as it is, nevertheless has 0 regret on any problem instance for which arm 1 is optimal. To rule out such counterexamples, we require the algorithm to perform reasonably well (but not necessarily optimally) across all problem instances.

Theorem 2.14. *Fix K, the number of arms. Consider an algorithm such that*

$$\mathbb{E}[R(t)] \leq O(C_{\mathcal{I},\alpha}\, t^{\alpha}) \quad \text{for each problem instance } \mathcal{I} \text{ and each } \alpha > 0. \tag{2.16}$$

Here the "constant" $C_{\mathcal{I},\alpha}$ can depend on the problem instance $\mathcal{I}$ and the α, but not on time t.

Fix an arbitrary problem instance $\mathcal{I}$. For this problem instance:

$$\text{There exists time } t_0 \text{ such that for any } t \geq t_0 \quad \mathbb{E}[R(t)] \geq C_{\mathcal{I}} \ln(t), \tag{2.17}$$

for some constant $C_{\mathcal{I}}$ that depends on the problem instance, but not on time t.

Remark 2.15. For example, Assumption (2.16) is satisfied for any algorithm with $\mathbb{E}[R(t)] \leq (\log t)^{1000}$.

Let us refine Theorem 2.14 and specify how the instance-dependent constant $C_{\mathcal{I}}$ in (2.17) can be chosen. In what follows, $\Delta(a) = \mu^* - \mu(a)$ be the "gap" of arm a.

Theorem 2.16. *For each problem instance $\mathcal{I}$ and any algorithm that satisfies (2.16),*

(a) the bound (2.17) holds with

$$C_{\mathcal{I}} = \sum_{a:\ \Delta(a)>0} \frac{\mu^*(1-\mu^*)}{\Delta(a)}.$$

(b) for each $\epsilon > 0$, the bound (2.17) holds with $C_{\mathcal{I}} = C^0_{\mathcal{I}} - \epsilon$, where

$$C^0_{\mathcal{I}} = \sum_{a:\ \Delta(a)>0} \frac{\Delta(a)}{\mathtt{KL}(\mu(a),\, \mu^*)} - \epsilon.$$

Remark 2.17. The lower bound from part (a) is similar to the upper bound achieved by UCB1 and Successive Elimination: $R(T) \leq \sum_{a:\ \Delta(a)>0} \frac{O(\log T)}{\Delta(a)}$. In particular, we see that the upper bound is optimal up to a constant factor when μ^* is bounded away from 0 and 1, *e.g.,* when $\mu^* \in [\,1/4, 3/4\,]$.

Remark 2.18. Part (b) is a stronger (*i.e.,* larger) lower bound which implies the more familiar form in (a). Several algorithms in the literature are known to come arbitrarily close to this lower bound. In particular, a version of Thompson Sampling (another standard algorithm discussed in Chapter 3) achieves regret

$$R(t) \leq (1+\delta)\, C^0_{\mathcal{I}}\ \ln(t) + C'_{\mathcal{I}}/\delta^2, \quad \forall \delta > 0,$$

where $C^0_{\mathcal{I}}$ is from part (b) and $C'_{\mathcal{I}}$ is some other instance-dependent constant.

2.7 Literature review and discussion

The $\Omega(\sqrt{KT})$ lower bound on regret is from Auer et al. (2002b). KL-divergence and its properties is "textbook material" from information theory, *e.g.,* see Cover and Thomas (1991). The outline and much of the technical details in the present exposition are based on the lecture notes from Kleinberg (2007). That said, we present a substantially simpler proof, in which we replace the general "chain rule" for KL-divergence with the special case of independent distributions (Theorem 2.4(b) in Section 2.1). This special case is much easier to formulate and apply, especially for those not deeply familiar with information theory. The proof of Lemma 2.8 for general K is modified accordingly. In particular, we define the "reduced" sample space Ω^* with only a small number of samples from the "bad" arm j, and apply the KL-divergence argument to carefully defined events in (2.15), rather than a seemingly more natural event $A = \{y_T = j\}$.

Lower bounds for non-adaptive exploration have been folklore in the community. The first published version traces back to Babaioff et al. (2014), to the best of our knowledge. They define a version of non-adaptive exploration and derive similar lower bounds as ours, but for a slightly different technical setting.

The logarithmic lower bound from Section 2.6 is due to Lai and Robbins (1985). Its proof is also based on the KL-divergence technique. Apart from the original paper, it can can also be found in (Bubeck and Cesa-Bianchi, 2012). Our exposition is more explicit on "unwrapping" what this lower bound means.

While these two lower bounds essentially resolve the basic version of multi-armed bandits, they do not suffice for many other versions. Indeed, some bandit problems posit auxiliary constraints on the problem instances, such as Lipschitzness or linearity (see Section 1.4), and the lower-bounding constructions need to respect these constraints. Typically such lower bounds do not depend on the number of actions (which may be very large or even infinite). In some other bandit problems the constraints are on the algorithm, *e.g.,* a limited inventory. Then much stronger lower bounds may be possible.

Therefore, a number of problem-specific lower bounds have been proved over the years. A representative, but likely incomplete, list is below:

- for dynamic pricing (Kleinberg and Leighton, 2003; Babaioff et al., 2015a) and Lipschitz bandits (Kleinberg, 2004; Slivkins, 2014; Kleinberg et al., 2019; Krishnamurthy et al., 2020); see Chapter 4 for definitions and algorithms, and Section 4.1.2 for a (simple) lower bound and its proof.

- for linear bandits (*e.g.,* Dani et al., 2007, 2008; Rusmevichientong and Tsitsiklis, 2010; Shamir, 2015) and combinatorial (semi-)bandits (*e.g.,* Audibert et al., 2014; Kveton et al., 2015c); see Chapter 7 for definitions and algorithms.

- for pay-per-click ad auctions (Babaioff et al., 2014; Devanur and Kakade, 2009). Ad auctions are parameterized by click probabilities of ads, which are a priori unknown but can be learned over time by a bandit algorithm. The said algorithm is constrained to be compatible with advertisers' incentives.

- for dynamic pricing with limited supply (Besbes and Zeevi, 2009; Babaioff et al., 2015a) and bandits with resource constraints (Badanidiyuru et al., 2018; Immorlica et al., 2022; Sankararaman and Slivkins, 2021); see Chapter 10 for definitions and algorithms.
- for best-arm identification (*e.g.,* Kaufmann et al., 2016; Carpentier and Locatelli, 2016).

Some lower bounds in the literature are derived from first principles, like in this chapter, *e.g.,* the lower bounds in Kleinberg and Leighton (2003); Kleinberg (2004); Babaioff et al. (2014); Badanidiyuru et al. (2018). Some other lower bounds are derived by reduction to more basic ones (*e.g.,* the lower bounds in Babaioff et al., 2015a; Kleinberg et al., 2019, and the one in Section 4.1.2). The latter approach focuses on constructing the problem instances and side-steps the lengthy KL-divergence arguments.

2.8 Exercises and hints

Exercise 2.1 (non-adaptive exploration). Prove Theorems 2.11 and 2.12. Specifically, consider an algorithm which satisfies non-adaptive exploration. Let N be the number of exploration rounds. Then:

(a) there is a problem instance such that $\mathbb{E}[R(T)] \geq \Omega(\ T \cdot \sqrt{K/\,\mathbb{E}[N]}\ )$.
(b) for each problem instance, randomly permuting the arms yields $\mathbb{E}[R(T)] \geq \mathbb{E}[N] \cdot \frac{1}{K} \sum_a \Delta(a)$.
(c) use parts (a,b) to derive Theorems 2.11 and 2.12.

Hint: For part (a), start with a deterministic algorithm. Consider each round separately and invoke Corollary 2.9. For a randomized algorithm, focus on event $\{N \leq 2\,\mathbb{E}[N]\}$; its probability is at least $1/2$ by Markov inequality. Part (b) can be proved from first principles.

To derive Theorem 2.12, use part (b) to lower-bound $\mathbb{E}[N]$, then apply part (a). Set $C = K^{1/3}$ in Theorem 2.12 to derive Theorems 2.11.

Chapter 3

Bayesian Bandits and Thompson Sampling

We introduce a Bayesian version of stochastic bandits, and discuss Thompson Sampling, an important algorithm for this version, known to perform well both in theory and in practice. The exposition is self-contained, introducing concepts from Bayesian statistics as needed.

Prerequisites: Chapter 1.

The Bayesian bandit problem adds the *Bayesian assumption* to stochastic bandits: the problem instance $\mathcal{I}$ is drawn initially from some known distribution $\mathbb{P}$. The time horizon T and the number of arms K are fixed. Then an instance of stochastic bandits is specified by the mean reward vector $\mu \in [0,1]^K$ and the reward distributions $(\mathcal{D}_a : a \in [K])$. The distribution $\mathbb{P}$ is called the *prior distribution*, or the *Bayesian prior*. The goal is to optimize *Bayesian regret*: expected regret for a particular problem instance $\mathcal{I}$, as defined in (1.1), in expectation over the problem instances:

$$\texttt{BR}(T) := \mathop{\mathbb{E}}_{\mathcal{I}\sim\mathbb{P}} \left[\mathbb{E}\left[R(T) \mid \mathcal{I} \right] \right] = \mathbb{E}_{\mathcal{I}\sim\mathbb{P}} \left[\mu^* \cdot T - \textstyle\sum_{t\in[T]} \mu(a_t) \right]. \tag{3.1}$$

Bayesian bandits follow a well-known approach from *Bayesian statistics*: posit that the unknown quantity is sampled from a known distribution, and optimize in expectation over this distribution. Note that a "worst-case" regret bound (an upper bound on $\mathbb{E}[R(T)]$ which holds for all problem instances) implies the same upper bound on Bayesian regret.

Simplifications. We make several assumptions to simplify presentation. First, the realized rewards come from a *single-parameter family* of distributions. There is a family of real-valued distributions $(\mathcal{D}_\nu,\ \nu \in [0,1])$, fixed and known to the algorithm, such that each distribution $\mathcal{D}_\nu$ has expectation ν. Typical examples are Bernoulli rewards and unit-variance Gaussians. The reward of each arm a is drawn from distribution $\mathcal{D}_{\mu(a)}$, where $\mu(a) \in [0,1]$ is the mean reward. We will keep the single-parameter family fixed and implicit in our notation. Then the problem instance is completely specified by the *mean reward vector* $\mu \in [0,1]^K$, and the prior $\mathbb{P}$ is simply a distribution over $[0,1]^K$ that μ is drawn from.

Second, unless specified otherwise, the realized rewards can only take finitely many different values, and the prior $\mathbb{P}$ has a finite support, denoted $\mathcal{F}$. Then we can focus on concepts and arguments essential to Thompson Sampling, rather than worry about the intricacies of integrals and probability densities. However, the definitions and lemmas stated below carry over to arbitrary priors and arbitrary reward distributions.

Third, the best arm a^* is unique for each mean reward vector in the support of $\mathbb{P}$. This is just for simplicity: this assumption can be easily removed at the cost of slightly more cumbersome notation.

3.1 Bayesian update in Bayesian bandits

An essential operation in Bayesian statistics is *Bayesian update*: updating the prior distribution given the new data. Let us discuss how this operation plays out for Bayesian bandits.

3.1.1 Terminology and notation

Fix round t. Algorithm's data from the first t rounds is a sequence of action-reward pairs, called *t-history*:

$$H_t = (\,(a_1, r_1)\,,\ \ldots\ ,(a_t, r_t)\,) \in (\mathcal{A} \times \mathbb{R})^t.$$

It is a random variable which depends on the mean reward vector μ, the algorithm, and the reward distributions (and the randomness in all three). A fixed sequence

$$H = ((a'_1, r'_1)\,,\ \ldots\ ,(a'_t, r'_t)) \in (\mathcal{A} \times \mathbb{R})^t \tag{3.2}$$

is called a *feasible t-history* if it satisfies $\Pr[H_t = H] > 0$ for some bandit algorithm; call such algorithm *H-consistent*. One such algorithm, called the *H-induced algorithm*, deterministically chooses arm a'_s in each round $s \in [t]$. Let $\mathcal{H}_t$ be the set of all feasible t-histories; it is finite, because each reward can only take finitely many values. In particular, $\mathcal{H}_t = (\mathcal{A} \times \{0,1\})^t$ for Bernoulli rewards and a prior $\mathbb{P}$ such that $\Pr[\mu(a) \in (0,1)] = 1$ for all arms a.

In what follows, fix a feasible t-history H. We are interested in the conditional probability

$$\mathbb{P}_H(\mathcal{M}) := \Pr\left[\,\mu \in \mathcal{M} \mid H_t = H\,\right], \qquad \forall \mathcal{M} \subset [0,1]^K. \tag{3.3}$$

This expression is well-defined for the H-induced algorithm, and more generally for any H-consistent bandit algorithm. We interpret $\mathbb{P}_H$ as a distribution over $[0,1]^K$.

Reflecting the standard terminology in Bayesian statistics, $\mathbb{P}_H$ is called the *(Bayesian) posterior distribution* after round t. The process of deriving $\mathbb{P}_H$ is called *Bayesian update* of $\mathbb{P}$ given H.

3.1.2 Posterior does not depend on the algorithm

A fundamental fact about Bayesian bandits is that distribution $\mathbb{P}_H$ does not depend on which H-consistent bandit algorithm has collected the history. Thus, w.l.o.g. it is the H-induced algorithm.

Lemma 3.1. *Distribution $\mathbb{P}_H$ is the same for all H-consistent bandit algorithms.*

The proof takes a careful argument; in particular, it is essential that the algorithm's action probabilities are determined by the history, and reward distribution is determined by the chosen action.

Proof. It suffices to prove the lemma for a singleton set $\mathcal{M} = \{\tilde{\mu}\}$, for any given vector $\tilde{\mu} \in [0,1]^K$. Thus, we are interested in the conditional probability of $\{\mu = \tilde{\mu}\}$. Recall that the reward distribution with mean reward $\tilde{\mu}(a)$ places probability $\mathcal{D}_{\tilde{\mu}(a)}(r)$ on every given value $r \in \mathbb{R}$.

Let us use induction on t. The base case is $t = 0$. To make it well-defined, let us define the 0-history as $H_0 = \emptyset$, so that $H = \emptyset$ to be the only feasible 0-history. Then, all algorithms are $\emptyset$-consistent, and the conditional probability $\Pr[\mu = \tilde{\mu} \mid H_0 = H]$ is simply the prior probability $\mathbb{P}(\tilde{\mu})$.

The main argument is the induction step. Consider round $t \geq 1$. Write H as concatenation of some feasible $(t-1)$-history H' and an action-reward pair (a, r). Fix an H-consistent bandit algorithm, and let

$$\pi(a) = \Pr\left[\,a_t = a \mid H_{t-1} = H'\,\right]$$

be the probability that this algorithm assigns to each arm a in round t given the history H'. Note that this probability does not depend on the mean reward vector μ.

$$\begin{aligned}\frac{\Pr[\mu=\tilde{\mu} \text{ and } H_t=H]}{\Pr[H_{t-1}=H']} &= \Pr\left[\,\mu=\tilde{\mu} \text{ and } (a_t,r_t)=(a,r) \mid H_{t-1}=H'\,\right]\\ &= \mathbb{P}_{H'}(\tilde{\mu})\cdot\Pr[(a_t,r_t)=(a,r)\mid \mu=\tilde{\mu} \text{ and } H_{t-1}=H']\\ &= \mathbb{P}_{H'}(\tilde{\mu})\\ &\quad \Pr\left[\,r_t=r \mid a_t=a \text{ and } \mu=\tilde{\mu} \text{ and } H_{t-1}=H'\,\right]\\ &\quad \Pr\left[\,a_t=a \mid \mu=\tilde{\mu} \text{ and } H_{t-1}=H'\,\right]\\ &= \mathbb{P}_{H'}(\tilde{\mu})\cdot\mathcal{D}_{\tilde{\mu}(a)}(r)\cdot\pi(a).\end{aligned}$$

Therefore,

$$\Pr[H_t=H]=\pi(a)\cdot\Pr[H_{t-1}=H']\sum_{\tilde{\mu}\in\mathcal{F}}\mathbb{P}_{H'}(\tilde{\mu})\cdot\mathcal{D}_{\tilde{\mu}(a)}(r).$$

It follows that

$$\mathbb{P}_H(\tilde{\mu})=\frac{\Pr[\mu=\tilde{\mu} \text{ and } H_t=H]}{\Pr[H_t=H]}=\frac{\mathbb{P}_{H'}(\tilde{\mu})\cdot\mathcal{D}_{\tilde{\mu}(a)}(r)}{\sum_{\tilde{\mu}\in\mathcal{F}}\mathbb{P}_{H'}(\tilde{\mu})\cdot\mathcal{D}_{\tilde{\mu}(a)}(r)}.$$

By the induction hypothesis, the posterior distribution $\mathbb{P}_{H'}$ does not depend on the algorithm. So, the expression above does not depend on the algorithm, either. □

It follows that $\mathbb{P}_H$ stays the same if the rounds are permuted:

Corollary 3.2. *$\mathbb{P}_H=\mathbb{P}_{H'}$ whenever $H'=\left(\left(a'_{\sigma(t)},\, r'_{\sigma(t)}\right):\ t\in[T]\right)$ for some permutation σ of $[t]$.*

Remark 3.3. Lemma 3.1 should not be taken for granted. Indeed, there are two very natural extensions of Bayesian update for which this lemma does *not* hold. First, suppose we condition on an arbitrary observable event. That is, fix a set $\mathcal{H}$ of feasible t-histories. For any algorithm with $\Pr[H_t\in\mathcal{H}]>0$, consider the posterior distribution given the event $\{H_t\in\mathcal{H}\}$:

$$\Pr[\mu\in\mathcal{M}\mid H_t\in\mathcal{H}],\quad \forall\mathcal{M}\subset[0,1]^K. \tag{3.4}$$

This distribution may depend on the bandit algorithm. For a simple example, consider a problem instance with Bernoulli rewards, three arms $\mathcal{A}=\{a,a',a''\}$, and a single round. Say $\mathcal{H}$ consists of two feasible 1-histories, $H=(a,1)$ and $H'=(a',1)$. Two algorithms, ALG and ALG′, deterministically choose arms a and a', respectively. Then the distribution (3.4) equals $\mathbb{P}_H$ under ALG, and $\mathbb{P}_{H'}$ under ALG′.

Second, suppose we condition on a *subset* of rounds. The algorithm's history for a subset $S\subset[T]$ of rounds, called *S-history*, is an ordered tuple

$$H_S=((a_t,r_t):\ t\in S)\in(\mathcal{A}\times\mathbb{R})^{|S|}. \tag{3.5}$$

For any feasible $|S|$-history H, the posterior distribution given the event $\{H_S=H\}$, denoted $\mathbb{P}_{H,S}$, is

$$\mathbb{P}_{H,S}(\mathcal{M}):=\Pr[\mu\in\mathcal{M}\mid H_S=H],\quad \forall\mathcal{M}\subset[0,1]^K. \tag{3.6}$$

However, this distribution may depend on the bandit algorithm, too. Consider a problem instance with Bernoulli rewards, two arms $\mathcal{A}=\{a,a'\}$, and two rounds. Let $S=\{2\}$ (*i.e.,* we only condition on what happens in the second round), and $H=(a,1)$. Consider two algorithms, ALG and ALG′, which choose different arms in the first round (say, a for ALG and a' for ALG′), and choose arm a in the second round if and only if they receive a reward of 1 in the first round. Then the distribution (3.6) additionally conditions on $H_1=(a,1)$ under ALG, and on on $H_1=(a',1)$ under ALG′.

3.1.3 Posterior as a new prior

The posterior $\mathbb{P}_H$ can be used as a prior for a subsequent Bayesian update. Consider a feasible $(t+t')$-history, for some t'. Represent it as a concatenation of H and another feasible t'-history H', where H comes first. Denote such concatenation as $H \oplus H'$. Thus, we have two events:

$$\{H_t = H\} \text{ and } \{H_S = H'\}, \text{ where } S = [t+t'] \setminus [t].$$

(Here H_S follows the notation from (3.5).) We could perform the Bayesian update in two steps: (i) condition on H and derive the posterior $\mathbb{P}_H$, and (ii) condition on H' using $\mathbb{P}_H$ as the new prior. In our notation, the resulting posterior can be written compactly as $(\mathbb{P}_H)_{H'}$. We prove that the "two-step" Bayesian update described above is equivalent to the "one-step" update given $H \oplus H'$. In a formula, $\mathbb{P}_{H\oplus H'} = (\mathbb{P}_H)_{H'}$.

Lemma 3.4. *Let H' be a feasible t'-history. Then $\mathbb{P}_{H\oplus H'} = (\mathbb{P}_H)_{H'}$. More explicitly:*

$$\mathbb{P}_{H\oplus H'}(\mathcal{M}) = \Pr_{\mu\sim\mathbb{P}_H}[\mu \in \mathcal{M} \mid H_{t'} = H'], \quad \forall \mathcal{M} \subset [0,1]^K. \tag{3.7}$$

One take-away is that $\mathbb{P}_H$ encompasses all pertinent information from H, as far as mean rewards are concerned. In other words, once $\mathbb{P}_H$ is computed, one can forget about $\mathbb{P}$ and H going forward.

The proof is a little subtle: it relies on the $(H \oplus H')$-induced algorithm for the main argument, and carefully applies Lemma 3.1 to extend to arbitrary bandit algorithms.

Proof. It suffices to prove the lemma for a singleton set $\mathcal{M} = \{\tilde{\mu}\}$, for any given vector $\tilde{\mu} \in \mathcal{F}$.

Let ALG be the $(H \oplus H')$-induced algorithm. Let $H_t^{\texttt{ALG}}$ denote its t-history, and let $H_S^{\texttt{ALG}}$ denote its S-history, where $S = [t+t'] \setminus [t]$. We will prove that

$$\mathbb{P}_{H\oplus H'}(\mu = \tilde{\mu}) = \Pr_{\mu\sim\mathbb{P}_H}[\mu = \tilde{\mu} \mid H_S^{\texttt{ALG}} = H']. \tag{3.8}$$

We are interested in two events:

$$\mathcal{E}_t = \{H_t^{\texttt{ALG}} = H\} \text{ and } \mathcal{E}_S = \{H_S^{\texttt{ALG}} = H'\}.$$

Write $\mathbb{Q}$ for $\Pr_{\mu\sim\mathbb{P}_H}$ for brevity. We prove that $\mathbb{Q}[\cdot] = \Pr[\cdot \mid \mathcal{E}_t]$ for some events of interest. Formally,

$$\begin{aligned}
\mathbb{Q}[\mu = \tilde{\mu}] &= \mathbb{P}[\mu = \tilde{\mu} \mid \mathcal{E}_t] && \textit{(by definition of } \mathbb{P}_H\textit{)}\\
\mathbb{Q}[\mathcal{E}_S \mid \mu = \tilde{\mu}] &= \Pr[\mathcal{E}_S \mid \mu = \tilde{\mu}, \mathcal{E}_t] && \textit{(by definition of } \texttt{ALG}\textit{)}\\
\mathbb{Q}[\mathcal{E}_S \text{ and } \mu = \tilde{\mu}] &= \mathbb{Q}[\mu = \tilde{\mu}] \cdot \mathbb{Q}[\mathcal{E}_S \mid \mu = \tilde{\mu}]\\
&= \mathbb{P}[\mu = \tilde{\mu} \mid \mathcal{E}_t] \cdot \Pr[\mathcal{E}_S \mid \mu = \tilde{\mu}, \mathcal{E}_t]\\
&= \Pr[\mathcal{E}_S \text{ and } \mu = \tilde{\mu} \mid \mathcal{E}_t].
\end{aligned}$$

Summing up over all $\tilde{\mu} \in \mathcal{F}$, we obtain:

$$\mathbb{Q}[\mathcal{E}_S] = \textstyle\sum_{\tilde{\mu}\in\mathcal{F}} \mathbb{Q}[\mathcal{E}_S \text{ and } \mu = \tilde{\mu}] = \sum_{\tilde{\mu}\in\mathcal{F}} \Pr[\mathcal{E}_S \text{ and } \mu = \tilde{\mu} \mid \mathcal{E}_t] = \Pr[\mathcal{E}_S \mid \mathcal{E}_t].$$

Now, the right-hand side of (3.8) is

$$\begin{aligned}
\mathbb{Q}[\mu = \tilde{\mu} \mid \mathcal{E}_S] &= \frac{\mathbb{Q}[\mu = \tilde{\mu} \text{ and } \mathcal{E}_S]}{\mathbb{Q}[\mathcal{E}_S]} = \frac{\Pr[\mathcal{E}_S \text{ and } \mu = \tilde{\mu} \mid \mathcal{E}_t]}{\Pr[\mathcal{E}_S \mid \mathcal{E}_t]} = \frac{\Pr[\mathcal{E}_t \text{ and } \mathcal{E}_S \text{ and } \mu = \tilde{\mu}]}{\Pr[\mathcal{E}_t \text{ and } \mathcal{E}_S]}\\
&= \Pr[\mu = \tilde{\mu} \mid \mathcal{E}_t \text{ and } \mathcal{E}_S].
\end{aligned}$$

The latter equals $\mathbb{P}_{H\oplus H'}(\mu = \tilde{\mu})$, proving (3.8).

It remains to switch from ALG to an arbitrary bandit algorithm. We apply Lemma 3.1 twice, to both sides of (3.8). The first application is simply that $\mathbb{P}_{H\oplus H'}(\mu = \tilde{\mu})$ does not depend on the bandit algorithm. The second application is for prior distribution $\mathbb{P}_H$ and feasible t'-history H'. Let $\texttt{ALG}'$ be the H'-induced algorithm $\texttt{ALG}'$, and let $H_{t'}^{\texttt{ALG}'}$ be its t'-history. Then

$$\Pr_{\mu\sim\mathbb{P}_H}[\mu = \tilde{\mu} \mid H_S^{\texttt{ALG}} = H'] = \Pr_{\mu\sim\mathbb{P}_H}[\mu = \tilde{\mu} \mid H_{t'}^{\texttt{ALG}'} = H'] = \Pr_{\mu\sim\mathbb{P}_H}[\mu = \tilde{\mu} \mid H_{t'} = H']. \tag{3.9}$$

The second equality is by Lemma 3.1. We defined $\texttt{ALG}'$ to switch from the S-history to the t'-history. Thus, we've proved that the right-hand side of (3.9) equals $\mathbb{P}_{H\oplus H'}(\mu = \tilde{\mu})$ for an arbitrary bandit algorithm. □

3.1.4 Independent priors

Bayesian update simplifies for for independent priors: essentially, each arm can be updated separately. More formally, the prior $\mathbb{P}$ is called *independent* if $(\mu(a) : a \in \mathcal{A})$ are mutually independent random variables.

Fix some feasible t-history H, as per (3.2). Let $S_a = \{s \in [t] : a'_s = a\}$ be the subset of rounds when a given arm a is chosen, according to H. The portion of H that concerns arm a is defined as an ordered tuple

$$\texttt{proj}(H; a) = ((a'_s, r'_s) : s \in S_a).$$

We think of $\texttt{proj}(H; a)$ as a projection of H onto arm a, and call it *projected history* for arm a. Note that it is itself a feasible $|S_a|$-history. Define the posterior distribution $\mathbb{P}_H^a$ for arm a:

$$\mathbb{P}_H^a(\mathcal{M}_a) := \mathbb{P}_{\texttt{proj}(H;a)}(\mu(a) \in \mathcal{M}_a), \quad \forall \mathcal{M}_a \subset [0, 1]. \tag{3.10}$$

$\mathbb{P}_H^a$ does not depend on the bandit algorithm, by Lemma 3.1. Further, for the H-induced algorithm

$$\mathbb{P}_H^a(\mathcal{M}_a) = \Pr[\mu(a) \in \mathcal{M}_a \mid \texttt{proj}(H_t; a) = \texttt{proj}(H; a)].$$

Now we are ready for a formal statement:

Lemma 3.5. *Assume the prior $\mathbb{P}$ is independent. Fix a subset $\mathcal{M}_a \subset [0, 1]$ for each arm $a \in \mathcal{A}$. Then*

$$\mathbb{P}_H(\cap_{a\in\mathcal{A}} \mathcal{M}_a) = \prod_{a\in\mathcal{A}} \mathbb{P}_H^a(\mathcal{M}_a).$$

Proof. The only subtlety is that we focus the H-induced bandit algorithm. Then the pairs $(\mu(a), \texttt{proj}(H_t; a))$, $a \in \mathcal{A}$ are mutually independent random variables. We are interested in these two events, for each arm a:

$$\begin{aligned}\mathcal{E}_a &= \{\mu(a) \in \mathcal{M}_a\}\\ \mathcal{E}_a^H &= \{\texttt{proj}(H_t; a) = \texttt{proj}(H; a)\}.\end{aligned}$$

Letting $\mathcal{M} = \cap_{a\in\mathcal{A}} \mathcal{M}_a$, we have:

$$\Pr[H_t = H \text{ and } \mu \in \mathcal{M}] = \Pr\left[\bigcap_{a\in\mathcal{A}} (\mathcal{E}_a \cap \mathcal{E}_a^H)\right] = \prod_{a\in\mathcal{A}} \Pr\left[\mathcal{E}_a \cap \mathcal{E}_a^H\right].$$

Likewise, $\Pr[H_t = H] = \prod_{a\in\mathcal{A}} \Pr[\mathcal{E}_a^H]$. Putting this together,

$$\begin{aligned}\mathbb{P}_H(\mathcal{M}) &= \frac{\Pr[H_t = H \text{ and } \mu \in \mathcal{M}]}{\Pr[H_t = H]} = \prod_{a\in\mathcal{A}} \frac{\Pr[\mathcal{E}_a \cap \mathcal{E}_a^H]}{\Pr[\mathcal{E}_a^H]}\\ &= \prod_{a\in\mathcal{A}} \Pr[\mu \in \mathcal{M} \mid \mathcal{E}_a^H] = \prod_{a\in\mathcal{A}} \mathbb{P}_H^a(\mathcal{M}_a).\end{aligned}$$

□

3.2 Algorithm specification and implementation

Consider a simple algorithm for Bayesian bandits, called *Thompson Sampling*. For each round t and arm a, the algorithm computes the posterior probability that a is the best arm, and samples a with this probability.

for *each round* $t = 1, 2, \ldots$ **do**
  Observe $H_{t-1} = H$, for some feasible $(t-1)$-history H;
  Draw arm a_t independently from distribution $p_t(\cdot\,|H)$, where

$$p_t(a \mid H) := \Pr[a^* = a \mid H_{t-1} = H] \quad \text{for each arm } a.$$

end

Algorithm 3.1: Thompson Sampling.

Remark 3.6. The probabilities $p_t(\cdot \mid H)$ are determined by H, by Lemma 3.1.

Thompson Sampling admits an alternative characterization:

for *each round* $t = 1, 2, \ldots$ **do**
  Observe $H_{t-1} = H$, for some feasible $(t-1)$-history H;
  Sample mean reward vector μ_t from the posterior distribution $\mathbb{P}_H$;
  Choose the best arm $\tilde{a}_t$ according to μ_t.
end

Algorithm 3.2: Thompson Sampling: alternative characterization.

It is easy to see that this characterization is in fact equivalent to the original algorithm.

Lemma 3.7. *For each round t, arms a_t and $\tilde{a}_t$ are identically distributed given H_t.*

Proof. Fix a feasible t-history H. For each arm a we have:

$$\begin{aligned} \Pr[\tilde{a}_t = a \mid H_{t-1} = H] &= \mathbb{P}_H(a^* = a) && \textit{(by definition of } \tilde{a}_t\textit{)} \\ &= p_t(a \mid H) && \textit{(by definition of } p_t\textit{).} \end{aligned}$$ □

The algorithm further simplifies when we have independent priors. By Lemma 3.5, it suffices to consider the posterior distribution $\mathbb{P}^a_H$ for each arm a separately, as per (3.10).

for *each round* $t = 1, 2, \ldots$ **do**
  Observe $H_{t-1} = H$, for some feasible $(t-1)$-history H;
  For each arm a, sample mean reward $\mu_t(a)$ independently from distribution $\mathbb{P}^a_H$;
  Choose an arm with largest $\mu_t(a)$.
end

Algorithm 3.3: Thompson Sampling for independent priors.

3.2.1 Computational aspects

While Thompson Sampling is mathematically well-defined, it may be computationally inefficient. Indeed, let us consider a brute-force computation for the round-t posterior $\mathbb{P}_H$:

$$\mathbb{P}_H(\tilde{\mu}) = \frac{\Pr[\mu = \tilde{\mu} \text{ and } H_t = H]}{\mathbb{P}(H_t = H)} = \frac{\mathbb{P}(\tilde{\mu}) \cdot \Pr[H_t = H \mid \mu = \tilde{\mu}]}{\sum_{\tilde{\mu} \in \mathcal{F}} \mathbb{P}(\tilde{\mu}) \cdot \Pr[H_t = H \mid \mu = \tilde{\mu}]}, \quad \forall \tilde{\mu} \in \mathcal{F}. \tag{3.11}$$

Since $\Pr[H_t = H \mid \mu = \tilde{\mu}]$ can be computed in time $O(t)$,[1] the probability $\mathbb{P}(H_t = H)$ can be computed in time $O(t \cdot |\mathcal{F}|)$. Then the posterior probabilities $\mathbb{P}_H(\cdot)$ and the sampling probabilities $p_t(\cdot \mid H)$ can be computed by a scan through all $\tilde{\mu} \in \mathcal{F}$. Thus, a brute-force implementation of Algorithm 3.1 or Algorithm 3.2 takes at least $O(t \cdot |\mathcal{F}|)$ running time in each round t, which may be prohibitively large.

A somewhat faster computation can be achieved via a *sequential* Bayesian update. After each round t, we treat the posterior $\mathbb{P}_H$ as a new prior. We perform a Bayesian update given the new data point $(a_t, r_t) = (a, r)$, to compute the new posterior $\mathbb{P}_{H \oplus (a,r)}$. This approach is sound by Lemma 3.4. The benefit in terms of the running time is that in each round the update is on the history of length 1. In particular, with similar brute-force approach as in (3.11), the per-round running time improves to $O(|\mathcal{F}|)$.

Remark 3.8. While we'd like to have both low regret and a computationally efficient implementation, either one of the two may be interesting: a slow algorithm can serve as a proof-of-concept that a given regret bound can be achieved, and a fast algorithm without provable regret bounds can still perform well in practice.

With independent priors, one can do the sequential Bayesian update for each arm a separately. More formally, fix round t, and suppose in this round $(a_t, r_t) = (a, r)$. One only needs to update the posterior for $\mu(a)$. Letting $H = H_t$ be the realized t-history, treat the current posterior $\mathbb{P}^a_H$ as a new prior, and perform a Bayesian update to compute the new posterior $\mathbb{P}^a_{H'}$, where $H' = H \oplus (a, r)$. Then:

$$\mathbb{P}^a_{H'}(x) = \Pr_{\mu(a) \sim \mathbb{P}^a_H}[\mu(a) = x \mid (a_t, r_t) = (a, r)] = \frac{\mathbb{P}^a_H(x) \cdot \mathcal{D}_x(r)}{\sum_{x \in \mathcal{F}_a} \mathbb{P}^a_H(x) \cdot \mathcal{D}_x(r)}, \quad \forall x \in \mathcal{F}_a, \tag{3.12}$$

where $\mathcal{F}_a$ is the support of $\mu(a)$. Thus, the new posterior $\mathbb{P}^a_{H'}$ can be computed in time $O(|\mathcal{F}_a|)$. This is an exponential speed-up compared $|\mathcal{F}|$ (in a typical case when $|\mathcal{F}| \approx \prod_a |\mathcal{F}_a|$).

Special cases. Some special cases admit much faster computation of the posterior $\mathbb{P}^a_H$ and much faster sampling therefrom. Here are two well-known special cases when this happens. For both cases, we relax the problem setting so that the mean rewards can take arbitrarily real values.

To simplify our notation, posit that there is only one arm a. Let $\mathbb{P}$ be the prior on its mean reward $\mu(a)$. Let H be a feasible t-history H, and let $\texttt{REW}_H$ denote the total reward in H.

Beta-Bernoulli Assume Bernoulli rewards. By Corollary 3.2, the posterior $\mathbb{P}_H$ is determined by the prior $\mathbb{P}$, the number of samples t, and the total reward $\texttt{REW}_H$. Suppose the prior is the uniform distribution on the $[0, 1]$ interval, denoted $\mathbb{U}$. Then the posterior $\mathbb{U}_H$ is traditionally called *Beta distribution* with parameters $\alpha = 1 + \texttt{REW}_H$ and $\beta = 1 + t$, and denoted $\texttt{Beta}(\alpha, \beta)$. For consistency, $\texttt{Beta}(1, 1) = \mathbb{U}$: if $t = 0$, the posterior $\mathbb{U}_H$ given the empty history H is simply the prior $\mathbb{U}$.

A *Beta-Bernoulli conjugate pair* is a combination of Bernoulli rewards and a prior $\mathbb{P} = \texttt{Beta}(\alpha_0, \beta_0)$ for some parameters $\alpha_0, \beta_0 \in \mathbb{N}$. The posterior $\mathbb{P}_H$ is simply $\texttt{Beta}(\alpha_0 + \texttt{REW}_H, \beta_0 + t)$. This is because $\mathbb{P} = \mathbb{U}_{H_0}$ for an appropriately chosen feasible history H_0, and $\mathbb{P}_H = \mathbb{U}_{H_0 \oplus H}$ by Lemma 3.4.

(Corollary 3.2 and Lemma 3.4 extend to priors $\mathbb{P}$ with infinite support, including Beta distributions.)

Gaussians A *Gaussian conjugate pair* is a combination of a Gaussian reward distribution and a Gaussian prior $\mathbb{P}$. Letting μ, μ_0 be their resp. means, σ, σ_0 be their resp. standard deviations, the posterior $\mathbb{P}_H$ is also a Gaussian whose mean and standard deviation are determined (via simple formulas) by the parameters $\mu, \mu_0, \sigma, \sigma_0$ and the summary statistics $\texttt{REW}_H, t$ of H.

Beta distributions and Gaussians are well-understood. In particular, very fast algorithms exist to sample from either family of distributions.

[1] Here and elsewhere, we count addition and multiplication as unit-time operations.

3.3 Bayesian regret analysis

Let us analyze Bayesian regret of Thompson Sampling, by connecting it to the upper and lower confidence bounds studied in Chapter 1. We prove:

Theorem 3.9. *Bayesian Regret of Thompson Sampling is* $\texttt{BR}(T) = O(\sqrt{KT\log(T)})$.

Let us recap some of the definitions from Chapter 1: for each arm a and round t,

$$\begin{aligned} r_t(a) &= \sqrt{2\cdot\log(T)\,/\,n_t(a)} && \textit{(confidence radius)} \\ \texttt{UCB}_t(a) &= \bar{\mu}_t(a) + r_t(a) && \textit{(upper confidence bound)} \\ \texttt{LCB}_t(a) &= \bar{\mu}_t(a) - r_t(a) && \textit{(lower confidence bound).} \end{aligned} \tag{3.13}$$

Here, $n_t(a)$ is the number of times arm a has been played so far, and $\bar{\mu}_t(a)$ is the average reward from this arm. As we've seen before, $\mu(a) \in [\,\texttt{LCB}_t(a),\ \texttt{UCB}_t(a)\,]$ with high probability.

The key lemma in the proof of Theorem 3.9 holds for a more general notion of the confidence bounds, whereby they can be arbitrary functions of the arm a and the t-history H_t: respectively, $U(a,\ H_t)$ and $L(a,\ H_t)$. There are two properties we want these functions to have, for some $\gamma > 0$ to be specified later:[2]

$$\mathbb{E}\left[\,[\,U(a,\ H_t) - \mu(a)\,]^-\,\right] \leq \tfrac{\gamma}{TK} \qquad \text{for all arms } a \text{ and rounds } t, \tag{3.14}$$

$$\mathbb{E}\left[\,[\,\mu(a) - L(a,\ H_t)\,]^-\,\right] \leq \tfrac{\gamma}{TK} \qquad \text{for all arms } a \text{ and rounds } t. \tag{3.15}$$

The first property says that the upper confidence bound U does not exceed the mean reward by too much *in expectation*, and the second property makes a similar statement about L. As usual, K denotes the number of arms. The confidence radius can be defined as $r(a,\ H_t) = \frac{U(a,\ H_t) - L(a,\ H_t)}{2}$.

Lemma 3.10. *Assume we have lower and upper bound functions that satisfy properties* (3.14) *and* (3.15)*, for some parameter $\gamma > 0$. Then Bayesian Regret of Thompson Sampling can be bounded as follows:*

$$\texttt{BR}(T) \leq 2\gamma + 2\sum_{t=1}^{T} \mathbb{E}\left[\,r(a_t,\ H_t)\,\right].$$

Proof. Fix round t. Interpreted as random variables, the chosen arm a_t and the best arm a^* are identically distributed given t-history H_t: for each feasible t-history H,

$$\Pr[a_t = a \mid H_t = H] = \Pr[a^* = a \mid H_t = H] \quad \text{for each arm } a.$$

It follows that

$$\mathbb{E}[\,U(a^*,\ H)\ \mid\ H_t = H\,] = \mathbb{E}[\,U(a_t,\ H)\ \mid\ H_t = H\,]. \tag{3.16}$$

Then Bayesian Regret suffered in round t is

$$\begin{aligned} \texttt{BR}_t &:= \mathbb{E}\left[\,\mu(a^*) - \mu(a_t)\,\right] \\ &= \mathop{\mathbb{E}}_{H\sim H_t}\left[\,\mathbb{E}\left[\,\mu(a^*) - \mu(a_t) \mid H_t = H\,\right]\right] \\ &= \mathop{\mathbb{E}}_{H\sim H_t}\left[\,\mathbb{E}\left[\,U(a_t,H) - \mu(a_t) + \mu(a^*) - U(a^*,H) \mid H_t = H\,\right]\right] && \textit{(by Eq. (3.16))} \\ &= \underbrace{\mathbb{E}\left[\,U(a_t,H_t) - \mu(a_t)\,\right]}_{\text{Summand 1}} + \underbrace{\mathbb{E}\left[\,\mu(a^*) - U(a^*,H_t)\,\right]}_{\text{Summand 2}}. \end{aligned}$$

[2] As a matter of notation, x^- is the negative portion of the number x, *i.e.,* $x^- = 0$ if $x \geq 0$, and $x^- = |x|$ otherwise.

We will use properties (3.14) and (3.15) to bound both summands. Note that we cannot *immediately* use these properties because they assume a fixed arm a, whereas both a_t and a^* are random variables.

$$\begin{aligned}
&\mathbb{E}\left[\,\mu(a^*) - U(a^*, H_t)\,\right] && \textit{(Summand 1)}\\
&\qquad \leq \mathbb{E}\left[\,(\mu(a^*) - U(a^*, H_t))^+\,\right]\\
&\qquad \leq \mathbb{E}\left[\sum_{\text{arms } a} \left[\,\mu(a) - U(a, H_t)\,\right]^+\right]\\
&\qquad = \textstyle\sum_{\text{arms } a} \mathbb{E}\left[\,(\,U(a, H_t) - \mu(a)\,)^-\,\right]\\
&\qquad \leq K \cdot \frac{\gamma}{KT} = \frac{\gamma}{T} && \textit{(by property (3.14)).}\\
&\mathbb{E}\left[\,U(a_t, H_t) - \mu(a_t)\,\right] && \textit{(Summand 2)}\\
&\qquad = \mathbb{E}\left[\,2r(a_t, H_t) + L(a_t, H_t) - \mu(a_t)\,\right] && \textit{(by definition of } r_t(\cdot)\textit{)}\\
&\qquad = \mathbb{E}\left[\,2r(a_t, H_t)\,\right] + \mathbb{E}\left[\,L(a_t, H_t) - \mu(a_t)\,\right]\\
&\mathbb{E}[L(a_t, H_t) - \mu(a_t)] \leq \mathbb{E}\left[\,(\,L(a_t, H_t) - \mu(a_t)\,)^+\,\right]\\
&\qquad \leq \mathbb{E}\left[\sum_{\text{arms } a} (\,L(a, H_t) - \mu(a)\,)^+\right]\\
&\qquad = \sum_{\text{arms } a} \mathbb{E}\left[\,(\,\mu(a) - L(a, H_t)\,)^-\,\right]\\
&\qquad \leq K \cdot \frac{\gamma}{KT} = \frac{\gamma}{T} && \textit{(by property (3.15)).}
\end{aligned}$$

Thus, $\texttt{BR}_t(T) \leq 2\frac{\gamma}{T} + 2\ \mathbb{E}[r(a_t, H_t)]$. The theorem follows by summing up over all rounds t. □

Remark 3.11. Thompson Sampling does not need to know what U and L are!

Remark 3.12. Lemma 3.10 does not rely on any specific structure of the prior. Moreover, it can be used to upper-bound Bayesian regret of Thompson Sampling for a particular *class* of priors whenever one has "nice" confidence bounds U and L for this class.

Proof of Theorem 3.9. Let us use the confidence bounds and the confidence radius from (3.13). Note that they satisfy properties (3.14) and (3.15) with $\gamma = 2$. By Lemma 3.10,

$$\texttt{BR}(T) \leq O\left(\sqrt{\log T}\right) \sum_{t=1}^{T} \mathbb{E}\left[\frac{1}{\sqrt{n_t(a_t)}}\right].$$

Moreover,

$$\begin{aligned}
\sum_{t=1}^{T} \sqrt{\frac{1}{n_t(a_t)}} &= \sum_{\text{arms } a} \sum_{\text{rounds } t:\ a_t = a} \frac{1}{\sqrt{n_t(a)}}\\
&= \sum_{\text{arms } a} \sum_{j=1}^{n_{T+1}(a)} \frac{1}{\sqrt{j}} = \sum_{\text{arms } a} O(\sqrt{n(a)}).
\end{aligned}$$

It follows that

$$\texttt{BR}(T) \leq O\left(\sqrt{\log T}\right) \sum_{\text{arms } a} \sqrt{n(a)} \leq O\left(\sqrt{\log T}\right) \sqrt{K \sum_{\text{arms } a} n(a)} = O\left(\sqrt{KT \log T}\right),$$

where the intermediate step is by the arithmetic vs. quadratic mean inequality. □

3.4 Thompson Sampling with no prior (and no proofs)

Thompson Sampling can also be used for the original problem of stochastic bandits, *i.e.,* the problem without a built-in prior $\mathbb{P}$. Then $\mathbb{P}$ is a "fake prior": just a parameter to the algorithm, rather than a feature of reality. Prior work considered two such "fake priors":

(i) independent uniform priors and 0-1 rewards.

(ii) independent standard Gaussian priors and unit-variance Gaussian rewards.

Remark 3.13. Under both approaches, the prior specifies the "shape" of the reward distributions: resp., Bernoulli and unit-variance Gaussian. However, this prior is just a parameter in the algorithm, and does *not* imply an assumption on the actual reward distributions. In particular, whenever some reward $r_t \notin \{0, 1\}$ is received under approach (i), one flips a random coin with expectation r_t, and pass the outcome of this coin flip to Thompson Sampling (so that the input is consistent with the prior). Approach (ii) treats the realized rewards *as if* they are generated by a unit-variance Gaussian.

Let us state the regret bounds for both approaches.

Theorem 3.14. *Thompson Sampling, with approach (i) or (ii), achieves expected regret*

$$\mathbb{E}[R(T)] \leq O(\sqrt{KT \log T}).$$

Theorem 3.15. *For each problem instance, Thompson Sampling with approach (i) achieves, for all $\epsilon > 0$,*

$$\mathbb{E}[R(T)] \leq (1+\epsilon)\, C\, \log(T) + \frac{f(\mu)}{\epsilon^2}, \quad \textit{where} \quad C = \sum_{\textit{arms } a:\, \Delta(a)<0} \frac{\mu(a^*) - \mu(a)}{\mathtt{KL}(\mu(a), \mu^*)}.$$

Here $f(\mu)$ depends on the mean reward vector μ, but not on ϵ or T.

The C term is the optimal constant in the regret bounds: it matches the constant in Theorem 2.16. This is a partial explanation for why Thompson Sampling is so good in practice. However, this is not a *full* explanation because the term $f(\mu)$ can be quite big, as far as one can prove.

3.5 Literature review and discussion

Thompson Sampling is the first bandit algorithm in the literature (Thompson, 1933). While it has been well-known for a long time, strong provable guarantees did not appear until recently. A detailed survey for various variants and developments can be found in Russo et al. (2018).

The material in Section 3.3 is from Russo and Van Roy (2014).[3] They refine this approach to obtain improved upper bounds for some specific classes of priors, including priors over linear and "generalized linear" mean reward vectors, and priors given by a Gaussian Process. Bubeck and Liu (2013) obtain $O(\sqrt{KT})$ regret for arbitrary priors, shaving off the $\log(T)$ factor from Theorem 3.9. Russo and Van Roy (2016) obtain regret bounds that scale with the entropy of the optimal-action distribution induced by the prior.

The prior-independent results in Section 3.4 are from (Agrawal and Goyal, 2012, 2013, 2017) and Kaufmann et al. (2012). The first "prior-independent" regret bound for Thompson Sampling, a weaker version of Theorem 3.15, has appeared in Agrawal and Goyal (2012). Theorem 3.14 is from Agrawal and Goyal (2013, 2017).[4] They also prove a matching lower bound for the Bayesian regret of Thompson Sampling for standard-Gaussian priors. Theorem 3.15 is from (Kaufmann et al., 2012; Agrawal and Goyal, 2013, 2017).[5]

[3] Lemma 3.10 follows from Russo and Van Roy (2014), making their technique more transparent.

[4] For standard-Gaussian priors, Agrawal and Goyal (2013, 2017) achieve a slightly stronger version, $O(\sqrt{KT \log K})$.

[5] Kaufmann et al. (2012) prove a slightly weaker version in which $\ln(T)$ is replaced with $\ln(T) + \ln\ln(T)$.

Bubeck et al. (2015); Zimmert and Lattimore (2019); Bubeck and Sellke (2020) extend Thompson Sampling to *adversarial bandits* (which are discussed in Chapter 6). In particular, variants of the algorithm have been used in some of the recent state-of-art results on bandits (Bubeck et al., 2015; Zimmert and Lattimore, 2019), building on the analysis technique from Russo and Van Roy (2016).

Chapter 4

Bandits with Similarity Information

We consider stochastic bandit problems in which an algorithm has auxiliary information on similarity between arms. We focus on *Lipschitz bandits*, where the similarity information is summarized via a Lipschitz constraint on the expected rewards. Unlike the basic model in Chapter 1, Lipschitz bandits remain tractable even if the number of arms is very large or infinite.

Prerequisites: Chapter 1; Chapter 2 (results and construction only).

A bandit algorithm may have auxiliary information on similarity between arms, so that "similar" arms have similar expected rewards. For example, arms can correspond to "items" (*e.g.,* documents) with feature vectors, and similarity between arms can be expressed as (some version of) distance between the feature vectors. Another example is dynamic pricing and similar problems, where arms correspond to offered prices for buying, selling or hiring; then similarity between arms is simply the difference between prices. In our third example, arms are configurations of a complex system, such as a server or an ad auction.

We capture these examples via an abstract model called *Lipschitz bandits*.[1] We consider stochastic bandits, as defined in Chapter 1. Specifically, the reward for each arm x is an independent sample from some fixed but unknown distribution whose expectation is denoted $\mu(x)$ and realizations lie in $[0,1]$. This basic model is endowed with auxiliary structure which expresses similarity. In the paradigmatic special case, called *continuum-armed bandits*, arms correspond to points in the interval $X = [0,1]$, and expected rewards obey a Lipschitz condition:

$$|\mu(x) - \mu(y)| \leq L \cdot |x - y| \quad \text{for any two arms } x, y \in X, \tag{4.1}$$

where L is a constant known to the algorithm.[2] In the general case, arms lie in an arbitrary metric space $(X, \mathcal{D})$ which is known to the algorithm, and the right-hand side in (4.1) is replaced with $\mathcal{D}(x, y)$.

The technical material is organized as follows. We start with fundamental results on continuum-armed bandits. Then we present the general case of Lipschitz bandits and recover the same results; along the way, we present sufficient background on metric spaces. We proceed to develop a more advanced algorithm which takes advantage of "nice" problem instances. Many auxiliary results are deferred to exercises, particularly those on lower bounds, metric dimensions, and dynamic pricing.

4.1 Continuum-armed bandits

To recap, continuum-armed bandits (*CAB*) is a version of stochastic bandits where the set of arms is the interval $X = [0,1]$ and mean rewards satisfy (4.1) with some known Lipschitz constant L. Note that we have infinitely many arms, and in fact, *continuously* many. While bandit problems with a very large, let alone infinite, number of arms are hopeless in general, CAB is tractable due to the Lipschitz condition.

[1] We discuss some non-Lipschitz models of similarity in the literature review. In particular, the basic versions of dynamic pricing and similar problems naturally satisfy a 1-sided version of Lipschitzness, which suffices for our purposes.

[2] A function $\mu : X \to \mathbb{R}$ which satisfies (4.1) is called *Lipschitz-continuous* on X, with *Lipschitz constant* L.

A problem instance is specified by reward distributions, time horizon T, and Lipschitz constant L. As usual, we are mainly interested in the mean rewards $\mu(\cdot)$ as far as reward distributions are concerned, while the exact shape thereof is mostly unimportant.[3]

4.1.1 Simple solution: fixed discretization

A simple but powerful technique, called *fixed discretization*, works as follows. We pick a fixed, finite set of arms $S \subset X$, called *discretization* of X, and use this set as an approximation for X. Then we focus only on arms in S, and run an off-the-shelf algorithm ALG for stochastic bandits, such as UCB1 or Successive Elimination, that only considers these arms. Adding more points to S makes it a better approximation of X, but also increases regret of ALG on S. Thus, S should be chosen so as to optimize this tradeoff.

The best arm in S is denoted $\mu^*(S) = \sup_{x \in S} \mu(x)$. In each round, algorithm ALG can only hope to approach expected reward $\mu^*(S)$, and additionally suffers *discretization error*

$$\texttt{DE}(S) = \mu^*(X) - \mu^*(S). \tag{4.2}$$

More formally, we can represent the algorithm's expected regret as

$$\begin{aligned}
\mathbb{E}[R(T)] &:= T \cdot \mu^*(X) - W(\texttt{ALG}) \\
&= (\, T \cdot \mu^*(S) - W(\texttt{ALG})\,) + T \cdot (\, \mu^*(X) - \mu^*(S)\,) \\
&= \mathbb{E}[R_S(T)] + T \cdot \texttt{DE}(S),
\end{aligned}$$

where $W(\texttt{ALG})$ is the expected total reward of the algorithm, and $R_S(T)$ is the regret relative to $\mu^*(S)$.

Let us assume that ALG attains the near-optimal regret rate from Chapter 1:

$$\mathbb{E}[R_S(T)] \leq c_{\texttt{ALG}} \cdot \sqrt{|S|\, T \log T} \quad \text{for any subset of arms } S \subset X, \tag{4.3}$$

where $c_{\texttt{ALG}}$ is an absolute constant specific to ALG. Then

$$\mathbb{E}[R(T)] \leq c_{\texttt{ALG}} \cdot \sqrt{|S|\, T \log T} + T \cdot \texttt{DE}(S). \tag{4.4}$$

This is a concrete expression for the tradeoff between the size of S and its discretization error.

Uniform discretization divides the interval $[0, 1]$ into intervals of fixed length ϵ, called *discretization step*, so that S consists of all integer multiples of ϵ. It is easy to see that $\texttt{DE}(S) \leq L\epsilon$. Indeed, if x^* is a best arm on X, and y is the closest arm to x^* that lies in S, it follows that $|x^* - y| \leq \epsilon$, and therefore $\mu(x^*) - \mu(y) \leq L\epsilon$. To optimize regret, we approximately equalize the two summands in (4.4).

Theorem 4.1. *Consider continuum-armed bandits with Lipschitz constant L and time horizon T. Uniform discretization with algorithm* ALG *satisfying (4.3) and discretization step $\epsilon = (TL^2/\log T)^{-1/3}$ attains*

$$\mathbb{E}[R(T)] \leq L^{1/3} \cdot T^{2/3} \cdot (1 + c_{\texttt{ALG}})(\log T)^{1/3}.$$

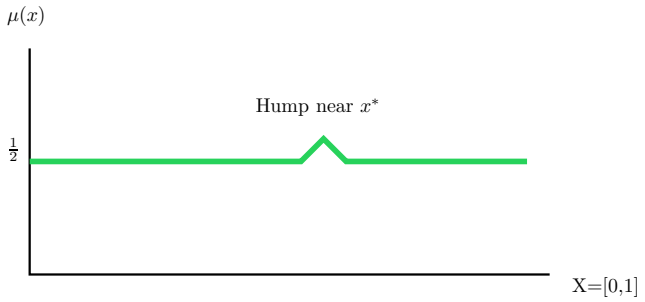
The main take-away here is the $\tilde{O}\left(L^{1/3} \cdot T^{2/3}\right)$ regret rate. The explicit constant and logarithmic dependence are less important.

[3] However, recall that some reward distributions allow for smaller confidence radii, *e.g.,* see Exercise 1.1.

4.1.2 Lower Bound

Uniform discretization is in fact optimal in the worst case: we have an $\Omega\left(L^{1/3} \cdot T^{2/3}\right)$ lower bound on regret. We prove this lower bound via a relatively simple reduction from the main lower bound, the $\Omega(\sqrt{KT})$ lower bound from Chapter 2, henceforth called `MainLB`.

The new lower bound involves problem instances with 0-1 rewards and the following structure. There is a unique best arm x^* with $\mu(x^*) = 1/2 + \epsilon$, where $\epsilon > 0$ is a parameter to be adjusted later in the analysis. All arms x have mean reward $\mu(x) = 1/2$ except those near x^*. The Lipschitz condition requires a smooth transition between x^* and the faraway arms; hence, we will have a "bump" around x^*.



Formally, we define a problem instance $\mathcal{I}(x^*, \epsilon)$ by

$$\mu(x) = \begin{cases} 1/2, & \text{all arms } x \text{ with } |x - x^*| \geq \epsilon/L \\ 1/2 + \epsilon - L \cdot |x - x^*|, & \text{otherwise.} \end{cases} \tag{4.5}$$

It is easy to see that any such problem instance satisfies the Lipschitz Condition (4.1). We will refer to $\mu(\cdot)$ as the *bump function*. We are ready to state the lower bound:

Theorem 4.2. *Let* `ALG` *be any algorithm for continuum-armed bandits with time horizon T and Lipschitz constant L. There exists a problem instance $\mathcal{I} = \mathcal{I}(x^*, \epsilon)$, for some $x^* \in [0, 1]$ and $\epsilon > 0$, such that*

$$\mathbb{E}\left[\, R(T) \mid \mathcal{I} \,\right] \geq \Omega\left(L^{1/3} \cdot T^{2/3}\right). \tag{4.6}$$

For simplicity of exposition, assume the Lipschitz constant is $L = 1$; arbitrary L is treated similarly.

Fix $K \in \mathbb{N}$ and partition arms into K disjoint intervals of length $\frac{1}{K}$. Use bump functions with $\epsilon = \frac{1}{2K}$ such that each interval either contains a bump or is completely flat. More formally, we use problem instances $\mathcal{I}(x^*, \epsilon)$ indexed by $a^* \in [K] := \{1, \ldots, K\}$, where the best arm is $x^* = (2\epsilon - 1) \cdot a^* + \epsilon$.

The intuition for the proof is as follows. We have these K intervals defined above. Whenever an algorithm chooses an arm x in one such interval, choosing the *center* of this interval is best: either this interval contains a bump and the center is the best arm, or all arms in this interval have the same mean reward $1/2$. But if we restrict to arms that are centers of the intervals, we have a family of problem instances of K-armed bandits, where all arms have mean reward $1/2$ except one with mean reward $1/2 + \epsilon$. This is precisely the family of instances from `MainLB`. Therefore, one can apply the lower bound from `MainLB`, and tune the parameters to obtain (4.6). To turn this intuition into a proof, the main obstacle is to prove that choosing the center of an interval is really the best option. While this is a trivial statement for the immediate round, we need to argue carefullt that choosing an arm elsewhere would not be advantageous later on.

Let us recap `MainLB` in a way that is convenient for this proof. Recall that `MainLB` considered problem instances with 0-1 rewards such that all arms a have mean reward $1/2$, except the best arm a^* whose mean reward is $1/2 + \epsilon$. Each instance is parameterized by best arm a^* and $\epsilon > 0$, and denoted $\mathcal{J}(a^*, \epsilon)$.

Theorem 4.3 (MainLB). *Consider stochastic bandits, with K arms and time horizon T (for any K, T). Let* ALG *be any algorithm for this problem. Pick any positive $\epsilon \leq \sqrt{cK/T}$, where c is an absolute constant from Lemma 2.8. Then there exists a problem instance $\mathcal{J} = \mathcal{J}(a^*, \epsilon)$, $a^* \in [K]$, such that*

$$\mathbb{E}\left[R(T) \mid \mathcal{J}\right] \geq \Omega(\epsilon T).$$

To prove Theorem 4.2, we show how to reduce the problem instances from MainLB to CAB in a way that we can apply MainLB and derive the claimed lower bound (4.6). On a high level, our plan is as follows: (i) take any problem instance $\mathcal{J}$ from MainLB and "embed" it into CAB, and (ii) show that any algorithm for CAB will, in fact, need to solve $\mathcal{J}$, (iii) tune the parameters to derive (4.6).

Proof of Theorem 4.2 ($L = 1$). We use the problem instances $\mathcal{I}(x^*, \epsilon)$ as described above. More precisely, we fix $K \in \mathbb{N}$, to be specified later, and let $\epsilon = \frac{1}{2K}$. We index the instances by $a^* \in [K]$, so that

$$x^* = f(a^*), \text{ where } f(a^*) := (2\epsilon - 1) \cdot a^* + \epsilon.$$

We use problem instances $\mathcal{J}(a^*, \epsilon)$ from MainLB, with K arms and the same time horizon T. The set of arms in these instances is denoted $[K]$. Each instance $\mathcal{J} = \mathcal{J}(a^*, \epsilon)$ corresponds to an instance $\mathcal{I} = \mathcal{I}(x^*, \epsilon)$ of CAB with $x^* = f(a^*)$. In particular, each arm $a \in [K]$ in $\mathcal{J}$ corresponds to an arm $x = f(a)$ in $\mathcal{I}$. We view $\mathcal{J}$ as a discrete version of $\mathcal{I}$. In particular, we have $\mu_{\mathcal{J}}(a) = \mu(f(a))$, where $\mu(\cdot)$ is the reward function for $\mathcal{I}$, and $\mu_{\mathcal{J}}(\cdot)$ is the reward function for $\mathcal{J}$.

Consider an execution of ALG on problem instance $\mathcal{I}$ of CAB, and use it to construct an algorithm ALG$'$ which solves an instance $\mathcal{J}$ of K-armed bandits. Each round in algorithm ALG$'$ proceeds as follows. ALG is called and returns some arm $x \in [0, 1]$. This arm falls into the interval $[f(a) - \epsilon,\ f(a) + \epsilon)$ for some $a \in [K]$. Then algorithm ALG$'$ chooses arm a. When ALG$'$ receives reward r, it uses r and x to compute reward $r_x \in \{0, 1\}$ such that $\mathbb{E}[r_x \mid x] = \mu(x)$, and feed it back to ALG. We summarize this in a table:

ALG for CAB instance $\mathcal{I}$	ALG$'$ for K-armed bandits instance $\mathcal{J}$
chooses arm $x \in [0, 1]$	
	chooses arm $a \in [K]$ with $x \in [f(a) - \epsilon,\ f(a) + \epsilon)$
	receives reward $r \in \{0, 1\}$ with mean $\mu_{\mathcal{J}}(a)$
receives reward $r_x \in \{0, 1\}$ with mean $\mu(x)$	

To complete the specification of ALG$'$, it remains to define reward r_x so that $\mathbb{E}[r_x \mid x] = \mu(x)$, and r_x can be computed using information available to ALG$'$ in a given round. In particular, the computation of r_x cannot use $\mu_{\mathcal{J}}(a)$ or $\mu(x)$, since they are not know to ALG$'$. We define r_x as follows:

$$r_x = \begin{cases} r & \text{with probability } p_x \in [0, 1] \\ X & \text{otherwise,} \end{cases} \tag{4.7}$$

where X is an independent toss of a Bernoulli random variable with expectation $1/2$, and probability p_x is to be specified later. Then

$$\begin{aligned} \mathbb{E}[r_x \mid x] &= p_x \cdot \mu_{\mathcal{J}}(a) + (1 - p_x) \cdot 1/2 \\ &= 1/2 + (\mu_{\mathcal{J}}(a) - 1/2) \cdot p_x \\ &= \begin{cases} 1/2 & \text{if } x \neq x^* \\ 1/2 + \epsilon p_x & \text{if } x = x^* \end{cases} \\ &= \mu(x) \end{aligned}$$

if we set $p_x = 1 - |x - f(a)|/\epsilon$ so as to match the definition of $\mathcal{I}(x^*, \epsilon)$ in Eq. (4.5).

For each round t, let x_t and a_t be the arms chosen by ALG and ALG$'$, resp. Since $\mu_{\mathcal{J}}(a_t) \geq 1/2$, we have

$$\mu(x_t) \leq \mu_{\mathcal{J}}(a_t).$$

It follows that the total expected reward of ALG on instance $\mathcal{I}$ cannot exceed that of ALG$'$ on instance $\mathcal{J}$. Since the best arms in both problem instances have the same expected rewards $1/2 + \epsilon$, it follows that

$$\mathbb{E}[R(T) \mid \mathcal{I}] \geq \mathbb{E}[R'(T) \mid \mathcal{J}],$$

where $R(T)$ and $R'(T)$ denote regret of ALG and ALG$'$, respectively.

Recall that algorithm ALG$'$ can solve any K-armed bandits instance of the form $\mathcal{J} = \mathcal{J}(a^*, \epsilon)$. Let us apply Theorem 4.3 to derive a lower bound on the regret of ALG$'$. Specifically, let us fix $K = (T/4c)^{1/3}$, so as to ensure that $\epsilon \leq \sqrt{cK/T}$, as required in Theorem 4.3. Then for some instance $\mathcal{J} = \mathcal{J}(a^*, \epsilon)$,

$$\mathbb{E}[R'(T) \mid \mathcal{J}] \geq \Omega(\sqrt{\epsilon T}) = \Omega(T^{2/3})$$

Thus, taking the corresponding instance $\mathcal{I}$ of CAB, we conclude that $\mathbb{E}[R(T) \mid \mathcal{I}] \geq \Omega(T^{2/3})$. □

4.2 Lipschitz bandits

A useful interpretation of continuum-armed bandits is that arms lie in a known metric space $(X, \mathcal{D})$, where $X = [0, 1]$ is a ground set and $\mathcal{D}(x, y) = L \cdot |x - y|$ is the metric. In this section we extend this problem and the uniform discretization approach to arbitrary metric spaces.

The general problem, called *Lipschitz bandits Problem*, is a stochastic bandit problem such that the expected rewards $\mu(\cdot)$ satisfy the Lipschitz condition relative to some known metric $\mathcal{D}$ on the set X of arms:

$$|\mu(x) - \mu(y)| \leq \mathcal{D}(x, y) \quad \text{for any two arms } x, y. \tag{4.8}$$

The metric space $(X, \mathcal{D})$ can be arbitrary, as far as this problem formulation is concerned (see Section 4.2.1). It represents an abstract notion of known similarity between arms. Note that w.l.o.g. $\mathcal{D}(x, y) \leq 1$. The set of arms X can be finite or infinite, this distinction is irrelevant to the essence of the problem. While some of the subsequent definitions are more intuitive for infinite X, we state them so that they are meaningful for finite X, too. A problem instance is specified by the metric space $(X, \mathcal{D})$, reward distributions, and the time horizon T; for reward distributions, we are mainly interested in mean rewards $\mu(\cdot)$.

That $\mathcal{D}$ is a metric is without loss of generality, in the following sense. Suppose the algorithm is given constraints $|\mu(x) - \mu(y)| \leq \mathcal{D}_0(x, y)$ for all arms $x \neq y$ and some numbers $\mathcal{D}_0(x, y) \in (0, 1]$. Then one could define $\mathcal{D}$ as the shortest-paths closure of $\mathcal{D}_0$, *i.e.,* $\mathcal{D}(x, y) = \min \sum_i \mathcal{D}_0(x_i, x_{i+1})$, where the $\min$ is over all finite sequences of arms $\sigma = (x_1, \ldots, x_{n_\sigma})$ which start with x and end with y.

4.2.1 Brief background on metric spaces

A metric space is a pair $(X, \mathcal{D})$, where X is a set (called the *ground set*) and $\mathcal{D}$ is a *metric* on X, *i.e.,* a function $\mathcal{D} : X \times X \to \mathbb{R}$ which satisfies the following axioms:

$$\begin{aligned}
&\mathcal{D}(x, y) \geq 0 && \textit{(non-negativity)},\\
&\mathcal{D}(x, y) = 0 \Leftrightarrow x = y && \textit{(identity of indiscernibles)},\\
&\mathcal{D}(x, y) = \mathcal{D}(y, x) && \textit{(symmetry)},\\
&\mathcal{D}(x, z) \leq \mathcal{D}(x, y) + \mathcal{D}(y, z) && \textit{(triangle inequality)}.
\end{aligned}$$

Intuitively, the metric represents "distance" between the elements of X. For $Y \subset X$, the pair $(Y, \mathcal{D})$ is also a metric space, where, by a slight abuse of notation, $\mathcal{D}$ denotes the same metric restricted Y. The *diameter* of Y is the maximal distance between any two points in Y, *i.e.,* $\sup_{x,y \in Y} \mathcal{D}(x,y)$. A metric space $(X, \mathcal{D})$ is called finite (resp., infinite) if so is X.

Some notable examples:

- $X = [0,1]^d$, $d \in \mathbb{N}$ and the metric is the p-norm, $p \geq 1$:

$$\ell_p(x,y) = \|x - y\|_p := \left(\sum_{i=1}^{d} (x_i - y_i)^p \right)^{1/p}.$$

  Most commonly are ℓ_1-metric (a.k.a. Manhattan metric) and ℓ_2-metric (a.k.a. Euclidean distance).

- $X = [0,1]$ and the metric is $\mathcal{D}(x,y) = |x-y|^{1/d}$, $d \geq 1$. In a more succinct notation, this metric space is denoted $\left([0,1],\, \ell_2^{1/d}\right)$.

- X is the set of nodes of a graph, and $\mathcal{D}$ is the *shortest-path metric*: $\mathcal{D}(x,y)$ is the length of a shortest path between nodes x and y.

- X is the set of leaves in a rooted tree, where each internal node u is assigned a weight $w(u) > 0$. The *tree distance* between two leaves x and y is $w(u)$, where u is the least common ancestor of x and y. (Put differently, u is the root of the smallest subtree containing both x and y.) In particular, *exponential tree distance* assigns weight c^h to each node at depth h, for some constant $c \in (0,1)$.

4.2.2 Uniform discretization

We generalize uniform discretization to arbitrary metric spaces as follows. We take an algorithm ALG satisfying (4.3), as in Section 4.1.1, and we run it on a fixed subset of arms $S \subset X$.

Definition 4.4. A subset $S \subset X$ is called an *ϵ-mesh*, $\epsilon > 0$, if every point $x \in X$ is within distance ϵ from S, in the sense that $\mathcal{D}(x,y) \leq \epsilon$ for some $y \in S$.

It is easy to see that the discretization error of an ϵ-mesh is $\texttt{DE}(S) \leq \epsilon$. The developments in Section 4.1.1 carry over word-by-word, up until Eq. (4.4). We summarize these developments as follows.

Theorem 4.5. *Consider Lipschitz bandits with time horizon T. Optimizing over the choice of an ϵ-mesh, uniform discretization with algorithm* ALG *satisfying (4.3) attains regret*

$$\mathbb{E}[R(T)] \leq \inf_{\epsilon > 0,\ \epsilon\text{-mesh } S} \epsilon T + c_{\texttt{ALG}} \cdot \sqrt{|S|\, T \log T}. \tag{4.9}$$

Remark 4.6. How do we *construct* a good ϵ-mesh? For many examples the construction is trivial, *e.g.,* a uniform discretization in continuum-armed bandits or in the next example. For an arbitrary metric space and a given $\epsilon > 0$, a standard construction starts with an empty set S, adds any arm that is within distance $> \epsilon$ from all arms in S, and stops when such arm does not exist. Now, consider different values of ϵ in exponentially decreasing order: $\epsilon = 2^{-i}$, $i = 1, 2, 3, \ldots$. For each such ϵ, compute an ϵ-mesh S_ϵ as specified above, and stop at the largest ϵ such that $\epsilon T \geq c_{\texttt{ALG}} \cdot \sqrt{|S_\epsilon|\, T \log T}$. This ϵ-mesh optimizes the right-hand side in (4.9) up to a constant factor, see Exercise 4.1

Example 4.7. To characterize the regret of uniform discretization more compactly, suppose the metric space is $X = [0,1]^d$, $d \in \mathbb{N}$ under the ℓ_p metric, $p \geq 1$. Consider a subset $S \subset X$ that consists of all points whose coordinates are multiples of a given $\epsilon > 0$. Then $|S| \leq \lceil 1/\epsilon \rceil^d$ points, and its discretization error is $\texttt{DE}(S) \leq c_{p,d} \cdot \epsilon$, where $c_{p,d}$ is a constant that depends only on p and d; *e.g.,* $c_{p,d} = d$ for $p = 1$. Plugging this into (4.4) and taking $\epsilon = (T \,/\, \log T)^{-1/(d+2)}$, we obtain

$$\mathbb{E}[R(T)] \leq (1 + c_{\texttt{ALG}}) \cdot T^{(d+1)/(d+2)} \, (c \log T)^{1/(d+2)}.$$

The $\tilde{O}(T^{(d+1)/(d+2)})$ regret rate in Example 4.7 extends to arbitrary metric spaces via the notion of *covering dimension*. In essence, it is the smallest d such that each $\epsilon > 0$ admits an ϵ-mesh of size $O(\epsilon^{-d})$.

Definition 4.8. Fix a metric space $(X, \mathcal{D})$. An *ϵ-covering*, $\epsilon > 0$ is a collection of subsets $X_i \subset X$ such that the diameter of each subset is at most ϵ and $X = \bigcup X_i$. The smallest number of subsets in an ϵ-covering is called the *covering number* and denoted $N_\epsilon(X)$. The *covering dimension*, with multiplier $c > 0$, is

$$\texttt{COV}_c(X) = \inf_{d \geq 0} \left\{ N_\epsilon(X) \leq c \cdot \epsilon^{-d} \quad \forall \epsilon > 0 \right\}. \tag{4.10}$$

Remark 4.9. Any ϵ-covering gives rise to an ϵ-mesh: indeed, just pick one arbitrary representative from each subset in the covering. Thus, there exists an ϵ-mesh S with $|S| = N_\epsilon(X)$. We define the covering numbers in terms of ϵ-coverings (rather than more directly in terms of ϵ-meshes) in order to ensure that $N_\epsilon(Y) \leq N_\epsilon(X)$ for any subset $Y \subset X$, and consequently $\texttt{COV}_c(Y) \leq \texttt{COV}_c(X)$. In words, covering numbers characterize complexity of a metric space, and a subset $Y \subset X$ cannot be more complex than X.

In particular, the covering dimension in Example 4.7 is d, with a large enough multiplier c. Likewise, the covering dimension of $\left([0,1],\, \ell_2^{1/d}\right)$ is d, for any $d \geq 1$; note that it can be fractional.

Remark 4.10. Covering dimension is often stated without the multiplier: $\texttt{COV}(X) = \inf_{c>0} \texttt{COV}_c(X)$. This version is meaningful (and sometimes more convenient) for infinite metric spaces. However, for finite metric spaces it is trivially 0, so we need to fix the multiplier for a meaningful definition. Furthermore, our definition allows for more precise regret bounds, both for finite and infinite metric spaces.

To de-mystify the infimum in Eq. (4.10), let us state a corollary: if $d = \texttt{COV}_c(X)$ then

$$N_\epsilon(X) \leq c' \cdot \epsilon^{-d} \qquad \forall c' > c,\ \epsilon > 0. \tag{4.11}$$

Thus, we take an ϵ-mesh S of size $|S| = N_\epsilon(X) \leq O(c/\epsilon^d)$. Taking $\epsilon = (T \,/\, \log T)^{-1/(d+2)}$, like in Example 4.7, and plugging this into into Eq. (4.4), we obtain the following theorem:

Theorem 4.11. *Consider Lipschitz bandits on a metric space $(X, \mathcal{D})$, with time horizon T. Let* ALG *be any algorithm satisfying (4.3). Fix any $c > 0$, and let $d = \texttt{COV}_c(X)$ be the covering dimension. Then there exists a subset $S \subset X$ such that running* ALG *on the set of arms S yields regret*

$$\mathbb{E}[R(T)] \leq (1 + c_{\texttt{ALG}}) \cdot T^{(d+1)/(d+2)} \cdot (c \, \log T)^{1/(d+2)}.$$

Specifically, S can be any ϵ-mesh of size $|S| \leq O(c/\epsilon^d)$, where $\epsilon = (T \,/\, \log T)^{-1/(d+2)}$; such S exists.

The upper bounds in Theorem 4.5 and Theorem 4.11 are the best possible in the worst case, up to $O(\log T)$ factors. The lower bounds follow from the same proof technique as Theorem 4.2, see the exercises in Section 4.5.2 for precise formulations and hints. A representative example is as follows:

Theorem 4.12. *Consider Lipschitz bandits on metric space $\left([0,1],\, \ell_2^{1/d}\right)$, for any $d \geq 1$, with time horizon T. For any algorithm, there exists a problem instance exists a problem instance $\mathcal{I}$ such that*

$$\mathbb{E}\left[\, R(T) \mid \mathcal{I} \,\right] \geq \Omega\left(T^{(d+1)/(d+2)} \right). \tag{4.12}$$

4.3 Adaptive discretization: the Zooming Algorithm

Despite the existence of a matching lower bound, the fixed discretization approach is wasteful. Observe that the discretization error of S is at most the minimal distance between S and the best arm x^*:

$$\texttt{DE}(S) \leq \mathcal{D}(S, x^*) := \min_{x \in S} \mathcal{D}(x, x^*).$$

So it should help to decrease $|S|$ while keeping $\mathcal{D}(S, x^*)$ constant. Thinking of arms in S as "probes" that the algorithm places in the metric space. If we know that x^* lies in a particular "region" of the metric space, then we do not need to place probes in other regions. Unfortunately, we do not know in advance where x^* is, so we cannot optimize S this way if S needs to be chosen in advance.

However, an algorithm could approximately learn the mean rewards over time, and adjust the placement of the probes accordingly, making sure that one has more probes in more "promising" regions of the metric space. This approach is called *adaptive discretization*. Below we describe one implementation of this approach, called the *zooming algorithm*. On a very high level, the idea is that we place more probes in regions that could produce better rewards, as far as the algorithm knowns, and less probes in regions which are known to yield only low rewards with high confidence. What can we hope to prove for this algorithm, given the existence of a matching lower bound for fixed discretization? The goal here is to attain the same worst-case regret as in Theorem 4.11, but do "better" on "nice" problem instances. Quantifying what we mean by "better" and "nice" here is an important aspect of the overall research challenge.

The zooming algorithm will bring together three techniques: the "UCB technique" from algorithm `UCB1`, the new technique of "adaptive discretization", and the "clean event" technique in the analysis.

4.3.1 Algorithm

The algorithm maintains a set $S \subset X$ of "active arms". In each round, some arms may be "activated" according to the "activation rule", and one active arm is selected to according to the "selection rule". Once an arm is activated, it cannot be "deactivated". This is the whole algorithm: we just need to specify the activation rule and the selection rule.

Confidence radius/ball. Fix round t and an arm x that is active at this round. Let $n_t(x)$ is the number of rounds before round t when this arm is chosen, and let $\mu_t(x)$ be the average reward in these rounds. The confidence radius of arm x at time t is defined as

$$r_t(x) = \sqrt{\frac{2 \log T}{n_t(x) + 1}}.$$

Recall that this is essentially the smallest number so as to guarantee with high probability that

$$|\mu(x) - \mu_t(x)| \leq r_t(x).$$

The *confidence ball* of arm x is a closed ball in the metric space with center x and radius $r_t(x)$.

$$\mathbf{B}_t(x) = \{y \in X : \ \mathcal{D}(x, y) \leq r_t(x)\}.$$

Activation rule. We start with some intuition. We will estimate the mean reward $\mu(x)$ of a given active arm x using only the samples from this arm. While samples from "nearby" arms could potentially improve the estimates, this choice simplifies the algorithm and the analysis, and does not appear to worsen the regret bounds. Suppose arm y is not active in round t, and lies very close to some active arm x, in the sense that

$\mathcal{D}(x,y) \ll r_t(x)$. Then the algorithm does not have enough samples of x to distinguish x and y. Thus, instead of choosing arm y the algorithm might as well choose arm x. We conclude there is no real need to activate y yet. Going further with this intuition, there is no real need to activate any arm that is covered by the confidence ball of any active arm. We would like to maintain the following invariant:

In each round, all arms are covered by confidence balls of the active arms. (4.13)

As the algorithm plays some arms over time, their confidence radii and the confidence balls get smaller, and some arm y may become uncovered. Then we simply activate it! Since immediately after activation the confidence ball of y includes the entire metric space, we see that the invariant is preserved.

Thus, the activation rule is very simple:

If some arm y becomes uncovered by confidence balls of the active arms, activate y.

With this activation rule, the zooming algorithm has the following "self-adjusting property". The algorithm "zooms in" on a given region R of the metric space (*i.e.,* activates many arms in R) if and only if the arms in R are played often. The latter happens (under any reasonable selection rule) if and only if the arms in R have high mean rewards.

Selection rule. We extend the technique from algorithm UCB1. If arm x is active at time t, we define

$$\texttt{index}_t(x) = \bar{\mu}_t(x) + 2r_t(x) \tag{4.14}$$

The selection rule is very simple:

Play an active arm with the largest index (break ties arbitrarily).

Recall that algorithm UCB1 chooses an arm with largest upper confidence bound (UCB) on the mean reward, defined as $\texttt{UCB}_t(x) = \mu_t(x) + r_t(x)$. So $\texttt{index}_t(x)$ is very similar, and shares the intuition behind UCB1: if $\texttt{index}_t(x)$ is large, then either $\mu_t(x)$ is large, and so x is likely to be a good arm, or $r_t(x)$ is large, so arm x has not been played very often, and should probably be explored more. And the '+' in (4.14) is a way to trade off exploration and exploitation. What is new here, compared to UCB1, is that $\texttt{index}_t(x)$ is a UCB not only on the mean reward of x, but also on the mean reward of any arm in the confidence ball of x.

To summarize, the algorithm is as follows:

Initialize: set of active arms $S \leftarrow \emptyset$.
for *each round* $t = 1, 2, \ldots$ **do**
  `// activation rule`
  if *some arm* y *is not covered by the confidence balls of active arms* **then**
    pick any such arm y and "activate" it: $S \leftarrow S \cap \{y\}$.
  `// selection rule`
  Play an active arm x with the largest $\texttt{index}_t(x)$.
end

Algorithm 4.1: Zooming algorithm for adaptive discretization.

4.3.2 Analysis: clean event

We define a "clean event" $\mathcal{E}$ much like we did in Chapter 1, and prove that it holds with high probability. The proof is more delicate than in Chapter 1, essentially because we cannot immediately take the Union Bound over all of X. The rest of the analysis would simply assume that this event holds.

We consider a $K \times T$ table of realized rewards, with T columns and a row for each arm x. The j-th column for arm x is the reward for the j-th time this arm is chosen by the algorithm. We assume, without loss of generality, that this entire table is chosen before round 1: each cell for a given arm is an independent draw from the reward distribution of this arm. The clean event is defined as a property of this reward table. For each arm x, the clean event is

$$\mathcal{E}_x = \{\ |\mu_t(x) - \mu(x)| \leq r_t(x) \quad \text{for all rounds } t \in [T+1]\ \}.$$

Here $[T] := \{1, 2, \ldots, T\}$. For convenience, we define $\mu_t(x) = 0$ if arm x has not yet been played by the algorithm, so that in this case the clean event holds trivially. We are interested in the event $\mathcal{E} = \cap_{x \in X} \mathcal{E}_x$.

To simplify the proof of the next claim, we assume that realized rewards take values on a finite set.

Claim 4.13. *Assume that realized rewards take values on a finite set. Then* $\Pr[\mathcal{E}] \geq 1 - \frac{1}{T^2}$.

Proof. By Hoeffding Inequality, $\Pr[\mathcal{E}_x] \geq 1 - \frac{1}{T^4}$ for each arm $x \in X$. However, one cannot immediately apply the Union Bound here because there may be too many arms.

Fix an instance of Lipschitz bandits. Let X_0 be the set of all arms that can possibly be activated by the algorithm on this problem instance. Note that X_0 is finite; this is because the algorithm is deterministic, the time horizon T is fixed, and, as we assumed upfront, realized rewards can take only finitely many values. (This is the only place where we use this assumption.)

Let N be the total number of arms activated by the algorithm. Define arms $y_j \in X_0$, $j \in [T]$, as follows

$$y_j = \begin{cases} j\text{-th arm activated}, & \text{if } j \leq N \\ y_N, & \text{otherwise.} \end{cases}$$

Here N and y_j's are random variables, the randomness coming from the reward realizations. Note that $\{y_1, \ldots, y_T\}$ is precisely the set of arms activated in a given execution of the algorithm. Since the clean event holds trivially for all arms that are not activated, the clean event can be rewritten as $\mathcal{E} = \cap_{j=1}^{T} \mathcal{E}_{y_j}$. In what follows, we prove that the clean event $\mathcal{E}_{y_j}$ happens with high probability for each $j \in [T]$.

Fix an arm $x \in X_0$ and fix $j \in [T]$. Whether the event $\{y_j = x\}$ holds is determined by the rewards of other arms. (Indeed, by the time arm x is selected by the algorithm, it is already determined whether x is the j-th arm activated!) Whereas whether the clean event $\mathcal{E}_x$ holds is determined by the rewards of arm x alone. It follows that the events $\{y_j = x\}$ and $\mathcal{E}_x$ are independent. Therefore, if $\Pr[y_j = x] > 0$ then

$$\Pr[\mathcal{E}_{y_j} \mid y_j = x] = \Pr[\mathcal{E}_x \mid y_j = x] = \Pr[\mathcal{E}_x] \geq 1 - \tfrac{1}{T^4}$$

Now we can sum over all $x \in X_0$:

$$\Pr[\mathcal{E}_{y_j}] = \sum_{x \in X_0} \Pr[y_j = x] \cdot \Pr[\mathcal{E}_{y_j} \mid x = y_j] \geq 1 - \tfrac{1}{T^4}$$

To complete the proof, we apply the Union bound over all $j \in [T]$:

$$\Pr[\mathcal{E}_{y_j}, j \in [T]] \geq 1 - \tfrac{1}{T^3}.$$ □

We assume the clean event $\mathcal{E}$ from here on.

4.3.3 Analysis: bad arms

Let us analyze the "bad arms": arms with low mean rewards. We establish two crucial properties: that active bad arms must be far apart in the metric space (Corollary 4.15), and that each "bad" arm cannot be played too often (Corollary 4.16). As usual, let $\mu^* = \sup_{x \in X} \mu(x)$ be the best reward, and let $\Delta(x) = \mu^* - \mu(x)$ denote the "gap" of arm x. Let $n(x) = n_{T+1}(x)$ be the total number of samples from arm x.

The following lemma encapsulates a crucial argument which connects the best arm and the arm played in a given round. In particular, we use the main trick from the analysis of `UCB1` and the Lipschitz property.

Lemma 4.14. *$\Delta(x) \leq 3\, r_t(x)$ for each arm x and each round t.*

Proof. Suppose arm x is played in this round. By the covering invariant, the best arm x^* was covered by the confidence ball of some active arm y, *i.e.,* $x^* \in \mathbf{B}_t(y)$. It follows that

$$\texttt{index}(x) \geq \texttt{index}(y) = \underbrace{\mu_t(y) + r_t(y)}_{\geq \mu(y)} + r_t(y) \geq \mu(x^*) = \mu^*$$

The last inequality holds because of the Lipschitz condition. On the other hand:

$$\texttt{index}(x) = \underbrace{\mu_t(x)}_{\leq \mu(x) + r_t(x)} + 2 \cdot r_t(x) \leq \mu(x) + 3 \cdot r_t(x)$$

Putting these two equations together: $\Delta(x) := \mu^* - \mu(x) \leq 3 \cdot r_t(x)$.

Now suppose arm x is not played in round t. If it has never been played before round t, then $r_t(x) > 1$ and the lemma follows trivially. Else, letting s be the last time when x has been played before round t, we see that $r_t(x) = r_s(x) \geq \Delta(x)/3$. □

Corollary 4.15. *For any two active arms x, y, we have $\mathcal{D}(x, y) > \frac{1}{3} \min\left(\Delta(x),\ \Delta(y)\right)$.*

Proof. W.l.o.g. assume that x has been activated before y. Let s be the time when y has been activated. Then $\mathcal{D}(x, y) > r_s(x)$ by the activation rule. And $r_s(x) \geq \Delta(x)/3$ by Lemma 4.14. □

Corollary 4.16. *For each arm x, we have $n(x) \leq O(\log T)\ \Delta^{-2}(x)$.*

Proof. Use Lemma 4.14 for $t = T$, and plug in the definition of the confidence radius. □

4.3.4 Analysis: covering numbers and regret

For $r > 0$, consider the set of arms whose gap is between r and $2r$:

$$X_r = \{x \in X : r \leq \Delta(x) < 2r\}.$$

Fix $i \in \mathbb{N}$ and let $Y_i = X_r$, where $r = 2^{-i}$. By Corollary 4.15, for any two arms $x, y \in Y_i$, we have $\mathcal{D}(x, y) > r/3$. If we cover Y_i with subsets of diameter $r/3$, then arms x and y cannot lie in the same subset. Since one can cover Y_i with $N_{r/3}(Y_i)$ such subsets, it follows that $|Y_i| \leq N_{r/3}(Y_i)$.

Using Corollary 4.16, we have:

$$R_i(T) := \sum_{x \in Y_i} \Delta(x) \cdot n_t(x) \leq \frac{O(\log T)}{\Delta(x)} \cdot N_{r/3}(Y_i) \leq \frac{O(\log T)}{r} \cdot N_{r/3}(Y_i).$$

Pick $\delta > 0$, and consider arms with $\Delta(\cdot) \leq \delta$ separately from those with $\Delta(\cdot) > \delta$. Note that the total regret from the former cannot exceed δ per round. Therefore:

$$\begin{aligned} R(T) &\leq \delta T + \sum_{i:\ r=2^{-i}>\delta} R_i(T) \\ &\leq \delta T + \sum_{i:\ r=2^{-i}>\delta} \frac{\Theta(\log T)}{r}\ N_{r/3}(Y_i) \qquad (4.15) \\ &\leq \delta T + O(c \cdot \log T) \cdot (\tfrac{1}{\delta})^{d+1} \qquad (4.16) \end{aligned}$$

where c is a constant and d is some number such that

$$N_{r/3}(X_r) \leq c \cdot r^{-d} \quad \forall r > 0.$$

The smallest such d is called the *zooming dimension*:

Definition 4.17. For an instance of Lipschitz MAB, the *zooming dimension* with multiplier $c > 0$ is

$$\inf_{d \geq 0} \left\{ N_{r/3}(X) \leq c \cdot r^{-d} \quad \forall r > 0 \right\}.$$

Choosing $\delta = (\frac{\log T}{T})^{1/(d+2)}$ in (4.16), we obtain $R(T) \leq O\left(T^{(d+1)/(d+2)}\ (c \log T)^{1/(d+2)} \right)$. Note that we make this chose in the analysis only; the algorithm does not depend on the δ.

Theorem 4.18. *Consider Lipschitz bandits with time horizon T. Assume that realized rewards take values on a finite set. For any given problem instance and any $c > 0$, the zooming algorithm attains regret*

$$\mathbb{E}[R(T)] \leq O\left(T^{(d+1)/(d+2)}\ (c \log T)^{1/(d+2)} \right),$$

where d is the zooming dimension with multiplier c.

While the covering dimension is a property of the metric space, the zooming dimension is a property of the problem instance: it depends not only on the metric space, but on the mean rewards. In general, the zooming dimension is at most as large as the covering dimension, but may be much smaller (see Exercises 4.5 and 4.6). This is because in the definition of the covering dimension one needs to cover all of X, whereas in the definition of the zooming dimension one only needs to cover set X_r

While the regret bound in Theorem 4.18 is appealingly simple, a more precise regret bound is given in (4.15). Since the algorithm does not depend on δ, this bound holds for all $\delta > 0$.

4.4 Literature review and discussion

Continuum-armed bandits have been introduced in Agrawal (1995), and further studied in (Kleinberg, 2004; Auer et al., 2007). Uniform discretization has been introduced in Kleinberg and Leighton (2003) for dynamic pricing, and in Kleinberg (2004) for continuum-armed bandits; Kleinberg et al. (2008b) observed that it easily extends to Lipschitz bandits. While doomed to $\tilde{O}(T^{2/3})$ regret in the worst case, uniform discretization yields $\tilde{O}(\sqrt{T})$ regret under strong concavity Kleinberg and Leighton (2003); Auer et al. (2007).

Lipschitz bandits have been introduced in Kleinberg et al. (2008b) and in a near-simultaneous and independent paper (Bubeck et al., 2011b). The zooming algorithm is from Kleinberg et al. (2008b), see Kleinberg et al. (2019) for a definitive journal version. Bubeck et al. (2011b) present a similar but technically different algorithm, with similar regret bounds.

Lower bounds for uniform discretization trace back to Kleinberg (2004), who introduces the proof technique in Section 4.1.2 and obtains Theorem 4.12. The other lower bounds follow easily from the same technique. The explicit dependence on L in Theorem 4.2 first appeared in Bubeck et al. (2011c), and the formulation in Exercise 4.4(b) is from Bubeck et al. (2011b). Our presentation in Section 4.1.2 roughly follows that in Slivkins (2014) and (Bubeck et al., 2011b).

A line of work that pre-dated and inspired Lipschitz bandits posits that the algorithm is given a "taxonomy" on arms: a tree whose leaves are arms, where arms in the same subtree being "similar" to one another (Kocsis and Szepesvari, 2006; Pandey et al., 2007a; Munos and Coquelin, 2007). Numerical similarity information is not revealed. While these papers report successful empirical performance of their algorithms on some examples, they do not lead to non-trivial regret bounds. Essentially, regret scales as the number of arms in the worst case, whereas in Lipschitz bandits regret is bounded in terms of covering numbers.

Covering dimension is closely related to several other "dimensions", such as Haussdorff dimension, capacity dimension, box-counting dimension, and Minkowski-Bouligand Dimension, that characterize the covering properties of a metric space in fractal geometry (*e.g.,* see Schroeder, 1991). Covering numbers and covering dimension have been widely used in machine learning to characterize the complexity of the hypothesis space in classification problems; however, we are not aware of a clear technical connection between this usage and ours. Similar but stronger notions of "dimension" of a metric space have been studied in theoretical computer science: the ball-growth dimension (*e.g.,* Karger and Ruhl, 2002; Abraham and Malkhi, 2005; Slivkins, 2007) and the doubling dimension (*e.g.,* Gupta et al., 2003; Talwar, 2004; Kleinberg et al., 2009a).[4] These notions allow for (more) efficient algorithms in many different problems: space-efficient distance representations such as metric embeddings, distance labels, and sparse spanners; network primitives such as routing schemes and distributed hash tables; approximation algorithms for optimization problems such as traveling salesman, k-median, and facility location.

4.4.1 Further results on Lipschitz bandits

Zooming algorithm. The analysis of the zooming algorithm, both in Kleinberg et al. (2008b, 2019) and in this chapter, goes through without some of the assumptions. First, there is no need to assume that the metric satisfies triangle inequality (although this assumption is useful for the intuition). Second, Lipschitz condition (4.8) only needs to hold for pairs (x, y) such that x is the best arm.[5] Third, no need to restrict realized rewards to finitely many possible values (but one needs a slightly more careful analysis of the clean event). Fourth, no need for a fixed time horizon: The zooming algorithm can achieve the same regret bound for all rounds at once, by an easy application of the "doubling trick" from Section 1.5.

[4] A metric has ball-growth dimension d if doubling the radius of a ball increases the number of points by at most $O(2^d)$. A metric has doubling dimension d if any ball can be covered with at most $O(2^d)$ balls of half the radius.

[5] A similar algorithm in Bubeck et al. (2011b) only requires the Lipschitz condition when both arms are near the best arm.

$c_{\mathcal{I}}$ \ $f_{\mathcal{I}}(t)$	worst-case	instance-dependent
worst-case	*e.g.,* $f(t) = \tilde{O}(t^{2/3})$ for CAB	*e.g.,* zooming dimension
instance-dependent	*e.g.,* $f(t) = \log(t)$ for $K < \infty$ arms; see Table 4.2 for $K = \infty$ arms	—

Table 4.1: Worst-case vs. instance-dependent regret rates of the form (4.17).

The zooming algorithm attains improved regret bounds for several special cases (Kleinberg et al., 2008b). First, if the maximal payoff is near 1. Second, when $\mu(x) = 1 - f(\mathcal{D}(x, S))$, where S is a "target set" that is not revealed to the algorithm. Third, if the realized reward from playing each arm x is $\mu(x)$ plus an independent noise, for several noise distributions; in particular, if rewards are deterministic.

The zooming algorithm achieves near-optimal regret bounds, in a very strong sense (Slivkins, 2014). The "raw" upper bound in (4.15) is optimal up to logarithmic factors, for any algorithm, any metric space, and any given value of this upper bound. Consequently, the upper bound in Theorem 4.18 is optimal, up to logarithmic factors, for any algorithm and any given value d of the zooming dimension that does not exceed the covering dimension. This holds for various metric spaces, *e.g.,* $\left([0,1],\, \ell_2^{1/d}\right)$ and $\left([0,1]^d,\, \ell_2\right)$.

The zooming algorithm, with similar upper and lower bounds, can be extended to the "contextual" version of Lipschitz bandits (Slivkins, 2014), see Sections 8.2 and 8.7 for details and discussion.

Worst-case vs. instance-dependent regret bounds. This distinction is more subtle than in stochastic bandits. Generically, we have regret bounds of the form

$$\mathbb{E}\left[\, R(t) \mid \mathcal{I} \,\right] \leq c_{\mathcal{I}} \cdot f_{\mathcal{I}}(t) + o(f_{\mathcal{I}}(t)) \quad \text{for each problem instance } \mathcal{I} \text{ and all rounds } t. \tag{4.17}$$

Here $f_{\mathcal{I}}(\cdot)$ defines the asymptotic shape of the regret bound, and $c_{\mathcal{I}}$ is the *leading constant* that cannot depend on time t. Both $f_{\mathcal{I}}$ and $c_{\mathcal{I}}$ may depend on the problem instance $\mathcal{I}$. Depending on a particular result, either of them could be worst-case, *i.e.,* the same for all problem instances on a given metric space. The subtlety is that the instance-dependent vs. worst-case distinction can be applied to $f_{\mathcal{I}}$ and $c_{\mathcal{I}}$ separately, see Table 4.1. Indeed, both are worst-case in this chapter's results on uniform discretization. In Theorem 4.18 for adaptive discretization, $f_{\mathcal{I}}(t)$ is instance-dependent, driven by the zooming dimension, whereas $c_{\mathcal{I}}$ is an absolute constant. In contrast, the $\log(t)$ regret bound for stochastic bandits with finitely many arms features a worst-case $f_{\mathcal{I}}$ and instance-dependent constant $c_{\mathcal{I}}$. (We are not aware of any results in which both $c_{\mathcal{I}}$ and $f_{\mathcal{I}}$ are instance-dependent.) Recall that we have essentially matching upper and lower bounds for uniform dicretization and for the dependence on the zooming dimension, the top two quadrants in Table 4.1. The bottom-left quadrant is quite intricate for infinitely many arms, as we discuss next.

Per-metric optimal regret rates. We are interested in regret rates (4.17), where $c_{\mathcal{I}}$ is instance-dependent, but $f_{\mathcal{I}} = f$ is the same for all problem instances on a given metric space; we abbreviate this as $O_{\mathcal{I}}(f(t))$. The upper/lower bounds for stochastic bandits imply that $O_{\mathcal{I}}(\log t)$ regret is feasible and optimal for finitely many arms, regardless of the metric space. Further, the $\Omega(T^{1-1/(d+2)})$ lower bound for $\left([0,1],\, \ell_2^{1/d}\right)$, $d \geq 1$ in Theorem 4.12 holds even if we allow an instance-dependent constant (Kleinberg, 2004). The construction in this result is similar to that in Section 4.1.2, but more complex. It needs to "work" for all times t, so it contains bump functions (4.5) for all scales ϵ simultaneously. This lower bound extends to metric spaces with covering dimension d, under a homogeneity condition (Kleinberg et al., 2008b, 2019).

However, better regret rates may be possible for arbitrary infinite metric spaces. The most intuitive example involves a "fat point" $x \in X$ such that cutting out any open neighborhood of x reduces the covering

If the metric completion of $(X, \mathcal{D})$ is ...	then regret can be ...	but not ...
finite	$O(\log t)$	$o(\log t)$
compact and countable	$\omega(\log t)$	$O(\log t)$
compact and uncountable		
$\texttt{MaxMinCOV} = 0$	$\tilde{O}\left(t^{\gamma}\right)$, $\gamma > 1/2$	$o(\sqrt{t})$
$\texttt{MaxMinCOV} = d \in (0, \infty)$	$\tilde{O}\left(t^{\gamma}\right)$, $\gamma > \frac{d+1}{d+2}$	$o\left(t^{\gamma}\right)$, $\gamma < \frac{d+1}{d+2}$
$\texttt{MaxMinCOV} = \infty$	$o(t)$	$O\left(t^{\gamma}\right)$, $\gamma < 1$
non-compact	$O(t)$	$o(t)$

Table 4.2: Per-metric optimal regret bounds for Lipschitz MAB

dimension by at least $\epsilon > 0$. Then one can obtain a regret rate "as if" the covering dimension were $d - \epsilon$. Kleinberg et al. (2008b, 2019) handle this phenomenon in full generality. For an arbitrary infinite metric space, they define a "refinement" of the covering dimension, denoted $\texttt{MaxMinCOV}(X)$, and prove matching upper and lower regret bounds "as if" the covering dimension were equal to this refinement. $\texttt{MaxMinCOV}(X)$ is always upper-bounded by $\texttt{COV}_c(X)$, and could be as low as 0 depending on the metric space. Their algorithm is a version of the zooming algorithm with quotas on the number of active arms in some regions of the metric space. Further, Kleinberg and Slivkins (2010); Kleinberg et al. (2019) prove that the transition from $O_{\mathcal{I}}(\log t)$ to $O_{\mathcal{I}}(\sqrt{t})$ regret is sharp and corresponds to the distinction between countable and uncountable set of arms. The full characterization of optimal regret rates is summarized in Table 4.2. This work makes deep connections between bandit algorithms, metric topology, and transfinite ordinal numbers.

Kleinberg and Slivkins (2010); Kleinberg et al. (2019) derive a similar characterization for a version of Lipschitz bandits with full feedback, *i.e.,* when the algorithm receives feedback for all arms. $O_{\mathcal{I}}(\sqrt{t})$ regret is feasible for any metric space of finite covering dimension. One needs an exponentially weaker version of covering dimension to induce regret bounds of the form $\tilde{O}_{\mathcal{I}}(t^{\gamma})$, for some $\gamma \in (1/2, 1)$.

Per-instance optimality. What is the best regret bound for a given instance of Lipschitz MAB? Magureanu et al. (2014) ask this question in the style of Lai-Robbins lower bound for stochastic bandits (Theorem 2.14). Specifically, they consider finite metric spaces, assume that the algorithm satisfies (2.16), the precondition in Theorem 2.14, and focus on optimizing the leading constant $c_{\mathcal{I}}$ in Eq. (4.17) with $f(t) = \log(t)$. For an arbitrary problem instance with finitely many arms, they derive a lower bound on $c_{\mathcal{I}}$, and provide an algorithm comes arbitrarily close to this lower bound. However, this approach may increase the $o(\log T)$ term in Eq. (4.17), possibly hurting the worst-case performance.

Beyond IID rewards. Uniform discretization easily extends for adversarial rewards (Kleinberg, 2004), and matches the regret bound in Theorem 4.11 (see Exercise 6.2). Adaptive discretization extends to adversarial rewards, too, albeit with much additional work: Podimata and Slivkins (2021) connect it to techniques from adversarial bandits, and generalize Theorem 4.18 for a suitable version of zooming dimension.

Some other variants with non-IID rewards have been studied. Maillard and Munos (2010) consider a full-feedback problem with adversarial rewards and Lipschitz condition in the Euclidean space $(\mathbb{R}^d, \ell_2)$, achieving a surprisingly strong regret bound of $O_d(\sqrt{T})$. Azar et al. (2014) consider a version in which the IID condition is replaced by more sophisticated ergodicity and mixing assumptions, and essentially recover the performance of the zooming algorithm. Slivkins (2014) extends the zooming algorithm to a version of Lipschitz bandits where expected rewards do not change too fast over time. Specifically, $\mu(x, t)$, the expected reward of a given arm x at time t, is Lipschitz relative to a known metric on pairs (x, t). Here round t is interpreted as a context in Lipschitz contextual bandits; see also Section 8.2 and the literature discussion in Section 8.7.

4.4.2 Partial similarity information

Numerical similarity information required for the Lipschitz bandits may be difficult to obtain in practice. A canonical example is the "taxonomy bandits" problem mentioned above, where an algorithm is given a taxonomy (a tree) on arms but not a metric which admits the Lipschitz condition (4.8). One goal here is to obtain regret bounds that are (almost) as good as if the metric were known.

Slivkins (2011) considers the metric implicitly defined by an instance of taxonomy bandits: the distance between any two arms is the "width" of their least common subtree S, where the width of S is defined as $W(S) := \max_{x,y \in S} |\mu(x) - \mu(y)|$. (Note that $W(S)$ is not known to the algorithm.) This is the best possible metric, *i.e.,* a metric with smallest distances, that admits the Lipschitz condition (4.8). Slivkins (2011) puts forward an extension of the zooming algorithm which partially reconstructs the implicit metric, and almost matches the regret bounds of the zooming algorithm for this metric. In doing so, it needs to deal with *another* exploration-exploitation tradeoff: between learning more about the widths and exploiting this knowledge to run the zooming algorithm. The idea is to have "active subtrees" S rather than "active arms", maintain a lower confidence bound (LCB) on $W(S)$, and use it instead of the true width. The LCB can be obtained any two sub-subtrees S_1, S_2 of S. Indeed, if one chooses arms from S at random according to some fixed distribution, then $W(S) \geq |\mu(S_1) - \mu(S_2)|$, where $\mu(S_i)$ is the expected reward when sampling from S_i, and with enough samples the empirical average reward from S_i is close to its expectation. The regret bound depends on the "quality parameter": essentially, how deeply does one need to look in each subtree S in order to find sub-subtrees S_1, S_2 that give a sufficiently good lower bound on $W(S)$. However, the algorithm does not need to know this parameter upfront. Bull (2015) considers a somewhat more general setting where multiple taxonomies on arms are available, and some of them may work better for this problem than others. He carefully traces out the conditions under which one can achieve $\tilde{O}(\sqrt{T})$ regret.

A similar issue arises when arms correspond to points in $[0, 1]$ but no Lipschitz condition is given. This setting can be reduced to "taxonomy bandits" by positing a particular taxonomy on arms, *e.g.,* the root corresponds to $[0, 1]$, its children are $[0, 1/2)$ and $[1/2, 1]$, and so forth splitting each interval into halves.

Ho et al. (2016) consider a related problem in the context of crowdsourcing markets. Here the algorithm is an employer who offers a quality-contingent contract to each arriving worker, and adjusts the contract over time. On an abstract level, this is a bandit problem in which arms are contracts: essentially, vectors of prices. However, there is no Lipschitz-like assumption. Ho et al. (2016) treat this problem as a version of "taxonomy bandits", and design a version of the zooming algorithm. They estimate the implicit metric in a problem-specific way, taking advantage of the structure provided by the employer-worker interactions, and avoid the dependence on the "quality parameter" from Slivkins (2011).

Another line of work studies the "pure exploration" version of "taxonomy bandits", where the goal is to output a "predicted best arm" with small instantaneous regret (Munos, 2011; Valko et al., 2013; Grill et al., 2015), see Munos (2014) for a survey. The main result essentially recovers the regret bounds for the zooming algorithm as if a suitable distance function were given upfront. The algorithm posits a parameterized family of distance functions, guesses the parameters, and runs a zooming-like algorithm for each guess.

Bubeck et al. (2011c) study a version of continuum-armed bandits with strategy set $[0, 1]^d$ and Lipschitz constant L that is not revealed to the algorithm, and match the regret rate in Theorem 4.2. This result is powered by an assumption that $\mu(\cdot)$ is twice differentiable, and a bound on the second derivative is known to the algorithm. Minsker (2013) considers the same strategy set, under metric $\|x - y\|_\infty^\beta$, where the "smoothness parameter" $\beta \in (0, 1]$ is not known. His algorithm achieves near-optimal instantaneous regret as if the β were known, under some structural assumptions.

4.4.3 Generic non-Lipschitz models for bandits with similarity

One drawback of Lipschitz bandits as a model is that the distance $\mathcal{D}(x, y)$ only gives a "worst-case" notion of similarity between arms x and y. In particular, the distances may need to be very large in order to accommodate a few outliers, which would make $\mathcal{D}$ less informative elsewhere.[6] With this criticism in mind, Srinivas et al. (2010); Krause and Ong (2011); Desautels et al. (2012) define a probabilistic model, called *Gaussian Processes Bandits*, where the expected payoff function is distributed according to a suitable Gaussian Process on X, thus ensuring a notion of "probabilistic smoothness" with respect to X.

Krishnamurthy et al. (2020) side-step Lipschitz assumptions by relaxing the benchmark. They define a new benchmark, which replaces each arm a with a low-variance distribution around this arm, called the *smoothed arm*, and compares the algorithm's reward to that of the best smoothed arm; call it the *smoothed benchmark*. For example, if the set of arms is $[0, 1]$, then the smoothed arm can be defined as the uniform distribution on the interval $[a - \epsilon, a + \epsilon]$, for some fixed $\epsilon > 0$. Thus, very sharp peaks in the mean rewards – which are impossible to handle via the standard best-arm benchmark without Lipschitz assumptions – are now smoothed over an interval. In the most general version, the smoothing distribution may be arbitrary, and arms may lie in an arbitrary "ambient space" rather than $[0, 1]$ interval. (The "ambient space" is supposed to be natural given the application domain; formally it is described by a metric on arms and a measure on balls in that metric.) Both fixed and adaptive discretization carries over to the smoothed benchmark, without any Lipschitz assumptions (Krishnamurthy et al., 2020). These results usefully extend to contextual bandits with policy sets, as defined in Section 8.4 (Krishnamurthy et al., 2020; Majzoubi et al., 2020).

Amin et al. (2011) and Combes et al. (2017) consider stochastic bandits with, essentially, an arbitrary known family $\mathcal{F}$ of mean reward functions, as per Section 1.4. Amin et al. (2011) posit a finite $|\mathcal{F}|$ and obtain a favourable regret bound when a certain complexity measure of $\mathcal{F}$ is small, and any two functions in $\mathcal{F}$ are sufficiently well-separated. However, their results do not subsume any prior work on Lipschitz bandits. Combes et al. (2017) extend the per-instance optimality approach from Magureanu et al. (2014), discussed above, from Lipschitz bandits to an arbitrary $\mathcal{F}$, under mild assumptions.

Unimodal bandits assume that mean rewards are *unimodal*: *e.g.,* when the set of arms is $X = [0, 1]$, there is a single best arm x^*, and mean rewards $\mu(x)$ increase for all arms $x < x^*$ and decrease for all arms $x > x^*$. For this setting, one can obtain $\tilde{O}(\sqrt{T})$ regret under some additional assumptions on $\mu(\cdot)$: smoothness (Cope, 2009), Lipschitzness (Yu and Mannor, 2011), or continuity (Combes and Proutière, 2014a). One can also consider a more general version of unimodality relative to a known partial order on arms (Yu and Mannor, 2011; Combes and Proutière, 2014b).

4.4.4 Dynamic pricing and bidding

A notable class of bandit problems has arms that correspond to monetary amounts, *e.g.,* offered prices for selling (*dynamic pricing*) or bying (*dynamic procurement*), offered wages for hiring, or bids in an auction (*dynamic bidding*). Most studied is the basic case when arms are real numbers, *e.g.,* prices rather than price vectors. All these problems satisfy a version of monotonicity, *e.g.,* decreasing the price cannot result in fewer sales. This property suffices for both uniform and adaptive discretization without any additional Lipschitz assumptions. We work this out for dynamic pricing, see the exercises in Section 4.5.4.

Dynamic pricing as a bandit problem has been introduced in Kleinberg and Leighton (2003), building on the earlier work (Blum et al., 2003). Kleinberg and Leighton (2003) introduce uniform discretization, and observe that it attains $\tilde{O}(T^{2/3})$ regret for stochastic rewards, just like in Section 4.1.1, and likewise for adversarial rewards, with an appropriate algorithm for adversarial bandits. Moreover, uniform discretization (with a different step ϵ) achieves $\tilde{O}(\sqrt{T})$ regret, if expected reward $\mu(x)$ is strongly concave as a function of price

[6] This concern is partially addressed by relaxing the Lipschitz condition in the analysis of the zooming algorithm.

x; this condition, known as *regular demands*, is standard in theoretical economics. Kleinberg and Leighton (2003) prove a matching $\Omega(T^{2/3})$ lower bound in the worst case, even if one allows an instance-dependent constant. The construction contains a version of bump functions (4.5) for all scales ϵ simultaneously, and predates a similar lower bound for continuum-armed bandits from Kleinberg (2004). That the zooming algorithm works without additional assumptions is straightforward from the original analysis in (Kleinberg et al., 2008b, 2019), but has not been observed until Podimata and Slivkins (2021).

$\tilde{O}(\sqrt{T})$ regret can be achieved in some auction-related problems when additional feedback is available to the algorithm. Weed et al. (2016) achieve $\tilde{O}(\sqrt{T})$ regret for dynamic bidding in first-price auctions, when the algorithm observes full feedback (*i.e.,* the minimal winning bid) whenever it wins the auction. Feng et al. (2018) simplifies the algorithm in this result, and extends it to a more general auction model. Both results extend to adversarial outcomes. Cesa-Bianchi et al. (2013) achieve $\tilde{O}(\sqrt{T})$ regret in a "dual" problem, where the algorithm optimizes the auction rather than the bids. Specifically, the algorithm adjusts the *reserve price* (the lowest acceptable bid) in a second-price auction. The algorithm receives more-than-bandit feedback: indeed, if a sale happens for a particular reserve price, then exactly the same sale would have happened for any smaller reserve price.

Dynamic pricing and related problems become more difficult when arms are price vectors, *e.g.,* when multiple products are for sale, the algorithm can adjust the price for each separately, and customer response for one product depends on the prices for others. In particular, one needs structural assumptions such as Lipschitzness, concavity or linearity. One exception is quality-contingent contract design (Ho et al., 2016), as discussed in Section 4.4.2, where a version of zooming algorithm works without additional assumptions.

Dynamic pricing and bidding is often studied in more complex environments with supply/budget constraints and/or forward-looking strategic behavior. We discuss these issues in Chapters 9 and 10. In particular, the literature on dynamic pricing with limited supply is reviewed in Section 10.5.5.

4.5 Exercises and hints

4.5.1 Construction of ϵ-meshes

Let us argue that the construction in Remark 4.6 attains the infimum in (4.9) up to a constant factor.

Let $\epsilon(S) = \inf\{\,\epsilon > 0 :\ S \text{ is an } \epsilon\text{-mesh}\,\}$, for $S \subset X$. Restate the right-hand side in (4.9) as

$$\inf_{S\subset X} f(S), \text{ where } f(S) = c_{\texttt{ALG}} \cdot \sqrt{|S|\,T\log T} + T\cdot\epsilon(S). \tag{4.18}$$

Recall that $\mathbb{E}[R(T)] \leq f(S)$ if we run algorithm ALG on set S of arms.

Recall the construction in Remark 4.6. Let $\mathcal{N}_i$ be the ϵ-mesh computed therein for a given $\epsilon = 2^{-i}$. Suppose the construction stops at some $i = i^*$. Thus, this construction gives regret $\mathbb{E}[R(T)] \leq f(N_{i^*})$.

Exercise 4.1. Compare $f(N_{i^*})$ against (4.18). Specifically, prove that

$$f(\mathcal{N}_{i^*}) \leq 16 \inf_{S\subset X} f(S). \tag{4.19}$$

The key notion in the proof is *ϵ-net*, $\epsilon > 0$: it is an ϵ-mesh where any two points are at distance $> \epsilon$.

(a) Observe that each ϵ-mesh computed in Remark 4.6 is in fact an ϵ-net.

(b) Prove that for any ϵ-mesh S and any ϵ'-net $\mathcal{N}$, $\epsilon' \geq 2\epsilon$, it holds that $|\mathcal{N}| \leq |S|$.
Use (a) to conclude that $\min_{i\in\mathbb{N}} f(\mathcal{N}_i) \leq 4\, f(S)$.

(c) Prove that $f(\mathcal{N}_{i^*}) \leq 4\, \min_{i\in\mathbb{N}} f(\mathcal{N}_i)$. Use (b) to conclude that (4.19) holds.

4.5.2 Lower bounds for uniform discretization

Exercise 4.2. Extend the construction and analysis in Section 4.1.2:

(a) ... from Lipschitz constant $L = 1$ to an arbitrary L, *i.e.,* prove Theorem 4.2.

(b) ... from continuum-armed bandits to $\left([0,1],\ \ell_2^{1/d}\right)$, *i.e.,* prove Theorem 4.12.

(c) ... to an arbitrary metric space $(X, \mathcal{D})$ and an arbitrary ϵ-net $\mathcal{N}$ therein (see Exercise 4.1): prove that for any algorithm there is a problem instance on $(X, \mathcal{D})$ such that

$$\mathbb{E}[R(T)] \geq \Omega\left(\min\left(\epsilon T,\ |\mathcal{N}|/\epsilon\right)\right) \quad \text{for any time horizon } T. \tag{4.20}$$

Exercise 4.3. Prove that the optimal uniform discretization from Eq. (4.9) is optimal up to $O(\log T)$ factors. Specifically, using the notation from Eq. (4.18), prove the following: for any metric space $(X, \mathcal{D})$, any algorithm and any time horizon T there is a problem instance on $(X, \mathcal{D})$ such that

$$\mathbb{E}[R(T)] \geq \widetilde{\Omega}\left(\inf_{S \subset X} f(S)\right).$$

Exercise 4.4 (Lower bounds via covering dimension). Consider Lipschitz bandits in a metric space $(X, \mathcal{D})$. Fix $d < \texttt{COV}_c(X)$, for some fixed absolute constant $c > 0$. Fix an arbitrary algorithm ALG. We are interested proving that this algorithm suffers lower bounds of the form

$$\mathbb{E}[R(T)] \geq \Omega\left(T^{(d+1)/(d+2)}\right). \tag{4.21}$$

The subtlety is, for which T can this be achieved?

(a) Assume that the covering property is nearly tight at a particular scale $\epsilon > 0$, namely:

$$N_\epsilon(X) \geq c' \cdot \epsilon^{-d} \quad \text{for some absolute constant } c'. \tag{4.22}$$

Prove that (4.21) holds for some problem instance and $T = c' \cdot \epsilon^{-d-2}$.

(b) Assume (4.22) holds for all $\epsilon > 0$. Prove that for each T there is a problem instance with (4.21).

(c) Prove that (4.21) holds for *some* time horizon T and some problem instance.

(d) Assume that $d < \texttt{COV}(X) := \inf_{c>0} \texttt{COV}_c(X)$. Prove that there are *infinitely many* time horizons T such that (4.21) holds for some problem instance $\mathcal{I}_T$. In fact, there is a distribution over these problem instances (each endowed with an infinite time horizon) such that (4.21) holds for *infinitely many* T.

Hints: Part (a) follows from Exercise 4.2(c), the rest follows from part (a). For part (c), observe that (4.22) holds for some ϵ. For part (d), observe that (4.22) holds for arbitrarily small $\epsilon > 0$, *e.g.,* with $c' = 1$, then apply part (a) to each such ϵ. To construct the distribution over problem instances $\mathcal{I}_T$, choose a sufficiently sparse sequence $(T_i : i \in \mathbb{N})$, and include each instance $\mathcal{I}_{T_i}$ with probability $\sim 1/\log(T_i)$, say. To remove the $\log T$ factor from the lower bound, carry out the argument above for some $d' \in (d, \texttt{COV}_c(X))$.

4.5.3 Examples and extensions

Exercise 4.5 (Covering dimension and zooming dimension).

(a) Prove that the covering dimension of $\big([0,1]^d, \ell_2\big)$, $d \in \mathbb{N}$ and $\Big([0,1],\, \ell_2^{1/d}\Big)$, $d \geq 1$ is d.

(b) Prove that the zooming dimension cannot exceed the covering dimension. More precisely: if $d = \texttt{COV}_c(X)$, then the zooming dimension with multiplier $3^d \cdot c$ is at most d.

(c) Construct an example in which the zooming dimension is much smaller than $\texttt{COV}_c(X)$.

Exercise 4.6 (Lipschitz bandits with a target set). Consider Lipschitz bandits on metric space $(X, \mathcal{D})$ with $\mathcal{D}(\cdot,\cdot) \leq 1/2$. Fix the best reward $\mu^* \in [3/4, 1]$ and a subset $S \subset X$ and assume that

$$\mu(x) = \mu^* - \mathcal{D}(x, S) \quad \forall x \in X, \qquad \text{where } \mathcal{D}(x,S) := \inf_{y \in S} \mathcal{D}(x,y).$$

In words, the mean reward is determined by the distance to some "target set" S.

(a) Prove that $\mu^* - \mu(x) \leq \mathcal{D}(x,S)$ for all arms x, and that this condition suffices for the analysis of the zooming algorithm, instead of the full Lipschitz condition (4.8).

(b) Assume the metric space is $([0,1]^d, \ell_2)$, for some $d \in \mathbb{N}$. Prove that the zooming dimension of the problem instance (with a suitably chosen multiplier) is at most the covering dimension of S.

Take-away: In part (b), the zooming algorithm achieves regret $\tilde{O}(T^{(b+1)/(b+2)})$, where b is the covering dimension of the target set S. Note that b could be much smaller than d, the covering dimension of the entire metric space. In particular, one achieves regret $\tilde{O}(\sqrt{T})$ if S is finite.

4.5.4 Dynamic pricing

Let us apply the machinery from this chapter to dynamic pricing. This problem naturally satisfies a monotonicity condition: essentially, you cannot sell less if you decrease the price. Interestingly, this condition suffices for our purposes, without any additional Lipschitz assumptions.

Problem protocol: Dynamic pricing

In each round $t \in [T]$:

1. Algorithm picks some price $p_t \in [0,1]$ and offers one item for sale at this price.
2. A new customer arrives with private value $v_t \in [0,1]$, not visible to the algorithm.
3. The customer buys the item if and only if $v_t \geq p_t$.
   The algorithm's reward is p_t if there is a sale, and 0 otherwise.

Thus, arms are prices in $X = [0,1]$. We focus on *stochastic* dynamic pricing: each private value v_t is sampled independently from some fixed distribution which is not known to the algorithm.

Exercise 4.7 (monotonicity). Observe that the sale probability $\Pr[\text{sale at price } p]$ is monotonically non-increasing in p. Use this monotonicity property to derive one-sided Lipschitzness:

$$\mu(p) - \mu(p') \leq p - p' \quad \text{for any two prices } p > p'. \tag{4.23}$$

Exercise 4.8 (uniform discretization). Use (4.23) to recover the regret bound for uniform discretization: with any algorithm ALG satisfying (4.3) and discretization step $\epsilon = (T/\log T)^{-1/3}$ one obtains

$$\mathbb{E}[R(T)] \leq T^{2/3} \cdot (1 + c_{\texttt{ALG}})(\log T)^{1/3}.$$

Exercise 4.9 (adaptive discretization). Modify the zooming algorithm as follows. The confidence ball of arm x is redefined as the interval $[x,\ x + r_t(x)]$. In the activation rule, when some arm is not covered by the confidence balls of active arms, pick the smallest (infimum) such arm and activate it. Prove that this modified algorithm achieves the regret bound in Theorem 4.18.

Hint: The invariant (4.13) still holds. The one-sided Lipschitzness (4.23) suffices for the proof of Lemma 4.14.

Chapter 5

Full Feedback and Adversarial Costs

For this one chapter, we shift our focus from bandit feedback to full feedback. As the IID assumption makes the problem "too easy", we introduce and study the other extreme, when rewards/costs are chosen by an adversary. We define and analyze two classic algorithms, *weighted majority* and *multiplicative-weights update*, a.k.a. `Hedge`.

Full feedback is defined as follows: in the end of each round, the algorithm observes the outcome not only for the chosen arm, but for all other arms as well. To be in line with the literature on such problems, we express the outcomes as *costs* rather than *rewards*. As IID costs are quite "easy" with full feedback, we consider the other extreme: costs can arbitrarily change over time, as if they are selected by an adversary.

The protocol for full feedback and adversarial costs is as follows:

Problem protocol: Bandits with full feedback and adversarial costs

---

Parameters: K arms, T rounds (both known).
In each round $t \in [T]$:

1. Adversary chooses costs $c_t(a) \geq 0$ for each arm $a \in [K]$.
2. Algorithm picks arm $a_t \in [K]$.
3. Algorithm incurs cost $c_t(a_t)$ for the chosen arm.
4. The costs of all arms, $c_t(a) : \ a \in [K]$, are revealed.

Remark 5.1. While some results rely on bounded costs, *e.g.,* $c_t(a) \leq 1$, we do not assume this by default.

One real-life scenario with full feedback is investments on a stock market. For a simple (and very stylized) example, recall one from the Introduction. Suppose each morning choose one stock and invest \$1 into it. At the end of the day, we observe not only the price of the chosen stock, but prices of all stocks. Based on this feedback, we determine which stock to invest for the next day.

A paradigmatic special case of bandits with full feedback is sequential prediction with experts advice. Suppose we need to predict labels for observations, and we are assisted with a committee of experts. In each round, a new observation arrives, and each expert predicts a correct label for it. We listen to the experts, and pick an answer to respond. We then observe the correct answer and costs/penalties of all other answers. Such a process can be described by the following protocol:

Problem protocol: Sequential prediction with expert advice

Parameters: K experts, T rounds, L labels, observation set $\mathcal{X}$ (all known).
For each round $t \in [T]$:

1. Adversary chooses observation $x_t \in \mathcal{X}$ and correct label $z_t^* \in [L]$.
   Observation x_t is revealed, label z_t^* is not.
2. The K experts predict labels $z_{1,t}, \ldots, z_{K,t} \in [L]$.
3. Algorithm picks an expert $e = e_t \in [K]$.
4. Correct label z_t^* is revealed.
5. Algorithm incurs cost $c_t = c\left(z_{e,t}, z_t^*\right)$, for some known *cost function* $c : [L] \times [L] \to [0, \infty)$.

The basic case is *binary costs*: $c(z, z^*) = \mathbf{1}_{\{z \neq z^*\}}$, *i.e.,* the cost is 0 if the answer is correct, and 1 otherwise. Then the total cost is simply the number of mistakes.

The goal is to do approximately as well as the best expert. Surprisingly, this can be done without any domain knowledge, as explained in the rest of this chapter.

Remark 5.2. Because of this special case, the general case of bandits with full feedback is usually called *online learning with experts*, and defined in terms of *costs* (as penalties for incorrect predictions) rather than *rewards*. We will talk about arms, actions and experts interchangeably throughout this chapter.

Remark 5.3 (IID costs). Consider the special case when the adversary chooses the cost $c_t(a) \in [0, 1]$ of each arm a from some fixed distribution $\mathcal{D}_a$, same for all rounds t. With full feedback, this special case is "easy": indeed, there is no need to explore, since costs of all arms are revealed after each round. With a naive strategy such as playing arm with the lowest average cost, one can achieve regret $O\left(\sqrt{T \log(KT)}\right)$. Further, there is a nearly matching lower regret bound $\Omega(\sqrt{T} + \log K)$. The proofs of these results are left as exercise. The upper bound can be proved by a simple application of clean event/confidence radius technique that we've been using since Chapter 1. The $\sqrt{T}$ lower bound follows from the same argument as the bandit lower bound for two arms in Chapter 2, as this argument does not rely on bandit feedback. The $\Omega(\log K)$ lower bound holds for a simple special case, see Theorem 5.8.

5.1 Setup: adversaries and regret

Let us elaborate on the types of adversaries one could consider, and the appropriate notions of regret. A crucial distinction is whether the cost functions $c_t(\cdot)$ depend on the algorithm's choices. An adversary is called *oblivious* if they don't, and *adaptive* if they do (*i.e.,* oblivious / adaptive to the algorithm). Like before, the "best arm" is an arm a with a lowest total cost, denoted $\texttt{cost}(a) = \sum_{t=1}^{T} c_t(a)$, and regret is the difference in total cost compared to this arm. However, defining this precisely is a little subtle, especially when the adversary is randomized. We explain all this in detail below.

Deterministic oblivious adversary. Thus, the costs $c_t(\cdot)$, $t \in [T]$ are deterministic and do not depend on the algorithm's choices. Without loss of generality, the entire "cost table" $\left(c_t(a) : a \in [K], t \in [T]\right)$ is chosen before round 1. The best arm is naturally defined as $\operatorname{argmin}_{a \in [K]} \texttt{cost}(a)$, and regret is defined as

$$R(T) = \texttt{cost}(\texttt{ALG}) - \min_{a \in [K]} \texttt{cost}(a), \tag{5.1}$$

where $\texttt{cost}(\texttt{ALG})$ denotes the total cost incurred by the algorithm. One drawback of such adversary is that it does not model IID costs, even though IID rewards are a simple special case of adversarial rewards.

Randomized oblivious adversary. The costs $c_t(\cdot)$, $t \in [T]$ do not depend on the algorithm's choices, as before, but can be randomized. Equivalently, the adversary fixes a distribution $\mathcal{D}$ over the cost tables before round 1, and then draws a cost table from this distribution. Then IID costs are indeed a simple special case. Since $\texttt{cost}(a)$ is now a random variable, there are two natural (and different) ways to define the "best arm":

- $\operatorname{argmin}_a \texttt{cost}(a)$: this is the best arm *in hindsight*, *i.e.,* after all costs have been observed. It is a natural notion if we start from the deterministic oblivious adversary.
- $\operatorname{argmin}_a \mathbb{E}[\texttt{cost}(a)]$: this is be best arm *in foresight*, *i.e.,* an arm you'd pick if you only know the distribution $\mathcal{D}$. This is a natural notion if we start from IID costs.

Accordingly, there are two natural versions of regret: with respect to the best-in-hindsight arm, as in (5.1), and with respect to the best-in-foresight arm,

$$R(T) = \texttt{cost}(\texttt{ALG}) - \min_{a \in [K]} \mathbb{E}[\texttt{cost}(a)]. \tag{5.2}$$

For IID costs, this notion coincides with the definition of regret from Chapter 1.

Remark 5.4. The notion in (5.1) is usually referred to simply as *regret* in the literature, whereas (5.2) is often called *pseudo-regret*. We will use this terminology when we need to distinguish between the two versions.

Remark 5.5. Pseudo-regret cannot exceed regret, because the best-in-foresight arm is a weaker benchmark. Some positive results for pseudo-regret carry over to regret, and some don't. For IID rewards/costs, the $\sqrt{T}$ upper regret bounds from Chapter 1 extend to regret, whereas the $\log(T)$ upper regret bounds do not extend in full generality, see Exercise 5.2 for details.

Adaptive adversary can change the costs depending on the algorithm's past choices. Formally, in each round t, the costs $c_t(\cdot)$ may depend on arms $a_1, \ldots, a_{t-1}$, but not on a_t or the realization of algorithm's internal randomness. This models scenarios when algorithm's actions may alter the environment that the algorithm operates in. For example:

- an algorithm that adjusts the layout of a website may cause users to permanently change their behavior, *e.g.,* they may gradually get used to a new design, and get dissatisfied with the old one.
- a bandit algorithm that selects news articles for a website may attract some users and repel some others, and/or cause the users to alter their reading preferences.
- if a dynamic pricing algorithm offers a discount on a new product, it may cause many people to buy this product and (eventually) grow to like it and spread the good word. Then more people would be willing to buy this product at full price.
- if a bandit algorithm adjusts the parameters of a repeated auction (*e.g.,* a reserve price), auction participants may adjust their behavior over time, as they become more familiar with the algorithm.

In game-theoretic applications, adaptive adversary can be used to model a game between an algorithm and a self-interested agent that responds to algorithm's moves and strives to optimize its own utility. In particular, the agent may strive to hurt the algorithm if the game is zero-sum. We will touch upon game-theoretic applications in Chapter 9.

An adaptive adversary is assumed to be randomized by default. In particular, this is because the adversary can adapt the costs to the algorithm's choice of arms in the past, and the algorithm is usually randomized. Thus, the distinction between regret and pseudo-regret comes into play again.

Crucially, which arm is best may depend on the algorithm's actions. For example, if an algorithm always chooses arm 1 then arm 2 is consistently much better, whereas *if the algorithm always played arm 2*, then arm 1 may have been better. One can side-step these issues by considering the *best-observed arm*: the best-in-hindsight arm according to the costs actually observed by the algorithm.

Regret guarantees relative to the best-observed arm are not always satisfactory, due to many problematic examples such as the one above. However, such guarantees are worth studying for several reasons. First, they *are* meaningful in some scenarios, *e.g.,* when algorithm's actions do not substantially affect the total cost of the best arm. Second, such guarantees may be used as a tool to prove results on oblivious adversaries (*e.g.,* see next chapter). Third, such guarantees are essential in several important applications to game theory, when a bandit algorithm controls a player in a repeated game (see Chapter 9). Finally, such guarantees often follow from the analysis of oblivious adversary with very little extra work.

Throughout this chapter, we consider an adaptive adversary, unless specified otherwise. We are interested in regret relative to the best-observed arm. For ease of comprehension, one can also interpret the same material as working towards regret guarantees against a randomized-oblivious adversary.

Let us (re-)state some notation: the best arm and its cost are

$$a^* \in \operatorname{argmin}_{a\in[K]} \texttt{cost}(a) \quad \text{and} \quad \texttt{cost}^* = \min_{a\in[K]} \texttt{cost}(a),$$

where $\texttt{cost}(a) = \sum_{t=1}^{T} c_t(a)$ is the total cost of arm a. Note that a^* and $\texttt{cost}^*$ may depend on randomness in rewards, and (for adaptive adversary) also on algorithm's actions.

As always, K is the number of actions, and T is the time horizon.

5.2 Initial results: binary prediction with experts advice

We consider *binary prediction with experts advice*, a special case where experts' predictions $z_{i,t}$ can only take two possible values. For example: is this image a face or not? Is it going to rain today or not?

Let us assume that there exists a *perfect expert* who never makes a mistake. Consider a simple algorithm that disregards all experts who made a mistake in the past, and follows the majority of the remaining experts:

> In each round t, pick the action chosen by the majority of the experts who did not err in the past.

We call this the *majority vote algorithm*. We obtain a strong guarantee for this algorithm:

Theorem 5.6. *Consider binary prediction with experts advice. Assuming a perfect expert, the majority vote algorithm makes at most* $\log_2 K$ *mistakes, where* K *is the number of experts.*

Proof. Let S_t be the set of experts who make no mistakes up to round t, and let $W_t = |S_t|$. Note that $W_1 = K$, and $W_t \geq 1$ for all rounds t, because the perfect expert is always in S_t. If the algorithm makes a mistake at round t, then $W_{t+1} \leq W_t/2$ because the majority of experts in S_t is wrong and thus excluded from S_{t+1}. It follows that the algorithm cannot make more than $\log_2 K$ mistakes. □

Remark 5.7. This simple proof introduces a general technique that will be essential in the subsequent proofs:

- Define a quantity W_t which measures the total remaining amount of "credibility" of the the experts. Make sure that by definition, W_1 is upper-bounded, and W_t does not increase over time. Derive a lower bound on W_T from the existence of a "good expert".
- Connect W_t with the behavior of the algorithm: prove that W_t decreases by a constant factor whenever the algorithm makes mistake / incurs a cost.

The guarantee in Theorem 5.6 is in fact optimal (the proof is left as Exercise 5.1):

Theorem 5.8. *Consider binary prediction with experts advice. For any algorithm, any T and any K, there is a problem instance with a perfect expert such that the algorithm makes at least $\Omega(\min(T, \log K))$ mistakes.*

Let us turn to the more realistic case where there is no perfect expert among the committee. The majority vote algorithm breaks as soon as all experts make at least one mistake (which typically happens quite soon).

Recall that the majority vote algorithm fully trusts each expert until his first mistake, and completely ignores him afterwards. When all experts may make a mistake, we need a more granular notion of trust. We assign a *confidence weight* $w_a \geq 0$ to each expert a: the higher the weight, the larger the confidence. We update the weights over time, decreasing the weight of a given expert whenever he makes a mistake. More specifically, in this case we multiply the weight by a factor $1 - \epsilon$, for some fixed parameter $\epsilon > 0$. We treat each round as a weighted vote among the experts, and we choose a prediction with a largest total weight. This algorithm is called *Weighted Majority Algorithm* (WMA).

parameter: $\epsilon \in [0, 1]$

Initialize the weights $w_i = 1$ for all experts.
For each round t:
  Make predictions using weighted majority vote based on w.
  For each expert i:
    If the i-th expert's prediction is correct, w_i stays the same.
    Otherwise, $w_i \leftarrow w_i(1 - \epsilon)$.

Algorithm 5.1: Weighted Majority Algorithm

To analyze the algorithm, we first introduce some notation. Let $w_t(a)$ be the weight of expert a before round t, and let $W_t = \sum_{a=1}^{K} w_t(a)$ be the total weight before round t. Let S_t be the set of experts that made incorrect prediction at round t. We will use the following fact about logarithms:

$$\ln(1 - x) < -x \qquad \forall x \in (0, 1). \tag{5.3}$$

From the algorithm, $W_1 = K$ and $W_{T+1} > w_t(a^*) = (1 - \epsilon)^{\texttt{cost}^*}$. Therefore, we have

$$\frac{W_{T+1}}{W_1} > \frac{(1 - \epsilon)^{\texttt{cost}^*}}{K}. \tag{5.4}$$

Since the weights are non-increasing, we must have

$$W_{t+1} \leq W_t \tag{5.5}$$

If the algorithm makes mistake at round t, then

$$\begin{aligned} W_{t+1} &= \sum_{a \in [K]} w_{t+1}(a) \\ &= \sum_{a \in S_t} (1 - \epsilon) w_t(a) + \sum_{a \notin S_t} w_t(a) \\ &= W_t - \epsilon \sum_{a \in S_t} w_t(a). \end{aligned}$$

Since we are using weighted majority vote, the incorrect prediction must have the majority vote:

$$\sum_{a \in S_t} w_t(a) \geq \tfrac{1}{2} W_t.$$

Therefore, if the algorithm makes mistake at round t, we have

$$W_{t+1} \leq (1 - \tfrac{\epsilon}{2}) W_t.$$

Combining with (5.4) and (5.5), we get

$$\frac{(1-\epsilon)^{\texttt{cost}^*}}{K} < \frac{W_{T+1}}{W_1} = \prod_{t=1}^{T} \frac{W_{t+1}}{W_t} \leq (1 - \tfrac{\epsilon}{2})^M,$$

where M is the number of mistakes. Taking logarithm of both sides, we get

$$\texttt{cost}^* \cdot \ln(1-\epsilon) - \ln K < M \cdot \ln(1 - \tfrac{\epsilon}{2}) < M \cdot (-\tfrac{\epsilon}{2}),$$

where the last inequality follows from Eq. (5.3). Rearranging the terms, we get

$$M < \texttt{cost}^* \cdot \tfrac{2}{\epsilon} \ln(\tfrac{1}{1-\epsilon}) + \tfrac{2}{\epsilon} \ln K < \tfrac{2}{1-\epsilon} \cdot \texttt{cost}^* + \tfrac{2}{\epsilon} \cdot \ln K,$$

where the last step follows from (5.3) with $x = \frac{\epsilon}{1-\epsilon}$. To summarize, we have proved:

Theorem 5.9. *The number of mistakes made by WMA with parameter $\epsilon \in (0,1)$ is at most*

$$\frac{2}{1-\epsilon} \cdot \texttt{cost}^* + \frac{2}{\epsilon} \cdot \ln K.$$

Remark 5.10. This bound is very meaningful if $\texttt{cost}^*$ is small, but it does not imply sublinear regret guarantees when $\texttt{cost}^* = \Omega(T)$. Interestingly, it recovers the $O(\ln K)$ number of mistakes in the special case with a perfect expert, *i.e.,* when $\texttt{cost}^* = 0$.

5.3 Hedge Algorithm

We improve over the previous section in two ways: we solve the general case, online learning with experts, and obtain regret bounds that are $o(T)$ and, in fact, optimal. We start with an easy observation that deterministic algorithms are not sufficient for this goal: essentially, any deterministic algorithm can be easily "fooled" (even) by a deterministic, oblivious adversary.

Theorem 5.11. *Consider online learning with K experts and binary costs. Any deterministic algorithm has total cost T for some deterministic, oblivious adversary, even if $\texttt{cost}^* \leq T/K$.*

The easy proof is left as Exercise 5.3. Essentially, the adversary knows exactly what the algorithm is going to do in the next round, and can rig the costs to hurt the algorithm.

Remark 5.12. Note that the special case of *binary* prediction with experts advice is much easier for deterministic algorithms. Indeed, it allows for an "approximation ratio" arbitrarily close to 2, as in Theorem 5.9, whereas in the general case the "approximation ratio" cannot be better than K.

We define a randomized algorithm for online learning with experts, called Hedge. This algorithm maintains a weight $w_t(a)$ for each arm a, with the same update rule as in WMA (generalized beyond 0-1 costs in a fairly natural way). We need to use a different rule to select an arm, because (i) we need this rule to be randomized, in light of Theorem 5.11, and (ii) the weighted majority rule is not even well-defined in the general case. We use another selection rule, which is also very natural: at each round, choose an arm with probability proportional to the weights. The complete specification is shown in Algorithm 5.2:

parameter: $\epsilon \in (0, 1/2)$

Initialize the weights as $w_1(a) = 1$ for each arm a.
For each round t:
  Let $p_t(a) = \frac{w_t(a)}{\sum_{a'=1}^{K} w_t(a')}$.
  Sample an arm a_t from distribution $p_t(\cdot)$.
  Observe cost $c_t(a)$ for each arm a.
  For each arm a, update its weight $w_{t+1}(a) = w_t(a) \cdot (1-\epsilon)^{c_t(a)}$.

Algorithm 5.2: Hedge algorithm for online learning with experts

Below we analyze Hedge, and prove $O\left(\sqrt{T \log K}\right)$ bound on expected regret. We use the same analysis to derive several important extensions, used in the subsequent sections on adversarial bandits.

Remark 5.13. The $O\left(\sqrt{T \log K}\right)$ regret bound is the best possible for regret. This can be seen on a simple example in which all costs are IID with mean $1/2$, see Exercise 5.2(b). Recall that we also have a $\Omega(\sqrt{T})$ bound for pseudo-regret, due to the lower-bound analysis for two arms in Chapter 2.

As in the previous section, we use the technique outlined in Remark 5.7, with $W_t = \sum_{a=1}^{K} w_t(a)$ being the total weight of all arms at round t. Throughout, $\epsilon \in (0, 1/2)$ denotes the parameter in the algorithm.

The analysis is not very long but intricate; some find it beautiful. We break it in several distinct steps, for ease of comprehension.

Step 1: easy observations

The weight of each arm after the last round is

$$w_{T+1}(a) = w_1(a) \prod_{t=1}^{T} (1-\epsilon)^{c_t(a)} = (1-\epsilon)^{\texttt{cost}(a)}.$$

Hence, the total weight of last round satisfies

$$W_{T+1} > w_{T+1}(a^*) = (1-\epsilon)^{\texttt{cost}^*}. \tag{5.6}$$

From the algorithm, we know that the total initial weight is $W_1 = K$.

Step 2: multiplicative decrease in W_t

We use polynomial upper bounds for $(1-\epsilon)^x$, $x > 0$. We use two variants, stated in a unified form:

$$(1-\epsilon)^x \leq 1 - \alpha x + \beta x^2 \quad \text{for all } x \in [0, u], \tag{5.7}$$

where the parameters α, β and u may depend on ϵ but not on x. The two variants are:

- a first-order (*i.e.,* linear) upper bound with $(\alpha, \beta, u) = (\epsilon, 0, 1)$;
- a second-order (*i.e.,* quadratic) upper bound with $\alpha = \ln\left(\frac{1}{1-\epsilon}\right)$, $\beta = \alpha^2$ and $u = \infty$.

We apply Eq. (5.7) to $x = c_t(a)$, for each round t and each arm a. We continue the analysis for both variants of Eq. (5.7) simultaneously: thus, we fix α and β and assume that $c_t(\cdot) \leq u$. Then

$$\begin{aligned}
\frac{W_{t+1}}{W_t} &= \sum_{a\in[K]} (1-\epsilon)^{c_t(a)} \cdot \frac{w_t(a)}{W_t} \\
&< \sum_{a\in[K]} (1 - \alpha\, c_t(a) + \beta\, c_t(a)^2) \cdot p_t(a) \\
&= \sum_{a\in[K]} p_t(a) - \alpha \sum_{a\in[K]} p_t(a)\, c_t(a) + \beta \sum_{a\in[K]} p_t(a)\, c_t(a)^2 \\
&= 1 - \alpha F_t + \beta G_t, \qquad (5.8)
\end{aligned}$$

where

$$\begin{aligned}
F_t &= \sum_{a\in[K]} p_t(a) \cdot c_t(a) = \mathbb{E}\left[\, c_t(a_t) \mid \vec{w}_t \,\right] \\
G_t &= \sum_{a\in[K]} p_t(a) \cdot c_t^2(a) = \mathbb{E}\left[\, c_t^2(a_t) \mid \vec{w}_t \,\right]. \qquad (5.9)
\end{aligned}$$

Here $\vec{w}_t = (w_t(a) : a \in [K])$ is the vector of weights at round t. Notice that the total expected cost of the algorithm is $\mathbb{E}[\texttt{cost}(\texttt{ALG})] = \sum_t \mathbb{E}[F_t]$.

A naive attempt

Using the (5.8), we can obtain:

$$\frac{(1-\epsilon)^{\texttt{cost}^*}}{K} \leq \frac{W_{T+1}}{W_1} = \prod_{t=1}^{T} \frac{W_{t+1}}{W_t} < \prod_{t=1}^{T} (1 - \alpha F_t + \beta G_t).$$

However, it is unclear how to connect the right-hand side to $\sum_t F_t$ so as to argue about $\texttt{cost}(\texttt{ALG})$.

Step 3: the telescoping product

Taking a logarithm on both sides of Eq. (5.8)) and using Eq. (5.3)), we obtain

$$\ln \frac{W_{t+1}}{W_t} < \ln(1 - \alpha F_t + \beta G_t) < -\alpha F_t + \beta G_t.$$

Inverting the signs and summing over t on both sides, we obtain

$$\begin{aligned}
\sum_{t\in[T]} (\alpha F_t - \beta G_t) &< -\sum_{t\in[T]} \ln \frac{W_{t+1}}{W_t} \\
&= -\ln \prod_{t\in[T]} \frac{W_{t+1}}{W_t} \\
&= -\ln \frac{W_{T+1}}{W_1} \\
&= \ln W_1 - \ln W_{T+1} \\
&< \ln K - \ln(1-\epsilon) \cdot \text{cost}^*, \qquad (5.10)
\end{aligned}$$

where we used (5.6) in the last step. Taking expectation on both sides, we obtain:

$$\alpha \cdot \mathbb{E}[\texttt{cost}(\texttt{ALG})] < \beta \sum_{t\in[T]} \mathbb{E}[G_t] + \ln K - \ln(1-\epsilon) \cdot \mathbb{E}[\texttt{cost}^*]. \tag{5.11}$$

We use this equation in two different ways, depending on the variant of Eq. (5.7). With $\alpha = \epsilon$ and $\beta = 0$ and $c_t(\cdot) \leq 1$, we obtain:

$$\mathbb{E}[\texttt{cost}(\texttt{ALG})] < \tfrac{\ln K}{\epsilon} + \underbrace{\tfrac{1}{\epsilon}\ln(\tfrac{1}{1-\epsilon})}_{\leq 1+\epsilon \text{ if } \epsilon \in (0,1/2)} \mathbb{E}[\texttt{cost}^*].$$

$$\mathbb{E}[\texttt{cost}(\texttt{ALG}) - \texttt{cost}^*] < \frac{\ln K}{\epsilon} + \epsilon\ \mathbb{E}[\texttt{cost}^*].$$

This yields the main regret bound for Hedge:

Theorem 5.14. *Assume all costs are at most* 1*. Consider an adaptive adversary such that* $\texttt{cost}^* \leq uT$ *for some known number* u*; trivially,* $u = 1$*. Then* Hedge *with parameter* $\epsilon = \sqrt{\frac{\ln K}{uT}}$ *satisfies*

$$\mathbb{E}[\texttt{cost}(\texttt{ALG}) - \texttt{cost}^*] < 2 \cdot \sqrt{uT \ln K}.$$

Remark 5.15. We also obtain a meaningful performance guarantee which holds with probability 1, rather than in expectation. Using the same parameters, $\alpha = \epsilon = \sqrt{\frac{\ln K}{T}}$ and $\beta = 0$, and assuming all costs are at most 1, Eq. (5.10) implies that

$$\textstyle\sum_{t\in[T]} p_t \cdot c_t - \texttt{cost}^* < 2 \cdot \sqrt{T \ln K}. \tag{5.12}$$

Step 4: unbounded costs

Next, we consider the case where the costs can be unbounded, but we have an upper bound on $\mathbb{E}[G_t]$. We use Eq. (5.11) with $\alpha = \ln(\frac{1}{1-\epsilon})$ and $\beta = \alpha^2$ to obtain:

$$\alpha\, \mathbb{E}[\texttt{cost}(\texttt{ALG})] < \alpha^2 \sum_{t\in[T]} \mathbb{E}[G_t] + \ln K + \alpha\, \mathbb{E}[\texttt{cost}^*].$$

Dividing both sides by α and moving terms around, we get

$$\mathbb{E}[\texttt{cost}(\texttt{ALG}) - \texttt{cost}^*] < \frac{\ln K}{\alpha} + \alpha \sum_{t\in[T]} \mathbb{E}[G_t] < \frac{\ln K}{\epsilon} + 3\epsilon \sum_{t\in[T]} \mathbb{E}[G_t],$$

where the last step uses the fact that $\epsilon < \alpha < 3\epsilon$ for $\epsilon \in (0, 1/2)$. Thus:

Theorem 5.16. *Assume* $\sum_{t\in[T]} \mathbb{E}[G_t] \leq uT$ *for some known number* u*, where* $\mathbb{E}[G_t] = \mathbb{E}\left[\, c_t^2(a_t) \,\right]$ *as per (5.9). Then* Hedge *with parameter* $\epsilon = \sqrt{\frac{\ln K}{3uT}}$ *achieves regret*

$$\mathbb{E}[\texttt{cost}(\texttt{ALG}) - \texttt{cost}^*] < 2\sqrt{3} \cdot \sqrt{uT \ln K}.$$

In particular, if $c_t(\cdot) \leq c$*, for some known* c*, then one can take* $u = c^2$*.*

In the next chapter we use this lemma to analyze a bandit algorithm.

Step 5: unbounded costs with small expectation and variance

Consider a randomized, oblivious adversary such that the costs are independent across rounds. Instead of bounding the actual costs $c_t(a)$, let us instead bound their expectation and variance:

$$\mathbb{E}\left[\, c_t(a) \,\right] \leq \mu \text{ and } \text{Var}\left[\, c_t(a) \,\right] \leq \sigma^2 \text{ for all rounds } t \text{ and all arms } a. \tag{5.13}$$

Then for each round t we have:

$$\begin{aligned}
\mathbb{E}\left[\, c_t(a)^2 \,\right] &= \text{Var}\left[\, c_t(a) \,\right] + \left(\, \mathbb{E}[c_t(a)] \,\right)^2 \leq \sigma^2 + \mu^2. \\
\mathbb{E}[G_t] &= \sum_{a \in [K]} p_t(a)\ \mathbb{E}\left[\, c_t(a)^2 \,\right] \leq \sum_{a \in [K]} p_t(a)\, (\mu^2 + \sigma^2) = \mu^2 + \sigma^2.
\end{aligned}$$

Thus, Theorem 5.16 with $u = \mu^2 + \sigma^2$ implies the following:

Corollary 5.17. *Consider online learning with experts, with a randomized, oblivious adversary. Assume the costs are independent across rounds, and satisfy (5.13) for some μ and σ known to the algorithm. Then* `Hedge` *with parameter $\epsilon = \sqrt{\ln K/(3T(\mu^2+\sigma^2))}$ has regret*

$$\mathbb{E}[\texttt{cost(ALG)} - \texttt{cost}^*] < 2\sqrt{3} \cdot \sqrt{T(\mu^2+\sigma^2)\ln K}.$$

5.4 Literature review and discussion

The weighted majority algorithm is from Littlestone and Warmuth (1994), and `Hedge` algorithm is from (Littlestone and Warmuth, 1994; Cesa-Bianchi et al., 1997; Freund and Schapire, 1997).[1] Sequential prediction with experts advise has a long history in economics and statistics, see Cesa-Bianchi and Lugosi (2006, Chapter 2) for deeper discussion and bibliographic notes.

The material in this chapter is presented in many courses and books on online learning. This chapter mostly follows the lecture plan from (Kleinberg, 2007, Week 1), but presents the analysis of `Hedge` a little differently, so as to make it immediately applicable to the analysis of `Exp3` and `Exp4` in the next chapter.

The problem of online learning with experts, and `Hedge` algorithm for this problem, are foundational in a very strong sense. `Hedge` serves as a subroutine in adversarial bandits (Chapter 6) and bandits with global constraints (Chapter 10). The multiplicative-weights technique is essential in several extensions of adversarial bandits, *e.g.,* the ones in Auer et al. (2002b) and Beygelzimer et al. (2011). `Hedge` also powers many applications "outside" of multi-armed bandits or online machine learning, particularly in the design of primal-dual algorithms (see Arora et al. (2012) for a survey) and via the "learning in games" framework (see Chapter 9 and bibliographic remarks therein). In many of these applications, `Hedge` can be replaced by any algorithm for online learning with experts, as long as it satisfies an appropriate regret bound.

While the regret bound for `Hedge` comes with a fairly small constant, it is still a multiplicative constant away from the best known lower bound. Sometimes one can obtain upper and lower bounds on regret that match exactly. This has been accomplished for $K = 2$ experts (Cover, 1965; Gravin et al., 2016) and for $K \leq 3$ experts with a geometric time horizon[2] (Gravin et al., 2016), along with an explicit specification of the optimal algorithm. Focusing on the regret of `Hedge`, Gravin et al. (2017) derived an improved the *lower* bound for each K, exactly matching a known upper bound from Cesa-Bianchi et al. (1997).

Various stronger notions of regret have been studied; we detail them in the next chapter (Section 6.6).

[1] Freund and Schapire (1997) handles the full generality of online learning with experts. Littlestone and Warmuth (1994) and Cesa-Bianchi et al. (1997) focus on binary prediction with experts advice, with slightly stronger guarantees.

[2] That is, the game stops in each round independently with probability δ.

Online learning with K experts can be interpreted as one with action set Δ_K, the set of all distributions over the K experts. This is a special case of *online linear optimization* (OLO) with a convex action set (see Chapter 7). In fact, Hedge can be interpreted as a special case of two broad families of algorithms for OLO, *follow the regularized leader* and *online mirror descent*; for background, see *e.g.,* Hazan (2015) and McMahan (2017). This OLO-based perspective has been essential in much of the recent progress.

5.5 Exercises and hints

Exercise 5.1. Consider binary prediction with expert advice, with a perfect expert. Prove Theorem 5.8: prove that any algorithm makes at least $\Omega(\min(T, \log K))$ mistakes in the worst case.

Take-away: The majority vote algorithm is worst-case-optimal for instances with a perfect expert.

Hint: For simplicity, let $K = 2^d$ and $T \geq d$, for some integer d. Construct a distribution over problem instances such that each algorithm makes $\Omega(d)$ mistakes in expectation. Recall that each expert e corresponds to a binary sequence $e \in \{0,1\}^T$, where e_t is the prediction for round t. Put experts in 1-1 correspondence with all possible binary sequences for the first d rounds. Pick the "perfect expert" u.a.r. among the experts.

Exercise 5.2 (IID costs and regret). Assume IID costs, as in Remark 5.3.

(a) Prove that $\min_a \mathbb{E}[\texttt{cost}(a)] \leq \mathbb{E}[\min_a \texttt{cost}(a)] + O(\sqrt{T \log(KT)})$.

Take-away: All $\sqrt{T}$-regret bounds from Chapter 1 carry over to regret.

Hint: Define the "clean event" as follows: the event in Hoeffding inequality holds for the cost sequence of each arm.

(b) Construct a problem instance with a deterministic adversary for which any algorithm suffers regret

$$\mathbb{E}[\texttt{cost}(\texttt{ALG}) - \min_{a \in [K]} \texttt{cost}(a)] \geq \Omega(\sqrt{T \log K}).$$

Hint: Assume all arms have 0-1 costs with mean $1/2$. Use the following fact about random walks:

$$\mathbb{E}[\min_a \texttt{cost}(a)] \leq \tfrac{T}{2} - \Omega(\sqrt{T \log K}). \tag{5.14}$$

Note: This example does not carry over to pseudo-regret. Since each arm has expected reward of $1/2$ in each round, any algorithm trivially achieves 0 pseudo-regret for this problem instance.

Take-away: The $O(\sqrt{T \log K})$ regret bound for Hedge is the best possible for regret. Further, $\log(T)$ upper regret bounds from Chapter 1 do not carry over to regret in full generality.

(c) Prove that algorithms UCB and Successive Elimination achieve logarithmic regret bound (1.11) even for regret, assuming that the best-in-foresight arm a^* is unique.

Hint: Under the "clean event", $\texttt{cost}(a) < T \cdot \mu(a) + O(\sqrt{T \log T})$ for each arm $a \neq a^*$, where $\mu(a)$ is the mean cost. It follows that a^* is also the best-in-hindsight arm, unless $\mu(a) - \mu(a^*) < O(\sqrt{T \log T})$ for some arm $a \neq a^*$ (in which case the claimed regret bound holds trivially).

Exercise 5.3. Prove Theorem 5.11: prove that any deterministic algorithm for the online learning problem with K experts and 0-1 costs can suffer total cost T for some deterministic-oblivious adversary, even if $\texttt{cost}^* \leq T/K$.

Take-away: With a deterministic algorithm, cannot even recover the guarantee from Theorem 5.9 for the general case of online learning with experts, let alone have $o(T)$ regret.

Hint: Fix the algorithm. Construct the problem instance by induction on round t, so that the chosen arm has cost 1 and all other arms have cost 0.

Chapter 6

Adversarial Bandits

This chapter is concerned with *adversarial bandits*: multi-armed bandits with adversarially chosen costs. In fact, we solve a more general formulation that explicitly includes expert advice. Our algorithm is based on a reduction to the full-feedback problem studied in Chapter 5.

Prerequisites: Chapter 5.

For ease of exposition, we focus on deterministic, oblivious adversary: that is, the costs for all arms and all rounds are chosen in advance. We are interested in regret with respect to the best-in-hindsight arm, as defined in Eq. (5.1). We assume bounded per-round costs: $c_t(a) \leq 1$ for all rounds t and all arms a.

We achieve regret bound $\mathbb{E}[R(T)] \leq O\left(\sqrt{KT \log K}\right)$. Curiously, this upper regret bound not only matches our result for IID bandits (Theorem 1.10), but in fact improves it a little bit, replacing the $\log T$ with $\log K$. This regret bound is essentially optimal, due to the $\Omega(\sqrt{KT})$ lower bound from Chapter 2.

Recap from Chapter 5

Let us recap the material on the full-feedback problem, reframing it slightly for this chapter. Recall that in the full-feedback problem, the cost of each arm is revealed after every round. A common interpretation is that each action corresponds to an "expert" that gives advice or makes predictions, and in each round the algorithm needs to choose which expert to follow. Hence, this problem is also known as the *online learning with experts*. We considered a particular algorithm for this problem, called `Hedge`. In each round t, it computes a distribution p_t over experts, and samples an expert from this distribution. We obtained the following regret bound (Theorem 5.16):

Theorem 6.1. *Consider online learning with N experts and T rounds. Consider an adaptive adversary and regret $R(T)$ relative to the best-observed expert. Suppose in any run of* `Hedge`*, with any parameter $\epsilon > 0$, it holds that $\sum_{t\in[T]} \mathbb{E}[G_t] \leq uT$ for some known $u > 0$, where $G_t = \sum_{\text{experts } e} p_t(e)\, c_t^2(e)$. Then*

$$\mathbb{E}[R(T)] \leq 2\sqrt{3} \cdot \sqrt{uT \log N} \qquad \text{provided that} \quad \epsilon = \epsilon_u := \sqrt{\tfrac{\ln N}{3uT}}.$$

We will distinguish between "experts" in the full-feedback problem and "actions" in the bandit problem. Therefore, we will consistently use "experts" for the former and "actions/arms" for the latter.

6.1 Reduction from bandit feedback to full feedback

Our algorithm for adversarial bandits is a reduction to the full-feedback setting. The reduction proceeds as follows. For each arm, we create an expert which always recommends this arm. We use `Hedge` with this set of experts. In each round t, we use the expert e_t chosen by `Hedge` to pick an arm a_t, and define "fake costs" $\widehat{c}_t(\cdot)$ on all experts in order to provide `Hedge` with valid inputs. This generic reduction is given below:

Given: set $\mathcal{E}$ of experts, parameter $\epsilon \in (0, \frac{1}{2})$ for `Hedge`.
In each round t,

1. Call `Hedge`, receive the probability distribution p_t over $\mathcal{E}$.
2. Draw an expert e_t independently from p_t.
3. *Selection rule*: use e_t to pick arm a_t (TBD).
4. Observe the cost $c_t(a_t)$ of the chosen arm.
5. Define "fake costs" $\widehat{c}_t(e)$ for all experts $x \in \mathcal{E}$ (TBD).
6. Return the "fake costs" to `Hedge`.

Algorithm 6.1: Reduction from bandit feedback to full feedback

We will specify *how* to select arm a_t using expert e_t, and *how* to define fake costs. The former provides for sufficient exploration, and the latter ensures that fake costs are unbiased estimates of the true costs.

6.2 Adversarial bandits with expert advice

The reduction defined above suggests a more general problem: what if experts can predict different arms in different rounds? This problem, called *bandits with expert advice*, is one that we will actually solve. We do it for three reasons: because it is a very interesting generalization, because we can solve it with very little extra work, and because separating experts from actions makes the solution clearer. Formally, the problem is defined as follows:

Problem protocol: Adversarial bandits with expert advice

Given: K arms, set $\mathcal{E}$ of N experts, T rounds (all known).
In each round $t \in [T]$:

1. adversary picks cost $c_t(a) \geq 0$ for each arm $a \in [K]$,
2. each expert $e \in \mathcal{E}$ recommends an arm $a_{t,e}$ (observed by the algorithm),
3. algorithm picks arm $a_t \in [K]$ and receives the corresponding cost $c_t(a_t)$.

The total cost of each expert is defined as $\texttt{cost}(e) = \sum_{t \in [T]} c_t(a_{t,e})$. The goal is to minimize regret relative to the best *expert*, rather than the best action:

$$R(T) = \texttt{cost}(\texttt{ALG}) - \min_{e \in \mathcal{E}} \texttt{cost}(e).$$

We focus on a deterministic, oblivious adversary: all costs $c_t(\cdot)$ are selected in advance. Further, we assume that the recommendations $a_{t,e}$ are also chosen in advance, *i.e.,* the experts cannot learn over time.

We solve this problem via the reduction in Algorithm 6.1, and achieve

$$\mathbb{E}[R(T)] \leq O\left(\sqrt{KT\log N}\right).$$

Note the logarithmic dependence on N: this regret bound allows to handle *lots* of experts.

This regret bound is essentially the best possible. Specifically, there is a nearly matching lower bound on regret that holds for any given triple of parameters K, T, N:

$$\mathbb{E}[R(T)] \geq \min\left(T,\ \Omega\left(\sqrt{KT\log(N)/\log(K)}\right)\right). \tag{6.1}$$

This lower bound can be proved by an ingenious (yet simple) reduction to the basic $\Omega(\sqrt{KT})$ lower regret bound for bandits, see Exercise 6.1.

6.3 Preliminary analysis: unbiased estimates

We have two notions of "cost" on experts. For each expert e at round t, we have the true cost $c_t(e) = c_t(a_{t,e})$ determined by the predicted arm $a_{t,e}$, and the *fake cost* $\widehat{c}_t(e)$ that is computed inside the algorithm and then fed to `Hedge`. Thus, our regret bounds for `Hedge` refer to the *fake regret* defined relative to the fake costs:

$$\widehat{R}_{\texttt{Hedge}}(T) = \widehat{\texttt{cost}}(\texttt{Hedge}) - \min_{e\in\mathcal{E}} \widehat{\texttt{cost}}(e),$$

where $\widehat{\texttt{cost}}(\texttt{Hedge})$ and $\widehat{\texttt{cost}}(e)$ are the total fake costs for `Hedge` and expert e, respectively.

We want the fake costs to be unbiased estimates of the true costs. This is because we will need to convert a bound on the fake regret $\widehat{R}_{\texttt{Hedge}}(T)$ into a statement about the true costs accumulated by `Hedge`. Formally, we ensure that

$$\mathbb{E}\left[\,\widehat{c}_t(e) \mid \vec{p}_t\,\right] = c_t(e) \quad \text{for all experts } e, \tag{6.2}$$

where $\vec{p}_t = (p_t(e) :\ \text{all experts } e)$. We use this as follows:

Claim 6.2. *Assuming Eq. (6.2), it holds that* $\mathbb{E}[R_{\texttt{Hedge}}(T)] \leq \mathbb{E}[\widehat{R}_{\texttt{Hedge}}(T)]$.

Proof. First, we connect true costs of `Hedge` with the corresponding fake costs.

$$\begin{aligned}
\mathbb{E}\left[\,\widehat{c}_t(e_t) \mid \vec{p}_t\,\right] &= \textstyle\sum_{e\in\mathcal{E}} \Pr\left[\,e_t = e \mid \vec{p}_t\,\right]\ \mathbb{E}\left[\,\widehat{c}_t(e) \mid \vec{p}_t\,\right] \\
&= \textstyle\sum_{e\in\mathcal{E}} p_t(e)\ c_t(e) && \textit{(use definition of } p_t(e) \textit{ and Eq. (6.2))} \\
&= \mathbb{E}\left[\,c_t(e_t) \mid \vec{p}_t\,\right].
\end{aligned}$$

Taking expectation of both sides, $\mathbb{E}[\widehat{c}_t(e_t)] = \mathbb{E}[c_t(e_t)]$. Summing over all rounds, it follows that

$$\mathbb{E}\left[\,\widehat{\texttt{cost}}(\texttt{Hedge})\,\right] = \mathbb{E}\left[\,\texttt{cost}(\texttt{Hedge})\,\right].$$

To complete the proof, we deal with the benchmark:

$$\mathbb{E}\left[\min_{e\in\mathcal{E}} \widehat{\texttt{cost}}(e)\right] \leq \min_{e\in\mathcal{E}} \mathbb{E}\left[\,\widehat{\texttt{cost}}(e)\,\right] = \min_{e\in\mathcal{E}} \mathbb{E}\left[\,\texttt{cost}(e)\,\right] = \min_{e\in\mathcal{E}} \texttt{cost}(e).$$

The first equality holds by (6.2), and the second equality holds because true costs $c_t(e)$ are deterministic. □

Remark 6.3. This proof used the "full power" of assumption (6.2). A weaker assumption $\mathbb{E}\left[\,\widehat{c}_t(e)\,\right] = \mathbb{E}[c_t(e)]$ would not have sufficed to argue about true vs. fake costs of `Hedge`.

6.4 Algorithm Exp4 and crude analysis

To complete the specification of Algorithm 6.1, we need to define fake costs $\widehat{c}_t(\cdot)$ and specify how to choose an arm a_t. For fake costs, we will use a standard trick in statistics called *Inverse Propensity Score* (IPS). Whichever way arm a_t is chosen in each round t given the probability distribution $\vec{p}_t$ over experts, this defines distribution q_t over arms:

$$q_t(a) := \Pr[\, a_t = a \mid \vec{p}_t \,] \quad \text{for each arm } a.$$

Using these probabilities, we define the fake costs on each arm as follows:

$$\widehat{c}_t(a) = \begin{cases} c_t(a_t)/q_t(a_t) & a_t = a, \\ 0 & \text{otherwise.} \end{cases}$$

The fake cost on each expert e is defined as the fake cost of the arm chosen by this expert: $\widehat{c}_t(e) = \widehat{c}_t(a_{t,e})$.

Remark 6.4. Algorithm 6.1 can use fake costs as defined above as long as it can compute probability $q_t(a_t)$.

Claim 6.5. *Eq. (6.2) holds if $q_t(a_{t,e}) > 0$ for each expert e.*

Proof. Let us argue about each arm a separately. If $q_t(a) > 0$ then

$$\mathbb{E}[\, \widehat{c}_t(a) \mid \vec{p}_t \,] = \Pr[\, a_t = a \mid \vec{p}_t \,] \cdot \frac{c_t(a_t)}{q_t(a)} + \Pr[\, a_t \neq a \mid \vec{p}_t \,] \cdot 0 = c_t(a).$$

For a given expert e plug in arm $a = a_{t,e}$, its choice in round t. □

So, if an arm a is selected by some expert in a given round t, the selection rule needs to choose this arm with non-zero probability, regardless of which expert is actually chosen by Hedge and what is this expert's recommendation. Further, if probability $q_t(a)$ is sufficiently large, then one can upper-bound fake costs and apply Theorem 6.1. On the other hand, we would like to follow the chosen expert e_t most of the time, so as to ensure low costs. A simple and natural way to achieve both objectives is to follow e_t with probability $1 - \gamma$, for some small $\gamma > 0$, and with the remaining probability choose an arm uniformly at random. This completes the specification of our algorithm, which is known as Exp4. We recap it in Algorithm 6.2.

Note that $q_t(a) \geq \gamma/K > 0$ for each arm a. According to Claim 6.5 and Claim 6.2, the expected true regret of Hedge is upper-bounded by its expected fake regret: $\mathbb{E}[R_{\texttt{Hedge}}(T)] \leq \mathbb{E}[\widehat{R}_{\texttt{Hedge}}(T)]$.

Remark 6.6. Fake costs $\widehat{c}_t(\cdot)$ depend on the probability distribution $\hat{p}_t$ chosen by Hedge. This distribution depends on the actions selected by Exp4 in the past, and these actions in turn depend on the experts chosen by Hedge in the past. To summarize, fake costs depend on the experts chosen by Hedge in the past. So, fake costs do not form an oblivious adversary, as far as Hedge is concerned. Thus, we need regret guarantees for Hedge against an adaptive adversary, even though the true costs are chosen by an oblivious adversary.

In each round t, our algorithm accumulates cost at most 1 from the low-probability exploration, and cost $c_t(e_t)$ from the chosen expert e_t. So the expected cost in this round is $\mathbb{E}[c_t(a_t)] \leq \gamma + \mathbb{E}[c_t(e_t)]$. Summing over all rounds, we obtain:

$$\begin{aligned} \mathbb{E}[\, \texttt{cost(Exp4)} \,] &\leq \mathbb{E}[\, \texttt{cost(Hedge)} \,] + \gamma T. \\ \mathbb{E}\left[\, R_{\texttt{Exp4}}(T) \,\right] &\leq \mathbb{E}\left[\, R_{\texttt{Hedge}}(T) \,\right] + \gamma T \leq \mathbb{E}\left[\, \widehat{R}_{\texttt{Hedge}}(T) \,\right] + \gamma T. \end{aligned} \tag{6.3}$$

Eq. (6.3) quantifies the sense in which the regret bound for Exp4 reduces to the regret bound for Hedge.

Given: set $\mathcal{E}$ of experts, parameter $\epsilon \in (0, \frac{1}{2})$ for `Hedge`, exploration parameter $\gamma \in [0, \frac{1}{2})$.
In each round t,

1. Call `Hedge`, receive the probability distribution p_t over $\mathcal{E}$.
2. Draw an expert e_t independently from p_t.
3. *Selection rule*: with probability $1-\gamma$ follow expert e_t; else pick an arm a_t uniformly at random.
4. Observe the cost $c_t(a_t)$ of the chosen arm.
5. Define fake costs for all experts e:

$$\widehat{c}_t(e) = \begin{cases} \frac{c_t(a_t)}{\Pr[a_t = a_{t,e} | \vec{p}_t]} & a_t = a_{t,e}, \\ 0 & \text{otherwise.} \end{cases}$$

6. Return the "fake costs" $\widehat{c}(\cdot)$ to `Hedge`.

Algorithm 6.2: Algorithm `Exp4` for adversarial bandits with experts advice

We can immediately derive a crude regret bound via Theorem 6.1. Observing $\widehat{c}_t(a) \leq 1/q_t(a) \leq K/\gamma$, we can take $u = (K/\gamma)^2$ in the theorem, and conclude that

$$\mathbb{E}\left[R_{\texttt{Exp4}}(T) \right] \leq O\left({}^{K}/_{\gamma} \cdot K^{1/2} \left(\log N\right)^{1/4} + \gamma T \right).$$

To (approximately) minimize expected regret, choose γ so as to equalize the two summands.

Theorem 6.7. *Consider adversarial bandits with expert advice, with a deterministic-oblivious adversary. Algorithm* `Exp4` *with parameters* $\gamma = T^{-1/4} K^{1/2} (\log N)^{1/4}$ *and* $\epsilon = \epsilon_u$, $u = K/\gamma$, *achieves regret*

$$\mathbb{E}\left[R(T) \right] = O\left(T^{3/4} K^{1/2} \left(\log N\right)^{1/4} \right).$$

Remark 6.8. We did not use any property of `Hedge` other than the regret bound in Theorem 6.1. Therefore, `Hedge` can be replaced with any other full-feedback algorithm with the same regret bound.

6.5 Improved analysis of `Exp4`

We obtain a better regret bound by analyzing the quantity

$$\widehat{G}_t := \textstyle\sum_{e \in \mathcal{E}} p_t(e)\, \widehat{c}_t^2(e).$$

We prove that $\mathbb{E}[G_t] \leq \frac{K}{1-\gamma}$, and use the regret bound for `Hedge`, Theorem 6.1, with $u = \frac{K}{1-\gamma}$. In contrast, the crude analysis presented above used Theorem 6.1 with $u = ({}^{K}/_{\gamma})^2$.

Remark 6.9. This analysis extends to $\gamma = 0$. In other words, the uniform exploration step in the algorithm is not necessary. While we previously used $\gamma > 0$ to guarantee that $q_t(a_{t,e}) > 0$ for each expert e, the same conclusion also follows from the fact that `Hedge` chooses each expert with a non-zero probability.

Lemma 6.10. *Fix parameter $\gamma \in [0, \frac{1}{2})$ and round t. Then $\mathbb{E}[\widehat{G}_t] \leq \frac{K}{1-\gamma}$.*

Proof. For each arm a, let $\mathcal{E}_a = \{e \in \mathcal{E} : a_{t,e} = a\}$ be the set of all experts that recommended this arm. Let

$$p_t(a) := \sum_{e \in \mathcal{E}_a} p_t(e)$$

be the probability that the expert chosen by `Hedge` recommends arm a. Then

$$q_t(a) = p_t(a)(1-\gamma) + \frac{\gamma}{K} \geq (1-\gamma)\, p_t(a).$$

For each expert e, letting $a = a_{t,e}$ be the recommended arm, we have:

$$\widehat{c}_t(e) = \widehat{c}_t(a) \leq \frac{c_t(a)}{q_t(a)} \leq \frac{1}{q_t(a)} \leq \frac{1}{(1-\gamma)\, p_t(a)}. \tag{6.4}$$

Each realization of $\widehat{G}_t$ satisfies:

$$\begin{aligned}
\widehat{G}_t &:= \sum_{e \in \mathcal{E}} p_t(e)\, \widehat{c}_t^2(e) \\
&= \sum_a \sum_{e \in \mathcal{E}_a} p_t(e) \cdot \widehat{c}_t(e) \cdot \widehat{c}_t(e) && \textit{(re-write as a sum over arms)} \\
&\leq \sum_a \sum_{e \in \mathcal{E}_a} \frac{p_t(e)}{(1-\gamma)\, p_t(a)}\, \widehat{c}_t(a) && \textit{(replace one } \widehat{c}_t(a) \textit{ with an upper bound (6.4))} \\
&= \frac{1}{1-\gamma} \sum_a \frac{\widehat{c}_t(a)}{p_t(a)} \sum_{e \in \mathcal{E}_a} p_t(e) && \textit{(move ``constant terms'' out of the inner sum)} \\
&= \frac{1}{1-\gamma} \sum_a \widehat{c}_t(a) && \textit{(the inner sum is just } p_t(a)\textit{)}
\end{aligned}$$

To complete the proof, take expectations over both sides and recall that $\mathbb{E}[\widehat{c}_t(a)] = c_t(a) \leq 1$. □

Let us complete the analysis, being slightly careful with the multiplicative constant in the regret bound:

$$\begin{aligned}
\mathbb{E}\left[\, \widehat{R}_{\texttt{Hedge}}(T) \,\right] &\leq 2\sqrt{3/(1-\gamma)} \cdot \sqrt{TK \log N} \\
\mathbb{E}\left[\, R_{\texttt{Exp4}}(T) \,\right] &\leq 2\sqrt{3/(1-\gamma)} \cdot \sqrt{TK \log N} + \gamma T && \textit{(by Eq. (6.3))} \\
&\leq 2\sqrt{3} \cdot \sqrt{TK \log N} + 2\gamma T && \textit{(since } \sqrt{1/(1-\gamma)} \leq 1+\gamma\textit{)} \qquad (6.5)
\end{aligned}$$

(To derive (6.5), we assumed w.l.o.g. that $2\sqrt{3} \cdot \sqrt{TK \log N} \leq T$.) This holds for any $\gamma > 0$. Therefore:

Theorem 6.11. *Consider adversarial bandits with expert advice, with a deterministic-oblivious adversary. Algorithm* `Exp4` *with parameters $\gamma \in [0, \frac{1}{2T})$ and $\epsilon = \epsilon_U$, $U = \frac{K}{1-\gamma}$, achieves regret*

$$\mathbb{E}[R(T)] \leq 2\sqrt{3} \cdot \sqrt{TK \log N} + 1.$$

6.6 Literature review and discussion

Exp4 stands for **exp**loration, **exp**loitation, **exp**onentiation, and **exp**erts. The specialization to adversarial bandits (without expert advice, *i.e.,* with experts that correspond to arms) is called Exp3, for the same reason. Both algorithms were introduced (and named) in the seminal paper (Auer et al., 2002b), along with several extensions. Their analysis is presented in various books and courses on online learning (e.g., Cesa-Bianchi and Lugosi, 2006; Bubeck and Cesa-Bianchi, 2012). Our presentation was most influenced by (Kleinberg, 2007, Week 8), but the reduction to Hedge is made more explicit.

Apart from the stated regret bound, Exp4 can be usefully applied in several extensions: contextual bandits (Chapter 8), shifting regret, and dynamic regret for slowly changing costs (both: see below).

The lower bound (6.1) for adversarial bandits with expert advice is due to Agarwal et al. (2012). We used a slightly simplified construction from Seldin and Lugosi (2016) in the hint for Exercise 6.1.

Running time. The running time of Exp4 is $O(N + K)$, so the algorithm becomes very slow when N, the number of experts, is very large. Good regret *and* good running time can be obtained for some important special cases with a large N. One approach is to replace Hedge with a different algorithm for online learning with experts which satisfies one or both regret bounds in Theorem 6.1. We follow this approach in the next chapter. On the other hand, the running time for Exp3 is very nice because in each round, we only need to do a small amount of computation to update the weights.

6.6.1 Refinements for the "standard" notion of regret

Much research has been done on various refined guarantees for adversarial bandits, using the notion of regret defined in this chapter. The most immediate ones are as follows:

- an algorithm that obtains a similar regret bound for adaptive adversaries (against the best-observed arm), and high probability. This is Algorithm EXP3.P.1 in the original paper Auer et al. (2002b).
- an algorithm with $O(\sqrt{KT})$ regret, shaving off the $\sqrt{\log K}$ factor and matching the lower bound up to constant factors (Audibert and Bubeck, 2010).
- While we have only considered a finite number of experts, similar results can be obtained for *infinite* classes of experts with some special structure. In particular, borrowing the tools from statistical learning theory, it is possible to handle classes of experts with a small VC-dimension.

Data-dependent regret bounds provide improvements if the realized costs are, in some sense, "nice", even though the extent of this "niceness" is not revealed to the algorithm. Such regret bounds come in many flavors, discussed below.

- *Small benchmark*: the total expected cost/reward of the best arm, denoted B. The $\tilde{O}(\sqrt{KT})$ regret rate can be improved to $\tilde{O}(\sqrt{KB})$, without knowing B in advance. The reward-maximizing version, when small B means that the best arm is that good, has been solved in the original paper (Auer et al., 2002b). In the cost-minimizing version, small B has an opposite meaning: the best arm *is* quite good. This version, a.k.a. *small-loss* regret bounds, has been much more challenging (Allenberg et al., 2006; Rakhlin and Sridharan, 2013a; Neu, 2015; Foster et al., 2016b; Lykouris et al., 2018b).
- *Small change in costs:* one can obtain regret bounds that are near-optimal in the worst case, and improve when the realized cost of each arm a does not change too much. Small change in costs can be quantified in terms of the total variation $\sum_{t,a} \left(c_t(a) - \texttt{cost}(a)/T \right)^2$ (Hazan and Kale, 2011), or

in terms of path-lengths $\sum_t |c_t(a) - c_{t-1}(a)|$ (Wei and Luo, 2018; Bubeck et al., 2019). However, these guarantees do not do much when only the change in *expected* costs is small, *e.g.,* for IID costs.

- *Best of both worlds:* algorithms that work well for both adversarial and stochastic bandits. Bubeck and Slivkins (2012) achieve an algorithm that is near-optimal in the worst case (like Exp3), and achieves logarithmic regret (like UCB1) if the costs are actually IID Further work refines and optimizes the regret bounds, achieves them with a more practical algorithm, and improves them for the adversarial case if the adversary is constrained (Seldin and Slivkins, 2014; Auer and Chiang, 2016; Seldin and Lugosi, 2017; Wei and Luo, 2018; Zimmert et al., 2019; Zimmert and Seldin, 2019).

- *Small change in expected costs.* A particularly clean model unifies the themes of "small change" and "best of both worlds" discussed above. It focuses on the total change in *expected* costs, denoted C and interpreted as an *adversarial corruption* of an otherwise stochastic problem instance. The goal here is regret bounds that are logarithmic when $C = 0$ (*i.e.,* for IID costs) and degrade gracefully as C increases, even if C is not known to the algorithm. Initiated in Lykouris et al. (2018a), this direction continued in (Gupta et al., 2019; Zimmert and Seldin, 2021) and a number of follow-ups.

6.6.2 Stronger notions of regret

Let us consider several benchmarks that are *stronger* than the best-in-hindsight arm in (5.1).

Shifting regret. One can compete with "policies" that can change the arm from one round to another, but not too often. More formally, an *S-shifting policy* is sequence of arms $\pi = (a_t : t \in [T])$ with at most S "shifts": rounds t such that $a_t \neq a_{t+1}$. *S-shifting regret* is defined as the algorithm's total cost minus the total cost of the best S-shifting policy:

$$R_S(T) = \texttt{cost}(\texttt{ALG}) - \min_{S\text{-shifting policies } \pi} \texttt{cost}(\pi).$$

Consider this as a bandit problem with expert advice, where each S-shifting policy is an expert. The number of experts $N \leq (KT)^S$; while it may be a large number, $\log(N)$ is not too bad! Using Exp4 and plugging $N \leq (KT)^S$ into Theorem 6.11, we obtain $\mathbb{E}[R_S(T)] = \tilde{O}(\sqrt{KST})$.

While Exp4 is computationally inefficient, Auer et al. (2002b) tackle shifting regret using a modification of Exp3 algorithm, with essentially the same running time as Exp3 and a more involved analysis. They obtain $\mathbb{E}[R_S(T)] = \tilde{O}(\sqrt{SKT})$ if S is known, and $\mathbb{E}[R_S(T)] = \tilde{O}(S\sqrt{KT})$ if S is not known.

For a fixed S, any algorithm suffers regret $\Omega(\sqrt{SKT})$ in the worst case (Garivier and Moulines, 2011).

Dynamic regret. The strongest possible benchmark is the best *current* arm: $c_t^* = \min_a c_t(a)$. We are interested in *dynamic regret*, defined as

$$R^*(T) = \min(\texttt{ALG}) - \textstyle\sum_{t\in[T]} c_t^*.$$

This benchmark is *too hard* in the worst case, without additional assumptions. In what follows, we consider a randomized oblivious adversary, and make assumptions on the rate of change in cost distributions.

Assume the adversary changes the cost distribution at most S times. One can re-use results for shifting regret, so as to obtain expected dynamic regret $\tilde{O}(\sqrt{SKT})$ when S is known, and $\tilde{O}(S\sqrt{KT})$ when S is not known. The former regret bound is optimal (Garivier and Moulines, 2011). The regret bound for unknown S can be improved to $\mathbb{E}[R^*(T)] = \tilde{O}(\sqrt{SKT})$, matching the optimal regret rate for known S, using more advanced algorithms (Auer et al., 2019; Chen et al., 2019).

Suppose expected costs change *slowly*, by at most ϵ in each round. Then one can obtain bounds on dynamic regret of the form $\mathbb{E}[R^*(T)] \leq C_{\epsilon,K} \cdot T$, where $C_{\epsilon,K} \ll 1$ is a "constant" determined by ϵ and

K. The intuition is that the algorithm pays a constant per-round "price" for keeping up with the changing costs, and the goal is to minimize this price as a function of K and ϵ. Stronger regret bound (*i.e.,* one with smaller $C_{\epsilon,K}$) is possible if expected costs evolve as a random walk.[1] One way to address these scenarios is to use an algorithm with S-shifting regret, for an appropriately chosen value of S, and restart it after a fixed number of rounds, *e.g.,* see Exercise 6.3. Slivkins and Upfal (2008); Slivkins (2014) provide algorithms with better bounds on dynamic regret, and obtain matching lower bounds. Their approach is an extension of the UCB1 algorithm, see Algorithm 1.5 in Chapter 1. In a nutshell, they add a term ϕ_t to the definition of $\texttt{UCB}_t(\cdot)$, where ϕ_t is a known high-confidence upper bound on each arm's change in expected rewards in t steps, and restart this algorithm every n steps, for a suitably chosen fixed n. E.g., $\phi_t = \epsilon t$ in general, and $\phi_t = O(\sqrt{t \log T})$ for the random walk scenario. In principle, ϕ_t can be arbitrary, possibly depending on an arm and on the time of the latest restart.

A more flexible model considers the *total variation* in cost distributions, $V = \sum_{t\in[T-1]} V_t$, where V_t is the amount of change in a given round t (defined as a total-variation distance between the cost distribution for rounds t and $t+1$). While V can be as large as KT in the worst case, the idea is to benefit when V is small. Compared to the previous two models, an arbitrary amount of per-round change is allowed, and a larger amount of change is "weighed" more heavily. The optimal regret rate is $\tilde{O}(V^{1/3}T^{2/3})$ when V is known; it can be achieved, *e.g.,* by Exp4 algorithm with restarts. The same regret rate can be achieved even without knowing V, using a more advanced algorithm (Chen et al., 2019). This result also obtains the best possible regret rate when each arm's expected costs change by at most ϵ per round, without knowing the ϵ.[2]

Swap regret. Let us consider a strong benchmark that depends not only on the costs but on the algorithm itself. Informally, what if we consider the sequence of actions chosen by the algorithm, and swap each occurrence of each action a with $\pi(a)$, according to some fixed *swapping policy* $\pi : [K] \mapsto [K]$. The algorithm competes with the best swapping policy. More formally, we define *swap regret* as

$$R_{\texttt{swap}}(T) = \texttt{cost}(\texttt{ALG}) - \min_{\text{swapping policies } \pi \in \mathcal{F}} \sum_{t\in[T]} c_t(\pi(a_t)), \tag{6.6}$$

where $\mathcal{F}$ is the class of all swapping policies. The standard definition of regret corresponds to a version of (6.6), where $\mathcal{F}$ is the set of all "constant" swapping functions (*i.e.,* those that map all arms to the same one).

The best known regret bound for swap regret is $\tilde{O}(K\sqrt{T})$ (Stoltz, 2005). Swapping regret has been introduced (Blum and Mansour, 2007), who also provided an explicit transformation that takes an algorithm with a "standard" regret bound, and transforms it into an algorithm with a bound on swap regret. Plugging in Exp3 algorithm, this approach results in $\tilde{O}(K\sqrt{KT})$ regret rate. For the full-feedback version, the same approach achieves $\tilde{O}(\sqrt{KT})$ regret rate, with a matching lower bound. All algorithmic results are for an oblivious adversary; the lower bound is for adaptive adversary.

Earlier literature in theoretical economics, starting from (Hart and Mas-Colell, 2000), designed algorithms for a weaker notion called *internal regret* (Foster and Vohra, 1997, 1998, 1999; Cesa-Bianchi and Lugosi, 2003). The latter is defined as a version of (6.6) where $\mathcal{F}$ consists of swapping policies that change only one arm. The main motivation was its connection to equilibria in repeated games, more on this in Section 9.5. Note that the swap regret is at most K times internal regret.

Counterfactual regret. The notion of "best-observed arm" is not entirely satisfying for adaptive adversaries, as discussed in Section 5.1. Instead, one can consider a *counterfactual* notion of regret, which asks what would have happened if the algorithm actually played this arm in every round. The benchmark is the

[1] Formally, the expected cost of each arm evolves as an independent random walk on $[0, 1]$ interval with reflecting boundaries.

[2] However, this approach does not appear to obtain better regret bounds more complicated variants of the "slow change" setting, such as when each arm's expected reward follows a random walk.

best fixed arm, but in this counterfactual sense. Sublinear regret is impossible against unrestricted adversaries. However, one can obtain non-trivial results against memory-restricted adversaries: ones that can use only m most recent rounds. In particular, one can obtain $\tilde{O}(mK^{1/3}T^{2/3})$ counterfactual regret for all $m < T^{2/3}$, without knowing m. This can be achieved using a simple "batching" trick: use some bandit algorithm ALG so that one round in the execution of ALG corresponds to a batch of τ consecutive rounds in the original problems. This result is due to Dekel et al. (2012); a similar regret bound for the full-feedback case has appeared in an earlier paper (Merhav et al., 2002).

Regret relative to the best *algorithm*. Given a (small) family $\mathcal{F}$ of algorithms, can one design a "meta-algorithm" which, on each problem instance, does almost as well as the best algorithm in $\mathcal{F}$? This is an extremely difficult benchmark, even if each algorithm in $\mathcal{F}$ has a very small set S of possible internal states. Even with $|S| \leq 3$ (and only 3 algorithms and 3 actions), no "meta-algorithm" can achieve expected regret better than $O(T \log^{-3/2} T)$ relative to the best algorithm in $\mathcal{F}$ (henceforth, *$\mathcal{F}$-regret*). Surprisingly, there is an intricate algorithm which obtains a similar upper bound on $\mathcal{F}$-regret, $O(\sqrt{K|S|} \cdot T \cdot \log^{-\Omega(1)} T)$. Both results are from Feige et al. (2017).

Agarwal et al. (2017c) bypass these limitations and design a "meta-algorithm" with much more favorable bounds on $\mathcal{F}$-regret. This comes at a cost of substantial assumptions on the algorithms' structure and a rather unwieldy fine-print in the regret bounds. Nevertheless, their regret bounds have been productively applied, *e.g.,* in Krishnamurthy et al. (2020).

6.7 Exercises and hints

Exercise 6.1 (lower bound). Consider adversarial bandits with experts advice. Prove the lower bound in (6.1) for any given (K, N, T). More precisely: construct a randomized problem instance for which any algorithm satisfies (6.1).

Hint: Split the time interval $1..T$ into $M = \frac{\ln N}{\ln K}$ non-overlapping sub-intervals of duration T/M . For each sub-interval, construct the randomized problem instance from Chapter 2 (independently across the sub-intervals). Each expert recommends the same arm within any given sub-interval; the set of experts includes all experts of this form.

Exercise 6.2 (fixed discretization). Let us extend the fixed discretization approach from Chapter 4 to adversarial bandits. Consider adversarial bandits with the set of arms $\mathcal{A} = [0, 1]$. Fix $\epsilon > 0$ and let S_ϵ be the ϵ-uniform mesh over $\mathcal{A}$, *i.e.,* the set of all points in $[0, 1]$ that are integer multiples of ϵ. For a subset $S \subset \mathcal{A}$, the optimal total cost is $\texttt{cost}^*(S) := \min_{a \in S} \texttt{cost}(a)$, and the discretization error is defined as $\texttt{DE}(S_\epsilon) = (\, \texttt{cost}^*(S) - \texttt{cost}^*(\mathcal{A}) \,)\, /T$.

(a) Prove that $\texttt{DE}(S_\epsilon) \leq L\epsilon$, assuming Lipschitz property:

$$|c_t(a) - c_t(a')| \leq L \cdot |a - a'| \quad \text{for all arms } a, a' \in \mathcal{A} \text{ and all rounds } t. \tag{6.7}$$

(b) Consider a version of dynamic pricing (see Section 4.5.4), where the values $v_1\,, \ \ldots\ , v_T$ are chosen by a deterministic, oblivious adversary. For compatibility, state the problem in terms of costs rather than rewards: in each round t, the cost is $-p_t$ if there is a sale, 0 otherwise. Prove that $\texttt{DE}(S_\epsilon) \leq \epsilon$.

Note: It is a special case of adversarial bandits with some extra structure which allows us to bound discretization error *without assuming Lipschitzness*.

(c) Assume that $\texttt{DE}(S_\epsilon) \leq \epsilon$ for all $\epsilon > 0$. Obtain an algorithm with regret $\mathbb{E}[R(T)] \leq O(T^{2/3} \log T)$.

Hint: Use algorithm Exp3 with arms $S \subset \mathcal{A}$, for a well-chosen subset S.

Exercise 6.3 (slowly changing costs). Consider a randomized oblivious adversary such that the expected cost of each arm changes by at most ϵ from one round to another, for some fixed and known $\epsilon > 0$. Use algorithm Exp4 to obtain dynamic regret

$$\mathbb{E}[R^*(T)] \leq O(T) \cdot (\epsilon \cdot K \log K)^{1/3}. \tag{6.8}$$

Hint: Recall the application of Exp4 to n-shifting regret, denote it $\texttt{Exp4}(n)$. Let $\texttt{OPT}_n = \min \texttt{cost}(\pi)$, where the $\min$ is over all n-shifting policies π, be the benchmark in n-shifting regret. Analyze the "discretization error": the difference between $\texttt{OPT}_n$ and $\texttt{OPT}^* = \sum_{t=1}^{T} \min_a c_t(a)$, the benchmark in dynamic regret. Namely: prove that $\texttt{OPT}_n - \texttt{OPT}^* \leq O(\epsilon T^2/n)$. Derive an upper bound on dynamic regret that is in terms of n. Optimize the choice of n.

Chapter 7

Linear Costs and Semi-Bandits

This chapter provides a joint introduction to several related lines of work: online routing, combinatorial (semi-)bandits, linear bandits, and online linear optimization. We study bandit problems with linear costs: actions are represented by vectors in $\mathbb{R}^d$, and their costs are linear in this representation. This problem is challenging even under full feedback, let alone bandit feedback; we also consider an intermediate regime called *semi-bandit feedback*. We start with an important special case called *online routing*, and its generalization, combinatorial semi-bandits. We solve both using a version of the bandits-to-`Hedge` reduction from Chapters 6. However, this solution is slow. To remedy this, we focus on the full-feedback problem, a.k.a. *online linear optimization*. We present a fundamental algorithm for this problem, called *Follow The Perturbed Leader*, which plugs nicely into the bandits-to-experts reduction and makes it computationally efficient.

Prerequisites: Chapters 5-6.

We consider *linear costs* throughout this chapter. As in the last two chapters, there are K actions and a fixed time horizon T, and each action $a \in [K]$ yields cost $c_t(a) \geq 0$ at each round $t \in [T]$. Actions are represented by low-dimensional real vectors; for simplicity, we assume that all actions lie within a unit hypercube: $a \in [0,1]^d$. Action costs are linear in a, namely: $c_t(a) = a \cdot v_t$ for some weight vector $v_t \in \mathbb{R}^d$ which is the same for all actions, but depends on the current time step.

Recap: bandits-to-experts reduction

We build on the bandits-to-experts reduction from Chapter 6. We will use it in a more abstract version, spelled out in Algorithm 7.1, with arbitrary "fake costs" and an arbitrary full-feedback algorithm. We posit that experts correspond to arms, *i.e.,* for each arm there is an expert that always recommends this arm.

We assume that "fake costs" are bounded from above and satisfy (6.2), and that `ALG` satisfies a regret bound against an adversary with bounded costs. For notation, an adversary is called u-*bounded* if $c_t(\cdot) \leq u$. While some steps in the algorithm are unspecified, the analysis from Chapter 6 carries over word-by-word *no matter how these missing steps are filled in*, and implies the following theorem.

Theorem 7.1. *Consider Algorithm 7.1 with algorithm* `ALG` *that achieves regret bound* $\mathbb{E}[R(T)] \leq f(T,K,u)$ *against adaptive,* u*-bounded adversary, for any given* $u > 0$ *that is known to the algorithm.*

Consider adversarial bandits with a deterministic, oblivious adversary. Assume "fake costs" satisfy

$$\mathbb{E}\left[\,\widehat{c}_t(x) \mid p_t\,\right] = c_t(x) \text{ and } \widehat{c}_t(x) \leq u/\gamma \qquad \text{for all experts } x \text{ and all rounds } t,$$

where u *is some number that is known to the algorithm. Then Algorithm 7.1 achieves regret*

$$\mathbb{E}[R(T)] \leq f(T,K,u/\gamma) + \gamma T.$$

Given: an algorithm ALG for online learning with experts, and parameter $\gamma \in (0, \frac{1}{2})$.
Problem: adversarial bandits with K arms and T rounds; "experts" correspond to arms.
In each round $t \in [T]$:

1. call ALG, receive an expert x_t chosen for this round,
   where x_t is an independent draw from some distribution p_t over the experts.
2. with probability $1 - \gamma$ follow expert x_t;
   else, chose arm via a version of "random exploration" (TBD)
3. observe cost c_t for the chosen arm, and perhaps some extra feedback (TBD)
4. define "fake costs" $\widehat{c}_t(x)$ for each expert x (TBD), and return them to ALG.

Algorithm 7.1: Reduction from bandit feedback to full feedback.

Corollary 7.2. *If* ALG *is* Hedge *in the theorem above, one can take* $f(T, K, u) = O(u \cdot \sqrt{T \ln K})$ *by Theorem 5.16. Then, setting* $\gamma = T^{-1/4} \sqrt{u \cdot \log K}$*, Algorithm 7.1 achieves regret*

$$\mathbb{E}[R(T)] \leq O\left(T^{3/4} \sqrt{u \cdot \log K} \right).$$

We instantiate this algorithm, *i.e.,* specify the missing pieces, to obtain a solution for some special cases of linear bandits that we define below.

7.1 Online routing problem

Let us consider an important special case of linear bandits called the *online routing problem*, a.k.a. *online shortest paths*. We are given a graph G with d edges, a source node u, and a destination node v. The graph can either be directed or undirected. We have costs on edges that we interpret as delays in routing, or lengths in a shortest-path problem. The cost of a path is the sum over all edges in this path. The costs can change over time. In each round, an algorithm chooses among "actions" that correspond to u-v paths in the graph. Informally, the algorithm's goal in each round is to find the "best route" from u to v: an u-v path with minimal cost (*i.e.,* minimal travel time). More formally, the problem is as follows:

Problem protocol: Online routing problem

Given: graph G, source node u, destination node v.
For each round $t \in [T]$:

1. Adversary chooses costs $c_t(e) \in [0, 1]$ for all edges e.
2. Algorithm chooses u-v-path $a_t \subset \texttt{Edges}(G)$.
3. Algorithm incurs cost $c_t(a_t) = \sum_{e \in a_t} a_e \cdot c_t(e)$ and receives feedback.

To cast this problem as a special case of "linear bandits", note that each path can be specified by a subset of edges, which in turn can be specified by a d-dimensional binary vector $a \in \{0, 1\}^d$. Here edges of the

graph are numbered from 1 to d, and for each edge e the corresponding entry a_e equals 1 if and only if this edge is included in the path. Let $v_t = (c_t(e) : \text{edges } e \in G)$ be the vector of edge costs at round t. Then the cost of a path can be represented as a linear product $c_t(a) = a \cdot v_t = \sum_{e \in [d]} a_e \; c_t(e)$.

There are three versions of the problem, depending on which feedback is received:

- *Bandit feedback:* only $c_t(a_t)$ is observed;
- *Semi-bandit feedback:* costs $c_t(e)$ for all edges $e \in a_t$ are observed;
- *Full feedback:* costs $c_t(e)$ for all edges e are observed.

Semi-bandit feedback is an intermediate "feedback regime" which we will focus on.

Full feedback. The full-feedback version can be solved with `Hedge` algorithm. Applying Theorem 5.16 with a trivial upper bound $c_t(\cdot) \leq d$ on action costs, and the trivial upper bound $K \leq 2^d$ on the number of paths, we obtain regret $\mathbb{E}[R(T)] \leq O(d\sqrt{dT})$.

Corollary 7.3. *Consider online routing with full feedback. Algorithm* `Hedge` *with with parameter* $\epsilon = 1/\sqrt{dT}$ *achieves regret* $\mathbb{E}[R(T)] \leq O(d\sqrt{dT})$.

This regret bound is optimal up $O(\sqrt{d})$ factor (Koolen et al., 2010). (The root-T dependence on T is optimal, as per Chapter 2.) An important drawback of using `Hedge` for this problem is the running time, which is exponential in d. We return to this issue in Section 7.3.

Semi-bandit feedback. We use the bandit-to-experts reduction (Algorithm 7.1) with `Hedge` algorithm, as a concrete and simple application of this machinery to linear bandits. We assume that the costs are selected by a deterministic oblivious adversary, and we do not worry about the running time.

As a preliminary attempt, we can use `Exp3` algorithm for this problem. However, expected regret would be proportional to square root of the number of actions, which in this case may be exponential in d.

Instead, we seek a regret bound of the form:

$$\mathbb{E}[R(T)] \leq \text{poly}(d) \cdot T^{\beta}, \quad \text{where } \beta < 1.$$

To this end, we use Algorithm 7.1 with `Hedge`. The "extra information" in the reduction is the semi-bandit feedback. Recall that we also need to specify the "random exploration" and the "fake costs".

For the "random exploration step", instead of selecting an action uniformly at random (as we did in `Exp4`), we select an edge e uniformly at random, and pick the corresponding path $a^{(e)}$ as the chosen action. We assume that each edge e belongs to some u-v path $a^{(e)}$; this is without loss of generality, because otherwise we can just remove this edge from the graph.

We define fake costs for each edge e separately; the fake cost of a path is simply the sum of fake costs over its edges. Let $\Lambda_{t,e}$ be the event that in round t, the algorithm chooses "random exploration", *and* in random exploration, it chooses edge e. Note that $\Pr[\Lambda_{t,e}] = \gamma/d$. The fake cost on edge e is

$$\widehat{c}_t(e) = \begin{cases} \frac{c_t(e)}{\gamma/d} & \text{if event } \Lambda_{t,e} \text{ happens} \\ 0 & \text{otherwise} \end{cases} \tag{7.1}$$

This completes the specification of an algorithm for the online routing problem with semi-bandit feedback; we will refer to this algorithm as `SemiBanditHedge`.

As in the previous lecture, we prove that fake costs provide unbiased estimates for true costs:

$$\mathbb{E}[\widehat{c}_t(e) \mid p_t] = c_t(e) \quad \text{for each round } t \text{ and each edge } e.$$

Since the fake cost for each edge is at most d/γ, it follows that $c_t(a) \leq d^2/\gamma$ for each action a. Thus, we can immediately use Corollary 7.2 with $u = d^2$. For the number of actions, let us use an upper bound $K \leq 2^d$. Then $u \log K \leq d^3$, and so:

Theorem 7.4. *Consider the online routing problem with semi-bandit feedback. Assume deterministic oblivious adversary. Algorithm* `SemiBanditHedge` *achieved regret* $\mathbb{E}[R(T)] \leq O(d^{3/2}\ T^{3/4})$.

Remark 7.5. Fake cost $\widehat{c}_t(e)$ is determined by the corresponding true cost $c_t(e)$ and event $\Lambda_{t,e}$ which does not depend on algorithm's actions. Therefore, fake costs are chosen by a (randomized) oblivious adversary. In particular, in order to apply Theorem 7.1 with a different algorithm `ALG` for online learning with experts, it suffices to have an upper bound on regret against an oblivious adversary.

7.2 Combinatorial semi-bandits

The online routing problem with semi-bandit feedback is a special case of *combinatorial semi-bandits*, where edges are replaced with d "atoms", and u-v paths are replaced with feasible subsets of atoms. The family of feasible subsets can be arbitrary (but it is known to the algorithm).

Problem protocol: Combinatorial semi-bandits

Given: set S of atoms, and a family $\mathcal{F}$ of feasible actions (subsets of S).

For each round $t \in [T]$:

1. Adversary chooses costs $c_t(e) \in [0,1]$ for all atoms e,
2. Algorithm chooses a feasible action $a_t \in \mathcal{F}$,
3. Algorithm incurs cost $c_t(a_t) = \sum_{e \in a_t} a_e \cdot c_t(e)$
   and observes costs $c_t(e)$ for all atoms $e \in a_t$.

The algorithm and analysis from the previous section does not rely on any special properties of u-v paths. Thus, they carry over word-by-word to combinatorial semi-bandits, replacing edges with atoms, and u-v paths with feasible subsets. We obtain the following theorem:

Theorem 7.6. *Consider combinatorial semi-bandits with deterministic oblivious adversary. Algorithm* `SemiBanditHedge` *achieved regret* $\mathbb{E}[R(T)] \leq O(d^{3/2}\ T^{3/4})$.

Let us list a few other notable special cases of combinatorial semi-bandits:

- *News Articles:* a news site needs to select a subset of articles to display to each user. The user can either click on an article or ignore it. Here, rounds correspond to users, atoms are the news articles, the reward is 1 if it is clicked and 0 otherwise, and feasible subsets can encode various constraints on selecting the articles.
- *Ads:* a website needs select a subset of ads to display to each user. For each displayed ad, we observe whether the user clicked on it, in which case the website receives some payment. The payment may depend on both the ad and on the user. Mathematically, the problem is very similar to the news articles: rounds correspond to users, atoms are the ads, and feasible subsets can encode constraints on which ads can or cannot be shown together. The difference is that the payments are no longer 0-1.

- *A slate of news articles:* Similar to the news articles problem, but the ordering of the articles on the webpage matters. This the news site needs to select a *slate* (an ordered list) of articles. To represent this problem as an instance of combinatorial semi-bandits, define each "atom" to mean "this news article is chosen for that slot". A subset of atoms is feasible if it defines a valid slate: *i.e.,* there is exactly one news article assigned to each slot.

Thus, combinatorial semi-bandits is a general setting which captures several motivating examples, and allows for a unified solution. Such results are valuable even if each of the motivating examples is only a very idealized version of reality, *i.e.,* it captures some features of reality but ignores some others.

Low regret *and* running time. Recall that `SemiBanditHedge` is slow: its running time per round is exponential in d, as it relies on `Hedge` with this many experts. We would like it to be *polynomial* in d.

One should not hope to accomplish this in the full generality of combinatorial bandits. Indeed, even if the costs on all atoms were known, choosing the best feasible action (a feasible subset of minimal cost) is a well-known problem of *combinatorial optimization*, which is NP-hard. However, combinatorial optimization allows for polynomial-time solutions in many interesting special cases. For example, in the online routing problem discussed above the corresponding combinatorial optimization problem is a well-known shortest-path problem. Thus, a natural approach is to assume that we have access to an *optimization oracle*: an algorithm which finds the best feasible action given the costs on all atoms, and express the running time of our algorithm in terms of the number of oracle calls.

In Section 7.3, we use this oracle to construct a new algorithm for combinatorial bandits with full feedback, called *Follow The Perturbed Leader* (`FTPL`). In each round, this algorithm inputs only the costs on the atoms, and makes only one oracle call. We derive a regret bound

$$\mathbb{E}[R(T)] \leq O(u\sqrt{dT}) \tag{7.2}$$

against an oblivious, u-bounded adversary such that the atom costs are at most u/d. (Recall that a regret bound against an oblivious adversary suffices for our purposes, as per Remark 7.5.)

We use algorithm `SemiBanditHedge` as before, but replace `Hedge` with `FTPL`; call the new algorithm `SemiBanditFTPL`. The analysis from Section 7.1 carries over to `SemiBanditFTPL`. We take $u = d^2/\gamma$ as a known upper bound on the fake costs of actions, and note that the fake costs of atoms are at most u/d. Thus, we can apply Theorem 7.1 for `FTPL` with fake costs, and obtain regret

$$\mathbb{E}[R(T)] \leq O(u\sqrt{dT}) + \gamma T.$$

Optimizing the choice of parameter γ, we immediately obtain the following theorem:

Theorem 7.7. *Consider combinatorial semi-bandits with deterministic oblivious adversary. Then algorithm* `SemiBanditFTPL` *with appropriately chosen parameter γ achieved regret*

$$\mathbb{E}[R(T)] \leq O\left(d^{5/4}\, T^{3/4}\right).$$

Remark 7.8. In terms of the running time, it is essential that the fake costs on atoms can be computed *fast*: this is because the normalizing probability in (7.1) is known in advance.

Alternatively, we could have defined fake costs on atoms e as

$$\widehat{c}_t(e) = \begin{cases} c_t(e)/\Pr[e \in a_t \mid p_t] & \text{if } e \in a_t \\ 0 & \text{otherwise.} \end{cases}$$

This definition leads to essentially the same regret bound (and, in fact, is somewhat better in practice). However, computing the probability $\Pr[e \in a_t \mid p_t]$ in a brute-force way requires iterating over all actions, which leads to running times exponential in d, similar to `Hedge`.

Remark 7.9. Solving a version with bandit feedback requires more work. The main challenge is to estimate fake costs for all atoms in the chosen action, whereas we only observe the total cost for the action. One solution is to construct a suitable *basis*: a subset of feasible actions, called *base actions*, such that each action can be represented as a linear combination thereof. Then a version of Algorithm 7.1, where in the "random exploration" step is uniform among the base actions, gives us fake costs for the base actions. The fake cost on each atom is the corresponding linear combination over the base actions. This approach works as long as the linear coefficients are small, and ensuring this property takes some work. This approach is worked out in Awerbuch and Kleinberg (2008), resulting in regret $\mathbb{E}[R(T)] \leq \tilde{O}(d^{10/3} \cdot T^{2/3})$.

7.3 Online Linear Optimization: Follow The Perturbed Leader

Let us turn our attention to *online linear optimization*, *i.e.,* bandits with full-feedback and linear costs. We do not restrict ourselves to combinatorial actions, and instead allow an arbitrary subset $\mathcal{A} \subset [0,1]^d$ of feasible actions. This subset is fixed over time and known to the algorithm. Recall that in each round t, the adversary chooses a "hidden vector" $v_t \in \mathbb{R}^d$, so that the cost for each action $a \in \mathcal{A}$ is $c_t(a) = a \cdot v_t$. We posit an upper bound on the costs: we assume that v_t satisfies $v_t \in [0, U/d]^d$, for some known parameter U, so that $c_t(a) \leq U$ for each action a.

Problem protocol: Online linear optimization

For each round $t \in [T]$:

1. Adversary chooses hidden vector $v_t \in \mathbb{R}^d$.
2. Algorithm chooses action $a = a_t \in \mathcal{A} \subset [0,1]^d$,
3. Algorithm incurs cost $c_t(a) = v_t \cdot a$ and observes v_t.

We design an algorithm, called Follow The Perturbed Leader (`FTPL`), that is computationally efficient and satisfies regret bound (7.2). In particular, this suffices to complete the proof of Theorem 7.7.

We assume than the algorithm has access to an *optimization oracle*: a subroutine which computes the best action for a given cost vector. Formally, we represent this oracle as a function M from cost vectors to feasible actions such that $M(v) \in \operatorname{argmin}_{a \in \mathcal{A}} a \cdot v$ (ties can be broken arbitrarily). As explained earlier, while in general the oracle is solving an NP-hard problem, polynomial-time algorithms exist for important special cases such as shortest paths. The implementation of the oracle is domain-specific, and is irrelevant to our analysis. We prove the following theorem:

Theorem 7.10. *Assume that $v_t \in [0, U/d]^d$ for some known parameter U. Algorithm* `FTPL` *achieves regret $\mathbb{E}[R(T)] \leq 2U \cdot \sqrt{dT}$. The running time in each round is polynomial in d plus one call to the oracle.*

Remark 7.11. The set of feasible actions $\mathcal{A}$ can be infinite, as long as a suitable oracle is provided. For example, if $\mathcal{A}$ is defined by a finite number of linear constraints, the oracle can be implemented via linear programming. Whereas `Hedge` is not even well-defined for infinitely many actions.

We use shorthand $v_{i:j} = \sum_{t=i}^{j} v_t \in \mathbb{R}^d$ to denote the total cost vector between rounds i and j.

Follow The Leader. Consider a simple, exploitation-only algorithm called *Follow The Leader*:

$$a_{t+1} = M(v_{1:t}).$$

Equivalently, we play an arm with the lowest average cost, based on the observations so far.

While this approach works fine for IID costs, it breaks for adversarial costs. The problem is synchronization: an oblivious adversary can force the algorithm to behave in a particular way, and synchronize its costs with algorithm's actions in a way that harms the algorithm. In fact, this can be done to any deterministic online learning algorithm, as per Theorem 5.11. For concreteness, consider the following example:

$$\begin{aligned}
&\mathcal{A} = \{(1,0),\ (0,1)\}\\
&v_1 = (\tfrac{1}{3}, \tfrac{2}{3})\\
&v_t = \begin{cases} (1,0) & \text{if } t \text{ is even,}\\ (0,1) & \text{if } t \text{ is odd.}\end{cases}
\end{aligned}$$

Then the total cost vector is

$$v_{1:t} = \begin{cases} (i+\frac{1}{3}, i-\frac{1}{3}) & \text{if } t = 2i,\\ (i+\frac{1}{3}, i+\frac{2}{3}) & \text{if } t = 2i+1.\end{cases}$$

Therefore, Follow The Leader picks action $a_{t+1} = (0,1)$ if t is even, and $a_{t+1} = (1,0)$ if t is odd. In both cases, we see that $c_{t+1}(a_{t+1}) = 1$. So the total cost for the algorithm is T, whereas any fixed action achieves total cost at most $1 + T/2$, so regret is, essentially, $T/2$.

Fix: perturb the history! Let us use randomization to side-step the synchronization issue discussed above. We perturb the history before handing it to the oracle. Namely, we pretend there was a 0-th round, with cost vector $v_0 \in \mathbb{R}^d$ sampled from some distribution $\mathcal{D}$. We then give the oracle the "perturbed history", as expressed by the total cost vector $v_{0:t-1}$, namely $a_t = M(v_{0:t-1})$. This modified algorithm is known as *Follow The Perturbed Leader* (`FTPL`).

Sample $v_0 \in \mathbb{R}^d$ from distribution $\mathcal{D}$;
for *each round* $t = 1, 2, \ldots$ **do**
  Choose arm $a_t = M(v_{0:t-1})$, where $v_{i:j} = \sum_{t=i}^{j} v_t \in \mathbb{R}^d$.
end

Algorithm 7.2: Follow The Perturbed Leader (`FTPL`).

Several choices for distribution $\mathcal{D}$ lead to meaningful analyses. For ease of exposition, we posit that each each coordinate of v_0 is sampled independently and uniformly from the interval $[-\frac{1}{\epsilon}, \frac{1}{\epsilon}]$. The parameter ϵ can be tuned according to T, U, and d; in the end, we use $\epsilon = \frac{\sqrt{d}}{U\sqrt{T}}$.

Analysis of the algorithm

As a tool to analyze `FTPL`, we consider a closely related algorithm called *Be The Perturbed Leader* (`BTPL`). Imagine that when we need to choose an action at time t, we already know the cost vector v_t, and in each round t we choose $a_t = M(v_{0:t})$. Note that `BTPL` is *not* an algorithm for online learning with experts; this is because it uses v_t to choose a_t.

The analysis proceeds in two steps. We first show that `BTPL` comes "close" to the optimal cost

$$\texttt{OPT} = \min_{a \in \mathcal{A}} \texttt{cost}(a) = v_{1:t} \cdot M(v_{1:t}),$$

and then we show that `FTPL` comes "close" to `BTPL`. Specifically, we will prove:

Lemma 7.12. *For each value of parameter $\epsilon > 0$,*
(i) $\texttt{cost}(\texttt{BTPL}) \leq \texttt{OPT} + \frac{d}{\epsilon}$
(ii) $\mathbb{E}[\texttt{cost}(\texttt{FTPL})] \leq \mathbb{E}[\texttt{cost}(\texttt{BTPL})] + \epsilon \cdot U^2 \cdot T$

Then choosing $\epsilon = \frac{\sqrt{d}}{U\sqrt{T}}$ gives Theorem 7.10. Curiously, note that part (i) makes a statement about realized costs, rather than expected costs.

Step I: `BTPL` **comes close to** `OPT`

By definition of the oracle M, it holds that

$$v \cdot M(v) \leq v \cdot a \quad \text{for any cost vector } v \text{ and feasible action } a. \tag{7.3}$$

The main argument proceeds as follows:

$$\begin{aligned}
\texttt{cost}(\texttt{BTPL}) + v_0 \cdot M(v_0) &= \sum_{t=0}^{T} v_t \cdot M(v_{0:T}) && \textit{(by definition of BTPL)} \\
&\leq v_{0:T} \cdot M(v_{0:T}) && \textit{(see Claim 7.13 below)} \qquad (7.4)\\
&\leq v_{0:T} \cdot M(v_{1:T}) && \textit{(by (7.3) with } a = M(v_{1:T})\textit{)} \\
&= v_0 \cdot M(v_{1:T}) + \underbrace{v_{1:T} \cdot M(v_{1:T})}_{\texttt{OPT}}.
\end{aligned}$$

Subtracting $v_0 \cdot M(v_0)$ from both sides, we obtain Lemma 7.12(i):

$$\texttt{cost}(\texttt{BTPL}) - \texttt{OPT} \leq \underbrace{v_0}_{\in[-\frac{1}{\epsilon},-\frac{1}{\epsilon}]^d} \cdot \underbrace{[M(v_{1:T}) - M(v_0)]}_{\in[-1,1]^d} \leq \frac{d}{\epsilon}.$$

The missing step (7.4) follows from the following claim, with $i = 0$ and $j = T$.

Claim 7.13. *For all rounds $i < j$, $\sum_{t=1}^{j} v_t \cdot M(v_{i:t}) \leq v_{i:j} \cdot M(v_{i:j})$.*

Proof. The proof is by induction on $j - i$. The claim is trivially satisfied for the base case $i = j$. For the inductive step:

$$\begin{aligned}
\sum_{t=i}^{j-1} v_t \cdot M(v_{i:t}) &\leq v_{i:j-1} \cdot M(v_{i:j-1}) && \textit{(by the inductive hypothesis)} \\
&\leq v_{i:j-1} \cdot M(v_{i:j}) && \textit{(by (7.3) with } a = M(v_{i:j})\textit{).}
\end{aligned}$$

Add $v_j \cdot M(v_{i:j})$ to both sides to complete the proof. □

Step II: `FTPL` **comes close to** `BTPL`

We compare the expected costs of `FTPL` and `BTPL` round per round. Specifically, we prove that

$$\mathbb{E}[\underbrace{v_t \cdot M(v_{0:t-1})}_{c_t(a_t) \text{ for } \texttt{FTPL}}] \leq \mathbb{E}[\underbrace{v_t \cdot M(v_{0:t})}_{c_t(a_t) \text{ for } \texttt{BTPL}}] + \epsilon U^2. \tag{7.5}$$

Summing up over all T rounds gives Lemma 7.12(ii).

It turns out that for proving (7.5) much of the structure in our problem is irrelevant. Specifically, we can denote $f(u) = v_t \cdot M(u)$ and $v = v_{1:t-1}$, and, essentially, prove (7.5) for arbitrary $f()$ and v.

Claim 7.14. *For any vectors $v \in \mathbb{R}^d$ and $v_t \in [0, U/d]^d$, and any function $f : \mathbb{R}^d \to [0, R]$,*

$$\left| \mathop{\mathbb{E}}_{v_0 \sim \mathcal{D}} [f(v_0 + v) - f(v_0 + v + v_t)] \right| \leq \epsilon UR.$$

In words: changing the input of function f from $v_0 + v$ to $v_0 + v + v_t$ does not substantially change the output, in expectation over v_0. What we actually prove is the following:

Claim 7.15. *Fix $v_t \in [0, U/d]^d$. There exists a random variable $v_0' \in \mathbb{R}^d$ such that (i) v_0' and $v_0 + v_t$ have the same marginal distribution, and (ii) $\Pr[v_0' \neq v_0] \leq \epsilon U$.*

It is easy to see that Claim 7.14 follows from Claim 7.15:

$$\begin{aligned} |\, \mathbb{E}\,[f(v_0 + v) - f(v_0 + v + v_t)]\,| &= \left|\, \mathbb{E}\left[f(v_0 + v) - f(v_0' + v)\right]\,\right| \\ &\leq \Pr[v_0' \neq v_0] \cdot R = \epsilon UR. \end{aligned}$$

It remains to prove Claim 7.15. First, let us prove this claim in one dimension:

Claim 7.16. *let X be a random variable uniformly distributed on the interval $[-\frac{1}{\epsilon}, \frac{1}{\epsilon}]$. We claim that for any $a \in [0, U/d]$ there exists a deterministic function $g(X, a)$ of X and a such that $g(X, a)$ and $X + a$ have the same marginal distribution, and $\Pr[g(X, a) \neq X] \leq \epsilon U/d$.*

Proof. Let us define

$$g(X, a) = \begin{cases} X & \text{if } X \in [v - \frac{1}{\epsilon}, \frac{1}{\epsilon}], \\ a - X & \text{if } X \in [-\frac{1}{\epsilon}, v - \frac{1}{\epsilon}). \end{cases}$$

It is easy to see that $g(X, a)$ is distributed uniformly on $[v - \frac{1}{\epsilon}, v + \frac{1}{\epsilon}]$. This is because $a - X$ is distributed uniformly on $[\frac{1}{\epsilon}, v + \frac{1}{\epsilon}]$ conditional on $X \in [-\frac{1}{\epsilon}, v - \frac{1}{\epsilon})$. Moreover,

$$\Pr[g(X, a) \neq X] \leq \Pr[X \notin [v - \tfrac{1}{\epsilon}, \tfrac{1}{\epsilon}]] = \epsilon v/2 \leq \tfrac{\epsilon U}{2d}. \qquad \square$$

To complete the proof of Claim 7.15, write $v_t = (v_{t,1}, v_{t,2}\,, \ \ldots \ , v_{t,d})$, and define $v_0' \in \mathbb{R}^d$ by setting its j-th coordinate to $Y(v_{0,j}, v_{t,j})$, for each coordinate j. We are done!

7.4 Literature review and discussion

This chapter touches upon several related lines of work: online routing, combinatorial (semi-)bandits, linear bandits, and online linear optimization. We briefly survey them below, along with some extensions.

Online routing and combinatorial (semi-)bandits. The study of *online routing*, a.k.a. *online shortest paths*, was initiated in Awerbuch and Kleinberg (2008), focusing on bandit feedback and achieving regret $\text{poly}(d) \cdot T^{2/3}$. Online routing with semi-bandit feedback was introduced in György et al. (2007), and the general problem of combinatorial bandits was initiated in Cesa-Bianchi and Lugosi (2012). Both papers achieve regret $\text{poly}(d) \cdot \sqrt{T}$, which is the best possible. Combinatorial semi-bandits admit improved dependence on d and other problem parameters, *e.g.,* for IID rewards (*e.g.,* Chen et al., 2013; Kveton et al., 2015c, 2014) and when actions are "slates" of search results (Kale et al., 2010).

Linear bandits. A more general problem of *linear bandits* allows an arbitrary action set $\mathcal{A} \subset [0,1]^d$, as in Chapter 7.3. Introduced in Awerbuch and Kleinberg (2008) and McMahan and Blum (2004), this problem has been studied in a long line of work. In particular, one can achieve $\text{poly}(d) \cdot \sqrt{T}$ regret (Dani et al., 2007), even with high probability against an adaptive adversary (Bartlett et al., 2008), and even via a computationally efficient algorithm if $\mathcal{A}$ is convex (Abernethy et al., 2008; Abernethy and Rakhlin, 2009). A detailed survey of this line of work can be found in Bubeck and Cesa-Bianchi (2012, Chapter 5).

In *stochastic* linear bandits, $\mathbb{E}[\, c_t(a)\,] = v \cdot a$ for each action $a \in \mathcal{A}$ and some fixed, unknown vector v. One way to realize the stochastic version as a special case of the adversarial version is to posit that in each round t, the hidden vector v_t is drawn independently from some fixed (but unknown) distribution.[1] Stochastic linear bandits have bee introduced in Auer (2002) and subsequently studied in (Abe et al., 2003; Dani et al., 2008; Rusmevichientong and Tsitsiklis, 2010) using the paradigm of "optimism under uncertainty" from Chapter 1. Generally, regret bounds admit better dependence on d compared to the adversarial version.

Several notable extensions of stochastic linear bandits have been studied. In *contextual* linear bandits, the action set is provided exogenously before each round, see Sections 8.3 for technical details and Section 8.7 for a literature review. In *generalized* linear bandits (starting from Filippi et al., 2010; Li et al., 2017), $\mathbb{E}[\, c_t(a)\,] = f(v \cdot a)$ for some known function f, building on the *generalized linear model* from statistics. In *sparse* linear bandits (starting from Abbasi-Yadkori et al., 2012; Carpentier and Munos, 2012), the dimension d is very large, and one takes advantage of the sparsity in the hidden vector v.

Online linear optimization. Follow The Perturbed Leader (FTPL) has been proposed in Hannan (1957), in the context of repeated games (as in Chapter 9). The algorithm was rediscovered in the computer science literature by Kalai and Vempala (2005), with an improved analysis.

While FTPL allows an arbitrary action set $\mathcal{A} \subset [0,1]^d$, a vast and beautiful theory is developed for a paradigmatic special case when $\mathcal{A}$ is convex. In particular, *Follow The Regularized Leader* (FTRL) is another generalization of Follow The Leader which chooses a strongly convex regularization function $\mathcal{R}_t : \mathcal{A} \to \mathbb{R}$ at each round t, and minimizes the sum: $a_t = \text{argmin}_{a \in \mathcal{A}} \mathcal{R}_t(a) + \sum_{s=1}^{t} c_s(a)$. The FTRL framework allows for a unified analysis and, depending on the choice of $\mathcal{R}_t$, instantiates to many specific algorithms and their respective guarantees, see the survey (McMahan, 2017). In fact, this machinery extends to convex cost functions, a subject called *online convex optimization*. More background on this subject can be found in books (Shalev-Shwartz, 2012) and (Hazan, 2015). A version of FTRL achieves $\text{poly}(d) \cdot \sqrt{T}$ regret for any convex $\mathcal{A}$ in a computationally efficient manner (Abernethy et al., 2008); in fact, it is a key ingredient in the $\text{poly}(d) \cdot \sqrt{T}$ regret algorithm for linear bandits.

[1] Alternatively, one could realize $c_t(a)$ as $v \cdot a$ plus independent noise ϵ_t. This version can also be represented as a special case of adversarial linear bandits, but in a more complicated way. Essentially, one more dimension is added, such that in this dimension each arm has coefficient 1, and each hidden vector v_t has coefficient ϵ_t.

Lower bounds and optimality. In linear bandits, $\mathrm{poly}(d) \cdot \sqrt{T}$ regret rates are inevitable in the worst case. This holds even under full feedback (Dani et al., 2007), even for stochastic linear bandits with "continuous" action sets (Dani et al., 2008), and even for stochastic combinatorial semi-bandits (*e.g.,* Audibert et al., 2014; Kveton et al., 2015c). In particular, the "price of bandit information", *i.e.,* the penalty in optimal regret bounds compared to the full-feedback version, is quite mild: the dependence on d increases from one polynomial to another. This is in stark contrast with K-armed bandits, where the dependence on K increases from logarithmic to polynomial.

A considerable literature strives to improve dependence on d and/or other structural parameters. This literature is concerned with both upper and lower bounds on regret, across the vast problem space of linear bandits. Some of the key distinctions in this problem space are as follows: feedback model (full, semi-bandit, or bandit); "type" of the adversary (stochastic, oblivious, or adaptive); structure of the action space (*e.g.,* arbitrary, convex, or combinatorial);[2] structure of the hidden vector (*e.g.,* sparsity). A detailed discussion of the "landscape" of optimal regret bounds is beyond our scope.

Combinatorial semi-bandits beyond additive costs. Several versions of combinatorial semi-bandits allow the atoms' costs to depend on the other atoms chosen in the same round. In much of this work, when a subset S of atoms is chosen by the algorithm, at most one atom $a \in S$ is then selected by "nature", and all other atoms receive reward/cost 0. In *multinomial-logit (MNL) bandits*, atoms are chosen probabilistically according to the MNL model, a popular choice model from statistics. Essentially, each atom a is associated with a fixed (but unknown) number v_a, and is chosen with probability $\mathbf{1}_{\{\,a\in S\,\}} \cdot v_a/(1+\sum_{a'\in S} v_{a'})$. MNL bandits are typically studied as a model for *dynamic assortment*, where S is the assortment of products offered for sale, *e.g.,* in (Caro and Gallien, 2007; Sauré and Zeevi, 2013; Rusmevichientong et al., 2010; Agrawal et al., 2019). Simchowitz et al. (2016) consider a more general choice model from theoretical economics, called *random utility model*. Here each atom $a \in S$ is assigned "utility" equal to v_a plus independent noise, and the atom with a largest utility is chosen.

A *cascade feedback* model posits that actions correspond to rankings of items such as search results, a user scrolls down to the first "relevant" item, clicks on it, and leaves. The reward is 1 if and only if some item is relevant (and therefore the user is satisfied). The study of bandits in this model has been initiated in (Radlinski et al., 2008). They allow relevance to be arbitrarily correlated across items, depending on the user's interests. In particular, the marginal importance of a given item may depend on other items in the ranking, and it may be advantageous to make the list more diverse. Streeter and Golovin (2008); Golovin et al. (2009) study a more general version with submodular rewards, and Slivkins et al. (2013) consider an extension to Lipschitz bandits. In all this work, one only achieves additive regret relative to the $(1-1/e)$-th fraction of the optimal reward. Kveton et al. (2015a,b); Zong et al. (2016) achieve much stronger guarantees, without the multiplicative approximation, assuming that relevance is independent across the items.[3]

In some versions of combinatorial semi-bandits, atoms are assigned rewards in each round independently of the algorithm's choices, but the "aggregate outcome" associated with a particular subset S of atoms chosen by the algorithm can be more general compared to the standard version. For example, Chen et al. (2016) allows the aggregate reward of S to be a function of the per-atom rewards in S, under some mild assumptions. Chen et al. (2013) allows other atoms to be "triggered", *i.e.,* included into S, according to some distribution determined by S.

[2] One could also posit a more refined structure, *e.g.,* assume a particular family of combinatorial subsets, such as {all subsets of a given size}, or {all paths in a graph}. Abernethy et al. (2008); Abernethy and Rakhlin (2009) express the "niceness" of a convex action set via the existence of an intricate object from convex optimization called "self-concordant barrier function".

[3] Kveton et al. (2015b) study a version of cascade feedback in which the aggregate reward for a ranked list of items is 1 if all items are "good" and 0 otherwise, and the feedback returns the first item that is "bad". The main motivation is network routing: an action corresponds to a path in the network, and it may be the case that only the first faulty edge is revealed to the algorithm.

Chapter 8

Contextual Bandits

In *contextual bandits*, rewards in each round depend on a *context*, which is observed by the algorithm prior to making a decision. We cover the basics of three prominent versions of contextual bandits: with a Lipschitz assumption, with a linearity assumption, and with a fixed policy class. We also touch upon offline learning from contextual bandit data. Finally, we discuss challenges that arise in large-scale applications, and a system design that address these challenges in practice.

Prerequisites: Chapters 1, 2, 6 (for background/perspective only), Chapter 4 (for Section 8.2).

We consider a generalization called *contextual bandits*, defined as follows:

Problem protocol: Contextual bandits

For each round $t \in [T]$:

1. algorithm observes a "context" x_t,
2. algorithm picks an arm a_t,
3. reward $r_t \in [0, 1]$ is realized.

The reward r_t in each round t depends both on the context x_t and the chosen action a_t. We make the IID assumption: reward r_t is drawn independently from some distribution parameterized by the (x_t, a_t) pair, but same for all rounds t. The expected reward of action a given context x is denoted $\mu(a|x)$. This setting allows a limited amount of "change over time", but this change is completely "explained" by the observable contexts. We assume contexts $x_1, x_2, \ldots$ are chosen by an oblivious adversary.

Several variants of this setting have been studied in the literature. We discuss three prominent variants in this chapter: with a Lipschitz assumption, with a linearity assumption, and with a fixed policy class.

Motivation. The main motivation is that a user with a known "user profile" arrives in each round, and the context is the user profile. The algorithm can personalize the user's experience. Natural application scenarios include choosing which news articles to showcase, which ads to display, which products to recommend, or which webpage layouts to use. Rewards in these applications are often determined by user clicks, possibly in conjunction with other observable signals that correlate with revenue and/or user satisfaction. Naturally, rewards for the same action may be different for different users.

Contexts can include other things apart from (and instead of) user profiles. First, contexts can include known features of the environment, such as day of the week, time of the day, season (*e.g.,* Summer, pre-Christmas shopping season), or proximity to a major event (*e.g.,* Olympics, elections). Second, some actions may be unavailable in a given round and/or for a given user, and a context can include the set of feasible actions. Third, actions can come with features of their own, and it may be convenient to include this information into the context, esp. if these features can change over time.

Regret relative to best response. For ease of exposition, we assume a fixed and known time horizon T. The set of actions and the set of all contexts are $\mathcal{A}$ and $\mathcal{X}$, resp.; $K = |\mathcal{A}|$ is the number of actions.

The (total) reward of an algorithm ALG is $\texttt{REW}(\texttt{ALG}) = \sum_{t=1}^{T} r_t$, so that the expected reward is

$$\mathbb{E}[\texttt{REW}(\texttt{ALG})] = \sum_{t=1}^{T} \mathbb{E}\left[\, \mu(a_t|x_t) \,\right].$$

A natural benchmark is the best-response policy, $\pi^*(x) = \max_{a\in\mathcal{A}} \mu(a|x)$. Then regret is defined as

$$R(T) = \texttt{REW}(\pi^*) - \texttt{REW}(\texttt{ALG}). \tag{8.1}$$

8.1 Warm-up: small number of contexts

One straightforward approach for contextual bandits is to apply a known bandit algorithm ALG such as UCB1: namely, run a separate copy of this algorithm for each context.

Initialization: For each context x, create an instance $\texttt{ALG}_x$ of algorithm ALG
for *each round* t **do**
  invoke algorithm $\texttt{ALG}_x$ with $x = x_t$
  "play" action a_t chosen by $\texttt{ALG}_x$, return reward r_t to $\texttt{ALG}_x$.
end

Algorithm 8.1: Contextual bandit algorithm for a small number of contexts

Let n_x be the number of rounds in which context x arrives. Regret accumulated in such rounds, denoted $R_x(T)$, satisfies $\mathbb{E}[R_x(T)] = O(\sqrt{K n_x \ln T})$. The total regret (from all contexts) is

$$\mathbb{E}[R(T)] = \textstyle\sum_{x\in\mathcal{X}} \mathbb{E}[R_x(T)] = \sum_{x\in\mathcal{X}} O(\sqrt{K n_x \ln T}) \leq O(\sqrt{KT\,|\mathcal{X}|\,\ln T}).$$

Theorem 8.1. *Algorithm 8.1 has regret* $\mathbb{E}[R(T)] = O(\sqrt{KT\,|\mathcal{X}|\,\ln T})$*, provided that the bandit algorithm* ALG *has regret* $\mathbb{E}[R_{\texttt{ALG}}(T)] = O(\sqrt{KT \log T})$.

Remark 8.2. The square-root dependence on $|\mathcal{X}|$ is slightly non-trivial, because a completely naive solution would give linear dependence. However, this regret bound is still very high if $|\mathcal{X}|$ is large, *e.g.,* if contexts are feature vectors with a large number of features. To handle contextual bandits with a large $|\mathcal{X}|$, we either assume some structure (as in Sections 8.2 and 8.3, or change the objective (as in Section 8.4).

8.2 Lipshitz contextual bandits

Let us consider contextual bandits with Lipschitz-continuity, as a simple end-to-end example of how structure allows to handle contextual bandits with a large number of contexts. We assume that contexts map into the $[0,1]$ interval (*i.e.,* $\mathcal{X} \subset [0,1]$) so that the expected rewards are Lipschitz with respect to the contexts:

$$|\mu(a|x) - \mu(a|x')| \leq L \cdot |x - x'| \quad \text{for any arm } a \in \mathcal{A} \text{ and any contexts } x, x' \in \mathcal{X}, \tag{8.2}$$

where L is the Lipschitz constant which is known to the algorithm.

One simple solution for this problem is given by uniform discretization of the context space. The approach is very similar to what we've seen for Lipschitz bandits and dynamic pricing; however, we need to be a little careful with some details: particularly, watch out for "discretized best response". Let S be the ϵ-*uniform mesh* on $[0,1]$, *i.e.,* the set of all points in $[0,1]$ that are integer multiples of ϵ. We take $\epsilon = 1/(d-1)$, where the integer d is the number of points in S, to be adjusted later in the analysis.

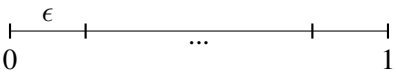


Figure 8.1: Discretization of the context space

We will use the contextual bandit algorithm from Section 8.1, applied to context space S; denote this algorithm as $\texttt{ALG}_S$. Let $f_S(x)$ be a mapping from context x to the closest point in S:

$$f_S(x) = \min(\operatorname*{argmin}_{x' \in S} |x - x'|)$$

(the $\min$ is added just to break ties). The overall algorithm proceeds as follows:

In each round t, "pre-process" the context x_t by replacing it with $f_S(x_t)$, and call $\texttt{ALG}_S$. (8.3)

The regret bound will have two summands: regret bound for $\texttt{ALG}_S$ and (a suitable notion of) discretization error. Formally, let us define the "discretized best response" $\pi^*_S : \mathcal{X} \to \mathcal{A}$:

$$\pi^*_S(x) = \pi^*(f_S(x)) \quad \text{for each context } x \in \mathcal{X}.$$

Then regret of $\texttt{ALG}_S$ and discretization error are defined as, resp.,

$$R_S(T) = \texttt{REW}(\pi^*_S) - \texttt{REW}(\texttt{ALG}_S)$$
$$\texttt{DE}(S) = \texttt{REW}(\pi^*) - \texttt{REW}(\pi^*_S).$$

It follows that the "overall" regret is the sum $R(T) = R_S(T) + \texttt{DE}(S)$, as claimed. We have $\mathbb{E}[R_S(T)] = O(\sqrt{KT|S|\ln T})$ from Lemma 8.1, so it remains to upper-bound the discretization error and adjust the discretization step ϵ.

Claim 8.3. $\mathbb{E}[\texttt{DE}(S)] \leq \epsilon LT$.

Proof. For each round t and the respective context $x = x_t$,

$$\begin{aligned} \mu(\pi^*_S(x) \mid f_S(x)) &\geq \mu(\pi^*(x) \mid f_S(x)) && \textit{(by optimality of } \pi^*_S\textit{)} \\ &\geq \mu(\pi^*(x) \mid x) - \epsilon L && \textit{(by Lipschitzness).} \end{aligned}$$

Summing this up over all rounds t, we obtain

$$\mathbb{E}[\texttt{REW}(\pi^*_S)] \geq \texttt{REW}[\pi^*] - \epsilon LT. \qquad \square$$

Thus, regret is

$$\mathbb{E}[R(T)] \leq \epsilon LT + O(\sqrt{\tfrac{1}{\epsilon} KT \ln T}) = O(T^{2/3}(LK \ln T)^{1/3}).$$

where for the last inequality we optimized the choice of ϵ.

Theorem 8.4. *Consider the Lipschitz contextual bandits problem with contexts in* $[0,1]$. *The uniform discretization algorithm (8.3) yields regret* $\mathbb{E}[R(T)] = O(T^{2/3}(LK \ln T)^{1/3})$.

An astute reader would notice a similarity with the uniform discretization result in Theorem 4.1. In fact, these two results admit a common generalization in which the Lipschitz condition applies to both contexts and arms, and arbitrary metrics are allowed. Specifically, the Lipschitz condition is now

$$|\mu(a|x) - \mu(a'|x')| \leq D_{\mathcal{X}}(x, x') + D_{\mathcal{A}}(a, a') \quad \text{for any arms } a, a' \text{ and contexts } x, x', \tag{8.4}$$

where $D_{\mathcal{X}}, D_{\mathcal{A}}$ are arbitrary metrics on contexts and arms, respectively, that are known to the algorithm. This generalization is fleshed out in Exercise 8.1.

8.3 Linear contextual bandits (no proofs)

We introduce the setting of *linear* contextual bandits, and define an algorithm for this setting called LinUCB.

Let us recap the setting of linear bandits from Chapter 7, specialized to stochastic bandits. One natural formulation is that each arm a is characterized by a feature vector $x_a \in [0,1]^d$, and the expected reward is linear in this vector: $\mu(a) = x_a \cdot \theta$, for some fixed but unknown vector $\theta \in [0,1]^d$. The tuple

$$x = \Big(x_a \in \{0,1\}^d : a \in \mathcal{A} \Big) \tag{8.5}$$

can be interpreted as a *static context*, *i.e.,* a context that does not change from one round to another.

In linear *contextual* bandits, contexts are of the form (8.5), and the expected rewards are linear:

$$\mu(a|x) = x_a \cdot \theta_a \quad \text{for all arms } a \text{ and contexts } x, \tag{8.6}$$

for some fixed but unknown vector $\theta = (\theta_a \in \mathcal{R}^d : a \in \mathcal{A})$. Note that we also generalize the unknown vector θ so that θ_a can depend on the arm a. Let Θ be the set of all feasible θ vectors, known to the algorithm.

This problem can be solved by a version of the UCB technique. Instead of constructing confidence bounds on the mean rewards of each arm, we do that for the θ vector. Namely, in each round t we construct a "confidence region" $C_t \subset \Theta$ such that $\theta \in C_t$ with high probability. Then we use C_t to construct an UCB on the mean reward of each arm given context x_t, and play an arm with the highest UCB. This algorithm, called LinUCB, is summarized in Algorithm 8.2.

for *each round* $t = 1, 2, \ldots$ **do**
  Form a confidence region $C_t \subset \Theta$ $\quad$ {*i.e.,* $\theta \in C_t$ with high probability}
  Observe context $x = x_t$ of the form (8.5)
  For each arm a, compute $\texttt{UCB}_t(a|x_t) = \sup_{\theta \in C_t} x_a \cdot \theta_a$
  Pick an arm a which maximizes $\texttt{UCB}_t(a|x_t)$.
end

Algorithm 8.2: LinUCB: UCB-based algorithm for linear contextual bandits

Suitably specified versions of LinUCB allow for rigorous regret bounds, and work well in experiments. The best known worst-case regret bound is $\mathbb{E}[R(T)] = \tilde{O}(d\sqrt{T})$, and there is a close lower bound $\mathbb{E}[R(T)] \geq \Omega(\sqrt{dT})$. The gap-dependent regret bound from Chapter 1 carries over, too: LinUCB enjoys $(d^2/\Delta) \cdot \text{polylog}(T)$ regret for problem instances with minimal gap at least Δ. Interestingly, the algorithm is known to work well in practice even for scenarios without linearity.

To completely specify the algorithm, one needs to specify what the confidence region is, and how to compute the UCBs; this is somewhat subtle, and there are multiple ways to do this. The technicalities of specification and analysis of LinUCB are beyond our scope.

8.4 Contextual bandits with a policy class

We now consider a more general contextual bandit problem where we do not make any assumptions on the mean rewards. Instead, we make the problem tractable by making restricting the benchmark in the definition of regret (*i.e.,* the first term in Eq. (8.1)). Specifically, we define a *policy* as a mapping from contexts to actions, and posit a known class of policies Π. Informally, algorithms only need to compete with the best policy in π. A big benefit of this approach is that it allows to make a clear connection to the "traditional" machine learning, and re-use some of its powerful tools.

We assume that contexts arrive as independent samples from some fixed distribution $\mathcal{D}$ over contexts. The expected reward of a given policy π is then well-defined:

$$\mu(\pi) = \mathop{\mathbb{E}}_{x\in\mathcal{D}} \left[\mu(\pi(x)) \mid x \right]. \tag{8.7}$$

This quantity, also known as *policy value*, gives a concrete way to compare policies to one another. The appropriate notion of regret, for an algorithm ALG, is relative to the best policy in a given policy class Π:

$$R_\Pi(T) = T \max_{\pi\in\Pi} \mu(\pi) - \texttt{REW}(\texttt{ALG}). \tag{8.8}$$

Note that the definition (8.1) can be seen a special case when Π is the class of all policies. For ease of presentation, we assume $K < \infty$ actions throughout; the set of actions is denoted $[K]$.

Remark 8.5 (Some examples). A policy can be based on a *score predictor*: given a context x, it assigns a numerical score $\nu(a|x)$ to each action a. Such score can represent an estimated expected reward of this action, or some other quality measure. The policy simply picks an action with a highest predicted reward. For a concrete example, if contexts and actions are represented as known feature vectors in $\mathbb{R}^d$, a *linear* score predictor is $\nu(a|x) = aMx$, for some fixed $d \times d$ weight matrix M.

A policy can also be based on a *decision tree*, a flowchart in which each internal node corresponds to a "test" on some attribute(s) of the context (*e.g.,* is the user male of female), branches correspond to the possible outcomes of that test, and each terminal node is associated with a particular action. Execution starts from the "root node" of the flowchart and follows the branches until a terminal node is reached.

Preliminary result. This problem can be solved with algorithm Exp4 from Chapter 6, with policies $\pi \in \Pi$ as "experts". Specializing Theorem 6.11 to the setting of contextual bandits, we obtain:

Theorem 8.6. *Consider contextual bandits with policy class Π. Algorithm* Exp4 *with expert set Π yields regret $\mathbb{E}[R_\Pi(T)] = O(\sqrt{KT \log |\Pi|})$. However, the running time per round is linear in $|\Pi|$.*

This is a very powerful regret bound: it works for an arbitrary policy class Π, and the logarithmic dependence on $|\Pi|$ makes it tractable even if the number of possible contexts is huge. Indeed, while there are $K^{|\mathcal{X}|}$ possible policies, for many important special cases $|\Pi| = K^c$, where c depends on the problem parameters but not on $|\mathcal{X}|$. This regret bound is essentially the best possible. Specifically, there is a nearly matching lower bound (similar to (6.1)), which holds for any given triple of parameters $K, T, |\Pi|$:

$$\mathbb{E}[R(T)] \geq \min\left(T,\ \Omega\left(\sqrt{KT \log(|\Pi|)/\log(K)}\right)\right). \tag{8.9}$$

However, the running time of Exp4 scales as $|\Pi|$ rather than $\log |\Pi|$, which makes the algorithm prohibitively slow in practice. In what follows, we achieve similar regret rates, but with faster algorithms.

Connection to a classification problem. We make a connection to a well-studied classification problem in "traditional" machine learning. This connection leads to faster algorithms, and is probably the main motivation for the setting of contextual bandits with policy sets.

To build up the intuition, let us consider the full-feedback version of contextual bandits, in which the rewards are observed for all arms.

Problem protocol: Contextual bandits with full feedback

For each round $t = 1, 2, \ldots$:
1. algorithm observes a "context" x_t,
2. algorithm picks an arm a_t,
3. rewards $\tilde{r}_t(a) \geq 0$ are observed for all arms $a \in \mathcal{A}$.

In fact, let us make the problem even easier. Suppose we already have N data points of the form $(x_t; \tilde{r}_t(a) : a \in \mathcal{A})$. What is the "best-in-hindsight" policy for this dataset? More precisely, what is a policy $\pi \in \Pi$ with a largest *realized policy value*

$$\tilde{r}(\pi) = \frac{1}{N} \sum_{t=1}^{N} \tilde{r}_t(\pi(x_i)). \tag{8.10}$$

This happens to be a well-studied problem called "cost-sensitive multi-class classification":

Problem: Cost-sensitive multi-class classification for policy class Π

Given: data points $(x_t; \tilde{r}_t(a) : a \in \mathcal{A})$, $t \in [N]$.
Find: policy $\pi \in \Pi$ with a largest realized policy value (8.10).

In the terminology of classification problems, each context x_t is an "example", and the arms correspond to different possible "labels" for this example. Each label has an associated reward/cost. We obtain a standard binary classification problem if for each data point t, there is one "correct label" with reward 1, and rewards for all other labels are 0. The practical motivation for finding the "best-in-hindsight" policy is that it is likely to be a good policy for future context arrivals. In particular, such policy is near-optimal under the IID assumption, see Exercise 8.2 for a precise formulation.

An algorithm for this problem will henceforth be called a *classification oracle* for policy class Π. While the exact optimization problem is NP-hard for many natural policy classes, practically efficient algorithms exist for several important policy classes such as linear classifiers, decision trees and neural nets.

A very productive approach for designing contextual bandit algorithms uses a classification oracle as a subroutine. The running time is then expressed in terms of the number of oracle calls, the implicit assumption being that each oracle call is reasonably fast. Crucially, algorithms can use any available classification oracle; then the relevant policy class Π is simply the policy class that the oracle optimizes over.

A simple oracle-based algorithm. Consider a simple explore-then-exploit algorithm that builds on a classification oracle. First, we explore uniformly for the first N rounds, where N is a parameter. Each round t of exploration gives a data point $(x_t, \tilde{r}_t(a) \in \mathcal{A})$ for the classification oracle, where the "fake rewards" $\tilde{r}_t(\cdot)$ are given by inverse propensity scoring:

$$\tilde{r}_t(a) = \begin{cases} r_t K & \text{if } a = a_t \\ 0, & \text{otherwise.} \end{cases} \tag{8.11}$$

We call the classification oracle and use the returned policy in the remaining rounds; see Algorithm 8.3.

Parameter: exploration duration N, classification oracle $\mathcal{O}$

1. Explore uniformly for the first N rounds: in each round, pick an arm u.a.r.
2. Call the classification oracle with data points $(x_t, \tilde{r}_t(a) \in \mathcal{A})$, $t \in [N]$ as per Eq. (8.11).
3. Exploitation: in each subsequent round, use the policy π_0 returned by the oracle.

Algorithm 8.3: Explore-then-exploit with a classification oracle

Remark 8.7. Algorithm 8.3 is modular in two ways: it can take an arbitrary classification oracle, and it can use any other unbiased estimator instead of Eq. (8.11). In particular, the proof below only uses the fact that Eq. (8.11) is an unbiased estimator with $\tilde{r}_t(a) \leq K$.

For a simple analysis, assume that the rewards are in $[0, 1]$ and that the oracle is *exact*, in the sense that it returns a policy $\pi \in \Pi$ that exactly maximizes $\mu(\pi)$.

Theorem 8.8. *Let $\mathcal{O}$ be an exact classification oracle for some policy class Π. Algorithm 8.3 parameterized with oracle $\mathcal{O}$ and $N = T^{2/3}(K \log(|\Pi|T))^{\frac{1}{3}}$ has regret*

$$\mathbb{E}[R_\Pi(T)] = O(T^{2/3})(K \log(|\Pi|T))^{\frac{1}{3}}.$$

This regret bound has two key features: logarithmic dependence on $|\Pi|$ and $\tilde{O}(T^{2/3})$ dependence on T.

Proof. Let us consider an arbitrary N for now. For a given policy π, we estimate its expected reward $\mu(\pi)$ using the realized policy value from (8.10), where the action costs $\tilde{r}_t(\cdot)$ are from (8.11). Let us prove that $\tilde{r}(\pi)$ is an unbiased estimate for $\mu(\pi)$:

$$\begin{aligned}
\mathbb{E}[\tilde{r}_t(a) \mid x_t] &= \mu(a \mid x_t) && \text{(for each action } a \in \mathcal{A}\text{)} \\
\mathbb{E}[\tilde{r}_t(\pi(x_t)) \mid x_t] &= \mu(\pi(x) \mid x_t) && \text{(plug in } a = \pi(x_t)\text{)} \\
\mathop{\mathbb{E}}_{x_t \sim D}[\tilde{r}_t(\pi(x_t))] &= \mathop{\mathbb{E}}_{x_t \sim D}[\mu(\pi(x_t)) \mid x_t] && \text{(take expectation over both } \tilde{r}_t \text{ and } x_t\text{)} \\
&= \mu(\pi), &&
\end{aligned}$$

which implies $\mathbb{E}[\tilde{r}(\pi)] = \mu(\pi)$, as claimed. Now, let us use this estimate to set up a "clean event":

$$\{ \mid \tilde{r}(\pi) - \mu(\pi) \mid \leq \texttt{conf}(N) \text{ for all policies } \pi \in \Pi \},$$

where the confidence term is $\texttt{conf}(N) = O(\sqrt{\frac{K \log(T|\Pi|)}{N}})$. We can prove that the clean event does indeed happen with probability at least $1 - \frac{1}{T}$, say, as an easy application of Chernoff Bounds. For intuition, the K is present in the confidence radius is because the "fake rewards" $\tilde{r}_t(\cdot)$ could be as large as K. The $|\Pi|$ is there (inside the $\log$) because we take a Union Bound across all policies. And the T is there because we need the "error probability" to be on the order of $\frac{1}{T}$.

Let $\pi^* = \pi^*_\Pi$ be an optimal policy. Since we have an exact classification oracle, $\tilde{r}(\pi_0)$ is maximal among all policies $\pi \in \Pi$. In particular, $\tilde{r}(\pi_0) \geq \tilde{r}(\pi^*)$. If the clean event holds, then

$$\mu(\pi^\star) - \mu(\pi_0) \leq 2\,\texttt{conf}(N).$$

Thus, each round in exploitation contributes at most $\texttt{conf}$ to expected regret. And each round of exploration contributes at most 1. It follows that $\mathbb{E}[R_\Pi(T)] \leq N + 2T\,\texttt{conf}(N)$. Choosing N so that $N = O(T\,\texttt{conf}(N))$, we obtain $N = T^{2/3}(K \log(|\Pi|T))^{\frac{1}{3}}$ and $\mathbb{E}[R_\Pi(T)] = O(N)$. □

Remark 8.9. If the oracle is only approximate – say, it returns a policy $\pi_0 \in \Pi$ which optimizes $c(\cdot)$ up to an additive factor of ϵ – it is easy to see that expected regret increases by an additive factor of ϵT. In practice, there may be a tradeoff between the approximation guarantee ϵ and the running time of the oracle.

Remark 8.10. A near-optimal regret bound can in fact be achieved with an *oracle-efficient* algorithm: one that makes only a small number of oracle calls. Specifically, one can achieve regret $O(\sqrt{KT \log(T|\Pi|)})$ with only $\tilde{O}(\sqrt{KT/\log|\Pi|})$ oracle calls across all T rounds. This sophisticated result can be found in (Agarwal et al., 2014). Its exposition is beyond the scope of this book.

8.5 Learning from contextual bandit data

Data collected by a contextual bandit algorithm can be analyzed "offline", separately from running the algorithm. Typical tasks are estimating the value of a given policy (*policy evaluation*), and learning a policy that performs best on the dataset (*policy training*). While these tasks are formulated in the terminology from Section 8.4, they are meaningful for all settings discussed in this chapter.

Let us make things more precise. Assume that a contextual bandit algorithm has been running for T rounds according to the following extended protocol:

Problem protocol: Contextual bandit data collection

For each round $t \in [N]$:

1. algorithm observes a "context" x_t,
2. algorithm picks a sampling distribution p_t over arms,
3. arm a_t is drawn independently from distribution p_t,
4. rewards $\tilde{r}_t(a)$ are realized for all arms $a \in \mathcal{A}$,
5. reward $r_t = \tilde{r}_t(a_t) \in [0,1]$ is recorded.

Thus, we have N data points of the form (x_t, p_t, a_t, r_t). The sampling probabilities p_t are essential to form the IPS estimates, as explained below. It is particularly important to record the sampling probability for the chosen action, $p_t(a_t)$. Policy evaluation and training are defined as follows:

Problem: Policy evaluation and training

Input: data points (x_t, p_t, a_t, r_t), $t \in [N]$.
Policy evaluation: estimate policy value $\mu(\pi)$ for a given policy π.
Policy training: find policy $\pi \in \Pi$ that maximizes $\mu(\pi)$ over a given policy class Π.

Inverse propensity scoring. Policy evaluation can be addressed via inverse propensity scoring (IPS), like in Algorithm 8.3. This approach is simple and does not rely on a particular model of the rewards such as linearity or Lipschitzness. We estimate the value of each policy π as follows:

$$\texttt{IPS}(\pi) = \sum_{t \in [N]:\ \pi(x_t) = a_t} \frac{r_t}{p_t(a_t)}. \tag{8.12}$$

Just as in the proof of Theorem 8.8, one can show that the IPS estimator is unbiased and accurate; accuracy holds with probability as long as the sampling probabilities are large enough.

Lemma 8.11. *The IPS estimator is unbiased:* $\mathbb{E}\left[\,\texttt{IPS}(\pi)\,\right] = \mu(\pi)$ *for each policy* π*. Moreover, the IPS estimator is accurate with high probability: for each* $\delta > 0$*, with probability at least* $1-\delta$

$$|\texttt{IPS}(\pi) - \mu(\pi)| \leq O\left(\sqrt{\tfrac{1}{p_0}\,\log(\tfrac{1}{\delta})/N}\right), \quad \textit{where } p_0 = \min_{t,a} p_t(a). \tag{8.13}$$

Remark 8.12. How many data points do we need to evaluate M policies simultaneously? To make this question more precise, suppose we have some fixed parameters $\epsilon, \delta > 0$ in mind, and want to ensure that

$$\Pr\left[\,|\texttt{IPS}(\pi) - \mu(\pi)| \leq \epsilon \quad \text{for each policy } \pi\,\right] > 1-\delta.$$

How large should N be, as a function of M and the parameters? Taking a union bound over (8.13), we see that it suffices to take

$$N \sim \frac{\sqrt{\log(M/\delta)}}{p_0 \cdot \epsilon^2}.$$

The logarithmic dependence on M is due to the fact that each data point t can be reused to evaluate many policies, namely all policies π with $\pi(x_t) = a_t$.

We can similarly compare $\texttt{IPS}(\pi)$ with the *realized* policy value $\tilde{r}(\pi)$, as defined in (8.10). This comparison does not rely on IID rewards: it applies whenever rewards are chosen by an oblivious adversary.

Lemma 8.13. *Assume rewards* $\tilde{r}_t(\cdot)$ *are chosen by a deterministic oblivious adversary. Then we have* $\mathbb{E}[\texttt{IPS}(\pi)] = \tilde{r}(\pi)$ *for each policy* π*. Moreover, for each* $\delta > 0$*, with probability at least* $1-\delta$ *we have:*

$$|\texttt{IPS}(\pi) - \tilde{r}(\pi)| \leq O\left(\sqrt{\tfrac{1}{p_0}\,\log(\tfrac{1}{\delta})/N}\right), \quad \textit{where } p_0 = \min_{t,a} p_t(a). \tag{8.14}$$

This lemma implies Lemma 8.11. It easily follows from a concentration inequality, see Exercise 8.3.

Policy training can be implemented similarly: by maximizing $\texttt{IPS}(\pi)$ over a given policy class Π. A maximizer π_0 can be found by a single call to the classification oracle for Π: indeed, call the oracle with "fake rewards" defined by estimates: $\rho_t(a) = \mathbf{1}_{\{\,a=a_t\,\}}\frac{r_t}{p_t(a_t)}$ for each arm a and each round t. The performance guarantee follows from Lemma 8.13:

Corollary 8.14. *Consider the setting in Lemma 8.13. Fix policy class* Π *and let* $\pi_0 \in \operatorname{argmax}_{\pi\in\Pi} \texttt{IPS}(\pi)$*. Then for each* $\delta > 0$*, with probability at least* $\delta > 0$ *we have*

$$\max_{\pi\in\Pi} \tilde{r}(\pi) - \tilde{r}(\pi_0) \leq O\left(\sqrt{\tfrac{1}{p_0}\,\log(\tfrac{|\Pi|}{\delta})/N}\right). \tag{8.15}$$

Model-dependent approaches. One could estimate the mean rewards $\mu(a|x)$ directly, *e.g.,* with linear regression, and then use the resulting estimates $\hat{\mu}(a|x)$ to estimate policy values:

$$\hat{\mu}(\pi) = \mathop{\mathbb{E}}_{x\sim\mathcal{D}}[\hat{\mu}(\pi(x) \mid x)].$$

Such approaches are typically motivated by some model of rewards, *e.g.,* linear regression is motivated by the linear model. Their validity depends on how well the data satisfies the assumptions in the model.

For a concrete example, linear regression based on the model in Section 8.3 constructs estimates $\hat{\theta}_a$ for latent vector θ_a, for each arm a. Then rewards are estimated as $\hat{\mu}(a|x) = x \cdot \theta_a$.

Model-dependent reward estimates can naturally be used for policy optimization:

$$\pi(x) = \operatorname*{argmax}_{a\in\mathcal{A}} \hat{\mu}(a|x).$$

Such policy can be good even if the underlying model is not. No matter how a policy is derived, it can be evaluated in a model-independent way using the IPS methodology described above.

8.6 Contextual bandits in practice: challenges and a system design

Implementing contextual bandit algorithms for large-scale applications runs into a number of engineering challenges and necessitates a design of a *system* along with the algorithms. We present a system for contextual bandits, called the Decision Service, building on the machinery from the previous two sections.

The key insight is the distinction between the *front-end* of the system, which directly interacts with users and needs to be extremely fast, and the *back-end*, which can do more powerful data processing at slower time scales. *Policy execution*, computing the action given the context, must happen in the front-end, along with data collection (*logging*). Policy evaluation and training usually happen in the back-end. When a better policy is trained, it can be deployed into the front-end. This insight leads to a particular methodology, organized as a loop in Figure 8.2. Let us discuss the four components of this loop in more detail.

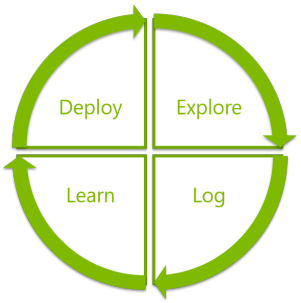


Figure 8.2: The learning loop.

Exploration Actions are chosen by the *exploration policy*: a fixed policy that runs in the front-end and combines exploration and exploitation. Conceptually, it takes one or more *default policies* as subroutines, and adds some exploration on top. A default policy is usually known to be fairly good; it could be a policy already deployed in the system and/or trained by a machine learning algorithm. One basic exploration policy is ϵ-*greedy*: choose an action uniformly at random with probability ϵ ("exploration branch"), and execute the default policy with the remaining probability ("exploitation branch"). If several default policies are given, the exploitation branch chooses uniformly at random among them; this provides an additional layer of exploration, as the default policies may disagree with one another.

If a default policy is based on a score predictor $\nu(a|x)$, as in Example 8.5, an exploration policy can give preference to actions with higher scores. One such exploration policy, known as *SoftMax*, assigns to each action a the probability proportional to $e^{\tau \cdot \nu(a|x)}$, where τ is the exploration parameter. Note that $\tau = 0$ corresponds to the uniform action selection, and increasing the τ favors actions with higher scores. Generically, exploration policy is characterized by its type (*e.g.,* ϵ-greedy or SoftMax), exploration parameters (such as the ϵ or the τ), and the default policy/policies.

Logging Logging runs in the front-end, and records the "data points" defined in Section 8.5. Each logged data point includes (x_t, a_t, r_t) – the context, the chosen arm, and the reward – and also $p_t(a_t)$, the probability of the chosen action. Additional information may be included to help with debugging and/or machine learning, *e.g.,* time stamp, current exploration policy, or full sampling distribution p_t.

In a typical high-throughput application such as a website, the reward (*e.g.,* a click) is observed long after the action is chosen – much longer than a front-end server can afford to "remember" a given user. Accordingly, the reward is logged separately. The *decision tuple*, comprising the context, the

chosen action, and sampling probabilities, is recorded by the front-end when the action is chosen. The reward is logged after it is observed, probably via a very different mechanism, and joined with the decision tuple in the back-end. To enable this join, the front-end server generates a unique "tuple ID" which is included in the decision tuple and passed along to be logged with the reward as the *outcome tuple*.

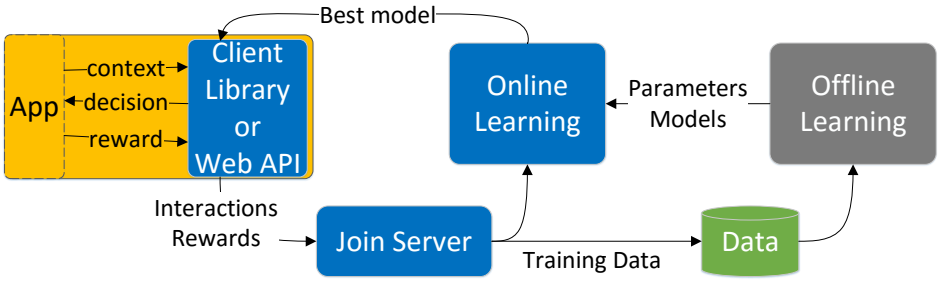
Logging often goes wrong in practice. The sampling probabilities p_t may be recorded incorrectly, or accidentally included as features. Features may be stored as references to a database which is updated over time (so the feature values are no longer the ones observed by the exploration policy). When optimizing an intermediate step in a complex system, the action chosen initially might be overridden by business logic, and the recorded action might incorrectly be this final action rather than the initially chosen action. Finally, rewards may be lost or incorrectly joined to the decision tuples.

Learning Policy training, discussed in Section 8.5, happens in the back-end, on the logged data. The main goal is to learn a better "default policy", and perhaps also better exploration parameters. The policy training algorithm should be *online*, allowing fast updates when new data points are received, so as to enable rapid iterations of the learning loop in Figure 8.2.

Offline learning uses logged data to simulate experiments on live users. Policy evaluation, for example, simulates deploying a given policy. One can also experiment with alternative exploration policies or parameterizations thereof. Further, one can try out other algorithms for policy training, such as those that need more computation, use different hyper-parameters, are based on estimators other than IPS, or lead to different policy classes. The algorithms being tested do not need to be approved for a live deployment (which may be a big hurdle), or implemented at a sufficient performance and reliability level for the said deployment (which tends to be very expensive).

Policy deployment New default policies and/or exploration parameters are deployed into the exploration policy, thereby completing the learning loop in Figure 8.2. As a safeguard, one can use human oversight and/or policy evaluation to compare the new policy with others before deploying it.[1] The deployment process should be automatic and frequent. The frequency of deployments depends on the delays in data collection and policy training, and on the need of human oversight. Also, some applications require a new default policy to improve over the old one by a statistically significant margin, which may require waiting for more data points.

The methodology described above leads to the following system design:



[1] A standard consideration in machine learning applies: a policy should not be evaluated on the training data. Instead, a separate dataset should be set aside for policy evaluation.

The front-end interacts with the application via a provided software library ("Client Library") or via an Internet-based protocol ("Web API"). The Client Library implements exploration and logging, and the Web API is an alternative interface to the said library. Decision tuples and outcome tuples are joined by the Join Server, and then fed into a policy training algorithm (the "Online Learning" box). Offline Learning is implemented as a separate loop which can operate at much slower frequencies.

Modularity of the design is essential, so as to integrate easily with an application's existing infrastructure. In particular, the components have well-defined interfaces, and admit multiple consistent implementations. This avoids costly re-implementation of existing functionality, and allows the system to improve seamlessly as better implementations become available.

Essential issues and extensions

Let discuss several additional issues and extensions that tend to be essential in applications. As a running example, consider optimization of a simple news website called `SimpleNews`. While this example is based on a successful product deployment, we ignore some real-life complications for the sake of clarity. Thus, `SimpleNews` displays exactly one headline to each user. When a user arrives, some information is available pertaining to the interaction with this particular user, *e.g.,* age, gender, geolocation, time of day, and possibly much more; such information is summarily called the *context*. The website chooses a news topic (*e.g.,* politics, sports, tech, etc.), possibly depending on the context, and the top headline for the chosen topic is displayed. The website's goal is to maximize the number of clicks on the headlines that it displays. For this example, we are only concerned with the choice of news topic: that is, we want to pick a news topic whose top headline is most likely to be clicked on for a given context.

Fragmented outcomes. A good practice is to log the entire observable outcome, not just the reward. For example, the outcome of choosing a news topic in `SimpleNews` includes the *dwell time*: the amount of time the user spent reading a suggested article (if any). Multiple outcome fragments may arrive at different times. For example, the dwell time `SimpleNews` is recorded long after the initial click on the article. Thus, multiple outcome tuples may be logged by the front-end. If the reward depends on multiple outcome fragments, it may be computed by the Join Server after the outcome tuples are joined, rather than in the front-end.

Reward metrics. There may be several reasonable ways to define rewards, especially with fragmented outcomes. For example, in `SimpleNews` the reward can include a bonus that depends on the dwell time, and there may be several reasonable choices for defining such a bonus. Further, the reward may depend on the context, *e.g.,* in `SimpleNews` some clicks may be more important than others, depending on the user's demographics. Thus, a reward metric used by the application is but a convenient proxy for the long-term objective such as cumulative revenue or long-term customer satisfaction. The reward metric may change over time when priorities change or when a better proxy for the long-term objective is found. Further, it may be desirable to consider several reward metrics at once, *e.g.,* for safeguards. Offline learning can be used to investigate the effects of switching to another reward metric in policy training.

Non-stationarity. While we've been positing a stationarity environment so far, applications exhibit only periods of near-stationarity in practice. To cope with a changing environment, we use a continuous loop in which the policies are re-trained and re-deployed quickly as new data becomes available. Therefore, the infrastructure and the policy training algorithm should process new data points sufficiently fast. Policy training should de-emphasize older data points over time. Enough data is needed within a period of near-stationarity (so more data is needed to adapt to faster rate of change).

Non-stationarity is partially mitigated if some of it is captured by the context. For example, users of `SimpleNews` may become more interested in sports during major sports events such as Olympics. If the

presence of such events is included as features in the context, the response to a particular news topic given a particular context becomes more consistent across time.

In a non-stationary environment, the goal of policy evaluation is no longer to estimate the expected reward (simply because this quantity changes over time) or predict rewards in the future. Instead, the goal is *counterfactual*: estimate the policy's performance if it were used when the exploration data was collected. This is a mathematically precise goal that is achievable (say) by the IPS estimator regardless of how the environment changes in the meantime. When algorithms for exploration and/or policy training are evaluated on the exploration data, the goal is counterfactual in a similar sense. This is very useful for comparing these algorithms with alternative approaches. One hopes that such comparison on the exploration data would be predictive of a similar comparison performed via a live A/B test.

Feature selection. Selecting which features to include in the context is a fundamental issue in machine learning. Adding more features tends to lead to better rewards in the long run, but may slow down the learning. The latter could be particular damaging if the environment changes over time and fast learning is essential. The same features can be represented in different ways, *e.g.,* a feature with multiple possible values can be represented either as one value or as a bit vector (*1-hot encoding*) with one bit for each possible value. Feature representation is essential, as general-purpose policy training algorithms tend to be oblivious to what features *mean*. A good feature representation may depend on a particular policy training algorithm. Offline learning can help investigate the effects of adding/removing features or changing their representation. For this purpose, additional observable features (not included in the context) can be logged in the decision tuple and passed along to offline learning.

Ease of adoption. Developers might give up on a new system, unless it is easy to adopt. Implementation should avoid or mitigate dependencies on particular programming languages or external libraries. The trial experience should be seamless, *e.g.,* sensible defaults should be provided for all components.

Road to deployment

Consider a particular application such as `SimpleNews`, call it `APP`. Deploying contextual bandits for this application should follow a process, even when the contextual bandit system such as the Decision Service is available. One should do some prep-work: frame the `APP` as a contextual bandit problem, verify that enough data would be available, and have a realistic game plan for integrating the system with the existing infrastructure. Next step is a pilot deployment on a small fraction of traffic. The goal here is to validate the system for `APP` and debug problems; deep integration and various optimizations can come later. Finally, the system is integrated into `APP`, and deployed on a larger fraction of traffic.

Framing the problem. Interactions between `APP` and its users should be interpreted as a sequence of small interactions with individual users, possibly overlapping in time. Each small interaction should follow a simple template: observe a *context*, make a *decision*, choosing from the available alternatives, and observe the *outcome* of this decision. The meaning of contexts, decisions and outcomes should be consistent throughout. The *context*, typically represented as a vector of features, should encompass the properties of the current user and/or task to be accomplished, and must be known to `APP`. The *decision* must be controlled by `APP`. The set of feasible actions should be known: either fixed in advance or specified by the context. It is often useful to describe actions with features of their own, a.k.a. *action-specific features*. The *outcome* consists of one or several events, all of which must be observable by `APP` not too long after the action is chosen. The outcome (perhaps jointly with the context) should define a *reward*: the short-term objective to be optimized. Thus, one should be able to fill in the table below:

	SimpleNews	Your APP
Context	(gender, location, time-of-day)	
Decision	a news topic to display	
Feasible actions	{politics, sports, tech, arts}	
Action-specific features	none	
Outcome	click or no click within 20 seconds	
Reward	1 if clicked, 0 otherwise	

As discussed above, feature selection and reward definition can be challenging. Defining *decisions* and *outcomes* may be non-trivial, too. For example, SimpleNews can be modified so that actions correspond to news articles, rather than news topics, and each *decision* consists of choosing a slate of news articles.

Scale and feasibility. To estimate the scale and feasibility of the learning problem in APP, one needs to estimate a few parameters: the number of features (#features), the number of feasible actions (#actions), a typical delay between making a decision and observing the corresponding outcome, and *the data rate*: the number of experimental units per a time unit. If the outcome includes a rare event whose frequency is crucial to estimate — *e.g.,* clicks are typically rare compared to non-clicks, — then we also need a rough estimate of this frequency. Finally, we need the *stationarity timescale*: the time interval during which the environment does not change too much, namely the distribution of arriving contexts (where a context includes the set of feasible actions and, if applicable, their features), and the expected rewards for each context-action pair. In the SimpleNews example, this corresponds to the distribution of arriving user profiles and the click probability for the top headline (for a given news topic, when presented to a typical user with a given user profile). To summarize, one should be able to fill in Table 8.1.

	SimpleNews	Your APP
#features	3	
#actions	4 news topics	
Typical delay	5 sec	
Data rate	100 users/sec	
Rare event frequency	typical click prob. 2-5%	
Stationarity timescale	one week	

Table 8.1: The scalability parameters (using SimpleNews as an example)

For policy training algorithms based on linear classifiers, a good rule-of-thumb is that the stationarity timescale should be much larger than the typical delay, and moreover we should have

$$\texttt{StatInterval} \times \texttt{DataRate} \times \texttt{RareEventFreq} \gg \#\texttt{actions} \times \#\texttt{features} \tag{8.16}$$

The left-hand side is the number of rare events in the timescale, and the right-hand side characterizes the complexity of the learning problem.

8.7 Literature review and discussion

Lipschitz contextual bandits. Contextual bandits with a Lipschitz condition on contexts have been introduced in Hazan and Megiddo (2007), along with a solution via uniform discretization of contexts and, essentially, the upper bound in Theorem 8.4. The extension to the more general Lipschitz condition (8.4) has been observed, simultaneously and independently, in Lu et al. (2010) and Slivkins (2014). The regret bound in Exercise 8.1 is optimal in the worst case (Lu et al., 2010; Slivkins, 2014).

Adaptive discretization improves over uniform discretization, much like it does in Chapter 4. The key is to discretize the context-arms pairs, rather than contexts and arms separately. This approach is implemented in Slivkins (2014), achieving regret bounds that are optimal in the worst case, and improve for "nice" problem instances. For precisely, there is a contextual version of the "raw" regret bound (4.15) in terms of the covering numbers, and an analog of Theorem 4.18 in terms of a suitable version of the zooming dimension. Both regret bounds are essentially the best possible. This approach extends to an even more general Lipschitz condition when the right-hand side of (8.2) is an arbitrary metric on context-arm pairs.

Slivkins (2014) applies this machinery to handle some adversarial bandit problems. Consider the special case of Lipschitz contextual bandits when the context x_t is simply the time t (here, it is crucial that the contexts are *not* assumed to arrive from fixed distribution). The Lipschitz condition can be written as

$$|\mu(a \mid t) - \mu(a \mid t')| \leq \mathcal{D}_a(t, t'), \tag{8.17}$$

where $\mathcal{D}_a$ is a metric on $[T]$ that is known to the algorithm, and possibly parameterized by the arm a. This condition describes a bandit problem with randomized adversarial rewards such that the expected rewards can only change slowly. The paradigmatic special cases are $\mathcal{D}_a(t,t') = \sigma_a \cdot |t - t'|$, bounded change in each round, and $\mathcal{D}_a(t,t') = \sigma_a \cdot \sqrt{|t - t'|}$. The latter case subsumes a scenario when mean rewards follow a random walk. More precisely, the mean reward of each arm a evolves as a random walk with step $\pm\sigma_a$ on the $[0,1]$ interval with reflecting boundaries. For the special cases, Slivkins (2014) achieves near-optimal bounds on "dynamic regret" (see Section 6.6), as a corollary of the "generic" machinery for adaptive discretization. In full generality, one has a metric space and a Lipschitz condition on context-arm-time triples, and the algorithm performs adaptive discretization over these triples.

Rakhlin et al. (2015); Cesa-Bianchi et al. (2017) tackle a version of Lipschitz contextual bandits in which the comparison benchmark is the best *Lipschitz policy*: a mapping π from contexts to actions which satisfies $D_{\mathcal{A}}(\pi(x), \pi(x')) \leq D_{\mathcal{X}}(x, x')$ for any two contexts x, x', where $D_{\mathcal{A}}$ and $D_{\mathcal{X}}$ are the metrics from (8.4). Several feedback models are considered, including bandit feedback and full feedback.

Krishnamurthy et al. (2020) consider a mash-up of Lipschitz bandits and contextual bandits with policy sets, in which the Lipschitz condition holds across arms for every given context, and regret is with respect to a given policy set. In addition to worst-case regret bounds that come from uniform discretization of the action space, Krishnamurthy et al. (2020) obtain instance-dependent regret bounds which generalize those for the zooming algorithm in Section 4.3. The algorithm is based on a different technique (from Dudík et al., 2011), and is not computationally efficient.

Linear contextual bandits have been introduced in Li et al. (2010), motivated by personalized news recommendations. Algorithm `LinUCB` was defined in Li et al. (2010), and analyzed in (Chu et al., 2011; Abbasi-Yadkori et al., 2011).[2] The details – how the confidence region is defined, the computational implementation, and even the algorithm's name – differ between from one paper to another. The name `LinUCB` stems from Li et al. (2010); we find it descriptive, and use as an umbrella term for the template in Algorithm 8.2. Good empirical performance of LinUCB has been observed in (Krishnamurthy et al., 2016), even

[2]The original analysis in Li et al. (2010) suffers from a subtle bug, as observed in Abbasi-Yadkori et al. (2011). The gap-dependent regret bound is from Abbasi-Yadkori et al. (2011); Dani et al. (2008) obtain a similar result for static contexts.

when the problem is not linear. The analysis of LinUCB in Abbasi-Yadkori et al. (2011) extends to a more general formulation, where the action set can be infinite and time-dependent. Specifically, the action set in a given round t is a bounded subset $D_t \subset [0,1]^d$, so that each arm $a \in D_t$ is identified with its feature vector: $\theta_a = a$; the context in round t is simply D_t.

The version with "static contexts" (*i.e.,* stochastic linear bandits) has been introduced in Auer (2002). The non-contextual version of LinUCB was suggested in Auer (2002), and analyzed in Dani et al. (2008). Auer (2002), as well Abe et al. (2003) and Rusmevichientong and Tsitsiklis (2010), present and analyze other algorithms for this problem which are based on the same paradigm of "optimism under uncertainty", but do not fall under LinUCB template.

If contexts are sufficiently "diverse", *e.g.,* if they come from a sufficiently "diffuse" distribution, then greedy (exploitation-only) algorithm work quite well. Indeed, it achieves the $\tilde{O}(\sqrt{KT})$ regret rate, which optimal in the worst case (Bastani et al., 2021; Kannan et al., 2018). Even stronger guarantees are possible if the unknown vector θ comes from a Bayesian prior with a sufficiently large variance, and one is interested in the Bayesian regret, *i.e.,* regret in expectation over this prior (Raghavan et al., 2023). The greedy algorithm matches the best possible Bayesian regret for a given problem instance, and is at most $\tilde{O}_K(T^{1/3})$ in the worst case. Moreover, LinUCB achieves Bayesian regret $\tilde{O}_K(T^{1/3})$ under the same assumptions.

Contextual bandits with policy classes. Theorem 8.6 is from Auer et al. (2002b), and the complimentary lower bound (8.9) is due to Agarwal et al. (2012). The oracle-based approach to contextual bandits was proposed in Langford and Zhang (2007), with an "epsilon-greedy"-style algorithm and regret bound similar to those in Theorem 8.8. Dudík et al. (2011) obtained a near-optimal regret bound, $O(\sqrt{KT\log(T|\Pi|)})$, via an algorithm that is oracle-efficient "in theory". This algorithm makes $\mathrm{poly}(T, K, \log|\Pi|)$ oracle calls and relies on the ellipsoid algorithm. Finally, a break-through result of (Agarwal et al., 2014) achieved the same regret bound via a "truly" oracle-efficient algorithm which makes only $\tilde{O}(\sqrt{KT/\log|\Pi|})$ oracle calls across all T rounds. Krishnamurthy et al. (2016) extend this approach to combinatorial semi-bandits.

Another recent break-through extends oracle-efficient contextual bandits to adversarial rewards (Syrgkanis et al., 2016a; Rakhlin and Sridharan, 2016; Syrgkanis et al., 2016b). The optimal regret rate for this problem is not yet settled: in particular, the best current upper bound has $\tilde{O}(T^{2/3})$ dependence on T, against the $\Omega(\sqrt{T})$ lower bound in Eq. (8.9).

Luo et al. (2018); Chen et al. (2019) design oracle-efficient algorithms with data-dependent bounds on dynamic regret (see the discussion of dynamic regret in Section 6.6). The regret bounds are in terms of the number of switches S and the total variation V, where S and V could be unknown, matching the regret rates (in terms of S, V, T) for the non-contextual case.

Classification oracles tend to be implemented via heuristics in practice (Agarwal et al., 2014; Krishnamurthy et al., 2016), since the corresponding classification problems are NP-hard for all/most policy classes that one would consider. However, the results on oracle-based contextual bandit algorithms (except for the ϵ-greedy-based approach) do not immediately extend to approximate classification oracles. Moreover, the oracle's performance in practice does not necessarily carry over to its performance inside a contextual bandit algorithm, as the latter typically calls the oracle on carefully constructed artificial problem instances.

Contextual bandits with realizability. Instead of a linear model, one could posit a more abstract assumption of *realizability*: that the expected rewards can be predicted perfectly by some function $\mathcal{X} \times \mathcal{A} \to [0,1]$, called *regressor*, from a given class $\mathcal{F}$ of regressors. This assumption leads to improved performance, in a strong provable sense, even though the worst-case regret bounds cannot be improved (Agarwal et al., 2012). One could further assume the existence of a *regression oracle* for $\mathcal{F}$: a computationally efficient algorithm which finds the best regressor in $\mathcal{F}$ for a given dataset. A contextual bandit algorithm can use such oracle as a subroutine, similarly to using a classification oracle for a given policy class (Foster et al., 2018; Foster and Rakhlin, 2020; Simchi-Levi and Xu, 2020). Compared to classification oracles, regression oracles tend

to be computationally efficient without any assumptions, both in theory and in practice.

Offline learning. "Offline" learning from contextual bandit data is a well-established approach initiated in Li et al. (2011), and further developed in subsequent work, (*e.g.,* Dudík et al., 2012, 2014; Swaminathan and Joachims, 2015; Swaminathan et al., 2017). In particular, Dudík et al. (2014) develops *doubly-robust* estimators which, essentially, combine the benefits of IPS and model-based approaches; Dudík et al. (2012) consider non-stationary policies (*i.e.,* algorithms); Swaminathan et al. (2017) consider policies for "combinatorial" actions, where each action is a slate of "atoms".

Practical aspects. The material in Section 8.6 is adapted from (Agarwal et al., 2016, 2017b). The Decision Service, a system for large-scale applications of contextual bandits, is available at GitHub.[3] At the time of this writing, the system is deployed at various places inside Microsoft, and is offered externally as a Cognitive Service on Microsoft Azure. [4] The contextual bandit algorithms and some of the core functionality is provided via *Vowpal Wabbit*,[5] an open-source library for machine learning.

Bietti et al. (2021) compare the empirical performance of various contextual bandit algorithms. Li et al. (2015) provide a case study of policy optimization and training in web search engine optimization.

8.8 Exercises and hints

Exercise 8.1 (Lipschitz contextual bandits). Consider the Lipschitz condition in (8.4). Design an algorithm with regret bound $\tilde{O}(T^{(d+1)/(d+2)})$, where d is the covering dimension of $\mathcal{X} \times \mathcal{A}$.

Hint: Extend the uniform discretization approach, using the notion of ϵ-mesh from Definition 4.4. Fix an ϵ-mesh $S_{\mathcal{X}}$ for $(\mathcal{X}, \mathcal{D}_{\mathcal{X}})$ and an ϵ-mesh $S_{\mathcal{A}}$ for $(\mathcal{A}, \mathcal{D}_{\mathcal{A}})$, for some $\epsilon > 0$ to be chosen in the analysis. Fix an optimal bandit algorithm `ALG` such as `UCB1`, with $S_{\mathcal{A}}$ as a set of arms. Run a separate copy $\texttt{ALG}_x$ of this algorithm for each context $x \in S_{\mathcal{X}}$.

Exercise 8.2 (Empirical policy value). Prove that realized policy value $\tilde{r}(\pi)$, as defined in (8.10), is close to the policy value $\mu(\pi)$. Specifically, observe that $\mathbb{E}[\tilde{r}(\pi)] = \mu(\pi)$. Next, fix $\delta > 0$ and prove that

$$|\mu(\pi) - \tilde{r}(\pi)| \leq \texttt{conf}(N) \quad \text{with probability at least } 1 - \delta/|\Pi|,$$

where the confidence term is $\texttt{conf}(N) = O\left(\sqrt{1/N \cdot \log(|\Pi|/\delta)}\right)$. Letting π_0 be the policy with a largest realized policy value, it follows that

$$\max_{\pi \in \Pi} \mu(\pi) - \mu(\pi_0) \leq \texttt{conf}(N) \quad \text{with probability at least } 1 - \delta$$

Exercise 8.3 (Policy evaluation). Use Bernstein Inequality to prove Lemma 8.13. In fact, prove a stronger statement: for each policy π and each $\delta > 0$, with probability at least $1 - \delta$ we have:

$$|\texttt{IPS}(\pi) - \tilde{r}(\pi)| \leq O\left(\sqrt{V \ \log(1/\delta)/N}\right), \quad \text{where } V = \max_t \sum_{a \in \mathcal{A}} \frac{1}{p_t(a)}. \tag{8.18}$$

[3] https://github.com/Microsoft/mwt-ds/.

[4] https://docs.microsoft.com/en-us/azure/cognitive-services/personalizer/.

[5] https://vowpalwabbit.org/.

Chapter 9

Bandits and Games

This chapter explores connections between bandit algorithms and game theory. We consider a bandit algorithm playing a repeated zero-sum game against an adversary (*e.g.,* another bandit algorithm). We are interested in convergence to an equilibrium: whether, in which sense, and how fast this happens. We present a sequence of results in this direction, focusing on best-response and regret-minimizing adversaries. Our analysis also yields a self-contained proof of von Neumann's "Minimax Theorem". We also present a simple result for general games.

Prerequisites: Chapters 5, 6.

Throughout this chapter, we consider the following setup. A bandit algorithm ALG plays a repeated game against another algorithm, called an *adversary* and denoted ADV. The game is characterized by a matrix M, and proceeds over T rounds. In each round t of the game, ALG chooses row i_t of M, and ADV chooses column j_t of M. They make their choices simultaneously, *i.e.,* without observing each other. The corresponding entry $M(i_t, j_t)$ specifies the cost for ALG. After the round, ALG observes the cost $M(i_t, j_t)$ and possibly some auxiliary feedback. Thus, the problem protocol is as follows:

Problem protocol: Repeated game between an algorithm and an adversary

In each round $t \in 1, 2, 3, \ldots, T$:

1. Simultaneously, ALG chooses a row i_t of M, and ADV chooses a column j_t of M.
2. ALG incurs cost $M(i_t, j_t)$, and observes feedback $\mathcal{F}_t = \mathcal{F}(t, i_t, j_t, M)$, where $\mathcal{F}$ is a fixed and known *feedback function*.

We are mainly interested in two feedback models:

$$\mathcal{F}_t = M(i_t, j_t) \qquad \text{(bandit feedback)},$$
$$\mathcal{F}_t = (\, M(i, j_t) : \text{all rows } i \,), \qquad \text{(full feedback, from ALG's perspective)}.$$

However, all results in this chapter hold for an arbitrary feedback function $\mathcal{F}$.

ALG's objective is to minimize its total cost, $\texttt{cost}(\texttt{ALG}) = \sum_{t=1}^{T} M(i_t, j_t)$.

We consider zero-sum games, unless noted otherwise, as the corresponding theory is well-developed and has rich applications. Formally, we posit that ALG's cost in each round t is also ADV's reward. In standard game-theoretic terms, each round is a *zero-sum game* between ALG and ADV, with *game matrix* M.[1]

[1] Equivalently, ALG and ADV incur costs $M(i_t, j_t)$ and $-M(i_t, j_t)$, respectively. Hence the term 'zero-sum game'.

Regret assumption. The only property of ALG that we will rely on is its regret. Specifically, we consider a bandit problem with the same feedback model $\mathcal{F}$, and for this bandit problem we consider regret $R(T)$ against an adaptive adversary, relative to the best-observed arm, as defined in Chapter 5.1. All results in this chapter are meaningful even if we only control *expected* regret $\mathbb{E}[R(T)]$, more specifically if $\mathbb{E}[R(t)] = o(t)$. Recall that this property is satisfied by algorithm Hedge for full feedback, and algorithm Exp3 for bandit feedback, as per Chapters 5.3 and 6).

Let us recap the definition of $R(T)$. Using the terminology from Chapter 5.1, the algorithm's cost for choosing row i in round t is $c_t(i) = M(i, j_t)$. Let $\texttt{cost}(i) = \sum_{t=1}^{T} c_t(i)$ be the total cost for always choosing row i, under the observed costs $c_t(\cdot)$. Then

$$\texttt{cost}^* := \min_{\text{rows } i} \texttt{cost}(i)$$

is the cost of the "best observed arm". Thus, as per Eq. (5.1),

$$R(T) = \texttt{cost}(\texttt{ALG}) - \texttt{cost}^*. \tag{9.1}$$

This is what we will mean by *regret* throughout this chapter.

Motivation. At face value, the setting in this chapter is about a repeated game between agents that use regret-minimizing algorithms. Consider algorithms for "actual" games such as chess, go, or poker. Such algorithm can have a "configuration" that can be tuned over time by a regret-minimizing "meta-algorithm" (which interprets configurations as actions, and wins/losses as payoffs). Further, agents in online commerce, such as advertisers in an ad auction and sellers in a marketplace, engage in repeated interactions such that each interaction proceeds as a game between agents according to some fixed rules, depending on bids, prices, or other signals submitted by the agents. An agent may adjust its behavior in this environment using a regret-minimizing algorithm. (However, the resulting repeated game is not zero-sum.)

Our setting, and particularly Theorem 9.5 in which both ALG and ADV minimize regret, is significant in several other ways. First, it leads to an important prediction about human behavior: namely, humans would approximately arrive at an equilibrium if they behave so as to minimize regret (which is a plausible behavioural model). Second, it provides a way to compute an approximate Nash equilibrium for a given game matrix. Third, and perhaps most importantly, the repeated game serves as a subroutine for a variety of algorithmic problems, see Section 9.5 for specific pointers. In particular, such subroutine is crucial for a bandit problem discussed in Chapter 10.

9.1 Basics: guaranteed minimax value

Game theory basics. Imagine that the game consists of a single round, *i.e.,* $T = 1$. Let $\Delta_{\texttt{rows}}$ and $\Delta_{\texttt{cols}}$ denote the set of all distributions over rows and colums of M, respectively. Let us extend the notation $M(i, j)$ to distributions over rows/columns: for any $p \in \Delta_{\texttt{rows}}$, $q \in \Delta_{\texttt{cols}}$, we define

$$M(p, q) := \mathop{\mathbb{E}}_{i \sim p,\, j \in q} [\, M(i, j)\,] = p^\top M q,$$

where distributions are interpreted as column vectors.

Suppose ALG chooses a row from some distribution $p \in \Delta_{\texttt{rows}}$ known to the adversary. Then the adversary can choose a distribution $q \in \Delta_{\texttt{cols}}$ so as to maximize its expected reward $M(p, q)$. (Any column j in the support of q would maximize $M(p, \cdot)$, too.) Accordingly, the algorithm should choose p so as to minimize its maximal cost,

$$f(p) := \sup_{q \in \Delta_{\texttt{cols}}} M(p, q) = \max_{\text{columns } j} M(p, j). \tag{9.2}$$

A distribution $p = p^*$ that minimizes $f(p)$ exactly is called a *minimax strategy*. At least one such p^* exists, as an argmin of a continuous function on a closed and bounded set. A minimax strategy achieves cost

$$v^* = \min_{p \in \Delta_{\texttt{rows}}} \max_{q \in \Delta_{\texttt{cols}}} M(p,q),$$

called the *minimax value* of the game M. Note that p^* guarantees cost at most v^* against any adversary:

$$M(p^*, j) \leq v^* \quad \forall \text{ column } j. \tag{9.3}$$

Arbitrary adversary. We apply the regret property of the algorithm and deduce that algorithm's expected average costs are approximately at least as good as the minimax value v^*. Indeed,

$$\texttt{cost}^* = \min_{\text{rows } i} \texttt{cost}(i) \leq \mathop{\mathbb{E}}_{i \sim p^*} [\, \texttt{cost}(i) \,] = \textstyle\sum_{t=1}^{T} M(p^*, j_t) \leq T\, v^*.$$

Recall that p^* is the minimax strategy, and the last inequality follows by Eq. (9.3). By definition of regret, we have:

Theorem 9.1. *For an arbitrary adversary* ADV *it holds that*

$$\tfrac{1}{T}\, \texttt{cost}(\texttt{ALG}) \leq v^* + R(T)/T.$$

In particular, if we (only) posit sublinear expected regret, $\mathbb{E}[R(t)] = o(t)$, then the expected average cost $\frac{1}{T}\, \mathbb{E}[\texttt{cost}(\texttt{ALG})]$ is asymptotically upper-bounded by v^*.

Best-response adversary. Assume a specific (and very powerful) adversary called the *best-response adversary*. Such adversary operates as follows: in each round t, it chooses a column j_t which maximizes its expected reward given the *history*

$$\mathcal{H}_t := (\ (i_1, j_1)\,,\ \ldots\ , (i_{t-1}, j_{t-1})\ )\,, \tag{9.4}$$

which encompasses what happened in the previous rounds. In a formula,

$$j_t = \min\left(\operatorname*{argmax}_{\text{columns } j} \mathbb{E}\left[M(i_t, j) \mid \mathcal{H}_t \right] \right), \tag{9.5}$$

where the expectation is over the algorithm's choice of row i_t.[2]

We consider the algorithm's *average play* (we give an abstract definition which we reuse later).

Definition 9.2. The average play of a given bandit algorithm (up to time T) is a distribution D over arms such that for each arm a, the coordinate $D(a)$ is the fraction of rounds in which this arm is chosen.

Thus, let $\bar{\imath} \in \Delta_{\texttt{rows}}$ be the algorithm's average play. It is useful to represent it as a vector over the rows and write $\bar{\imath} := \frac{1}{T} \sum_{t \in [T]} \mathbf{e}^{\texttt{row}}_{i_t}$, where $\mathbf{e}^{\texttt{row}}_i$ is the i-th unit vector over rows. We argue that the expected average play, $\mathbb{E}[\bar{\imath}]$, performs well against an arbitrary column j. This is easy to prove:

$$\begin{aligned}
\mathbb{E}[M(i_t,\, j)] &= \mathbb{E}[\ \mathbb{E}[M(i_t,\, j) \mid \mathcal{H}_t]\ ] \\
&\leq \mathbb{E}[\ \mathbb{E}[M(i_t,\, j_t) \mid \mathcal{H}_t]\ ] && \textit{(by best response)} \\
&= \mathbb{E}[M(i_t,\, j_t)] \\
M(\ \mathbb{E}[\bar{\imath}], j\ ) &= \tfrac{1}{T} \textstyle\sum_{t=1}^{T} \mathbb{E}[M(i_t,\, j)] && \textit{(by linearity of } M(\cdot,\cdot)\textit{)} \\
&\leq \tfrac{1}{T} \textstyle\sum_{t=1}^{T} \mathbb{E}[\ M(i_t, j_t)\ ] \\
&= \tfrac{1}{T}\, \mathbb{E}[\texttt{cost}(\texttt{ALG})].
\end{aligned} \tag{9.6}$$

[2] Note that the column j in the argmax need not be unique, hence the $\min$ in front of the argmax. Any other tie-breaking rule would work, too. Such column j also maximizes the expected reward $\mathbb{E}\left[M(i_t, \cdot) \mid \mathcal{H}_t \right]$ over all distributions $q \in \Delta_{\texttt{cols}}$.

Plugging this into Theorem 9.1, we prove the following:

Theorem 9.3. *If* ADV *is the best-response adversary, as in (9.5), then*

$$M(\,\mathbb{E}[\bar{\imath}], q\,) \leq \tfrac{1}{T}\,\mathbb{E}[\texttt{cost}(\texttt{ALG})] \leq v^* + \mathbb{E}[R(T)]/T \qquad \forall q \in \Delta_{\texttt{cols}}.$$

Thus, if $\mathbb{E}[R(t)] = o(t)$ then ALG's expected average play $\mathbb{E}[\bar{\imath}]$ asymptotically achieves the minimax property of p^*, as expressed by Eq. (9.3).

9.2 The minimax theorem

A fundamental fact about the minimax value is that it equals the maximin value:

$$\min_{p\in\Delta_{\texttt{rows}}} \max_{q\in\Delta_{\texttt{cols}}} M(p,q) = \max_{q\in\Delta_{\texttt{cols}}} \min_{p\in\Delta_{\texttt{rows}}} M(p,q). \tag{9.7}$$

In other words, the $\max$ and the $\min$ can be switched. The maximin value is well-defined, in the sense that the $\max$ and the $\min$ exist, for the same reason as they do for the minimax value.

The maximin value emerges naturally in the single-round game if one switches the roles of ALG and ADV so that the former controls the columns and the latter controls the rows (and M represents algorithm's rewards rather than costs). Then a *maximin strategy* – a distribution $q = q^* \in \Delta_{\texttt{cols}}$ that maximizes the right-hand size of (9.7) – arises as the algorithm's best response to a best-responding adversary. Moreover, we have an analog of Eq. (9.3):

$$M(p,q^*) \geq v^\sharp \quad \forall p \in \Delta_{\texttt{rows}}, \tag{9.8}$$

where $v^\sharp$ be the right-hand side of (9.7). In words, maximin strategy q^* guarantees reward at least $v^\sharp$ against any adversary. Now, since $v^* = v^\sharp$ by Eq. (9.7), we can conclude the following:

Corollary 9.4. $M(p^*,q^*) = v^*$*, and the pair* (p^*,q^*) *forms a* Nash equilibrium*, in the sense that*

$$p^* \in \operatorname*{argmin}_{p\in\Delta_{\texttt{rows}}} M(p,q^*) \text{ and } q^* \in \operatorname*{argmax}_{q\in\Delta_{\texttt{cols}}} M(p^*,q). \tag{9.9}$$

With this corollary, Eq. (9.7) is a celebrated early result in mathematical game theory, known as the *minimax theorem*. Surprisingly, it admits a simple alternative proof based on the existence of sublinear-regret algorithms and the machinery developed earlier in this chapter.

Proof of Eq. (9.7). The $\geq$ direction is easy:

$$M(p,q) \geq \min_{p'\in\Delta_{\texttt{rows}}} M(p',q) \qquad \forall q \in \Delta_{\texttt{cols}}$$

$$\max_{q\in\Delta_{\texttt{cols}}} M(p,q) \geq \max_{q\in\Delta_{\texttt{cols}}} \min_{p'\in\Delta_{\texttt{rows}}} M(p',q).$$

The $\leq$ direction is the difficult part. Let us consider a full-feedback version of the repeated game studied earlier. Let ALG be any algorithm with subliner expected regret, *e.g.,* Hedge algorithm from Chapter 5.3, and let ADV be the best-response adversary. Define

$$h(q) := \inf_{p\in\Delta_{\texttt{rows}}} M(p,q) = \min_{\text{rows } i} M(i,q) \tag{9.10}$$

for each distribution $q \in \Delta_{\texttt{cols}}$, similarly to how $f(p)$ is defined in (9.2). We need to prove that

$$\min_{p \in \Delta_{\texttt{rows}}} f(p) \leq \max_{q \in \Delta_{\texttt{cols}}} h(q). \tag{9.11}$$

Let us take care of some preliminaries. Let $\bar{\jmath}$ be the average play of ADV, as per Definition 9.2. Then:

$$\begin{aligned} &\mathbb{E}[\texttt{cost}(i)] = \textstyle\sum_{t=1}^{T} \mathbb{E}[\, M(i, j_t)\,] = T\ \mathbb{E}[\, M(i, \bar{\jmath})\,] = T\, M(i,\, \mathbb{E}[\bar{\jmath}]) \qquad \text{for each row } i \\ &\tfrac{1}{T}\ \mathbb{E}[\texttt{cost}^*] \leq \tfrac{1}{T} \min_{\text{rows } i} \mathbb{E}[\texttt{cost}(i)] = \min_{\text{rows } i} M(i,\, \mathbb{E}[\bar{\jmath}]) = \min_{p \in \Delta_{\texttt{rows}}} M(p,\, \mathbb{E}[\bar{\jmath}]) = h(\mathbb{E}[\bar{\jmath}]). \end{aligned} \tag{9.12}$$

The crux of the argument is as follows:

$$\begin{aligned} \min_{p \in \Delta_{\texttt{rows}}} f(p) &\leq f(\,\mathbb{E}[\bar{\imath}]\,) \\ &\leq \tfrac{1}{T}\ \mathbb{E}[\texttt{cost}(\texttt{ALG})] && \textit{(by best response, see (9.6))} \\ &= \tfrac{1}{T}\ \mathbb{E}[\texttt{cost}^*] + \tfrac{\mathbb{E}[R(T)]}{T} && \textit{(by definition of regret)} \\ &\leq h(\mathbb{E}[\bar{\jmath}]) + \tfrac{\mathbb{E}[R(T)]}{T} && \textit{(using } h(\cdot)\textit{, see (9.12))} \\ &\leq \max_{q \in \Delta_{\texttt{cols}}} h(q) + \tfrac{\mathbb{E}[R(T)]}{T}. \end{aligned}$$

Now, taking $T \to \infty$ implies Eq. (9.11) because $\mathbb{E}[R(T)]/T \to 0$, completing the proof. □

9.3 Regret-minimizing adversary

The most interesting version of our game is when the adversary itself is a regret-minimizing algorithm (possibly in a different feedback model). More formally, we posit that after each round t the adversary observes its reward $M(i_t, j_t)$ and auxiliary feedback $\mathcal{F}'_t = \mathcal{F}'(t, i_t, j_t, M)$, where $\mathcal{F}'$ is some fixed feedback model. We consider the regret of ADV in this feedback model, denote it $R'(T)$. We use the minimax theorem to prove that $(\bar{\imath}, \bar{\jmath})$, the average play of ALG and ADV, approximates the Nash equilibrium (p^*, q^*).

First, let us express $\texttt{cost}^*$ in terms of the function $h(\cdot)$ from Eq. (9.10):

$$\begin{aligned} &\texttt{cost}(i) = \textstyle\sum_{t=1}^{T} M(i, j_t) = T\, M(i, \bar{\jmath}) \qquad \text{for each row } i \\ &\tfrac{1}{T}\ \texttt{cost}^* = \min_{\text{rows } i} M(i,\ \bar{\jmath}) = h(\bar{\jmath}). \end{aligned} \tag{9.13}$$

Let us analyze ALG's costs:

$$\begin{aligned} \tfrac{1}{T}\ \texttt{cost}(\texttt{ALG}) - \tfrac{R(T)}{T} &= \tfrac{1}{T}\ \texttt{cost}^* && \textit{(regret for ALG)} \\ &= h(\bar{\jmath}) && \textit{(using } h(\cdot)\textit{, see (9.13))} \\ &\leq h(q^*) = v^\sharp && \textit{(by definition of maximin strategy } q^*\textit{)} \\ &= v^* && \textit{(by Eq. (9.7))} \end{aligned} \tag{9.14}$$

Similarly, analyze the rewards of ADV. Let $\texttt{rew}(j) = \sum_{t=1}^{T} M(i_t, j)$ be the total reward collected by the adversary for always choosing column j, and let

$$\texttt{rew}^* := \max_{\text{columns } j} \texttt{rew}(j)$$

be the reward of the "best observed column". Let's take care of some formalities:

$$\begin{aligned} \texttt{rew}(j) &= \textstyle\sum_{t=1}^{T} M(i_t, j) = T\, M(\bar{\imath},\, j) \qquad \text{for each column } j \\ \tfrac{1}{T}\,\texttt{rew}^* &= \max_{\text{columns } j} M(\bar{\imath},\, j) = \max_{q \in \Delta_{\texttt{cols}}} M(\bar{\imath},\, q) = f(\bar{\imath}). \end{aligned} \tag{9.15}$$

Now, let us use the regret of ADV:

$$\begin{aligned} \tfrac{1}{T}\,\texttt{cost}(\texttt{ALG}) + \tfrac{R'(T)}{T} &= \tfrac{1}{T}\,\texttt{rew}(\texttt{ADV}) + \tfrac{R'(T)}{T} && \textit{(zero-sum game)} \\ &= \tfrac{1}{T}\,\texttt{rew}^* && \textit{(regret for ADV)} \\ &= f(\bar{\imath}) && \textit{(using } f(\cdot)\textit{, see (9.15))} \\ &\geq f(p^*) = v^* && \textit{(by definition of minimax strategy } p^*\textit{)} \end{aligned} \tag{9.16}$$

Putting together (9.14) and (9.16), we obtain

$$v^* - \tfrac{R'(T)}{T} \leq f(\bar{\imath}) - \tfrac{R'(T)}{T} \leq \tfrac{1}{T}\,\texttt{cost}(\texttt{ALG}) \leq h(\bar{\jmath}) + \tfrac{R(T)}{T} \leq v^* + \tfrac{R(T)}{T}.$$

Thus, we have the following theorem:

Theorem 9.5. *Let $R'(T)$ be the regret of* ADV*. Then the average costs/rewards converge, in the sense that*

$$\tfrac{1}{T}\,\texttt{cost}(\texttt{ALG}) = \tfrac{1}{T}\,\texttt{rew}(\texttt{ADV}) \in \left[v^* - \tfrac{R'(T)}{T},\, v^* + \tfrac{R(T)}{T}\right].$$

*Moreover, the average play $(\bar{\imath}, \bar{\jmath})$ forms an ϵ_T-*approximate Nash equilibrium*, with $\epsilon_T := \frac{R(T)+R'(T)}{T}$:*

$$\begin{aligned} M(\bar{\imath},\, q) &\leq v^* + \epsilon_T \qquad \forall q \in \Delta_{\texttt{cols}}, \\ M(p,\, \bar{\jmath}) &\geq v^* - \epsilon_T \qquad \forall p \in \Delta_{\texttt{rows}}. \end{aligned}$$

The guarantees in Theorem 9.5 are strongest if ALG and ADV admit high-probability regret bounds, *i.e.,* if

$$R(T) \leq \tilde{R}(T) \text{ and } R'(T) \leq \tilde{R}'(T) \tag{9.17}$$

with probability at least $1-\gamma$, for some functions $\tilde{R}(t)$ and $\tilde{R}'(t)$. Then the guarantees in Theorem 9.5 hold, with $R(T)$ and $R'(T)$ replaced with, resp., $\tilde{R}(T)$ and $\tilde{R}'(T)$, whenever (9.17) holds. In particular, algorithm Hedge and a version of algorithm Exp3 achieve high-probability regret bounds, resp., for full feedback with $R(T) = O(\sqrt{T \log K})$, for bandit feedback with $R(T) = O(\sqrt{TK \log K})$, where K is the number of actions (Freund and Schapire, 1997; Auer et al., 2002b).

Further, we recover similar but weaker guarantees even if we only have a bound on *expected* regret:

Corollary 9.6. $\frac{1}{T}\,\mathbb{E}[\texttt{cost}(\texttt{ALG})] = \frac{1}{T}\,\mathbb{E}[\texttt{rew}(\texttt{ADV})] \in \left[v^* - \frac{\mathbb{E}[R'(T)]}{T},\, v^* + \frac{\mathbb{E}[R(T)]}{T}\right]$.

Moreover, the expected average play $(\mathbb{E}[\bar{\imath}],\, \mathbb{E}[\bar{\jmath}])$ forms an $\mathbb{E}[\epsilon_T]$-approximate Nash equilibrium.

This is because $\mathbb{E}[\, M(\bar{\imath},\, q)\,] = M(\mathbb{E}[\bar{\imath}], q)$ by linearity, and similarly for $\bar{\jmath}$.

Remark 9.7. Convergence of $\texttt{cost}(\texttt{ALG})$ to the corresponding equilibrium cost v^* is characterized by the *convergence rate* $\frac{|\texttt{cost}(\texttt{ALG}) - v^*|}{T}$, as a function of T. Plugging in generic $\tilde{O}(\sqrt{T})$ regret bounds into Theorem 9.5 yields convergence rate $\tilde{O}\left(\frac{1}{\sqrt{T}}\right)$.

9.4 Beyond zero-sum games: coarse correlated equilibrium

What can we prove if the game is not zero-sum? While we would like to prove convergence to a Nash Equilibrium, like in Theorem 9.5, this does not hold in general. However, one can prove a weaker notion of convergence, as we explain below. Consider distributions over row-column pairs, let $\Delta_{\texttt{pairs}}$ be the set of all such distributions. We are interested in the average distribution defined as follows:

$$\bar{\sigma} := (\sigma_1 + \ldots + \sigma_T)/T \in \Delta_{\texttt{pairs}}, \qquad \text{where } \sigma_t := p_t \times q_t \in \Delta_{\texttt{pairs}} \tag{9.18}$$

We argue that $\bar{\sigma}$ is, in some sense, an approximate equilibrium.

Imagine there is a "coordinator" who takes some distribution $\sigma \in \Delta_{\texttt{pairs}}$, draws a pair (i, j) from this distribution, and recommends row i to ALG and column j to ADV. Suppose each player has only two choices: either "commit" to following the recommendation before it is revealed, or "deviate" and not look at the recommendation. The equilibrium notion that we are interested here is that each player wants to "commit" given that the other does.

Formally, assume that ADV "commits". The expected costs for ALG are

$$\begin{aligned} U_\sigma &:= \mathop{\mathbb{E}}_{(i,j)\sim\sigma} M(i,j) && \text{if ALG "commits",} \\ U_\sigma(i_0) &:= \mathop{\mathbb{E}}_{(i,j)\sim\sigma} M(i_0,j) && \text{if ALG "deviates" and chooses row } i_0 \text{ instead.} \end{aligned}$$

Distribution $\sigma \in \Delta_{\texttt{pairs}}$ is a *coarse correlated equilibrium* (CCE) if $U_\sigma \geq U_\sigma(i_0)$ for each row i_0, and a similar property holds for ADV.

We are interested in the approximate version of this property: $\sigma \in \Delta_{\texttt{pairs}}$ is an ϵ-approximate CCE if

$$U_\sigma \geq U_\sigma(i_0) - \epsilon \qquad \text{for each row } i_0 \tag{9.19}$$

and similarly for ADV. It is easy to see that distribution $\bar{\sigma}$ achieves this with $\epsilon = \frac{\mathbb{E}[R(T)]}{T}$. Indeed,

$$\begin{aligned} U_{\bar{\sigma}} &:= \mathop{\mathbb{E}}_{(i,j)\sim\bar{\sigma}} [\, M(i,j)\,] = \tfrac{1}{T} \sum_{t=1}^{T} \mathop{\mathbb{E}}_{(i,j)\sim\sigma_t} [\, M(i,j)\,] = \tfrac{1}{T}\, \mathbb{E}[\texttt{cost}(\texttt{ALG})] \\ U_{\bar{\sigma}}(i_0) &:= \mathop{\mathbb{E}}_{(i,j)\sim\bar{\sigma}} [\, M(i_0,j)\,] = \tfrac{1}{T} \sum_{t=1}^{T} \mathop{\mathbb{E}}_{j\sim q_t} [\, M(i_0,j)\,] = \tfrac{1}{T}\, \mathbb{E}[\texttt{cost}(i_0)]. \end{aligned}$$

Hence, $\bar{\sigma}$ satisfies Eq. (9.19) with $\epsilon = \frac{\mathbb{E}[R(T)]}{T}$ by definition of regret. Thus:

Theorem 9.8. *Distribution $\bar{\sigma}$ defined in (9.18) forms an ϵ_T-approximate CCE, where $\epsilon_T = \frac{\mathbb{E}[R(T)]}{T}$.*

9.5 Literature review and discussion

The results in this chapter stem from extensive literature on learning in repeated games, an important subject in theoretical economics. A deeper discussion of this subject from the online machine learning perspective can be found in Cesa-Bianchi and Lugosi (2006, Chapter 7) and the bibliographic notes therein. Empirical evidence that regret-minimization is a plausible model of self-interested behavior can be found in recent studies (Nekipelov et al., 2015; Nisan and Noti, 2017).

9.5.1 Zero-sum games

Some of the early work concerns *fictitious play*: two-player zero-sum games where each player best-responds to the empirical play of the opponent (*i.e.,* the historical frequency distribution over the arms). Introduced in Brown (1949), fictitious play was proved to converge at the rate $O(t^{-1/n})$ for $n \times n$ game matrices (Robinson, 1951). This convergence rate is the best possible (Daskalakis and Pan, 2014).

The repeated game between two regret-minimizing algorithms serves as a subroutine for a variety of algorithmic problems. This approach can be traced to Freund and Schapire (1996, 1999). It has been used as a unifying algorithmic framework for several problems: boosting Freund and Schapire (1996), linear programs Arora et al. (2012), maximum flow Christiano et al. (2011), and convex optimization Abernethy and Wang (2017); Wang and Abernethy (2018). In conjunction with a specific way to define the game matrix, this approach can solve a variety of constrained optimization problems, with application domains ranging from differential privacy to algorithmic fairness to learning from revealed preferences (Rogers et al., 2015; Hsu et al., 2016; Roth et al., 2016; Kearns et al., 2018; Agarwal et al., 2017a; Roth et al., 2017). In Chapter 10, this approach is used to solve bandits with global constraints.

Refinements. Our analysis can be refined for algorithm `Hedge` and/or the best-response adversary:

- The best-response adversary can be seen as a regret-minimizing algorithm which satisfies Theorem 9.5 with $\mathbb{E}[R'(T)] \leq 0$; see Exercise 9.2.
- The `Hedge` vs. `Hedge` game satisfies the approximate Nash property from Theorem 9.5 *with probability* 1, albeit in a modified feedback model; see Exercise 9.3(b).
- The `Hedge` vs. Best Response game also satisfies the approximate Nash property with probability 1; see Exercise 9.3(c).

The last two results can be used for computing an approximate Nash equilibrium for a known matrix M.

The results in this chapter admit multiple extensions, some of which are discussed below.

- Theorem 9.5 can be extended to *stochastic games*, where in each round t, the game matrix M_t is drawn independently from some fixed distribution over game matrices (see Exercise 9.4).
- One can use algorithms with better regret bounds if the game matrix M allows it. For example, `ALG` can be an algorithm for Lipschitz bandits if all functions $M(\cdot, j)$ satisfy a Lipschitz condition.[3]
- Theorem 9.5 and the minimax theorem can be extended to infinite action sets, as long as `ALG` and `ADV` admit $o(T)$ expected regret for the repeated game with a given game matrix; see Exercise 9.5 for a precise formulation.

[3]That is, if $|M(i,j) - M(i',j)| \leq D(i,i')$ for all rows i, i' and all columns j, for some known metric D on rows.

Faster convergence. Rakhlin and Sridharan (2013b); Daskalakis et al. (2015) obtain $\tilde{O}(\frac{1}{t})$ convergence rates for repeated zero-sum games with full feedback. (This is a big improvement over the $\tilde{O}(t^{-1/2})$ convergence results in this chapter, see Remark 9.7.) Wei and Luo (2018) obtains $\tilde{O}(t^{-3/4})$ convergence rate for repeated zero-sum games with bandit feedback. These results are for specific classes of algorithms, and rely on improved analyses of the repeated game.

Last-iterate convergence. While all convergence results in this chapter are only in terms of the *average* play, it is very tempting to ask whether the actual play (i_t, j_t) converges, too. In the literature, such results are called *last-iterate convergence* and *topological convergence*.

With `Hedge` and some other standard algorithms, the results are mixed. On the one hand, strong negative results are known, with a detailed investigation of the non-converging behavior (Bailey and Piliouras, 2018; Mertikopoulos et al., 2018; Cheung and Piliouras, 2019). On the other hand, `Hedge` admits a strong positive result for *congestion games*, a well-studied family of games where a player's utility for a given action depends only on the number of other players which choose an "overlapping" action. In fact, there is a family of regret-minimizing algorithms which generalize `Hedge` such that we have last-iterate convergence in repeated congestion games if each player uses some algorithm from this family (Kleinberg et al., 2009b).

A recent flurry of activity, starting from Daskalakis et al. (2018), derives general results on last-iterate convergence (see Daskalakis and Panageas, 2019; Golowich et al., 2020; Wei et al., 2021, and references therein). These results apply to the main setting in this chapter: arbitrary repeated zero-sum games with two players and finitely many actions, and extend beyond that under various assumptions. All these results require full feedback, and hinge upon two specific, non-standard regret-minimizing algorithms.

9.5.2 Beyond zero-sum games

Correlated equilibria. Coarse correlated equilibrium, introduced in (Moulin and Vial, 1978), is a classic notion in theoretical economics. The simple argument in Section 9.4 appears to be folklore.

A closely related notion called *correlated equilibrium* (Aumann, 1974), posits that

$$\mathop{\mathbb{E}}_{(i,j)\sim\sigma}[\, M(i,j) - M(i_0,j) \mid i \,] \geq 0 \qquad \text{for each row } i_0,$$

and similarly for `ADV`. This is a stronger notion, in the sense that any correlated equilibrium is a coarse correlated equilibrium, but not vice versa. One obtains an approximate correlated equilibria, in a sense similar to Theorem 9.8, if `ALG` and `ADV` have sublinear *internal regret* (Hart and Mas-Colell, 2000, also see Section 6.6 in this book). Both results easily extend to games with multiple players. We specialized the discussion to the case of two players for ease of presentation only.

Smooth games is a wide class of multi-player games which admit strong guarantees about the self-interested behavior being "not too bad" (Roughgarden, 2009; Syrgkanis and Tardos, 2013; Lykouris et al., 2016).[4] Repeated smooth games admit strong convergence guarantees: social welfare of average play converges over time, for arbitrary regret-minimizing algorithms (Blum et al., 2008; Roughgarden, 2009). Moreover, one can achieve faster convergence, at the rate $\tilde{O}(\frac{1}{t})$, under some assumptions on the algorithms' structure (Syrgkanis et al., 2015; Foster et al., 2016a).

Cognitive radios. An application to *cognitive radios* has generated much interest, starting from Lai et al. (2008); Liu and Zhao (2010); Anandkumar et al. (2011), and continuing to, *e.g.,* Avner and Mannor (2014);

[4] More formally, the *price of anarchy* – the ratio between the social welfare (agents' total reward) in the best centrally coordinated solution and the worst Nash equilibrium – can be usefully upper-bounded in terms of the game parameters. More background can be found in the textbook Roughgarden (2016), as well as in many recent classes on algorithmic game theory.

Rosenski et al. (2016); Boursier and Perchet (2019); Bubeck et al. (2020). In this application, multiple radios transmit simultaneously in a shared medium. Each radio can switch among different available channels. Whenever two radios transmit on the same channel, a *collision* occurs, and the transmission does not get through. Each radio chooses channels over time using a multi-armed bandit algorithm. The whole system can be modeled as a repeated game between bandit algorithms.

This line of work has focused on designing algorithms which work well in the repeated game, rather than studying the repeated game between arbitrary algorithms. Various assumptions are made regarding whether and to which extent communication among algorithms is allowed, the algorithms can be synchronized with one another, and collisions are detected. It is typically assumed that each radio transmits continuously.

9.6 Exercises and hints

Exercise 9.1 (game-theory basics).

(a) Prove that a distribution $p \in \Delta_{\texttt{rows}}$ satisfies Eq. (9.3) if and only if it is a minimax strategy.

Hint: $M(p^*, q) \leq f(p^*)$ if p^* is a minimax strategy; $f(p) \leq v^*$ if p satisfies (9.3).

(b) Prove that $p \in \Delta_{\texttt{rows}}$ and $q \in \Delta_{\texttt{cols}}$ form a Nash equilibrium if and only if p is a minimax strategy and q is a maximin strategy.

Exercise 9.2 (best-response adversary). The best-response adversary, as defined in Eq. (9.5), can be seen as an algorithm in the setting of Chapter 9.3. (More formally, this algorithm knows ALG and the game matrix M, and receives auxiliary feedback $\mathcal{F}_t = j_t$.) Prove that its expected regret satisfies $\mathbb{E}[R'(T)] \leq 0$.

Take-away: Thus, Theorem 9.5 applies to the best-response adversary, with $\mathbb{E}[R'(T)] \leq 0$.

Exercise 9.3 (Hedge vs. Hedge). Suppose both ALG and ADV are implemented by algorithm Hedge. Assume that M is a $K \times K$ matrix with entries in $[0, 1]$, and the parameter in Hedge is $\epsilon = \sqrt{(\ln K)/(2T)}$.

(a) Consider the full-feedback model. Prove that the average play $(\bar{\imath}, \bar{\jmath})$ forms a δ_T-approximate Nash equilibrium with high probability (*e.g.,* with probability at least $1-T^{-2}$), where $\delta_T = O\left(\sqrt{\frac{\ln(KT)}{T}}\right)$.

(b) Consider a modified feedback model: in each round t, ALG is given costs $M(\cdot, q_t)$, and ADV is given rewards $M(p_t, \cdot)$, where $p_t \in \Delta_{\texttt{rows}}$ and $q_t \in \Delta_{\texttt{cols}}$ are the distributions chosen by ALG and ADV, respectively. Prove that the average play $(\bar{\imath}, \bar{\jmath})$ forms a δ_T-approximate Nash equilibrium.

(c) Suppose ADV is the best-response adversary, as per Eq. (9.5), and ALG is Hedge with full feedback. Prove that $(\bar{\imath}, \bar{\jmath})$ forms a δ_T-approximate Nash equilibrium.

Hint: Follow the steps in the proof of Theorem 9.5, but use the probability-1 performance guarantee for Hedge (Eq. (5.12) on page 67) instead of the standard definition of regret (9.1).

For part (a), define $\widetilde{\texttt{cost}}(\texttt{ALG}) := \sum_{t\in[T]} M(p_t, j_t)$ and $\widetilde{\texttt{rew}}(\texttt{ADV}) := \sum_{t\in[T]} M(i_t, q_t)$, and use Eq. (5.12) to conclude that they are close to $\texttt{cost}^*$ and $\texttt{rew}^*$, respectively. Apply Azuma-Hoeffding inequality to prove that both $\widetilde{\texttt{cost}}(\texttt{ALG})$ and $\widetilde{\texttt{rew}}(\texttt{ADV})$ are close to $\sum_{t\in[T]} M(p_t, q_t)$.

For part (b), define $\widetilde{\texttt{cost}}(\texttt{ALG}) := \widetilde{\texttt{rew}}(\texttt{ADV}) := \sum_{t\in[T]} M(p_t, q_t)$, and use Eq. (5.12) to conclude that it is close to both $\texttt{cost}^*$ and $\texttt{rew}^*$.

For part (c), define $\widetilde{\texttt{cost}}(\texttt{ALG}) = \widetilde{\texttt{rew}}(\texttt{ADV})$ and handle Hedge as in part (b). Modify (9.16), starting from $\widetilde{\texttt{rew}}(\texttt{ADV})$, to handle the best-response adversary.

Exercise 9.4 (stochastic games). Consider an extension to stochastic games: in each round t, the game matrix M_t is drawn independently from some fixed distribution over game matrices. Assume all matrices have the same dimensions (number of rows and columns). Suppose both ALG and ADV are regret-minimizing algorithms, as in Section 9.3, and let $R(T)$ and $R'(T)$ be their respective regret. Prove that the average play $(\bar{\imath}, \bar{\jmath})$ forms a δ_T-approximate Nash equilibrium for the expected game matrix $M = \mathbb{E}[M_t]$, with

$$\delta_T = \tfrac{R(T)+R'(T)}{T} + \texttt{err}, \text{ where } \texttt{err} = \left| \textstyle\sum_{t\in[T]} M_t(i_t, j_t) - M(i_t, j_i) \right|.$$

Hint: The total cost/reward is now $\sum_{t\in[T]} M_t(i_t, j_t)$. Transition from this to $\sum_{t\in[T]} M(i_t, j_t)$ using the error term err, and follow the steps in the proof of Theorem 9.5.

Note: If all matrix entries are in the $[a, b]$ interval, then $\texttt{err} \in [0, b-a]$, and by Azuma-Hoeffing inequality $\texttt{err} < O(b-a)\sqrt{\log(T)/T}$ with probability at most $1 - T^{-2}$.

Exercise 9.5 (infinite action sets). Consider an extension of the repeated game to infinite action sets. Formally, ALG and ADV have action sets I, J, resp., and the game matrix M is a function $I \times J \to [0, 1]$. I and J can be arbitrary sets with well-defined probability measures such that each singleton set is measurable. Assume there exists a function $\tilde{R}(T) = o(T)$ such that expected regret of ALG is at most $\tilde{R}(T)$ for any ADV, and likewise expected regret of ADV is at most $\tilde{R}(T)$ for any ALG.

(a) Prove an appropriate versions of the minimax theorem:

$$\inf_{p\in\Delta_{\texttt{rows}}} \sup_{j\in J} \int M(p, j)\, \mathrm{d}p = \sup_{q\in\Delta_{\texttt{cols}}} \inf_{i\in I} \int M(i, q)\, \mathrm{d}q,$$

where $\Delta_{\texttt{rows}}$ and $\Delta_{\texttt{cols}}$ are now the sets of all probability measures on I and J.

(b) Formulate and prove an appropriate version of Theorem 9.5.

Hint: Follow the steps in Sections 9.2 and 9.3 with minor modifications. Distributions over rows and columns are replaced with probability measures over I and J. Maxima and minima over I and J are replaced with $\sup$ and $\inf$. Best response returns a distribution over Y (rather than a particular column).

Chapter 10

Bandits with Knapsacks

Bandits with Knapsacks (BwK) is a general framework for bandit problems with global constraints such as supply constraints in dynamic pricing. We define and motivate the framework, and solve it using the machinery from Chapter 9. We also describe two other algorithms for BwK, based on "successive elimination" and "optimism under uncertainty" paradigms from Chapter 1.

Prerequisites: Chapters 1, 5, 6, 9.

10.1 Definitions, examples, and discussion

Motivating example. We start with a motivating example: *dynamic pricing with limited supply*. The basic version of this problem is as follows. The algorithm is a seller, with a limited inventory of B identical items for sale. There are T rounds. In each round t, the algorithm chooses a price $p_t \in [0,1]$ and offers one item for sale at this price. A new customer arrives, having in mind some value v_t for this item (known as *private value*). We posit that v_t is drawn independently from some fixed but unknown distribution. The customer buys the item if and only if $v_t \geq p_t$, and leaves. The algorithm stops after T rounds or after there are no more items to sell, whichever comes first. The algorithm's goal is to maximize revenue from the sales; there is no premium or rebate for the left-over items. Recall that the special case $B = T$ (*i.e.,* unlimited supply of items) falls under "stochastic bandits", where arms corresponds to prices. However, with $B < T$ we have a "global" constraint: a constraint that binds across all rounds and all actions.

More generally, the algorithm may have $n > 1$ products in the inventory, with a limited supply of each. In each round t, the algorithm chooses a price $p_{t,i}$ for each product i, and offers one copy of each product for sale. A new customer arrives, with a vector of private values $v_t = (v_{t,1}, \ldots, v_{t,n})$, and buys each product i such that $v_{t,i} \geq p_{t,i}$. The vector v_t is drawn independently from some distribution.[1] We interpret this scenario as a stochastic bandits problem, where actions correspond to price vectors $p_t = (p_{t,1}, \ldots, p_{t,n})$, and we have a separate "global constraint" on each product.

General framework. We introduce a general framework for bandit problems with global constraints, called "bandits with knapsacks", which subsumes dynamic pricing and many other examples. In this framework, there are several constrained *resources* being consumed by the algorithm, such as the inventory of products in the dynamic pricing problem. One of these resources is time: each arm consumes one unit of the "time resource" in each round, and its budget is the time horizon T. The algorithm stops when the total consumption of some resource i exceeds its respective budget B_i.

[1] If the values are independent across products, *i.e.,* $v_{t,1}, \ldots, v_{t,n}$ are mutually independent random variables, then the problem can be decoupled into n separate per-product problems. However, in general the values may be correlated.

Problem protocol: Bandits with Knapsacks (BwK)

Parameters: K arms, d resources with respective budgets $B_1, \ldots, B_d \in [0, T]$.
In each round $t = 1, 2, 3 \ldots$:

1. Algorithm chooses an arm $a_t \in [K]$.
2. Outcome vector $\vec{o}_t = (r_t; c_{t,1}, \ldots, c_{t,d}) \in [0,1]^{d+1}$ is observed,
   where r_t is the algorithm's reward, and $c_{t,i}$ is consumption of each resource i.

Algorithm stops when the total consumption of some resource i exceeds its budget B_i.

In each round, an algorithm chooses an arm, receives a reward, and also consumes some amount of each resource. Thus, the outcome of choosing an arm is now a $(d+1)$-dimensional vector rather than a scalar. As a technical assumption, the reward and consumption of each resource in each round lie in $[0, 1]$. We posit the "IID assumption", which now states that for each arm a the outcome vector is sampled independently from a fixed distribution over outcome vectors. Formally, an instance of BwK is specified by parameters T, K, d, budgets $B_1, \ldots, B_d$, and a mapping from arms to distributions over outcome vectors. The algorithm's goal is to maximize its *adjusted total reward*: the total reward over all rounds but the very last one.

The name "bandits with knapsacks" comes from an analogy with the well-known *knapsack problem* in algorithms. In that problem, one has a knapsack of limited size, and multiple items each of which has a value and takes a space in the knapsack. The goal is to assemble the knapsack: choose a subset of items that fits in the knapsacks so as to maximize the total value of these items. Similarly, in dynamic pricing each action p_t has "value" (the revenue from this action) and "size in the knapsack" (namely, the number of items sold). However, in BwK the "value" and "size" of a given action are not known in advance.

Remark 10.1. The special case $B_1 = \ldots = B_d = T$ is just "stochastic bandits", as in Chapter 1.

Remark 10.2. An algorithm can continue while there are sufficient resources to do so, even if it almost runs out of some resources. Then the algorithm should only choose "safe" arms if at all possible, where an arm is called "safe" if playing this arm in the current round cannot possibly cause the algorithm to stop.

Discussion. Compared to stochastic bandits, BwK is more challenging in several ways. First, resource consumption during exploration may limit the algorithm's ability to exploit in the future rounds. A stark consequence is that Explore-first algorithm fails if the budgets are too small, see Exercise 10.1(a). Second, per-round expected reward is no longer the right objective. An arm with high per-round expected reward may be undesirable because of high resource consumption. Instead, one needs to think about the *total* expected reward over the entire time horizon. Finally, learning the best arm is no longer the right objective! Instead, one is interested in the best fixed *distribution* over arms. This is because a fixed distribution over arms can perform much better than the best fixed arm. All three challenges arise even when $d = 2$ (one resource other than time), $K = 2$ arms, and $B > \Omega(T)$.

To illustrate the distinction between the best fixed distribution and the best fixed arm, consider the following example. There are two arms $a \in \{1, 2\}$ and two resources $i \in \{1, 2\}$ other than time. In each round, each arm a yields reward 1, and consumes $\mathbf{1}_{\{a=i\}}$ units of each resource i. For intuition, plot resource consumption on a plane, so that there is a "horizontal" am which consumes only the "horizontal" resource, and a "vertical" arm which consumes only the "vertical" resource, see Figure 10.1. Both resources have budget B, and time horizon is $T = 2B$. Then always playing the same arm gives the total reward of B, whereas alternating the two arms gives the total reward of $2B$. Choosing an arm uniformly (and independently) in each round yields the same expected total reward of $2B$, up to a low-order error term.

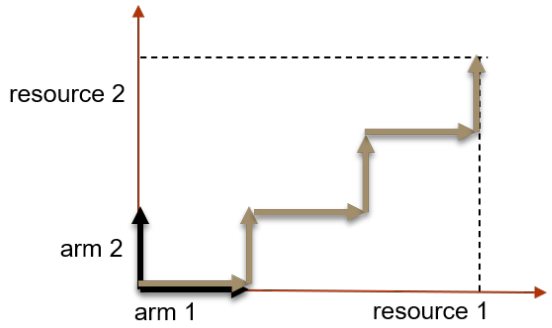


Figure 10.1: Example: alternating arm is twice as good as a fixed arm

Benchmarks. We compare a BwK algorithm against an *all-knowing benchmark*: informally, the best thing one could do if one knew the problem instance. Without resources, "the best thing one could do" is always play the best arm. For BwK, there are three reasonable benchmarks one could consider: best arm, best distribution over arms, and best algorithm. These benchmarks are defined in a uniform way:

$$\texttt{OPT}_{\mathcal{A}}(\mathcal{I}) = \sup_{\text{algorithms } \texttt{ALG}\in\mathcal{A}} \mathbb{E}\left[\,\texttt{REW}(\texttt{ALG} \mid \mathcal{I})\,\right], \tag{10.1}$$

where $\mathcal{I}$ is a problem instance, $\texttt{REW}(\texttt{ALG} \mid \mathcal{I})$ is the adjusted total reward of algorithm ALG on this problem instance, and $\mathcal{A}$ is a class of algorithms. Thus:

- if $\mathcal{A}$ is the class of all BwK algorithms, we obtain the *best algorithm benchmark*, denoted OPT; this is the main benchmark in this chapter.
- if algorithms in $\mathcal{A}$ are fixed distributions over arms, *i.e.,* they draw an arm independently from the same distribution in each round, we obtain the *fixed-distribution benchmark*, denoted $\texttt{OPT}_{\texttt{FD}}$.
- if algorithms in $\mathcal{A}$ are fixed arms, *i.e.,* if they choose the same arm in each round, we obtain the *fixed-arm benchmark*, denoted $\texttt{OPT}_{\texttt{FA}}$.

Obviously, $\texttt{OPT}_{\texttt{FA}} \leq \texttt{OPT}_{\texttt{FD}} \leq \texttt{OPT}$. Generalizing the example above, $\texttt{OPT}_{\texttt{FD}}$ could be up to d times larger than $\texttt{OPT}_{\texttt{FA}}$, see Exercise 10.2. In fact, similar examples exist for the main motivating applications of BwK (Badanidiyuru et al., 2013, 2018). The difference between OPT and $\texttt{OPT}_{\texttt{FD}}$ is not essential in this chapter's technical presentation (but it is for some of the other results spelled out in the literature review).

10.2 Examples

We illustrate the generality of BwK with several examples. In all these examples, "time resource" is the last component of the outcome vector.

- *Dynamic pricing.* Dynamic pricing with a single product is a special case of BwK with two resources: time (*i.e.,* the number of customers) and supply of the product. Actions correspond to chosen prices p. If the price is accepted, reward is p and resource consumption is 1. Thus, the outcome vector is

$$\vec{o}_t = \begin{cases} (p,1,1) & \text{if price } p \text{ is accepted} \\ (0,0,1) & \text{otherwise.} \end{cases} \tag{10.2}$$

- *Dynamic pricing for hiring*, a.k.a. *dynamic procurement*. A contractor on a crowdsourcing market has a large number of similar tasks, and a fixed amount of money, and wants to hire some workers to perform these tasks. In each round t, a worker shows up, the algorithm chooses a price p_t, and offers a contract for one task at this price. The worker has a value v_t in mind, and accepts the offer (and performs the task) if and only if $p_t \geq v_t$. The goal is to maximize the number of completed tasks.

  This problem as a special case of BwK with two resources: time (*i.e.,* the number of workers) and contractor's budget. Actions correspond to prices p; if the offer is accepted, the reward is 1 and the resource consumption is p. So, the outcome vector is

$$\vec{o}_t = \begin{cases} (1, p, 1) & \text{if price } p \text{ is accepted} \\ (0, 0, 1) & \text{otherwise.} \end{cases} \tag{10.3}$$

- *Pay-per-click ad allocation.* There is an advertising platform with pay-per-click ads (advertisers pay only when their ad is clicked). For any ad a there is a known per-click reward r_a: the amount an advertiser would pay to the platform for each click on this ad. If shown, each ad a is clicked with some fixed but unknown probability q_a. Each advertiser has a limited budget of money that he is allowed to spend on her ads. In each round, a user shows up, and the algorithm chooses an ad. The algorithm's goal is to maximize the total reward.

  This problem is a special case of BwK with one resource for each advertiser (her budget) and the "time" resource (*i.e.,* the number of users). Actions correspond to ads. Each ad a generates reward r_a if clicked, in which case the corresponding advertiser spends r_a from her budget. In particular, for the special case of one advertiser the outcome vector is:

$$\vec{o}_t = \begin{cases} (r_a, r_a, 1) & \text{if ad } a \text{ is clicked} \\ (0, 0, 1) & \text{otherwise.} \end{cases} \tag{10.4}$$

- *Repeated auctions.* An auction platform such as eBay runs many instances of the same auction to sell B copies of the same product. At each round, a new set of bidders arrives, and the platform runs a new auction to sell an item. The auction s parameterized by some parameter θ: *e.g.,* the second price auction with the reserve price θ. In each round t, the algorithm chooses a value $\theta = \theta_t$ for this parameter, and announces it to the bidders. Each bidder is characterized by the value for the item being sold; in each round, the tuple of bidders' values is drawn from some fixed but unknown distribution over such tuples. Algorithm's goal is to maximize the total profit from sales.

  This is a special case of BwK with two resources: time (*i.e.,* the number of auctions) and the limited supply of the product. Arms correspond to feasible values of parameter θ. The outcome vector is:

$$\vec{o}_t = \begin{cases} (p_t, 1, 1) & \text{if an item is sold at price } p_t \\ (0, 0, 1) & \text{otherwise.} \end{cases}$$

  The price p_t is determined by the parameter θ and the bids in this round.

- *Dynamic bidding on a budget.* Let's look at a repeated auction from a bidder's perspective. It may be a complicated auction that the bidder does not fully understand. In particular, the bidder often not know the best bidding strategy, but may hope to learn it over time. Accordingly, we consider the following setting. In each round t, one item is offered for sale. An algorithm chooses a bid b_t and observes whether it receives an item and at which price. The outcome (whether we win an item and

at which price) is drawn from a fixed but unknown distribution. The algorithm has a limited budget and aims to maximize the number of items bought.

This is a special case of BwK with two resources: time (*i.e.,* the number of auctions) and the bidder's budget. The outcome vector is

$$\vec{o}_t = \begin{cases} (1, p_t, 1) & \text{if the bidder wins the item and pays } p_t \\ (0, 0, 1) & \text{otherwise.} \end{cases}$$

The payment p_t is determined by the chosen bid b_t, other bids, and the rules of the auction.

10.3 LagrangeBwK: a game-theoretic algorithm for BwK

We present an algorithm for BwK, called LagrangeBwK, which builds on the zero-sum games framework developed in Chapter 9. On a high level, our approach consists of four steps:

Linear relaxation We consider a relaxation of BwK in which a fixed distribution D over arms is used in all rounds, and outcomes are equal to their expected values. This relaxation can be expressed as a linear program for optimizing D, whose per-round value is denoted $\texttt{OPT}_{\texttt{LP}}$ We prove that $\texttt{OPT}_{\texttt{LP}} \geq \texttt{OPT}/T$.

Lagrange game We consider the Lagrange function $\mathcal{L}$ associated with this linear program. We focus on the *Lagrange game*: a zero-sum game, where one player chooses an arm a, the other player chooses a resource i, and the payoff is $\mathcal{L}(a, i)$. We prove that the value of this game is $\texttt{OPT}_{\texttt{LP}}$.

Repeated Lagrange game We consider a repeated version of this game. In each round t, the payoffs are given by Lagrange function $\mathcal{L}_t$, which is defined by this round's outcomes in a similar manner as $\mathcal{L}$ is defined by the expected outcomes. Each player is controlled by a regret-minimizing algorithm. The analysis from Chapter 9 connects the average play in this game with its Nash equilibrium.

Reward at the stopping time The final step argues that the reward at the stopping time is large compared to the "relevant" value of the Lagrange function (which in turn is large enough because of the Nash property). Interestingly, this step only relies on the definition of $\mathcal{L}$, and holds for any algorithm.

Conceptually, these steps connect BwK to the linear program, to the Lagrange game, to the repeated game, and back to BwK, see Figure 10.2. We flesh out the details in what follows.

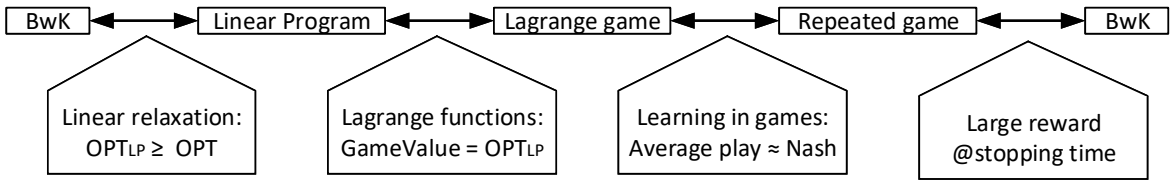


Figure 10.2: The approach in LagrangeBwK.

Preliminaries

We will use the following notation. Let $r(a)$ be the expected per-round reward of arm a, and $c_i(a)$ be the expected per-round consumption of a given resource i. The sets of rounds, arms and resources are denoted $[T]$, $[K]$ and $[d]$, respectively. The distributions over arms and resources are denoted Δ_K and Δ_d. The adjusted total reward of algorithm ALG is denoted REW(ALG).

Let $B = \min_{i\in[d]} B_i$ be the smallest budget. Without loss of generality, we rescale the problem so that all budgets are B. For this, divide the per-round consumption of each resource i by B_i/B. In particular, the per-round consumption of the time resource is now B/T.

We posit that one of the arms, called the *null arm*, brings no reward and consumes no resource except the "time resource". Playing this arm is tantamount to skipping a round. The presence of such arm is essential for several key steps in the analysis. In dynamic pricing the largest possible price is usually assumed to result in no sale, and therefore can be identified with the null arm.

Remark 10.3. Even when skipping rounds is not allowed, existence of the null arm comes without loss of generality. This is because, whenever the null arm is chosen, an algorithm can proceed to the next round *internally*, without actually outputting an action. After T rounds have passed from the algorithm's perspective, the algorithm can choose arms arbitrarily, which can only increase the total reward.

Remark 10.4. Without loss of generality, the outcome vectors are chosen as follows. In each round t, the *outcome matrix* $\mathbf{M}_t \in [0,1]^{K\times(d+1)}$ is drawn from some fixed distribution. Rows of $\mathbf{M}_t$ correspond to arms: for each arm $a \in [K]$, the a-th row of $\mathbf{M}_t$ is

$$\mathbf{M}_t(a) = (r_t(a);\, c_{t,1}(a)\,,\; \ldots\; , c_{t,d}(a)),$$

so that $r_t(a)$ is the reward and $c_{t,i}(a)$ is the consumption of each resource i. The round-t outcome vector is defined as the a_t-th row of this matrix: $\vec{o}_t = \mathbf{M}_t(a_t)$. Only this row is revealed to the algorithm.

Step 1: Linear relaxation

Let us define a relaxation of BwK as follows. Fix some distribution D over arms. Suppose this distribution is used in a given round, *i.e.,* the arm is drawn independently from D. Let $r(D)$ be the expected reward, and $c_i(D)$ be the expected per-round consumption of each resource i:

$$r(D) = \textstyle\sum_{a\in[K]}\; D(a)\; r(a) \quad \text{and} \quad c_i(D) = \sum_{a\in[K]}\; D(a)\; c_i(a).$$

In the relaxation distribution D is used in each round, and the reward and resource-i consumption are deterministically equal to $r(D)$ and $c_i(D)$, respectively. We are only interested in distributions D such that the algorithm does not run out of resources until round T. The problem of choosing D so as to maximize the per-round reward in the relaxation can be formulated as a linear program:

$$\begin{array}{ll} \text{maximize} & r(D) \\ \text{subject to} & \\ & D \in \Delta_K \\ & T\cdot c_i(D) \le B \qquad \forall i \in [d]. \end{array} \tag{10.5}$$

The value of this linear program is denoted $\texttt{OPT}_{\texttt{LP}}$. We claim that the corresponding total reward, $T\cdot\texttt{OPT}_{\texttt{LP}}$, is an upper bound on OPT, the best expected reward achievable in BwK.

Claim 10.5. $T\cdot\texttt{OPT}_{\texttt{LP}} \ge \texttt{OPT}$.

Proof. Fix some algorithm ALG for BwK. Let τ be the round after which this algorithm stops. Without loss of generality, if $\tau < T$ then ALG continues to play the null arm in all rounds $\tau, \tau+1, \ldots, T$.

Let $X_t(a) = \mathbf{1}_{\{a_t = a\}}$ be the indicator variable of the event that ALG chooses arm a in round t. Let $D \in \Delta_K$ be the algorithm's expected average play, as per Definition 9.2: *i.e.,* for each arm a, $D(a)$ is the expected fraction of rounds in which this arm is chosen.

First, we claim that the expected total adjusted reward is $\mathbb{E}[\texttt{REW}(\texttt{ALG})] = r(D)$. Indeed,

$$\begin{aligned}
\mathbb{E}[r_t] &= \textstyle\sum_{a\in[K]} \Pr[a_t = a] \cdot \mathbb{E}[r_t \mid a_t = a] \\
&= \textstyle\sum_{a\in[K]} \mathbb{E}[X_t(a)] \cdot r(a). \\
\mathbb{E}[\texttt{REW}] &= \sum_{t\in[T]} \mathbb{E}[r_t] = \sum_{a\in[K]} r(a) \cdot \sum_{t\in[T]} \mathbb{E}[X_t(a)] = \sum_{a\in[K]} r(a) \cdot T \cdot D(a) = T \cdot r(D).
\end{aligned}$$

Similarly, the expected total consumption of each resource i is $\sum_{t\in[T]} \mathbb{E}[c_{t,i}] = T \cdot c_i(D)$.

Since the (modified) algorithm does not stop until time T, we have $\sum_{t\in[T]} c_{i,t} \leq B$, and consequently $c_i(D) \leq B/T$. Therefore, D is a feasible solution for the linear program (10.5). It follows that $\mathbb{E}[\texttt{REW}(\texttt{ALG})] = r(D) \leq \texttt{OPT}_{\texttt{LP}}$. □

Step 2: Lagrange functions

Consider the Lagrange function associated with the linear program (10.5). For our purposes, this function inputs a distribution D over arms and a distribution λ over resources,

$$\mathcal{L}(D, \lambda) := r(D) + \sum_{i\in[d]} \lambda_i \left[1 - \tfrac{T}{B}\, c_i(D)\right]. \tag{10.6}$$

Define the *Lagrange game*: a zero-sum game, where the *primal player* chooses an arm a, the *dual player* chooses a resource i, and the payoff is given by the Lagrange function:

$$\mathcal{L}(a, i) = r(a) + 1 - \tfrac{T}{B}\, c_i(a). \tag{10.7}$$

The primal player receives this number as a reward, and the dual player receives it as cost. The two players move simultaneously, without observing one another.

Remark 10.6. The terms *primal player* and *dual player* are inspired by the duality in linear programming. For each linear program (LP), a.k.a. *primal* LP, there is an associated *dual* LP. Variables in (10.5) correspond to arms, and variables in its dual LP correspond to resources. Thus, the primal player chooses among the variables in the primal LP, and the dual player chooses among the variables in the dual LP.

The Lagrangian game is related to the linear program, as expressed by the following lemma.

Lemma 10.7. *Suppose (D^*, λ^*) is a mixed Nash equilibrium for the Lagrangian game. Then*

(a) $1 - \frac{T}{B}\, c_i(D^*) \geq 0$ *for each resource i, with equality if $\lambda_i^* > 0$.*

(b) D^* *is an optimal solution for the linear program (10.5).*

(c) The minimax value of the Lagrangian game equals the LP value: $\mathcal{L}(D^*, \lambda^*) = \texttt{OPT}_{\texttt{LP}}$.

Remark 10.8. Lagrange function is a standard notion in mathematical optimization. For an arbitrary linear program (with at least one solution and a finite LP value), the function satisfies a max-min property:

$$\min_{\lambda\in\mathbb{R}^d,\ \lambda\geq 0}\ \max_{D\in\Delta_K} \mathcal{L}(D, \lambda) = \max_{D\in\Delta_K}\ \min_{\lambda\in\mathbb{R}^d,\ \lambda\geq 0} \mathcal{L}(D, \lambda) = \texttt{OPT}_{\texttt{LP}}. \tag{10.8}$$

Because of the special structure of (10.5), we obtain the same property with $\lambda \in \Delta_d$.

Remark 10.9. In what follows, we only use part (c) of Lemma 10.7. Parts (ab) serve to prove (c), and are stated for intuition only. The property in part (a) is known as complementary slackness. The proof of Lemma 10.7 is a standard linear programming argument, not something about multi-armed bandits.

Proof of Lemma 10.7. By the definition of the mixed Nash equilibrium,

$$\mathcal{L}(D^*, \lambda) \geq \mathcal{L}(D^*, \lambda^*) \geq \mathcal{L}(D, \lambda^*) \qquad \forall D \in \Delta_K, \lambda \in \Delta_d. \tag{10.9}$$

Part (a). First, we claim that $Y_i := 1 - \frac{T}{B}\, c_i(D^*) \geq 0$ for each resource i with $\lambda_i^* = 1$.

To prove this claim, assume i is not the time resource (otherwise $c_i(D^*) = B/T$, and we are done). Fix any arm a, and consider the distribution D over arms which assigns probability 0 to arm a, puts probability $D^*(\texttt{null}) + D^*(a)$ on the null arm, and coincides with D^* on all other arms. Using Eq. (10.9),

$$\begin{aligned}
0 &\leq \mathcal{L}(D^*, \lambda^*) - \mathcal{L}(D, \lambda^*) \\
&= [r(D^*) - r(D)] - \tfrac{T}{B}\, [c_i(D^*) - c_i(D)] \\
&= D^*(a) \left[r(a) - \tfrac{T}{B}\, c_i(a) \right] \\
&\leq D^*(a) \left[1 - \tfrac{T}{B}\, c_i(a) \right], \quad \forall \in [K].
\end{aligned}$$

Summing over all arms, we obtain $Y_i \geq 0$, claim proved.

Second, we claim that $Y_i \geq 0$ all resources i. Suppose this is not the case. Focus on resource i with the smallest $Y_i < 0$; note that $\lambda_i^* < 1$ by the first claim. Consider putting all probability on this resource: we have $\mathcal{L}(D^*, i) < 0 = \mathcal{L}(D^*, \texttt{time}) \leq \mathcal{L}(D^*, \lambda^*)$, contradicting (10.9).

Third, assume that $\lambda_i^* > 0$ and $Y_i > 0$ for some resource i. Then $\mathcal{L}(D^*, \lambda^*) > r(D^*)$. Now, consider distribution λ which puts probability 1 on the dummy resource. Then $\mathcal{L}(D^*, \lambda) = r(D^*) < \mathcal{L}(D^*, \lambda^*)$, contradicting (10.9). Thus, $\lambda_i^* = 0$ implies $Y_i > 0$.

Part (bc). By part (a), D^* is a feasible solution to (10.5), and $\mathcal{L}(D^*, \lambda^*) = r(D^*)$. Let D be some other feasible solution for (10.5). Plugging in the feasibility constraints for D, we have $\mathcal{L}(D, \lambda^*) \geq r(D)$. Then

$$r(D^*) = \mathcal{L}(D^*, \lambda^*) \geq \mathcal{L}(D, \lambda^*) \geq r(D).$$

So, D^* is an optimal solution to the LP. In particular, $\texttt{OPT}_{\texttt{LP}} = r(D^*) = \mathcal{L}(D^*, \lambda^*)$. □

Step 3: Repeated Lagrange game

The round-t outcome matrix $\mathbf{M}_t$, as defined in Remark 10.4, defines the respective Lagrange function $\mathcal{L}_t$:

$$\mathcal{L}_t(a, i) = r_t(a) + 1 - \tfrac{T}{B}\, c_{t,i}(a), \quad a \in [K],\ i \in [d]. \tag{10.10}$$

Note that $\mathbb{E}[\mathcal{L}_t(a, i)] = \mathcal{L}(a, i)$, so we will refer to $\mathcal{L}$ as the *expected* Lagrange function.

Remark 10.10. The function defined in (10.10) is a Lagrange function for the appropriate "round-t version" of the linear program (10.5). Indeed, consider the expected outcome matrix $\mathbf{M}_{\texttt{exp}} := \mathbb{E}[\mathbf{M}_t]$, which captures the expected per-round rewards and consumptions,[2] and therefore defines the LP. Let us instead plug in an *arbitrary* outcome matrix $\mathbf{M} \in [0,1]^{K \times (d+1)}$ instead of $\mathbf{M}_{\texttt{exp}}$. Formally, let $\texttt{LP}_{\mathbf{M}}$ be a version of (10.5) when $\mathbf{M}_{\texttt{exp}} = \mathbf{M}$, and let $\mathcal{L}_{\mathbf{M}}$ be the corresponding Lagrange function. Then $\mathcal{L}_t = \mathcal{L}_{\mathbf{M}_t}$ for each round t, *i.e.,* $\mathcal{L}_t$ is the Lagrange function for the version of the LP induced by the round-t outcome matrix $\mathbf{M}_t$.

[2] The a-th row of $\mathbf{M}_{\texttt{exp}}$ is $(r(a);\ c_1(a)\,,\ \ldots\ ,\ c_d(a))$, for each arm $a \in [K]$.

The *repeated Lagrange game* is a game between two algorithms, the *primal algorithm* $\texttt{ALG}_1$ and the *dual algorithm* $\texttt{ALG}_2$, which proceeds over T rounds. In each round t, the primal algorithm chooses arm a_t, the dual algorithm chooses a resource i_t, and the payoff — primal player's reward and dual player's cost — equals $\mathcal{L}_t(a_t, i_t)$. The two algorithms make their choices simultaneously, without observing one another.

Our algorithm, called LagrangeBwK, is very simple: it is a repeated Lagrangian game in which the primal algorithm receives bandit feedback, and the dual algorithm receives full feedback. The pseudocode, summarized in Algorithm 10.1, is self-contained: it specifies the algorithm even without defining repeated games and Lagrangian functions. The algorithm is *implementable*, in the sense that the outcome vector $\vec{o}_t$ revealed in each round t of the BwK problem suffices to generate the feedback for both $\texttt{ALG}_1$ and $\texttt{ALG}_2$.

Given : time horizon T, budget B, number of arms K, number of resources d.
Bandit algorithm $\texttt{ALG}_1$: action set $[K]$, maximizes rewards, bandit feedback.
Bandit algorithm $\texttt{ALG}_2$: action set $[d]$, minimizes costs, full feedback.
for *round* $t = 1, 2, \ldots$ *(until stopping)* **do**
  $\texttt{ALG}_1$ returns arm $a_t \in [K]$, algorithm $\texttt{ALG}_2$ returns resource $i_t \in [d]$.
  Arm a_t is chosen, outcome vector $\vec{o}_t = (r_t(a_t); c_{t,1}(a_t), \ \ldots \ , c_{t,d}(a_t)) \in [0,1]^{d+1}$ is observed.
  The payoff $\mathcal{L}_t(a_t, i_t)$ from (10.10) is reported to $\texttt{ALG}_1$ as reward, and to $\texttt{ALG}_2$ as cost.
  The payoff $\mathcal{L}_t(a_t, i)$ is reported to $\texttt{ALG}_2$ for each resource $i \in [d]$.
end

Algorithm 10.1: Algorithm LagrangeBwK

Let us apply the machinery from Section 9 to the repeated Lagrangian game. For each algorithm $\texttt{ALG}_j$, $j \in \{1, 2\}$, we are interested in its regret $R_j(T)$ relative to the best-observed action, as defined in Chapter 5.1. We will call it *adversarial regret* throughout this chapter, to distinguish from the regret in BwK. For each round $\tau \in [T]$, let $\bar{a} \in \Delta_K$ and $\bar{\imath} \in \Delta_d$ be the average play of $\texttt{ALG}_1$ and $\texttt{ALG}_2$, resp., up to round τ, as per Definition 9.2. Now, Exercise 9.4, applied for the time horizon $\tau \in [T]$, implies the following:

Lemma 10.11. *For each round $\tau \in [T]$, the average play $(\bar{a}_\tau, \bar{\imath}_\tau)$ forms a δ_τ-approximate Nash equilibrium for the expected Lagrange game defined by $\mathcal{L}$, where*

$$\tau \cdot \delta_\tau = R_1(\tau) + R_2(\tau) + \texttt{err}_\tau, \text{ with error term } \texttt{err}_\tau := \left| \sum_{t \in [\tau]} \mathcal{L}_t(i_t, j_t) - \mathcal{L}(i_t, j_t) \right|.$$

Corollary 10.12. $\mathcal{L}(\bar{a}_\tau, i) \geq \texttt{OPT}_{\texttt{LP}} - \delta_\tau$ *for each resource i.*

Step 4: Reward at the stopping time

We focus on the *stopping time* τ, the first round when the total consumption of some resource i exceeds its budget; call i the *stopping resource*. We argue that REW is large compared to $\mathcal{L}(\bar{a}_\tau, i)$ (which plugs nicely into Corollary 10.12). In fact, this step holds for any BwK algorithm.

Lemma 10.13. *For an arbitrary* BwK *algorithm* ALG*,*

$$\texttt{REW}(\texttt{ALG}) \geq \tau \cdot \mathcal{L}(\bar{a}_\tau, i) + (T - \tau) \cdot \texttt{OPT}_{\texttt{LP}} - \texttt{err}^*_{\tau,i},$$

where $\texttt{err}^*_{\tau,i} := \left| \tau \cdot r(\bar{a}_\tau) - \sum_{t \in [\tau]} r_t \right| + \frac{T}{B} \left| \tau \cdot c_i(\bar{a}_\tau) - \sum_{t \in [\tau]} c_{i,t} \right|$ *is the error term.*

Proof. Note that $\sum_{t\in[\tau]} c_{t,i} > B$ because of the stopping. Then:

$$\begin{aligned}\tau \cdot \mathcal{L}(\bar{a}_\tau, i) &= \tau \cdot \left(r(\bar{a}) + 1 - \tfrac{T}{B} c_i(\bar{a})\right) \\ &\leq \sum_{t\in[\tau]} r_t + \tau - \tfrac{T}{B} \sum_{t\in[\tau]} c_{i,t} + \texttt{err}^*_{\tau,i} \\ &\leq \texttt{REW} + \tau - T + \texttt{err}^*_{\tau,i}.\end{aligned}$$

The Lemma follows since $\texttt{OPT}_{\texttt{LP}} \leq 1$. □

Plugging in Corollary 10.12, the analysis is summarized as follows:

$$\texttt{REW}(\texttt{LagrangeBwK}) \geq T \cdot \texttt{OPT}_{\texttt{LP}} - \left[\, R_1(\tau) + R_2(\tau) + \texttt{err}_\tau + \texttt{err}^*_{\tau,i} \,\right], \tag{10.11}$$

where τ is the stopping time and i is the stopping resource.

Wrapping up

It remains to bound the error/regret terms in (10.11). Since the payoffs in the Lagrange game lie in the range $[a, b] := [1 - \frac{T}{B}, 2]$, all error/regret terms are scaled up by the factor $b - a = 1 + \frac{T}{B}$.

Fix an arbitrary failure probability $\delta > 0$. An easy application of Azuma-Hoeffding Bounds implies that

$$\texttt{err}_\tau + \texttt{err}^*_{\tau,i} \leq O(b-a) \cdot \sqrt{TK \log\left(\tfrac{dT}{\delta}\right)} \qquad \forall \tau \in [T],\ i \in [d],$$

with probability at least $1 - \delta$. (Apply Azuma-Hoeffding to each $\texttt{err}_\tau$ and $\texttt{err}^*_{\tau,i}$ separately.)

Let use use algorithms $\texttt{ALG}_1$ and $\texttt{ALG}_2$ that admit high-probability upper bounds on adversarial regret. For $\texttt{ALG}_1$, we use algorithm EXP3.P.1 from Auer et al. (2002b), and for $\texttt{ALG}_2$ we use a version of algorithm Hedge from Chapter 5.3. With these algorithms, with probability at least $1-\delta$ it holds that, for each $\tau \in [T]$,

$$\begin{aligned}R_1(\tau) &\leq O(b-a) \cdot \sqrt{TK \log\left(\tfrac{T}{\delta}\right)}, \\ R_2(\tau) &\leq O(b-a) \cdot \sqrt{T \log\left(\tfrac{dT}{\delta}\right)}.\end{aligned}$$

Plugging this into (10.11), we obtain the main result for LagrangeBwK.

Theorem 10.14. *Suppose algorithm* LagrangeBwK *is used with* EXP3.P.1 *as* $\texttt{ALG}_1$*, and* Hedge *as* $\texttt{ALG}_2$*. Then the following regret bound is achieved, with probability at least* $1 - \delta$*:*

$$\texttt{OPT} - \texttt{REW} \leq O\left(T/B\right) \cdot \sqrt{TK \ln\left(\tfrac{dT}{\delta}\right)}.$$

This regret bound is optimal in the worst case, up to logarithmic factors, in the regime when $B = \Omega(T)$. This is because of the $\Omega(\sqrt{KT})$ lower bound for stochastic bandits. However, as we will see next, this regret bound is not optimal when $\min(B, \texttt{OPT}) \ll T$.

LagrangeBwK achieves optimal $\tilde{O}(KT)$ regret on problem instances with zero resource consumption, *i.e.,* the T/B factor in the regret bound vanishes, see Exercise 10.4.

10.4 Optimal algorithms and regret bounds (no proofs)

The optimal regret bound is in terms of (unknown) optimum $\texttt{OPT}$ and budget B rather than time horizon T:

$$\texttt{OPT} - \mathbb{E}[\texttt{REW}] \leq \tilde{O}\left(\sqrt{K \cdot \texttt{OPT}} + \texttt{OPT}\sqrt{K/B}\right). \tag{10.12}$$

This regret bound is essentially optimal for any given triple (K, B, T): no algorithm can achieve better regret, up to logarithmic factors, over all problem instances with these (K, B, T).[3] The first summand is essentially regret from stochastic bandits, and the second summand is due to the global constraints. The dependence on d is only logarithmic.

We obtain regret $\tilde{O}(\sqrt{KT})$ when $B > \Omega(T)$, like in Theorem 10.14. We have an improvement when $\texttt{OPT}/\sqrt{B} \ll \sqrt{T}$: *e.g.,* $\texttt{OPT} \leq B$ in the dynamic pricing example described above, so we obtain regret $\tilde{O}(\sqrt{KB})$ if there are only K feasible prices. The bad case is when budget B is small, but $\texttt{OPT}$ is large.

Below we outline two algorithms that achieve the optimal regret bound in (10.12).[4] These algorithms build on techniques from IID bandits: resp., Successive Elimination and Optimism under Uncertainty (see Chapter 1). We omit their analyses, which are very detailed and not as lucid as the one for LagrangeBwK.

Prepwork

We consider outcome matrices: formally, these are matrices in $[0,1]^{K\times(d+1)}$. The round-t outcome matrix $\mathbf{M}_t$ is defined as per Remark 10.4, and the expected outcome matrix is $\mathbf{M}_{\texttt{exp}} = \mathbb{E}[\mathbf{M}_t]$.

Both algorithms maintain a *confidence region* on $\mathbf{M}_{\texttt{exp}}$: a set of outcome matrices which contains $\mathbf{M}_{\texttt{exp}}$ with probability at least $1 - T^{-2}$. In each round t, the confidence region $\texttt{ConfRegion}_t$ is recomputed based on the data available in this round. More specifically, a confidence interval is computed separately for each "entry" of $\mathbf{M}_{\texttt{exp}}$, and $\texttt{ConfRegion}_t$ is defined as a product set of these confidence intervals.

Given a distribution D over arms, consider its value in the linear program (10.5):

$$\texttt{LP}(D \mid B, \mathbf{M}_{\texttt{exp}}) = \begin{cases} r(D) & \text{if } c_i(D) \leq B/T \text{ for each resource } i, \\ 0 & \text{otherwise.} \end{cases}$$

We use a flexible notation which allows to plug in arbitrary outcome matrix $\mathbf{M}_{\texttt{exp}}$ and budget B.

Algorithm I: Successive Elimination with Knapsacks

We look for optimal *distributions* over arms. A distribution D is called *potentially optimal* if it optimizes $\texttt{LP}(D \mid B, \mathbf{M})$ for some $\mathbf{M} \in \texttt{ConfRegion}_t$. In each round, we choose a potentially optimal distribution, which suffices for exploitation. But *which* potentially optimal distribution to choose so as to ensure sufficient exploration? Intuitively, we would like to explore each arm as much as possible, given the constraint that we can only use potentially optimal distributions. We settle for something almost as good: we choose an arm a uniformly at random, and then explore it as much as possible, in the sense that we choose a potentially optimal distribution D that maximizes $D(a)$. See Algorithm 10.2 for the pseudocode.

Remark 10.15. The step of maximizing $D(b_t)$ does not have a computationally efficient implementation (more precisely, such implementation is not known for the general case of BwK).

[3] More precisely, no algorithm can achieve regret $\Omega(\min(\texttt{OPT}, \texttt{reg}))$, where $\texttt{reg} = \sqrt{K \cdot \texttt{OPT}} + \texttt{OPT}\sqrt{K/B}$.

[4] More precisely, Algorithm 10.2 achieves the regret bound (10.12) with an extra multiplicative term of $\sqrt{d}$.

for *round* $t = 1, 2, \ldots$ *(until stopping)* **do**
  $S_t \leftarrow$ the set of all potentially optimal distributions over arms.
  Pick arm b_t uniformly at random.
  Pick a distribution $D = D_t$ so as to maximize $D(b_t)$ over all $D \in S_t$.
  Pick arm $a_t \sim D_t$.
end

Algorithm 10.2: Successive Elimination with Knapsacks

This algorithm can be seen as an extension of Successive Elimination. Recall that in Successive Elimination, we start with all arms being "active" and permanently de-activate a given arm a once we have high-confidence evidence that some other arm is better. The idea is that each arm that is currently active can potentially be an optimal arm given the evidence collected so far. In each round we choose among arms that are still "potentially optimal", which suffices for the purpose of exploitation. And choosing *uniformly* (or round-robin) among the potentially optimal arms suffices for the purpose of exploration.

Algorithm II: Optimism under Uncertainty

For each round t and each distribution D over arms, define the Upper Confidence Bound as

$$\texttt{UCB}_t(D \mid B) = \sup_{\mathbf{M} \in \texttt{ConfRegion}_t} \texttt{LP}(D \mid B, \mathbf{M}). \tag{10.13}$$

The algorithm is very simple: in each round, the algorithm picks distribution D which maximizes the UCB. An additional trick is to pretend that all budgets are scaled down by the same factor $1 - \epsilon$, for an appropriately chosen parameter ϵ. This trick ensures that the algorithm does not run out of resources too soon due to randomness in the outcomes or to the fact that the distributions D_t do not quite achieve the optimal value for $\texttt{LP}(D)$. The algorithm, called `UcbBwK`, is as follows:

Rescale the budget: $B' \leftarrow B(1 - \epsilon)$, where $\epsilon = \tilde{\Theta}(\sqrt{K/B})$
Initialization: pull each arm once.
for *all subsequent rounds* t **do**
  In each round t, pick distribution $D = D_t$ with highest $\texttt{UCB}_t(\cdot \mid B')$.
  Pick arm $a_t \sim D_t$.
end

Algorithm 10.3: `UcbBwK`: Optimism under Uncertainty with Knapsacks.

Remark 10.16. For a given distribution D, the supremum in (10.13) is obtained when upper confidence bounds are used for rewards, and lower confidence bounds are used for resource consumption:

$$\texttt{UCB}_t(D \mid B') = \begin{cases} r^{\texttt{UCB}}(D) & \text{if } c_i^{\texttt{LCB}}(D) \leq B'/T \text{ for each resource } i, \\ 0 & \text{otherwise.} \end{cases}$$

Accordingly, choosing a distribution with maximal `UCB` can be implemented by a linear program:

$$\begin{aligned} \text{maximize} \quad & r^{\texttt{UCB}}(D) \\ \text{subject to} \quad & \\ & D \in \Delta_K \\ & c_i^{\texttt{LCB}}(D) \leq B'/T. \end{aligned} \tag{10.14}$$

10.5 Literature review and discussion

The general setting of BwK is introduced in Badanidiyuru et al. (2013, 2018), along with an optimal solution (10.12), the matching lower bound, and a detailed discussion of various motivational examples. (The lower bound is also implicit in earlier work of Devanur et al. (2019).) LagrangeBwK, the algorithm this chapter focuses on, is from a subsequent paper (Immorlica et al., 2022).[5] This algorithm is in fact a "reduction" from BwK to bandits (see Section 10.5.1), and also "works" for the adversarial version (see Section 10.5.4)

Various motivating applications of BwK have been studied separately: dynamic pricing (Besbes and Zeevi, 2009; Babaioff et al., 2015a; Besbes and Zeevi, 2012; Wang et al., 2014), dynamic procurement (Badanidiyuru et al., 2012; Singla and Krause, 2013), and pay-per-click ad allocation (Slivkins, 2013; Combes et al., 2015). Much of this work preceded and inspired Badanidiyuru et al. (2013).

The optimal regret bound in (10.12) has been achieved by three different algorithms: the two in Section 10.4 and one other algorithm from Badanidiyuru et al. (2013, 2018). The successive elimination-based algorithm (Algorithm 10.2) is from (Badanidiyuru et al., 2013, 2018), and UcbBwK (Algorithm 10.3) is from a follow-up paper of Agrawal and Devanur (2014, 2019). The third algorithm, also from (Badanidiyuru et al., 2013, 2018), is a "primal-dual" algorithm superficially similar to LagrangeBwK. Namely, it decouples into two online learning algorithms: a "primal" algorithm which chooses among arms, and a "dual" algorithm similar to $\texttt{ALG}_2$, which chooses among resources. However, the primal and dual algorithms are not playing a repeated game in any meaningful sense. Moreover, the primal algorithm is very problem-specific: it interprets the dual distribution as a vector of costs over resources, and chooses arms with largest reward-to-cost ratios, estimated using "optimism under uncertainty".

Some of the key ideas in BwK trace back to earlier work.[6] First, focusing on total expected rewards rather than per-round expected rewards, approximating total expected rewards with a linear program, and using "optimistic" estimates of the LP values in a UCB-style algorithm goes back to Babaioff et al. (2015a). They studied the special case of dynamic pricing with limited supply, and applied these ideas to fixed arms (not distributions over arms). Second, repeated Lagrange games, in conjunction with regret minimization in zero-sum games, have been used as an algorithmic tool to solve various convex optimization problems (different from BwK), with application domains ranging from differential privacy to algorithmic fairness to learning from revealed preferences (Rogers et al., 2015; Hsu et al., 2016; Roth et al., 2016; Kearns et al., 2018; Agarwal et al., 2017a; Roth et al., 2017). All these papers deal with deterministic games (*i.e.,* same game matrix in all rounds). Most related are (Roth et al., 2016, 2017), where a repeated Lagrangian game is used as a subroutine (the "inner loop") in an online algorithm; the other papers solve an offline problem. Third, estimating an optimal "dual" vector from samples and using this vector to guide subsequent "primal" decisions is a running theme in the work on *stochastic packing* problems (Devanur and Hayes, 2009; Agrawal et al., 2014; Devanur et al., 2019; Feldman et al., 2010; Molinaro and Ravi, 2012). These are full information problems in which the costs and rewards of decisions in the past and present are fully known, and the only uncertainty is about the future. Particularly relevant is the algorithm of Devanur et al. (2011), in which the dual vector is adjusted using multiplicative updates, as in LagrangeBwK and the primal-dual algorithm from Badanidiyuru et al. (2013, 2018).

BwK with only one constrained resource and unlimited number of rounds tends to be an easier problem, avoiding much of the complexity of the general case. In particular, $\texttt{OPT}_{\texttt{FD}} = \texttt{OPT}_{\texttt{FA}}$, *i.e.,* the best distribution over arms is the same as the best fixed arm. György et al. (2007) and subsequently Tran-Thanh et al. (2010, 2012); Ding et al. (2013) obtain instance-dependent $\text{polylog}(T)$ regret bounds under various assumptions.

[5] Cardoso et al. (2018), in a simultaneous and independent work, put forward a similar algorithm, and analyze it under full feedback for the setting of "bandit convex optimization with knapsacks" (see Section 10.5.2).

[6] In addition to the zero-sum games machinery from Chapter 9 and stochastic bandit techniques from Chapter 1.

10.5.1 Reductions from BwK to bandits

Taking a step back from bandits with knapsacks, recall that global constrains is just one of several important "dimensions" in the problem space of multi-armed bandits. It is desirable to unify results on BwK with those on other "problem dimensions". Ideally, results on bandits would seamlessly translate into BwK. In other words, solving some extension of BwK should reduce to solving a similar extension of bandits. Such results are called *reductions* from one problem to another. LagrangeBwK and UcbBwK algorithms give rise to two different reductions from BwK to bandits, discussed below.

LagrangeBwK takes an arbitrary "primal" algorithm $\texttt{ALG}_1$, and turns it into an algorithm for BwK. To obtain an extension to a particular extension of BwK, algorithm $\texttt{ALG}_1$ should work for this extension in bandits, and achieve a high-probability bound on its "adversarial regret" $R_1(\cdot)$ (provided that per-round rewars/costs lie in $[0,1]$). This regret bound $R_1(\cdot)$ then plugs into Eq. (10.11) and propagates through the remaining steps of the analysis. Then with probability at least $1-\delta$ one has

$$\texttt{OPT} - \texttt{REW} \leq O\left(\,{}^{T}\!/_{B}\,\right) \cdot \left(R_1(T) + \sqrt{TK\ln(dT/\delta)} \right). \tag{10.15}$$

Immorlica et al. (2022) makes this observation and uses it to obtain several extensions: to contextual bandits, combinatorial semi-bandits, bandit convex optimization, and full feedback. (These and other extensions are discussed in detail in Section 10.5.2.)

The reduction via UcbBwK is different: it inputs a *lemma* about stochastic bandits, not an algorithm. This lemma works with an abstract *confidence radius* $\texttt{rad}_t(\cdot)$, which generalizes that from Chapter 1. For each arm a and round t, $\texttt{rad}_t(a)$ is a function of algorithm's history which is an upper confidence bound for $|r(a) - \hat{r}_t(a)|$ and $|c_i(a) - \hat{c}_{t,i}(a)|$, where $\hat{r}_t(a)$ and $\hat{c}_{t,i}(a)$ are some estimates computable from the data. The lemma asserts that for a particular version of confidence radius *and any bandit algorithm* it holds that

$$\textstyle\sum_{t\in S} \texttt{rad}_t(a_t) \leq \sqrt{\beta\,|S|} \quad \text{for all subsets of rounds } S \subset [T], \tag{10.16}$$

where $\beta \ll K$ is some application-specific parameter. (The left-hand side is called the *confidence sum*.) Eq. (10.16) holds with $\beta = K$ for stochastic bandits; this follows from the analysis in Section 1.3.2, and stems from the original analysis of UCB1 algorithm in Auer et al. (2002a). Similar results are known for several extensions of stochastic bandits: this is typically a key step in the analysis of any extension of UCB1 algorithm. Whenever Eq. (10.16) holds for some extension of stochastic bandits, one can define a version of UcbBwK algorithm which uses $\hat{r}(a)+\texttt{rad}_t(a)$ and $\hat{c}_{t,i}-\texttt{rad}_t(a)$ as, resp., UCB on $r(a)$ and LCB on $c_i(a)$ in (10.14). Plugging (10.16) into the original analysis of UcbBwK in Agrawal and Devanur (2014, 2019) yields

$$\texttt{OPT} - \mathbb{E}[\texttt{REW}] \leq O(\sqrt{\beta T})(1 + \texttt{OPT}/B). \tag{10.17}$$

Sankararaman and Slivkins (2021) make this observation explicit, and apply it to derive extensions to combinatorial semi-bandits, linear contextual bandits, and multinomial-logit bandits.

Extensions via either approach take little or no extra work (when there is a result to reduce from), whereas many papers, detailed in Section 10.5.2, target one extension each. The resulting regret bounds are usually optimal in the regime when $\min(B, \texttt{OPT}) > \Omega(T)$. Moreover, the LagrangeBwK reduction also "works" for the adversarial version (see Section 10.5.4).

However, these results come with several important caveats. First, the regret bounds can be suboptimal if $\min(B, \texttt{OPT}) \ll \Omega(T)$, and may be improved upon via application-specific results. Second, algorithm $\texttt{ALG}_1$ in LagrangeBwK needs a regret bound against an adaptive adversary, even though for BwK we only need an regret bound against stochastic outcomes. Third, this regret bound needs to hold for the rewards specified by the Lagrange functions, rather than the rewards in the underlying BwK problem (and the latter may have some useful properties that do not carry over to the former). Fourth, most results obtained via the UcbBwK reduction do not come with a computationally efficient implementation.

10.5.2 Extensions of BwK

BwK with generalized resources. Agrawal and Devanur (2014, 2019) consider a version of BwK with a more abstract version of resources. First, they remove the distinction between rewards and resource consumption. Instead, in each round t there is an outcome vector $o_t \in [0,1]^d$, and the "final outcome" is the average $\bar{o}_T = \frac{1}{T}\sum_{t\in[T]} o_t$. The total reward is determined by $\bar{o}_T$, in a very flexible way: it is $T \cdot f(\bar{o}_T)$, for an arbitrary Lipschitz concave function $f : [0,1]^d \mapsto [0,1]$ known to the algorithm. Second, the resource constraints can be expressed by an arbitrary convex set $S \subset [0,1]^d$ which $\bar{o}_T$ must belong to. Third, they allow *soft constraints*: rather than requiring $\bar{o}_t \in S$ for all rounds t (*hard constraints*), they upper-bound the distance between $\bar{o}_t$ and S. They obtain regret bounds that scale as $\sqrt{T}$, with distance between $\bar{o}_t$ and S scaling as $1/\sqrt{T}$. Their results extend to the hard-constraint version by rescaling the budgets, as in Algorithm 10.3, as long as the constraint set S is downward closed.

Contextual bandits with knapsacks. Contextual bandits with knapsacks (*cBwK*) is a common generalization of BwK and contextual bandits (background on the latter can be found in Chapter 8). In each round t, an algorithm observes context x_t before it chooses an arm. The pair $(x_t, \mathbf{M}_t)$, where $\mathbf{M}_t \in [0,1]^{K\times(d+1)}$ is the round-t outcome matrix, is chosen independently from some fixed distribution (which is included in the problem instance). The algorithm is given a set Π of policies (mappings from arms to actions), as in Section 8.4. The benchmark is the best all-knowing algorithm restricted to policies in Π: $\texttt{OPT}_\Pi$ is the expected total reward of such algorithm, similarly to (10.1). The following regret bound can be achieved:

$$\texttt{OPT}_\Pi - \mathbb{E}[\texttt{REW}] \leq \tilde{O}(1 + \texttt{OPT}_\Pi/B)\,\sqrt{KT\log|\Pi|}. \tag{10.18}$$

The $\sqrt{KT\log(|\Pi|)}$ dependence on K, T and Π is optimal for contextual bandits, whereas the $(1+\texttt{OPT}_\Pi/B)$ term is due BwK. In particular, this regret bound is optimal in the regime $B > \Omega(\texttt{OPT}_\Pi)$.

Badanidiyuru et al. (2014) achieve (10.18), with an extra factor of $\sqrt{d}$, unifying Algorithm 10.2 and the *policy elimination* algorithm for contextual bandits (Dudík et al., 2011). Like policy elimination, the algorithm in Badanidiyuru et al. (2014) does not come with a computationally efficient implementation. Subsequently, Agrawal et al. (2016) obtained (10.18) using an "oracle-efficient" algorithm: it uses a classification oracle for policy class Π as a subroutine, and calls this oracle only $\tilde{O}(d\sqrt{KT\log|\Pi|})$ times. Their algorithm builds on the contextual bandit algorithm from Agarwal et al. (2014), see Remark 8.10.

LagrangeBwK reduction can be applied, achieving regret bound

$$\texttt{OPT}_\Pi - \mathbb{E}[\texttt{REW}] \leq \tilde{O}(T/B)\,\sqrt{KT\log|\Pi|}.$$

This regret rate matches (10.18) when $\texttt{OPT}_\Pi > \Omega(T)$, and is optimal in the regime $B > \Omega(T)$. The "primal" algorithm $\texttt{ALG}_1$ is Exp4.P from Beygelzimer et al. (2011), the high-probability version of algorithm Exp4 from Section 6.4. Like Exp4, this algorithm is not computationally efficient.

In a stark contrast with BwK, non-trivial regret bounds are not necessarily achievable when B and $\texttt{OPT}_\Pi$ are small. Indeed, $o(\texttt{OPT})$ worst-case regret is impossible in the regime $\texttt{OPT}_\Pi \leq B \leq \sqrt{KT}/2$ (Badanidiyuru et al., 2014). Whereas $o(\texttt{OPT})$ regret bound holds for BwK whenever $B = \omega(1)$, as per (10.12).

Slivkins et al. (2023) and Han et al. (2023) pursue an alternative approach whereby one posits *realizability* and applies a regression oracle (see Chapter 8.7). They combine LagrangeBwK and SquareCB, the regression-based algorithm for contextual bandits from Foster and Rakhlin (2020).

Linear contextual BwK. In the extension to *linear* contextual bandits (see Section 8.3), the expected outcome matrix is linear in the context x_t, and all policies are allowed. Formally, each context is a matrix: $x_t \in [0,1]^{K\times m}$, where rows correspond to arms, and each arm's context has dimension m. The linear assumption states that $\mathbb{E}[\mathbf{M}_t \mid x_t] = x_t\,\mathbf{W}$, for some matrix $\mathbf{W} \in [0,1]^{(d+1)\times m}$ that is fixed over time, but

not known to the algorithm. Agrawal and Devanur (2016) achieve regret bound

$$\texttt{OPT} - \mathbb{E}[\texttt{REW}] \leq \tilde{O}(m\sqrt{T})(1 + \texttt{OPT}/B) \qquad \text{in the regime } B > mT^{3/4}. \tag{10.19}$$

The $\tilde{O}(m\sqrt{T})$ dependence is the best possible for linear bandits (Dani et al., 2008), whereas the $(1+\texttt{OPT}/B)$ term and the restriction to $B > mT^{3/4}$ is due to BwK. In particular, this regret bound is optimal, up to logarithmic factors, in the regime $B > \Omega(\max(\texttt{OPT},\, m\,T^{3/4}))$.

Eq. (10.19) is immediately obtained via UcbBwK reduction, albeit without a computationally efficient implementation (Sankararaman and Slivkins, 2021).

Combinatorial semi-bandits with knapsacks. In the extension to combinatorial semi-bandits (see Section 7.2), there is a finite set S of *atoms*, and a collection $\mathcal{F}$ of feasible subsets of S. Arms correspond to the subsets in $\mathcal{F}$. When an arm $a = a_t \in \mathcal{F}$ is chosen in some round t, each atom $i \in a$ collects a reward and consumes resources, with its outcome vector $v_{t,i} \in [0,1]^{d+1}$ chosen independently from some fixed but unknown distribution. The "total" outcome vector $\vec{o}_t$ is a sum over atoms: $\vec{o}_t = \sum_{i \in a} v_{t,i}$. We have *semi-bandit feedback*: the outcome vector $v_{t,i}$ is revealed to the algorithm, for each atom $i \in a$. The central theme is that, while the number of arms, $K = |\mathcal{F}|$, may be exponential in the number of atoms, $m = |S|$, one can achieve regret bounds that are polynomial in m.

Combinatorial semi-bandits with knapsacks is a special case of linear cBwK, as defined above, where the context x_t is the same in all rounds, and defines the collection $\mathcal{F}$. (For each arm $a \in \mathcal{F}$, the a-th row of x_t is a binary vector that represents a.) Thus, the regret bound (10.19) for linear cBwK applies. Sankararaman and Slivkins (2018) achieve an improved regret bound when the set system $\mathcal{F}$ is a matroid. They combine Algorithm 10.3 with the *randomized rounding* techniques from approximation algorithms. In the regime when $\min(B, \texttt{OPT}) > \Omega(T)$, these two regret bounds become, respectively, $\tilde{O}(m\sqrt{T})$ and $\tilde{O}(\sqrt{mT})$. The $\sqrt{mT}$ regret is optimal, up to constant factors, even without resources (Kveton et al., 2014).

Both reductions from Section 10.5.1 apply. LagrangeBwK reduction achieves regret $\tilde{O}(T/B)\sqrt{mT}$ (Immorlica et al., 2022). UcbBwK reduction achieves regret $\tilde{O}(m\sqrt{T})(1 + \texttt{OPT}/B)$ via a computationally inefficient algorithm (Sankararaman and Slivkins, 2021).

Multinomial-logit Bandits with Knapsacks. The setup starts like in combinatorial semi-BwK. There is a ground set of N *atoms*, and a fixed family $\mathcal{F} \subset 2^{[N]}$ of feasible actions. In each round, each atom a has an outcome $\vec{o}_t(a) \in [0,1]^{d+1}$, and the outcome matrix $(\, \vec{o}_t(a) : a \in [N] \,)$ is drawn independently from some fixed but unknown distribution. The aggregate outcome is formed in a different way: when a given subset $A_t \in \mathcal{F}$ is chosen by the algorithm in a given round t, at most one atom $a_t \in A_t$ is chosen stochastically by "nature", and the aggregate outcome is then $\vec{o}_t(A_t) := \vec{o}_t(a)$; otherwise, the algorithm skips this round. A common interpretation is *dynamic assortment*: that the atoms correspond to products, the chosen action $A_t \in \mathcal{F}$ is the bundle of products offered to the customer; then at most one product from this bundle is actually purchased. As usual, the algorithm continues until some resource (incl. time) is exhausted.

The selection probabilities are defined via the multinomial-logit model. For each atom a there is a hidden number $v_a \in [0,1]$, interpreted as the customers' valuation of the respective product, and

$$\Pr[\,\text{atom } a \text{ is chosen} \mid A_t\,] = \begin{cases} \frac{v_a}{1+\sum_{a' \in A_t} v_{a'}} & \text{if } a \in A_t \\ 0 & \text{otherwise.} \end{cases}$$

The set of possible bundles is $\mathcal{F} = \{\, A \subset [N] :\ \mathbf{W} \cdot \texttt{bin}(A) \leq \vec{b} \,\}$, for some (known) totally unimodular matrix $\mathbf{W} \in \mathbb{R}^{N \times N}$ and a vector $\vec{b} \in \mathbb{R}^N$, where $\texttt{bin}(A) \in \{0,1\}^N$ is a binary-vector representation.

Multinomial-logit (MNL) bandits, the special case without resources, was studied in connection with dynamic assortment, *e.g.,* in Caro and Gallien (2007); Sauré and Zeevi (2013); Rusmevichientong et al. (2010); Agrawal et al. (2019).

MNL-BwK was introduced in Cheung and Simchi-Levi (2017) and solved via a computationally inefficient algorithm. Sankararaman and Slivkins (2021) solve this problem via the UcbBwK reduction (also computationally inefficiently). Aznag et al. (2021) obtains a computationally efficient version of UcbBwK. These algorithms achieve regret rates similar to (10.17), with varying dependence on problem parameters.

Bandit convex optimization (BCO) with knapsacks. BCO is a version of multi-armed bandits where the action set is a convex set $\mathcal{X} \subset \mathbb{R}^K$, and in each round t, there is a concave function $f_t : \mathcal{X} \to [0,1]$ such that the reward for choosing action $\vec{x} \in \mathcal{X}$ in this round is $f_t(\vec{x})$. BCO is a prominent topic in bandits, starting from (Kleinberg, 2004; Flaxman et al., 2005).

We are interested in a common generalization of BCO and BwK, called *BCO with knapsacks*. For each round t and each resource i, one has convex functions $g_{t,i} : \mathcal{X} \to [0,1]$, such that the consumption of this resource for choosing action $\vec{x} \in \mathcal{X}$ in this round is $g_{t,i}(\vec{x})$. The tuple of functions $(f_t; g_{t,1}, \ldots, g_{t,d})$ is sampled independently from some fixed distribution (which is not known to the algorithm). LagrangeBwK reduction in Immorlica et al. (2022) yields a regret bound of the form $\frac{T}{B}\sqrt{T} \cdot \text{poly}(K \log T)$, building on the recent breakthrough in BCO in Bubeck et al. (2017).[7]

Experts with knapsacks. In the full-feedback version of BwK, the entire outcome matrix $\mathbf{M}_t$ is revealed after each round t. Essentially, one can achieve regret bound (10.12) with K replaced by $\log K$ using UcbBwK algorithm (Algorithm 10.3) with a slightly modified analysis. LagrangeBwK reduction achieves regret $O(T/B) \cdot \sqrt{T \log(dKT)}$ if the "primal" algorithm $\texttt{ALG}_1$ is Hedge from Section 5.3.

10.5.3 Beyond the worst case

Characterization for $O(\log T)$ regret. Going beyond worst-case regret bounds, it is natural to ask about smaller, instance-dependent regret rates, akin to $O(\log T)$ regret rates for stochastic bandits from Chapter 1. Sankararaman and Slivkins (2021) find that $O(\log T)$ regret rates are possible for BwK if and only if two conditions hold: there is only one resource other than time (*i.e.,* $d = 2$), and the best distribution over arms reduces to the best fixed arm (*best-arm-optimality*). If either condition fails, any algorithm is doomed to $\Omega(\sqrt{T})$ regret in a wide range of problem instances.[8] Here both upper and lower bounds are against the fixed-distribution benchmark ($\texttt{OPT}_{\texttt{FD}}$).

Assuming $d = 2$ and best-arm optimality, $O(\log T)$ regret against $\texttt{OPT}_{\texttt{FD}}$ is achievable with UcbBwK algorithm (Sankararaman and Slivkins, 2021). In particular, the algorithm does not know in advance whether best-arm-optimality holds, and attains the optimal worst-case regret bound for all instances, best-arm-optimal or not. The instance-dependent parameter in this regret bound generalizes the *gap* from stochastic bandits (see Remark 1.1), call it *reward-gap* in the context of BwK. The definition uses Lagrange functions from Eq. (10.6):

$$G_{\texttt{LAG}}(a) := \texttt{OPT}_{\texttt{LP}} - \mathcal{L}(a, \lambda^*) \qquad \textit{(Lagrangian gap of arm } a\textit{)}, \tag{10.20}$$

where λ^* is a minimizer dual vector in Eq. (10.8), and $G_{\texttt{LAG}} := \min_{a \notin \{a^*, \texttt{null}\}} G_{\texttt{LAG}}(a)$. The regret bound scales as $O(K G_{\texttt{LAG}}^{-1} \log T)$, which is optimal in $G_{\texttt{LAG}}$, under a mild additional assumption that the expected consumption of the best arm is not very close to B/T. Otherwise, regret is $\mathcal{O}(K G_{\texttt{LAG}}^{-2} \log T)$.

While the $O(\log T)$ regret result is most meaningful as a part of the characterization, it also stands on its own, even though it requires $d = 2$, best-arm-optimality, and a reasonably small number of arms K.

[7] The regret bound is simply $O(T/B)$ times the state-of-art regret bound from Bubeck et al. (2017). Cardoso et al. (2018) obtain a similar regret bound for the full feedback version, in a simultaneous and independent work relative to Immorlica et al. (2022).

[8] The precise formulation of this lower bound is somewhat subtle. It starts with any problem instance $\mathcal{I}_0$ with three arms, under mild assumptions, such that either $d > 2$ or best-arm optimality fails with some margin. Then it constructs two problem instances $\mathcal{I}, \mathcal{I}'$ which are ϵ-perturbations of $\mathcal{I}_0$, $\epsilon = O(1/\sqrt{T})$, in the sense that expected rewards and resource consumptions of each arm differ by at most ϵ. The guarantee is that any algorithm suffers regret $\Omega(\sqrt{(T)})$ on either $\mathcal{I}$ or $\mathcal{I}'$.

Indeed, BwK problems with $d = 2$ and small K capture the three challenges of BwK from the discussion in Section 10.1 and, as spelled out in Sankararaman and Slivkins (2021, Appendix A), arise in many motivating applications. Moreover, best-arm-optimality is a typical, non-degenerate case, in some rigorous sense.

Other results on $O(\log T)$ regret. Several other results achieve $O(\log T)$ regret in BwK, with various assumptions and caveats, cutting across the characterization discussed above.

Wu et al. (2015) assume deterministic resource consumption, whereas all motivating examples of BwK require consumption to be stochastic, and in fact correlated with rewards (*e.g.,* dynamic pricing consumes supply only if a sale happens). They posit $d = 2$ and no other assumptions, whereas "best-arm optimality" is necessary for stochastic resource consumption.

Flajolet and Jaillet (2015) assume "best-arm-optimality" (it is implicit in the definition of their generalization of reward-gap). Their algorithm inputs an instance-dependent parameter which is normally not revealed to the algorithm (namely, an exact value of some continuous-valued function of mean rewards and consumptions). For $d = 2$, regret bounds for their algorithm scale with $c_{\min}$, minimal expected consumption among arms: as $c_{\min}^{-4}$ for the $O(\log T)$ bound, and as $c_{\min}^{-2}$ for the worst-case bound. Their analysis extents to $d > 2$, with regret $c_{\min}^{-4}\, K^K/\texttt{gap}^6$ and without a worst-case regret bound.

Vera et al. (2020) study a contextual version of BwK with two arms, one of which does nothing. This formulation is well-motivated for contextual BwK, but meaningless when specialized to BwK.

Li et al. (2021) do not make any assumptions, but use additional instance-dependent parameters (*i.e.,* other than their generalization of reward-gap). These parameters spike up and yield $\Omega(\sqrt{T})$ regret whenever the $\Omega(\sqrt{T})$ lower bounds from Sankararaman and Slivkins (2021) apply. Their algorithm does not appear to achieve any non-trivial regret bound in the worst case.

Finally, as mentioned before, György et al. (2007); Tran-Thanh et al. (2010, 2012); Ding et al. (2013); Rangi et al. (2019) posit only one constrained resource and $T = \infty$.

Simple regret, which tracks algorithm's performance in a given round, can be small in all but a few rounds. Like in stochastic bandits, simple regret can be at least ϵ in at most $\tilde{\mathcal{O}}(K/\epsilon^2)$ rounds, for all $\epsilon > 0$ simultaneously.[9] In fact, this is achieved by UcbBwK algorithm (Sankararaman and Slivkins, 2021).

Simple regret for BwK is defined, for a given round t, as $\texttt{OPT}/T - r(X_t)$, where X_t is the distribution over arms chosen by the algorithm in this round. The benchmark $\texttt{OPT}/T$ generalizes the best-arm benchmark from stochastic bandits. If each round corresponds to a user and the reward is this user's utility, then $\texttt{OPT}/T$ is the "fair share" of the total reward. Thus, with UcbBwK, all but a few users receive close to their fair share. This holds if $B > \Omega(T) \gg K$, without any other assumptions.

10.5.4 Adversarial bandits with knapsacks

In the adversarial version of BwK, each outcome matrices $\mathbf{M}_t$ is chosen by an adversary. Let us focus on the oblivious adversary, so that the entire sequence $\mathbf{M}_1, \mathbf{M}_2, \ldots, \mathbf{M}_T$ is fixed before round 1. All results below are from Immorlica et al. (2022), unless mentioned otherwise. The version with IID outcomes that we've considered before will be called *Stochastic BwK*.

Hardness of the problem. Adversarial BwK is a much harder problem compared to the stochastic version. The new challenge is that the algorithm needs to decide how much budget to save for the future, without being able to predict it. An algorithm must compete, during any given time segment $[1, \tau]$, with a distribution D_τ over arms that maximizes the total reward on this time segment. However, these distributions may be

[9] For stochastic bandits, this is implicit in the analysis in Chapter 1.3.2 and the original analysis of UCB1 algorithm in Auer et al. (2002a). This result is achieved along with the worst-case and logarithmic regret bounds.

very different for different τ. For example, one distribution D_τ may exhaust some resources by time τ, whereas another distribution $D_{\tau'}$, $\tau' > \tau$ may save some resources for later.

Due to this hardness, one can only approximate optimal rewards up to a multiplicative factor, whereas sublinear regret is no longer possible. To state this point more concretely, consider the ratio $\texttt{OPT}_{\texttt{FD}}/\mathbb{E}[\texttt{REW}]$, called *competitive ratio*, where $\texttt{OPT}_{\texttt{FD}}$ is the fixed-distribution benchmark (as defined in Section 10.1). A very strong lower bound holds: no algorithm can achieve competitive ratio better than $O(\log T)$ on all problem instances. The lower-bounding construction involves only two arms and only one resource, and forces the algorithm to make a huge commitment without knowing the future.

It is instructive to consider a simple example in which the competitive ratio is at least $\frac{5}{4} - o(1)$ for any algorithm. There are two arms and one resource with budget $T/2$. Arm 1 has zero rewards and zero consumption. Arm 2 has consumption 1 in each round, and offers reward $1/2$ in each round of the first half-time ($T/2$ rounds). In the second half-time, it offers either reward 1 in all rounds, or reward 0 in all rounds. Thus, there are two problem instances that coincide for the first half-time and differ in the second half-time. The algorithm needs to choose how much budget to invest in the first half-time, without knowing what comes in the second. Any choice leads to competitive ratio at least $5/4$ on one of the two instances.

A more elaborate version of this example, with $\Omega(T/B)$ phases rather than two, proves that competitive ratio can be no better than $\texttt{OPT}_{\texttt{FD}}/\mathbb{E}[\texttt{REW}] \geq \Omega(\log T/B)$ in the worst case.[10]

The best distribution is arguably a good benchmark for this problem. The best-algorithm benchmark is arguably *too harsh*: essentially, the ratio $\texttt{OPT}/\mathbb{E}[\texttt{REW}]$ cannot be better than T/B in the worst case.[11] The fixed-arm benchmark $\texttt{OPT}_{\texttt{FA}}$ can be arbitrarily worse compared to $\texttt{OPT}_{\texttt{FD}}$ (see Exercise 10.5). Moreover it is *uninteresting*: $\texttt{OPT}_{\texttt{FA}}/\mathbb{E}[\texttt{REW}] \geq \Omega(K)$ for some problem instances, matched by a trivial algorithm that chooses one arm uniformly at random and plays it forever.

Algorithmic results. One can achieve a near-optimal competitive ratio,

$$(\texttt{OPT}_{\texttt{FD}} - \texttt{reg})/\mathbb{E}[\texttt{REW}] \leq O_d(\log T), \tag{10.21}$$

up to a regret term $\texttt{reg} = O(1 + \frac{\texttt{OPT}_{\texttt{FD}}}{dB})\sqrt{TK\log(Td)}$. This is achieved using a version of LagrangeBwK algorithm, with two important differences: the time resource is not included in the outcome matrices, and the T/B ratio in the Lagrange function (10.10) is replaced by a parameter $\gamma \in (0, T/B]$, sampled at random from some exponential scale. When γ is sampled close to $\texttt{OPT}_{\texttt{FD}}/B$, the algorithm obtains a constant competitive ratio.[12] A completely new analysis of LagrangeBwK is needed, as the zero-games framework from Chapter 9 is no longer applicable. The initial analysis in Immorlica et al. (2022) obtains (10.21) with competitive ratio $\frac{d+1}{2}\ln T$. Kesselheim and Singla (2020) refine this analysis to (10.21) with competitive ratio $O\left(\log(d)\,\log(T)\right)$. They also prove that such competitive ratio is optimal up to constant factors.

One can also achieve an $O(d\log T)$ competitive ratio with high probability:

$$\Pr\left[\,(\texttt{OPT}_{\texttt{FD}} - \texttt{reg})/\texttt{REW} \leq O(d\log T)\,\right] \geq 1 - T^{-2}, \tag{10.22}$$

with a somewhat larger regret term reg. This result uses LagrangeBwK as a subroutine, and its analysis as a key lemma. The algorithm is considerably more involved: instead of guessing the parameter γ upfront, the guess is iteratively refined over time.

[10] More precisely, for any $B \leq T$ and any algorithm, there is a problem instance with budget B and time horizon T such that $\texttt{OPT}_{\texttt{FD}}/\mathbb{E}[\texttt{REW}] \geq \frac{1}{2}\ln\lceil T/B\rceil + \zeta - \tilde{O}(1/\sqrt{B})$, where $\zeta = 0.577...$ is the Euler-Masceroni constant.

[11] Balseiro and Gur (2019) construct this lower bound for dynamic bidding in second-price auctions (see Section 10.2). Their guarantee is as follows: for any time horizon T, any constants $0 < \gamma < \rho < 1$, and any algorithm there is a problem instance with budget $B = \rho T$ such that $\texttt{OPT}/\mathbb{E}[\texttt{REW}] \geq \gamma - o(T)$. The construction uses only $K = T/B$ distinct bids.

Immorlica et al. (2022) provide a simpler but weaker guarantee with only $K = 2$ arms: for any time horizon T, any budget $B < \sqrt{T}$ and any algorithm, there is a problem instance with $\texttt{OPT}/\mathbb{E}[\texttt{REW}] \geq T/B^2$.

[12] On instances with zero resource consumption, this algorithm achieves $\tilde{O}(\sqrt{KT})$ regret, for any choice of parameter γ.

It remains open, at the time of this writing, whether constant-factor competitive ratio is possible when $B > \Omega(T)$, whether against $\texttt{OPT}_{\texttt{FD}}$ or against OPT. Balseiro and Gur (2019) achieve competitive ratio T/B for dynamic bidding in second-price auctions, and derive a matching lower bound (see Footnote 11). Their positive result makes some convexity assumptions, and holds (even) against OPT.

LagrangeBwK reduction discussed in Section 10.5.1 applies to adversarial BwK as well. We immediately obtain extensions to the same settings as before: contextual BwK, combinatorial semi-BwK, BCO with knapsacks, and BwK with full feedback. For each setting, one obtains the competitive ratios in (10.21) and (10.22), with some problem-specific regret terms.

Kesselheim and Singla (2020) consider a more general version of the stopping rule: the algorithm stops at time t if $\|(C_{t,1}, \ldots, C_{t,d}\|_p > B$, where $p \geq 1$ and $C_{t,i}$ is the total consumption of resource i at time t. The case $p = \infty$ corresponds to BwK. They obtain the same competitive ratio, $O(\log(d)\,\log(T))$, using a version of LagrangeBwK with a different "dual" algorithm and a different analysis.

Problem variants with sublinear regret. Several results achieve sublinear regret, *i.e.,* a regret bound that is sublinear in T, via various simplifications.

Rangi et al. (2019) consider the special case when there is only one constrained resource, including time. They assume a known lower bound $c_{\min} > 0$ on realized per-round consumption of each resource, and their regret bound scales as $1/c_{\min}$. They also achieve polylog(T) instance-dependent regret for the stochastic version using the same algorithm (matching results from prior work on the stochastic version).

Several papers posit a relaxed benchmark: they only compare to distributions over actions which satisfy the time-averaged resource constraint *in every round*. Sun et al. (2017) handles BwK with $d = 2$ resources; their results extend to contextual BwK with policy sets, via a computationally inefficient algorithm. *Online convex optimization with constraints* (Mahdavi et al., 2012, 2013; Chen et al., 2017; Neely and Yu, 2017; Chen and Giannakis, 2018) assumes that the action set is a convex subset of $\mathbb{R}^m$, $m \in \mathbb{N}$, and in each round rewards are concave and consumption of each resource is convex (as functions from actions). In all this work, resource constraints only apply at the last round, and more-than-bandit feedback is observed.[13]

10.5.5 Paradigmaric application: Dynamic pricing with limited supply

In the basic version, the algorithm is a seller with B identical copies of some product, and there are T rounds. In each round $t \in [T]$, the algorithm chooses some price $p_t \in [0, 1]$ and offers one copy for sale at this price. The outcome is either a sale or no sale; the corresponding outcome vectors are shown in Eq. (10.2). The customer response is summarized by the probability of making a sale at a given price p, denoted $S(p)$, which is assumed to be the same in all rounds, and non-increasing in p.[14] The function $S(\cdot)$ is the *demand curve*, a well-known notion in Economics. The problem was introduced in Besbes and Zeevi (2009). The unconstrained case ($B = T$) is discussed in Section 4.4.4.

The problem can be solved via *discretization*: choose a finite subset $P \subset [0, 1]$ of prices, and run a generic BwK algorithm with action set P. The generic guarantees for BwK provide a regret bound against $\texttt{OPT}(P)$, the expected total reward of the best all-knowing algorithm restricted to prices in P. One needs to choose P so as to balance the BwK regret bound (which scales with $\sqrt{|P|}$) and the *discretization error* $\texttt{OPT} - \texttt{OPT}(P)$ (or a similar difference in the LP-values, whichever is more convenient). Bounding the discretization error is a new, non-trivial step, separate from the analysis of the BwK algorithm. With this approach, Badanidiyuru et al. (2013, 2018) achieve regret bound $\tilde{O}(B^{2/3})$ against OPT. Note that, up to

[13] Full feedback is observed for the resource consumption, and (except in Chen and Giannakis (2018)) the algorithm also observes either full feedback on rewards or the rewards gradient around the chosen action.

[14] In particular, suppose one customer arrives at time t, with private value v_t, and buys if and only if $v_t \geq p_t$. If v_t is drawn independently from some distribution D, then $S(p) = \Pr_{v \sim D}[v \geq p]$. Considering the sales probability directly is more general, *e.g.,* it allows for multiple customers to be present at a given round.

logarithmic factors, this regret bound is driven by B rather than T. This regret rate is optimal for dynamic pricing, for any given B, T: a complimentary $\Omega(B^{2/3})$ lower bound has been proved in Babaioff et al. (2015a), even against the best fixed price. Interestingly, the optimal regret rate is attained using a generic BwK algorithm: other than the choice of discretization P, the algorithm is not adapted to dynamic pricing.

Earlier work focused on competing with the best fixed price, *i.e.,* $\texttt{OPT}_{\texttt{FA}}$. Babaioff et al. (2015a) achieved $\tilde{O}(B^{2/3})$ regret against $\texttt{OPT}_{\texttt{FA}}$, along with the lower bound mentioned above. Their algorithm is a simplified version of UcbBwK: in each round, it chooses a price with a largest UCB on the expected total reward; this algorithm is run on a pre-selected subset of prices. The initial result from Besbes and Zeevi (2009) assumes $B > \Omega(T)$ and achieves regret $\tilde{O}(T^{3/4})$, using the explore-first technique (see Chapter 1).

Consider the same problem under *regular demands*, a standard assumption in theoretical economics which states that $R(p) = p \cdot S(p)$, the expected revenue at price p, is a concave function of p. Then the best fixed price is close to the best algorithm: $\texttt{OPT}_{\texttt{FA}} \geq \texttt{OPT} - \tilde{O}(\sqrt{B})$ (Yan, 2011). In particular, the $\tilde{O}(B^{2/3})$ regret bound from Babaioff et al. (2015a) carries over to OPT. Further, Babaioff et al. (2015a) achieve $\tilde{O}(c_S \cdot \sqrt{B})$ regret provided that $B/T \leq c'_S$, where c_S and $c'_S > 0$ are some positive constants determined by the demand curve S. They also provide a matching $\Omega(c_S \cdot \sqrt{T})$ lower bound, even if an S-dependent constant c_S is allowed in $\Omega(\cdot)$. A simultaneous and independent work, Wang et al. (2014) attains a similar upper bound via a different algorithm, under additional assumptions that $B > \Omega(T)$ and the demand curve S is Lipschitz. The initial result in Besbes and Zeevi (2009) achieved $\tilde{O}(T^{2/3})$ regret under the same assumptions. Wang et al. (2014) also proves a $\Omega(\sqrt{T})$ lower bound, but without an S-dependent constant.

Both lower bounds mentioned above, *i.e.,* $\Omega(B^{2/3})$ against $\texttt{OPT}_{\texttt{FA}}$ and $\Omega(c_S \cdot \sqrt{B})$ for regular demands, are proved in Babaioff et al. (2015a) via a reduction to the respective lower bounds from Kleinberg and Leighton (2003) for the unconstrained case ($B = T$). The latter lower bounds encapsulate most of the "heavy lifting" in the analysis.

Dynamic pricing with $n \geq 2$ products in the inventory is less understood. As in the single-product case, one can use discretization and run an optimal BwK algorithm on a pre-selected finite subset P of price vectors. If the demand curve is Lipschitz, one can bound the discretization error, and achieve regret rate on the order of $T^{(n+1)/(n+2)}$, see Exercise 10.3(a). However, it is unclear how to bound the discretization error without Lipschitz assumptions. The only known result in this direction is when there are multiple feasible bundles of goods, and in each round the algorithm chooses a bundle and a price. Then the technique from the single-product case applies, and one obtains a regret bound of $\tilde{O}\left(n\, B^{2/3}\, (N\ell)^{1/3}\right)$, where N is the number of bundles, each bundle consists of at most ℓ items, and prices are in the range $[0, \ell]$ (Badanidiyuru et al., 2013, 2018). Dynamic pricing with $n \geq 2$ products was first studied in (Besbes and Zeevi, 2012). They provide several algorithms with non-adaptive exploration for the regime when all budgets are $\Omega(T)$. In particular, they attain regret $\tilde{O}(T^{1-1/(n+3)})$ when demands are Lipschitz (as a function of prices) and expected revenue is concave (as a function of demands).

While this discussion focuses on regret-minimizing formulations of dynamic pricing, Bayesian and parametric formulations versions have a rich literature in Operations Research and Economics (Boer, 2015).

10.5.6 Rewards vs. costs

BwK as defined in this chapter does not readily extend to a version with costs rather than rewards. Indeed, it does not make sense to stop a cost-minimizing algorithm once it runs out of resources — because sich algorithm would seek high-consumption arms in order to stop early! So, a cost-minimizing version of BwK must require the algorithm to continue till the time horizon T. Likewise, an algorithm cannot be allowed to skip rounds (otherwise it would just skip *all* rounds). Consequently, the null arm — which is now an arm with maximal cost and no resource consumption — is not guaranteed to exist. In fact, this is the version of BwK studied in Sun et al. (2017) and the papers on online convex optimization (discussed in Section 10.5.4).

10.6 Exercises and hints

(Assume Stochastic BwK unless specified otherwise.)

Exercise 10.1 (Explore-first algorithm for BwK). Fix time horizon T and budget B. Consider an algorithm ALG which explores uniformly at random for the first N steps, where N is fixed in advance, then chooses some distribution D over arms and draws independently from this distribution in each subsequent rounds.

(a) Assume $B < \sqrt{T}$. Prove that there exists a problem instance on which ALG suffers linear regret:

$$\texttt{OPT} - \mathbb{E}[\texttt{REW}] > \Omega(T).$$

Hint: Posit one resource other than time, and three arms:

- the *bad arm*, with deterministic reward 0 and consumption 1;
- the *good arm*, with deterministic reward 1 and expected consumption $\frac{B}{T}$;
- the *decoy arm*, with deterministic reward 1 and expected consumption $2\frac{B}{T}$.

Use the following fact: given two coins with expectations $\frac{B}{T}$ and $\frac{B}{T} + c/\sqrt{N}$, for a sufficiently low absolute constant c, after only N tosses of each coin, for any algorithm it is a constant-probability event that this algorithm cannot tell one coin from another.

(b) Assume $B > \Omega(T)$. Choose N and D so that ALG achieves regret $\texttt{OPT} - \mathbb{E}[\texttt{REW}] < \tilde{O}(T^{2/3})$.

Hint: Choose D as a solution to the "optimistic" linear program (10.14), with rescaled budget $B' = B(1 - \sqrt{K/T})$. Compare D to the value of the original linear program (10.5) with budget B', and the latter to the value of (10.5) with budget B.

Exercise 10.2 (Best distribution vs. best fixed arm). Recall that $\texttt{OPT}_{\texttt{FA}}$ and $\texttt{OPT}_{\texttt{FD}}$ are, resp., the fixed-arm benchmark and the fixed-distribution benchmark, as defined in Section 10.1. Let d be the number of resources, including time.

(a) Construct an example such that $\texttt{OPT}_{\texttt{FD}} \geq d \cdot \texttt{OPT}_{\texttt{FA}} - o(\texttt{OPT}_{\texttt{FA}})$.

Hint: Extend the $d = 2$ example from Section 10.1.

(b) Prove that $\texttt{OPT} \leq d \cdot \texttt{OPT}_{\texttt{FA}}$.

Hint: (10.5) has an optimal solution with support size at most d. Use the pigeonhole principle!

Exercise 10.3 (Discretization in dynamic pricing). Consider dynamic pricing with limited supply of d products: actions are price vectors $p \in [0,1]^d$, and $c_i(p) \in [0,1]$ is the expected per-round amount of product i sold at price vector p. Let $P_\epsilon := [0,1]^d \cap \epsilon\,\mathbb{N}^d$ be a uniform mesh of prices with step $\epsilon \in (0,1)$. Let $\texttt{OPT}(P_\epsilon)$ and $\texttt{OPT}_{\texttt{FA}}(P_\epsilon)$ be the resp. benchmark restricted to the price vectors in P_ϵ.

(a) Assume that $c_i(p)$ is Lipschitz in p, for each product i:

$$|c_i(p) - c_i(p')| \leq L \cdot \|p - p\|_1, \quad \forall p, p' \in [0,1]^d.$$

Prove that the discretization error is $\texttt{OPT} - \texttt{OPT}(P_\epsilon) \leq O(\epsilon d L)$. Using an optimal BwK algorithm with appropriately action set P_ϵ, obtain regret rate $\texttt{OPT} - \mathbb{E}[\texttt{REW}] < \tilde{O}(T^{(d+1)/(d+2)})$.

Hint: To bound the discretization error, use the approach from Exercise 10.1; now the deviations in rewards/consumptions are due to the change in p.

(b) For the single-product case ($d = 1$), consider the fixed-arm benchmark, and prove that the resp. discretization error is $\texttt{OPT}_{\texttt{FA}} - \texttt{OPT}_{\texttt{FA}}(P_\epsilon) \leq O(\epsilon \sqrt{B})$.

Hint: Consider "approximate total reward" at price p as $V(p) = p \cdot \min(B, T \cdot S(p))$. Prove that the expected total reward for always using price p lies between $V(p) - \tilde{O}(\sqrt{B})$ and $V(p)$.

Exercise 10.4 (`LagrangeBwK` with zero resource consumption). Consider a special case of `BwK` with $d \geq 2$ resources and zero resource consumption. Prove that `LagrangeBwK` achieves regret $\tilde{O}(\sqrt{KT})$.

Hint: Instead of the machinery from Chapter 9, use the regret bound for the primal algorithm, and the fact that there exists an optimal solution for (10.5) with support size 1.

Exercise 10.5 ($\texttt{OPT}_{\texttt{FA}}$ for adversarial `BwK`). Consider Adversarial `BwK`. Prove that $\texttt{OPT}_{\texttt{FA}}$ can be arbitrarily worse than $\texttt{OPT}_{\texttt{FD}}$. Specifically, fix arbitrary time horizon T, budget $B < T/2$, and number of arms K, and construct a problem instance with $\texttt{OPT}_{\texttt{FA}} = 0$ and $\texttt{OPT}_{\texttt{FD}} > \Omega(T)$.

Hint: Make all arms have reward 0 and consumption 1 in the first B rounds.

Chapter 11

Bandits and Agents

In many scenarios, multi-armed bandit algorithms interact with self-interested parties, a.k.a. *agents*. The algorithm can affects agents' incentives, and agents' decisions in response to these incentives can influence the algorithm's objectives. We focus this chapter on a particular scenario, *incentivized exploration*, motivated by exploration in recommendation systems, and we survey some other scenarios in the literature review.

Prerequisites: Chapter 1; Chapter 2 (results only, just for perspective).

Consider a population of self-interested agents that make decisions under uncertainty. They *explore* to acquire new information and *exploit* this information to make good decisions. Collectively they need to balance these two objectives, but their incentives are skewed toward exploitation. This is because exploration is costly, but its benefits are spread over many agents in the future. Thus, we ask, *How to incentivize self-interested agents to explore when they prefer to exploit?*

Our motivation comes from recommendation systems. Users therein consume information from the previous users, and produce information for the future. For example, a decision to dine in a particular restaurant may be based on the existing reviews, and may lead to some new subjective observations about this restaurant. This new information can be consumed either directly (via a review, photo, tweet, etc.) or indirectly through aggregations, summarizations or recommendations, and can help others make similar choices in similar circumstances in a more informed way. This phenomenon applies very broadly, to the choice of a product or experience, be it a movie, hotel, book, home appliance, or virtually any other consumer's choice. Similar issues, albeit with higher stakes, arise in health and lifestyle decisions such as adjusting exercise routines or selecting a doctor or a hospital. Collecting, aggregating and presenting users' observations is a crucial value proposition of numerous businesses in the modern economy.

When self-interested individuals (*agents*) engage in the information-revealing decisions discussed above, individual and collective incentives are misaligned. If a social planner were to direct the agents, she would trade off exploration and exploitation so as to maximize the social welfare. However, when the decisions are made by the agents rather than enforced by the planner, each agent's incentives are typically skewed in favor of exploitation, as (s)he would prefer to benefit from exploration done by others. Therefore, the society as a whole may suffer from insufficient amount of exploration. In particular, if a given alternative appears suboptimal given the information available so far, however sparse and incomplete, then this alternative may remain unexplored forever (even though it may be the best).

Let us consider a simple example in which the agents fail to explore. Suppose there are two actions $a \in \{1, 2\}$ with deterministic rewards μ_1, μ_2 that are initially unknown. Each μ_a is drawn independently from a known Bayesian prior such that $\mathbb{E}[\mu_1] > \mathbb{E}[\mu_2]$. Agents arrive sequentially: each agent chooses an action, observes its reward and reveals it to all subsequent agents. Then the first agent chooses action 1 and reveals μ_1. If $\mu_1 > \mathbb{E}[\mu_2]$, then all future agents also choose arm 1. So, action 2 never gets chosen. This is very wasteful if the prior assigns a large probability to the event $\{\mu_2 \gg \mu_1 > \mathbb{E}[\mu_2]\}$.

Our problem, called **incentivized exploration**, asks how to incentivize the agents to explore. We consider a *principal* who cannot control the agents, but can communicate with them, *e.g.,* recommend an action and observe the outcome later on. Such a principal would typically be implemented via a website, either one dedicated to recommendations and feedback collection (*e.g.,* Yelp, Waze), or one that actually provides the product or experience being recommended (*e.g.,* Netflix, Amazon). While the principal would often be a for-profit company, its goal for our purposes would typically be well-aligned with the social welfare.

We posit that the principal creates incentives *only* via communication, rather monetary incentives such as rebates or discounts. Incentives arise due *information asymmetry*: the fact that the principal collects observations from the past agents and therefore has more information than any one agent. Accordingly, agents realize that (s)he may benefit from following the principal's recommendations, even these recommendations sometimes include exploration.

Incentivizing exploration is a non-trivial task even in the simple example described above, and even if there are only two agents. This is *Bayesian persuasion*, a well-studied problem in theoretical economics. When rewards are noisy and incentives are not an issue, the problem reduces to stochastic bandits, as studied in Chapter 1 . Essentially, incentivized exploration needs to solve both problems simultaneously. We design algorithms which create the desired incentives, and (with some caveats) match the performance of the optimal bandit algorithms.

11.1 Problem formulation: incentivized exploration

We capture the problem discussed above as a bandit problem with auxiliary constraints that arise due to incentives. The problem formulation has two distinct parts: the "algorithmic" part, which is essentially about Bayesian bandits, and the "economic" part, which is about agents' knowledge and incentives. Such "two-part" structure is very common in the area of *algorithmic economics*. Each part individually is very standard, according to the literature on bandits and theoretical economics, respectively. It is their combination that leads to a novel and interesting problem.

An algorithm (henceforth, the *principal*) interacts with self-interested decision-makers (henceforth, *agents*) over time. There are T rounds and K possible actions, a.k.a. arms; we use $[T]$ and $[K]$ to denote, resp., the set of all rounds and the set of all actions. In each round $t \in [T]$, the principal recommends an arm $\texttt{rec}_t \in [K]$. Then an agent arrives, observes the recommendation $\texttt{rec}_t$, chooses an arm a_t, receives a reward $r_t \in [0,1]$ for this arm, and leaves forever. Rewards come from a known parameterized family $(\mathcal{D}_x : \ x \in [0,1])$ of reward distributions such that $\mathbb{E}[\mathcal{D}_x] = x$. Specifically, each time a given arm a is chosen, the reward is realized as an independent draw from $\mathcal{D}_x$ with mean reward $x = \mu_a$. The mean reward vector $\mu \in [0,1]^K$ is drawn from a Bayesian prior $\mathcal{P}$. Prior $\mathcal{P}$ are known, whereas μ is not.

Problem protocol: Incentivized exploration

Parameters: K arms, T rounds, common prior $\mathcal{P}$, reward distributions $(\mathcal{D}_x : \ x \in [0,1])$.

Initialization: the mean rewards vector $\mu \in [0,1]^K$ is drawn from the prior $\mathcal{P}$.

In each round $t = 1, 2, 3, \ \ldots \ , T$:

1. Algorithm chooses its recommended arm $\texttt{rec}_t \in [K]$.
2. Agent t arrives, receives recommendation $\texttt{rec}_t$, and chooses arm $a_t \in [K]$.
3. (Agent's) reward $r_t \in [0,1]$ is realized as an independent draw from D_x, where $x = \mu_{a_t}$.
4. Action a_t and reward r_t are observed by the algorithm.

Remark 11.1. We allow *correlated priors*, *i.e.,* random variables $\mu_a : a \in [K]$ can be correlated. An important special case is *independent priors*, when these random variables are mutually independent.

Remark 11.2. If all agents are guaranteed to *comply*, *i.e.,* follow the algorithm's recommendations, then the problem protocol coincides with Bayesian bandits, as defined in Chapter 3.

What does agent t know before (s)he chooses an action? Like the algorithm, (s)he knows the parameterized reward distribution $(\mathcal{D}_x)$ and the prior $\mathcal{P}$, but not the mean reward vector μ. Moreover, (s)he knows the principal's recommendation algorithm, the recommendation $\texttt{rec}_t$, and the round t. However, (s)he does not observe what happened with the previous agents.

Remark 11.3. All agents share the same beliefs about the mean rewards (as expressed by the prior $\mathcal{P}$), and these beliefs are *correct*, in the sense that μ is actually drawn from $\mathcal{P}$. While idealized, these two assumptions are very common in theoretical economics.

For each agent t, we put forward a constraint that compliance is in this agent's best interest. We condition on the event that a particular arm a is being recommended, and the event that all previous agents have complied. The latter event, denoted $\mathcal{E}_{t-1} = \{ a_s = \texttt{rec}_s : s \in [t-1] \}$, ensures that an agent has well-defined beliefs about the behavior of the previous agents.

Definition 11.4. An algorithm is called *Bayesian incentive-compatible* (*BIC*) if for all rounds t we have

$$\mathbb{E}\left[\mu_a - \mu_{a'} \mid \texttt{rec}_t = a,\ \mathcal{E}_{t-1} \right] \geq 0, \tag{11.1}$$

where a, a' are any two distinct arms such that $\Pr\left[\texttt{rec}_t = a,\ \mathcal{E}_{t-1} \right] > 0$.

We are (only) interested in BIC algorithms. We posit that all agents comply with such algorithm's recommendations. Accordingly, a BIC algorithm is simply a bandit algorithm with an auxiliary BIC constraint.

Remark 11.5. The definition of BIC follows one of the standard paradigms in theoretical economics: identify the desirable behavior (in our case, following algorithm's recommendations), and require that this behavior maximizes each agent's expected reward, according to her beliefs. Further, to define the agents' beliefs, posit that all uncertainty is realized as a random draw from a Bayesian prior, that the prior and the principal's algorithm are known to the agents, and that all previous agents follow the desired behavior.

Algorithm's objective is to maximize the total reward over all rounds. A standard performance measure is *Bayesian regret*, *i.e.,* pseudo-regret in expectation over the Bayesian prior. We are also interested in comparing BIC bandit algorithms with *optimal* bandit algorithms.

Preliminaries. We focus the technical developments on the special case of $K = 2$ arms (which captures much of the complexity of the general case). Let $\mu_a^0 = \mathbb{E}[\mu_a]$ denote the prior mean reward for arm a. W.l.o.g., $\mu_1^0 \geq \mu_2^0$, *i.e.,* arm 1 is (weakly) preferred according to the prior. For a more elementary exposition, let us assume that the realized rewards of each arm can only take finitely many possible values.

Let $\mathcal{S}_{1,n}$ denote an ordered tuple of n independent samples from arm 1. (Equivalently, $\mathcal{S}_{1,n}$ comprises the first n samples from arm 1.) Let $\texttt{AVG}_{1,n}$ be the average reward in these n samples.

Throughout this chapter, we use a more advanced notion of conditional expectation given a random variable. Letting X be a real-valued random variable, and let Y be another random variable with an arbitrary (not necessarily real-valued) but finite support $\mathcal{Y}$. The conditional expectation of X given Y is a itself a random variable, $\mathbb{E}[X \mid Y] := F(Y)$, where $F(y) = \mathbb{E}[X \mid Y = y]$ for all $y \in \mathcal{Y}$. The conditional expectation given an event E can be expressed as $\mathbb{E}[X|E] = \mathbb{E}[X|\mathbf{1}_E]$. We are particularly interested in $\mathbb{E}[\cdot \mid \mathcal{S}_{1,n}]$, the posterior mean reward after n samples from arm 1.

We will repeatedly use the following fact, a version of the *law of iterated expectation*.

Fact 11.6. *Suppose random variable Z is determined by Y and some other random variable Z_0 such that X and Z_0 are independent (think of Z_0 as algorithm's random seed). Then* $\mathbb{E}[\, \mathbb{E}[X|Y] \mid Z] = \mathbb{E}[X|Z]$.

11.2 How much information to reveal?

How much information should the principal reveal to the agents? Consider two extremes: recommending an arm without providing any supporting information, and revealing the entire history. We argue that the former suffices, whereas the latter does not work.

Recommendations suffice. Let us consider a more general model: in each round t, an algorithm ALG sends the t-th agent an arbitrary message σ_t, which includes a recommended arm $\texttt{rec}_t \in [K]$. The message lies in some fixed, but otherwise arbitrary, space of possible messages; to keep exposition elementary, assume that this space is finite. A suitable BIC constraint states that recommendation $\texttt{rec}_t$ is optimal given message σ_t and compliance of the previous agents. In a formula,

$$\texttt{rec}_t \in \operatorname*{argmax}_{a\in[K]} \mathbb{E}\left[\, \mu_a \mid \sigma_t,\ \mathcal{E}_{t-1}\,\right], \qquad \forall t \in [T].$$

Given an algorithm ALG as above, consider another algorithm ALG$'$ which only reveals recommendation $\texttt{rec}_t$ in each round t. It is easy to see that ALG$'$ is BIC, as per (11.1). Indeed, fix round t and arm a such that $\Pr\left[\,\texttt{rec}_t = a,\ \mathcal{E}_{t-1}\,\right] > 0$. Then

$$\mathbb{E}\left[\, \mu_a - \mu_{a'} \mid \sigma_t,\ \texttt{rec}_t = a,\ \mathcal{E}_{t-1}\,\right] \geq 0 \qquad \forall a' \in [K].$$

We obtain (11.1) by integrating out the message σ_t, *i.e.,* by taking conditional expectation of both sides given $\{\texttt{rec}_t = a,\ \mathcal{E}_{t-1}\}$; formally, (11.1) follows by Fact 11.6.

Thus, it suffices to issue recommendations, without any supporting information. This conclusion, along with the simple argument presented above, is a version of a well-known technique from theoretical economics called Myerson's *direct revelation principle*. While surprisingly strong, it relies on several subtle assumptions implicit in our model. We discuss these issues more in Section 11.6.

Full revelation does not work. Even though recommendations suffice, does the principal need to bother designing and deploying a bandit algorithm? In theoretical terms, does the principal need to *explore*? An appealing alternative is to reveal the full history, perhaps along with some statistics, and let the agents choose for themselves. Being myopic, the agents would follow the *Bayesian-greedy* algorithm, a Bayesian version of the "greedy" bandit algorithm which always "exploits" and never "explores".

Formally, suppose in each round t, the algorithm reveals a message σ_t which includes the history, $H_t = \{(a_s, r_s) : s \in [t-1]\}$. Posterior mean rewards are determined by H_t:

$$\mathbb{E}[\mu_a \mid \sigma_t] = \mathbb{E}[\mu_a \mid H_t] \quad \text{for all arms } a,$$

because the rest of the message can only be a function of H_t, the algorithm's random seed, and possibly other inputs that are irrelevant. Consequently, agent t chooses an arm

$$a_t \in \operatorname*{argmax}_{a\in[K]} \mathbb{E}\left[\, \mu_a \mid H_t\,\right]. \tag{11.2}$$

Up to tie-breaking, this defines an algorithm, which we call GREEDY. We make no assumptions on how the ties are broken, and what else is included in algorithm's messages. In contrast with (11.1), the expectation in (11.2) is well-defined without $\mathcal{E}_{t-1}$ or any other assumption about the choices of the previous agents, because these choices are already included in the history H_t.

GREEDY performs terribly on a variety of problem instances, suffering Bayesian regret $\Omega(T)$. (Recall that bandit algorithms can achieve regret $\tilde{O}(\sqrt{T})$ on all problem instances, as per Chapter 1.) The root cause of this inefficiency is that GREEDY may never try arm 2. For the special case of deterministic rewards, this happens with probability $\Pr[\mu_1 \leq \mu_2^0]$, since μ_1 is revealed in round 1 and arm 2 is never chosen if $\mu_1 \leq \mu_2^0$. With a different probability, this result carries over to the general case.

Theorem 11.7. *With probability at least $\mu_1^0 - \mu_2^0$,* GREEDY *never chooses arm 2.*

Proof. In each round t, the key quantity is $Z_t = \mathbb{E}[\mu_1 - \mu_2 \mid H_t]$. Indeed, arm 2 is chosen if and only if $Z_t < 0$. Let τ be the first round when GREEDY chooses arm 2, or $T+1$ if this never happens. We use martingale techniques to prove that

$$\mathbb{E}[Z_\tau] = \mu_1^0 - \mu_2^0. \tag{11.3}$$

We obtain Eq. (11.3) via a standard application of the optional stopping theorem; it can be skipped by readers who are not familiar with martingales. We observe that τ is a *stopping time* relative to the sequence $\mathcal{H} = (\, H_t : t \in [T+1] \,)$, and $(\, Z_t : t \in [T+1] \,)$ is a martingale relative to $\mathcal{H}$. [1] The optional stopping theorem asserts that $\mathbb{E}[Z_\tau] = \mathbb{E}[Z_1]$ for any martingale Z_t and any bounded stopping time τ. Eq. (11.3) follows because $\mathbb{E}[Z_1] = \mu_1^0 - \mu_2^0$.

On the other hand, by Bayes' theorem it holds that

$$\mathbb{E}[Z_\tau] = \Pr[\tau \leq T]\ \mathbb{E}[Z_\tau \mid \tau \leq T] + \Pr[\tau > T]\ \mathbb{E}[Z_\tau \mid \tau > T] \tag{11.4}$$

Recall that $\tau \leq T$ implies that GREEDY chooses arm 2 in round τ, which in turn implies that $Z_\tau \leq 0$ by definition of GREEDY. It follows that $\mathbb{E}[Z_\tau \mid \tau \leq T] \leq 0$. Plugging this into Eq. (11.4), we find that

$$\mu_1^0 - \mu_2^0 = \mathbb{E}[Z_\tau] \leq \Pr[\tau > T].$$

And $\{\tau > T\}$ is precisely the event that GREEDY never tries arm 2. □

This is a very general result: it holds for arbitrary priors. Under some mild assumptions, the algorithm never tries arm 2 *when it is in fact the best arm*, leading to $\Omega(T)$ Bayesian regret.

Corollary 11.8. *Consider independent priors such that $\Pr[\mu_1 = 1] < (\mu_1^0 - \mu_2^0)/2$. Pick any $\alpha > 0$ such that $\Pr[\mu_1 \geq 1 - 2\,\alpha] \leq (\mu_1^0 - \mu_2^0)/2$. Then* GREEDY *suffers Bayesian regret*

$$\mathbb{E}[R(T)] \geq T \cdot \left(\tfrac{\alpha}{2}\ (\mu_1^0 - \mu_2^0)\ \Pr[\mu_2 > 1 - \alpha] \right).$$

Proof. Let $\mathcal{E}_1$ be the event that $\mu_1 < 1 - 2\alpha$ and GREEDY never chooses arm 2. By Theorem 11.7 and the definition of α, we have $\Pr[\mathcal{E}_1] \geq (\mu_1^0 - \mu_2^0)/2$.

Let $\mathcal{E}_2$ be the event that $\mu_2 > 1 - \alpha$. Under event $\mathcal{E}_1 \cap \mathcal{E}_2$, each round contributes $\mu_2 - \mu_1 \geq \alpha$ to regret, so $\mathbb{E}\,[\, R(T) \mid \mathcal{E}_1 \cap \mathcal{E}_2 \,] \geq \alpha\, T$. Since event $\mathcal{E}_1$ is determined by the draw of μ_1 and the realized rewards of arm 1, it is independent from $\mathcal{E}_2$. It follows that

$$\begin{aligned} \mathbb{E}[R(T)] &\geq \mathbb{E}[R(T) \mid \mathcal{E}_1 \cap \mathcal{E}_2] \cdot \Pr[\mathcal{E}_1 \cap \mathcal{E}_2] \\ &\geq \alpha T \cdot (\mu_1^0 - \mu_2^0)/2 \cdot \Pr[\mathcal{E}_2]. \end{aligned}$$

□

Here's a less quantitative but perhaps cleaner implication:

Corollary 11.9. *Consider independent priors. Assume that each arm's prior has a positive density,* i.e., *for each arm a, the prior on $\mu_a \in [0,1]$ has probability density function that is strictly positive on $[0,1]$. Then* GREEDY *suffers Bayesian regret at least $c_{\mathcal{P}} \cdot T$, where the constant $c_{\mathcal{P}} > 0$ depends only on the prior $\mathcal{P}$.*

[1] The latter follows from a general fact that sequence $\mathbb{E}[X \mid H_t]$, $t \in [T+1]$ is a martingale w.r.t. $\mathcal{H}$ for any random variable X with $\mathbb{E}\,[\,|X|\,] \infty$. It is known as *Doob martingale* for X.

11.3 Basic technique: hidden exploration

The basic technique to ensure incentive-compatibility is to *hide a little exploration in a lot of exploitation*. Focus on a single round of a bandit algorithm. Suppose we observe a realization of some random variable $\texttt{sig} \in \Omega_{\texttt{sig}}$, called the *signal*.[2] With a given probability $\epsilon > 0$, we recommend an arm what we actually want to choose, as described by the (possibly randomized) *target function* $a_{\texttt{trg}} : \Omega_{\texttt{sig}} \to \{1, 2\}$. The basic case is just choosing arm $a_{\texttt{trg}} = 2$. With the remaining probability we *exploit*, *i.e.,* choose an arm that maximizes $\mathbb{E}\left[\, \mu_a \mid \texttt{sig} \,\right]$. Thus, the technique, called `HiddenExploration`, is as follows:

Parameters: probability $\epsilon > 0$, function $a_{\texttt{trg}} : \Omega_{\texttt{sig}} \to \{1, 2\}$.
Input: signal realization $S \in \Omega_{\texttt{sig}}$.
Output: recommended arm `rec`.

With probability $\epsilon > 0$, `// exploration branch`
  $\texttt{rec} \leftarrow a_{\texttt{trg}}(S)$
else `// exploitation branch`
  $\texttt{rec} \leftarrow \min\left(\arg\max_{a \in \{1,2\}} \mathbb{E}\left[\, \mu_a \mid \texttt{sig} = S \,\right]\right)$ `// tie ⇒ choose arm 1`

Algorithm 11.1: `HiddenExploration` with signal `sig`.

We are interested in the (single-round) BIC property: for any two distinct arms a, a'

$$\Pr[\texttt{rec} = a] > 0 \;\;\Rightarrow\;\; \mathbb{E}\left[\, \mu_a - \mu_{a'} \mid \texttt{rec} = a \,\right] \geq 0 \tag{11.5}$$

We prove that `HiddenExploration` satisfies this property when the exploration probability ϵ is sufficiently small, so that the exploration branch is offset by exploitation. A key quantity here is a random variable which summarizes the meaning of signal `sig`:

$$G := \mathbb{E}[\mu_2 - \mu_1 \mid \texttt{sig}] \qquad \textit{(posterior gap)}.$$

Lemma 11.10. *Algorithm 11.1 is BIC, for any target function $a_{\texttt{trg}}$, as long as $\epsilon \leq \frac{1}{3}\, \mathbb{E}\left[G \cdot \mathbf{1}_{\{\, G>0 \,\}}\right]$.*

Remark 11.11. A suitable $\epsilon > 0$ exists if and only if $\Pr[G > 0] > 0$. Indeed, if $\Pr[G > 0] > 0$ then $\Pr[G > \delta] = \delta' > 0$ for some $\delta > 0$, so

$$\mathbb{E}\left[G \cdot \mathbf{1}_{\{\, G>0 \,\}}\right] \geq \mathbb{E}\left[G \cdot \mathbf{1}_{\{\, G>\delta \,\}}\right] = \Pr[G > \delta] \cdot \mathbb{E}[G \mid G > \delta] \geq \delta \cdot \delta' > 0.$$

The rest of this section proves Lemma 11.10. We start with an easy observation: for any algorithm, it suffices to guarantee the BIC property when arm 2 is recommended.

Claim 11.12. *Assume (11.5) holds for arm $\texttt{rec} = 2$. Then it also holds for $\texttt{rec} = 1$.*

Proof. If arm 2 is never recommended, then the claim holds trivially since $\mu_1^0 \geq \mu_2^0$. Now, suppose both arms are recommended with some positive probability. Then

$$0 \geq \mathbb{E}[\mu_2 - \mu_1] = \textstyle\sum_{a \in \{1,2\}} \mathbb{E}[\mu_2 - \mu_1 \mid \texttt{rec} = a] \Pr[\texttt{rec} = a].$$

Since $\mathbb{E}[\mu_2 - \mu_1 \mid \texttt{rec} = 2] > 0$ by the BIC assumption, $\mathbb{E}[\mu_2 - \mu_1 \mid \texttt{rec} = 1] < 0$. □

[2] Think of `sig` as the algorithm's history, but it is instructive to keep presentation abstract. For elementary exposition, we assume that the universe $\Omega_{\texttt{sig}}$ is finite. Otherwise we require a more advanced notion of conditional expectation.

Thus, we need to prove (11.5) for $\texttt{rec} = 2$, *i.e.,* that

$$\mathbb{E}\left[\,\mu_2 - \mu_1 \mid \texttt{rec} = 2\,\right] > 0. \tag{11.6}$$

(We note that $\Pr[\mu_2 - \mu_1] > 0$, *e.g.,* because $\Pr[G > 0] > 0$, as per Remark 11.11).

Denote the event $\{\texttt{rec} = 2\}$ with $\mathcal{E}_2$. By Fact 11.6, $\mathbb{E}\left[\,\mu_2 - \mu_1 \mid \mathcal{E}_2\,\right] = \mathbb{E}[G \mid \mathcal{E}_2]$. [3]

We focus on the posterior gap G from here on. More specifically, we work with expressions of the form $F(\mathcal{E}) := \mathbb{E}\left[\,G \cdot \mathbf{1}_{\mathcal{E}}\,\right]$, where $\mathcal{E}$ is some event. Proving Eq. (11.6) is equivalent to proving that $F(\mathcal{E}_2) > 0$; we prove the latter in what follows.

We will use the following fact:

$$F(\mathcal{E} \cup \mathcal{E}') = F(\mathcal{E}) + F(\mathcal{E}') \quad \text{for any disjoint events } \mathcal{E}, \mathcal{E}'. \tag{11.7}$$

Letting $\mathcal{E}_{\texttt{explore}}$ (resp., $\mathcal{E}_{\texttt{exploit}}$) be the event that the algorithm chooses exploration branch (resp., exploitation branch), we can write

$$F(\mathcal{E}_2) = F(\mathcal{E}_{\texttt{explore}} \text{ and } \mathcal{E}_2) + F(\mathcal{E}_{\texttt{exploit}} \text{ and } \mathcal{E}_2). \tag{11.8}$$

We prove that this expression is non-negative by analyzing the exploration and exploitation branches separately. For the exploitation branch, the events $\{\mathcal{E}_{\texttt{exploit}} \text{ and } \mathcal{E}_2\}$ and $\{\mathcal{E}_{\texttt{exploit}} \text{ and } G > 0\}$ are the same by algorithm's specification. Therefore,

$$\begin{aligned}
F(\mathcal{E}_{\texttt{exploit}} \text{ and } \mathcal{E}_2) &= F(\mathcal{E}_{\texttt{exploit}} \text{ and } G > 0) \\
&= \mathbb{E}[G \mid \mathcal{E}_{\texttt{exploit}} \text{ and } G > 0] \cdot \Pr[\mathcal{E}_{\texttt{exploit}} \text{ and } G > 0] && \textit{(by definition of } F\textit{)} \\
&= \mathbb{E}[G \mid G > 0] \cdot \Pr[G > 0] \cdot (1 - \epsilon) && \textit{(by independence)} \\
&= (1 - \epsilon) \cdot F(G > 0) && \textit{(by definition of } F\textit{)}.
\end{aligned}$$

For the exploration branch, recall that $F(\mathcal{E})$ is non-negative for any event $\mathcal{E}$ with $G \geq 0$, and non-positive for any event $\mathcal{E}$ with $G \leq 0$. Therefore,

$$\begin{aligned}
F(\mathcal{E}_{\texttt{explore}} \text{ and } \mathcal{E}_2) &= F(\mathcal{E}_{\texttt{explore}} \text{ and } \mathcal{E}_2 \text{ and } G < 0) + F(\mathcal{E}_{\texttt{explore}} \text{ and } \mathcal{E}_2 \text{ and } G \geq 0) && \textit{(by (11.7))} \\
&\geq F(\mathcal{E}_{\texttt{explore}} \text{ and } \mathcal{E}_2 \text{ and } G < 0) \\
&= F(\mathcal{E}_{\texttt{explore}} \text{ and } G < 0) - F(\mathcal{E}_{\texttt{explore}} \text{ and } \neg\mathcal{E}_2 \text{ and } G < 0) && \textit{(by (11.7))} \\
&\geq F(\mathcal{E}_{\texttt{explore}} \text{ and } G < 0) \\
&= \mathbb{E}[G \mid \mathcal{E}_{\texttt{explore}} \text{ and } G < 0] \cdot \Pr[\mathcal{E}_{\texttt{explore}} \text{ and } G < 0] && \textit{(by defn. of } F\textit{)} \\
&= \mathbb{E}[G \mid G < 0] \cdot \Pr[G < 0] \cdot \epsilon && \textit{(by independence)} \\
&= \epsilon \cdot F(G < 0) && \textit{(by defn. of } F\textit{)}.
\end{aligned}$$

Putting this together and plugging into (11.8), we have

$$F(\mathcal{E}_2) \geq \epsilon \cdot F(G < 0) + (1 - \epsilon) \cdot F(G > 0). \tag{11.9}$$

Now, applying (11.7) yet again we see that $F(G < 0) + F(G > 0) = \mathbb{E}[\mu_2 - \mu_1]$. Plugging this back into (11.9) and rearranging, it follows that $F(\mathcal{E}_2) > 0$ whenever

$$F(G > 0) > \epsilon\left(\,2F(G > 0) + \mathbb{E}[\mu_1 - \mu_2]\,\right).$$

In particular, $\epsilon < \frac{1}{3} \cdot F(G > 0)$ suffices. This completes the proof of Lemma 11.10.

[3] This is the only step in the analysis where it is essential that both the exploration and exploitation branches (and therefore event $\mathcal{E}_2$) are determined by the signal $\texttt{sig}$.

11.4 Repeated hidden exploration

Let us develop the hidden exploration technique into an algorithm for incentivized exploration. We take an arbitrary bandit algorithm ALG, and consider a repeated version of HiddenExploration (called RepeatedHE), where the exploration branch executes one call to ALG. We interpret calls to ALG as exploration. To get started, we include N_0 rounds of "initial exploration", where arm 1 is chosen. The exploitation branch conditions on the history of all previous exploration rounds:

$$\mathcal{S}_t = (\,(s, a_s, r_s) : \text{all exploration rounds } s < t\,). \tag{11.10}$$

Parameters: $N_0 \in \mathbb{N}$, exploration probability $\epsilon > 0$
In the first N_0 rounds, recommend arm 1. `// initial exploration`
In each subsequent round t,
  With probability ϵ `// explore`
    call ALG, let $\texttt{rec}_t$ be the chosen arm, feed reward r_t back to ALG.
  else `// exploit`
    $\texttt{rec}_t \leftarrow \min\big(\arg\max_{a\in\{1,2\}} \mathbb{E}[\mu_a \mid \mathcal{S}_t]\big)$. `// S_t from (11.10)`

Algorithm 11.2: RepeatedHE with bandit algorithm ALG.

Remark 11.13. RepeatedHE can be seen as a reduction from bandit algorithms to BIC bandit algorithms. The simplest version always chooses arm 2 in exploration rounds, and (only) provides non-adaptive exploration. For better regret bounds, ALG needs to perform adaptive exploration, as per Chapter 1.

Each round $t > N_0$ can be interpreted as HiddenExploration with signal $\mathcal{S}_t$, where the "target function" executes one round of algorithm ALG. Note that $\texttt{rec}_t$ is determined by $\mathcal{S}_t$ and the random seed of ALG, as required by the specification of HiddenExploration. Thus, Lemma 11.10 applies, and yields the following corollary in terms of $G_t = \mathbb{E}[\mu_2 - \mu_1 \mid \mathcal{S}_t]$, the posterior gap given signal $\mathcal{S}_t$.

Corollary 11.14. RepeatedHE *is BIC if* $\epsilon < \frac{1}{3}\,\mathbb{E}\left[G_t \cdot \mathbf{1}_{\{G_t>0\}}\right]$ *for each time* $t > N_0$.

For the final BIC guarantee, we show that it suffices to focus on $t = N_0 + 1$.

Theorem 11.15. RepeatedHE *with exploration probability* $\epsilon > 0$ *and* N_0 *initial samples of arm* 1 *is BIC as long as* $\epsilon < \frac{1}{3}\,\mathbb{E}\left[G \cdot \mathbf{1}_{\{G>0\}}\right]$, *where* $G = G_{N_0+1}$.

Proof. The only remaining piece is the claim that the quantity $\mathbb{E}\left[G_t \cdot \mathbf{1}_{\{G_t>0\}}\right]$ does not decrease over time. This claim holds for any sequence of signals $(\mathcal{S}_1, \mathcal{S}_2, \ldots, S_T)$ such that each signal S_t is determined by the next signal S_{t+1}.

Fix round t. Applying Fact 11.6 twice, we obtain

$$\mathbb{E}[G_t \mid G_t > 0] = \mathbb{E}[\mu_2 - \mu_1 \mid G_t > 0] = \mathbb{E}[G_{t+1} \mid G_t > 0].$$

(The last equality uses the fact that S_{t+1} determines S_t.) Then,

$$\begin{aligned}
\mathbb{E}\left[G_t \cdot \mathbf{1}_{\{G_t>0\}}\right] &= \mathbb{E}[G_t \mid G_t > 0] \cdot \Pr[G_t > 0] \\
&= \mathbb{E}[G_{t+1} \mid G_t > 0] \cdot \Pr[G_t > 0] \\
&= \mathbb{E}\left[G_{t+1} \cdot \mathbf{1}_{\{G_t>0\}}\right] \\
&\leq \mathbb{E}\left[G_{t+1} \cdot \mathbf{1}_{\{G_{t+1}>0\}}\right].
\end{aligned}$$

The last inequality holds because $x \cdot \mathbf{1}_{\{\cdot\}} \leq x \cdot \mathbf{1}_{\{x>0\}}$ for any $x \in R$. □

Remark 11.16. The theorem focuses on the posterior gap G given N_0 initial samples from arm 1. The theorem requires parameters $\epsilon > 0$ and N_0 to satisfy some condition that depends only on the prior. Such parameters exist if and only if $\Pr[G > 0] > 0$ for some N_0 (for precisely the same reason as in Remark 11.11). The latter condition is in fact necessary, as we will see in Section 11.5.

Performance guarantees for RepeatedHE completely separated from the BIC guarantee, in terms of results as well as proofs. Essentially, RepeatedHE learns at least as fast as an appropriately slowed-down version of ALG. There are several natural ways to formalize this, in line with the standard performance measures for multi-armed bandits. For notation, let $\texttt{REW}^{\texttt{ALG}}(n)$ be the total reward of ALG in the first n rounds of its execution, and let $\texttt{BR}^{\texttt{ALG}}(n)$ be the corresponding Bayesian regret.

Theorem 11.17. *Consider* RepeatedHE *with exploration probability $\epsilon > 0$ and N_0 initial samples. Let N be the number of exploration rounds $t > N_0$.* [4] *Then:*

(a) If ALG *always chooses arm* 2*,* RepeatedHE *chooses arm* 2 *at least N times.*

(b) The expected reward of RepeatedHE *is at least* $\frac{1}{\epsilon}\ \mathbb{E}\left[\ \texttt{REW}^{\texttt{ALG}}(N)\ \right]$.

(c) Bayesian regret of RepeatedHE *is* $\texttt{BR}(T) \leq N_0 + \frac{1}{\epsilon}\ \mathbb{E}\left[\ \texttt{BR}^{\texttt{ALG}}(N)\ \right]$.

Proof Sketch. Part (a) is obvious. Part (c) trivially follows from part (b). The proof of part (b) invokes Wald's identify and the fact that the expected reward in "exploitation" is at least as large as in "exploration" for the same round. □

Remark 11.18. We match the Bayesian regret of ALG up to factors N_0, $\frac{1}{\epsilon}$, which depend only on the prior $\mathcal{P}$ (and not on the time horizon or the realization of the mean rewards). In particular, we can achieve $\tilde{O}(\sqrt{T})$ regret for all problem instances, *e.g.,* using algorithm UCB1 from Chapter 1. If a smaller regret rate $f(T) = o(T)$ is achievable for a given prior using some other algorithm ALG, we can match it, too. However, the prior-dependent factors can be arbitrarily large, depending on the prior.

11.5 A necessary and sufficient assumption on the prior

We need to restrict the prior $\mathcal{P}$ so as to give the algorithm a fighting chance to convince some agents to try arm 2. (Recall that $\mu_1^0 \geq \mu_2^0$.) Otherwise the problem is just hopeless. For example, if μ_1 and $\mu_1 - \mu_2$ are independent, then samples from arm 1 have no bearing on the conditional expectation of $\mu_1 - \mu_2$, and therefore cannot possibly incentivize any agent to try arm 2.

We posit that arm 2 *can* appear better after seeing sufficiently many samples of arm 1. Formally, we consider the posterior gap given n samples from arm 1:

$$G_{1,n} := \mathbb{E}[\,\mu_2 - \mu_1 \mid \mathcal{S}_{1,n}], \tag{11.11}$$

where $\mathcal{S}_{1,n}$ denotes an ordered tuple of n independent samples from arm 1. We focus on the property that this random variable can be positive:

$$\Pr\left[G_{1,n} > 0\right] > 0 \quad \text{for some prior-dependent constant } n = n_{\mathcal{P}} < \infty. \tag{11.12}$$

For independent priors, this property can be simplified to $\Pr[\mu_2^0 > \mu_1] > 0$. Essentially, this is because $G_{1,n} = \mu_2^0 - \mathbb{E}[\mu_1 \mid \mathcal{S}_{1,n}]$ converges to $\mu_2^0 - \mu_1$ as $n \to \infty$.

Recall that Property (11.12) is sufficient for RepeatedHE, as per Remark 11.16. We prove that it is necessary for BIC bandit algorithms.

[4] Note that $\mathbb{E}[N] = \epsilon(T - N_0)$, and $|N - \mathbb{E}[N]| \leq O(\sqrt{T \log T})$ with high probability.

Theorem 11.19. *Suppose ties in Definition 11.4 are always resolved in favor of arm* 1 *(*i.e., *imply* $\texttt{rec}_t = 1$*). Absent (11.12), any BIC algorithm never plays arm* 2*.*

Proof. Suppose Property (11.12) does not hold. Let ALG be a strongly BIC algorithm. We prove by induction on t that ALG cannot recommend arm 2 to agent t.

This is trivially true for $t = 1$. Suppose the induction hypothesis is true for some t. Then the decision whether to recommend arm 2 in round $t + 1$ (*i.e.,* whether $a_{t+1} = 2$) is determined by the first t outcomes of arm 1 and the algorithm's random seed. Letting $U = \{a_{t+1} = 2\}$, we have

$$\begin{aligned}
\mathbb{E}[\mu_2 - \mu_1 \mid U] &= \mathbb{E}\left[\ \mathbb{E}[\mu_2 - \mu_1 \mid \mathcal{S}_{1,t}] \ \mid U\ \right] && \text{(by Fact 11.6)} \\
&= \mathbb{E}[G_{1,t} | U] && \text{(by definition of } G_{1,t}\text{)} \\
&\leq 0 && \text{(since (11.12) does not hold).}
\end{aligned}$$

The last inequality holds because the negation of (11.12) implies $\Pr[G_{1,t} \leq 0] = 1$. This contradicts ALG being BIC, and completes the induction proof. □

11.6 Literature review and discussion: incentivized exploration

The study of incentivized exploration has been initiated in Kremer, Mansour, and Perry (2014) and Che and Hörner (2018). The model in this chapter was introduced in Kremer et al. (2014), and studied under several names, *e.g.,* "BIC bandit exploration" in Mansour et al. (2020) and "Bayesian Exploration" in Mansour et al. (2022). All results in this chapter are from Mansour, Slivkins, and Syrgkanis (2020), specialized to $K = 2$, with slightly simplified algorithms and a substantially simplified presentation. While Mansour et al. (2020) used a version of HiddenExploration as a common technique in several results, we identify it as an explicit "building block" with standalone guarantees, and use it as a subroutine in the algorithm and a lemma in the overall analysis. The version in Mansour et al. (2020) runs in phases of fixed duration, which consist of exploitation rounds with a few exploration rounds inserted uniformly at random.

Incentivized exploration is connected to theoretical economics in three different ways. First, it adopts the BIC paradigm, as per Remark 11.5. Second, the game between the principal and a single agent in our model has been studied, under the name *Bayesian Persuasion*, in a long line of work starting from Kamenica and Gentzkow (2011), see Kamenica (2019) for a survey. This is an idealized model for many real-life scenarios in which a more informed "principal" wishes to persuade the "agent" to take an action which benefits the principal. A broader theme here is the design of "information structures": signals received by players in a game (Bergemann and Morris, 2019). A survey (Slivkins, 2023) elucidates the connection between this work and incentivized exploration. Third, the field of *social learning* studies self-interested agents that jointly learn over time in a shared environment (Golub and Sadler, 2016). In particular, *strategic experimentation* studies models similar to incentivized exploration, but without a coordinator (Hörner and Skrzypacz, 2017).

The basic model defined in this chapter was studied, and largely resolved, in (Kremer et al., 2014; Mansour et al., 2020, 2022; Cohen and Mansour, 2019; Sellke and Slivkins, 2022). While very idealized, this model is very rich, leading to a variety of results and algorithms. Results extend to $K > 2$, and come in several "flavors" other than Bayesian regret: to wit, optimal policies for deterministic rewards, regret bounds for all realizations of the prior, and (sample complexity of) exploring all arms that can be explored. The basic model has been extended in various ways (Mansour et al., 2020, 2022; Bahar et al., 2016, 2019; Immorlica et al., 2019, 2020; Simchowitz and Slivkins, 2023). Generally, the model can be made more realistic in three broad directions: generalize the *exploration* problem (in all ways that one can generalize multi-armed

bandits), generalize the *persuasion* problem (in all ways that one can generalize Bayesian persuasion), and relax the standard (yet strong) assumptions about agents' behavior.

Several papers start with a similar motivation, but adopt substantially different technical models: time-discounted rewards (Bimpikis et al., 2018); continuous information flow and a continuum of agents (Che and Hörner, 2018); incentivizing exploration using money (Frazier et al., 2014; Chen et al., 2018); incentivizing the agents to "participate" even if they knew as much as the algorithm (Bahar et al., 2020); not expecting the agents to comply with recommendations, and instead treating recommendations as "instrumental variables" in statistics (Kallus, 2018; Ngo et al., 2021).

Similar issues, albeit with much higher stakes, arise in medical decisions: selecting a doctor or a hospital, choosing a drug or a treatment, or deciding whether to participate in a medical trial. An individual can consult information from similar individuals in the past, to the extent that such information is available, and later he can contribute his experience as a review or as an outcome in a medical trial. A detailed discussion of the connection to medical trials can be found in Mansour et al. (2020).

In what follows, we spell out the results on the basic model, and briefly survey the extensions.

Diverse results in the basic model. The special case of deterministic rewards and two arms has been optimally solved in the original paper of Kremer et al. (2014). That is, they design a BIC algorithm which exactly optimizes the expected reward, for a given Bayesian prior, among all BIC algorithms. This result has been extended to $K > 2$ arms in Cohen and Mansour (2019), under additional assumptions.

`RepeatedHE` comes with no guarantees on pseudo-regret for each realization of the prior. Kremer et al. (2014) and Mansour et al. (2020) provide such guarantees, via different algorithms: Kremer et al. (2014) achieve $\tilde{O}(T^{2/3})$ regret, and Mansour et al. (2020) achieve regret bounds with a near-optimal dependence on T, both in the $\tilde{O}(\sqrt{T})$ worst-case sense and in the $O(\log T)$ instance-dependent sense. Both algorithms suffer from prior-dependent factors similar to those in Remark 11.18.

`RepeatedHE` and the optimal pseudo-regret algorithm from Mansour et al. (2020) extend to $K > 2$ arms under independent priors. `RepeatedHE` also works for correlated priors, under a version of assumption (11.12); however, it is unclear whether this assumption is necessary. Both algorithms require a *warm-start*: some pre-determined number of samples from each arm. Regret bounds for both algorithms suffer exponential dependence on K in the worst case. Very recently, Sellke and Slivkins (2022) improved this dependence to $\mathrm{poly}(K)$ for Bayesian regret and independent priors (more on this below).

While all these algorithms are heavily tailored to incentivized exploration, Sellke and Slivkins (2022) revisit Thompson Sampling, the Bayesian bandit algorithm from Chapter 3. They prove that Thompson Sampling is BIC for independent priors (and any K), when initialized with prior $\mathcal{P}$ and a sufficient warm-start. If prior mean rewards are the same for all arms, then Thompson Sampling is BIC even without the warm-start. Recall that Thompson Sampling achieves $\tilde{O}(\sqrt{KT})$ Bayesian regret for any prior (Russo and Van Roy, 2014). It is unclear whether other "organic" bandit algorithms such as `UCB1` can be proved BIC under similar assumptions.

Call an arm *explorable* if it can be explored with some positive probability by some BIC algorithm. In general, not all arms are explorable. Mansour et al. (2022) design an algorithm which explores all explorable arms, and achieves regret $O(\log T)$ relative to the best explorable arm (albeit with a very large instance-dependent constant). Interestingly, the set of all explorable arms is not determined in advance: instead, observing a particular realization of one arm may "unlock" the possibility of exploring another arm. In contrast, for independent priors explorability is completely determined by the arms' priors, and admits a simple characterization (Sellke and Slivkins, 2022): each arm a is explorable if and only if

$$\Pr\left[\,\mu_{a'} < \mu_a^0\,\right] > 0 \quad \text{for all arms } a' \neq a. \tag{11.13}$$

All explorable arms can be explored, *e.g.,* via the K-arms extension of `RepeatedHE` mentioned above.

Sample complexity. How many rounds are needed to sample each explorable arm even once? This is arguably the most basic objective in incentivized exploration, call it *sample complexity*. While Mansour et al. (2020, 2022) give rather crude upper bounds for correlated priors, Sellke and Slivkins (2022) obtain tighter results for independent priors. Without loss of generality, one can assume that all arms are explorable, *i.e.,* focus on the arms which satisfy (11.13). If all per-arm priors belong to some collection $\mathcal{C}$, one can cleanly decouple the dependence on the number of arms K and the dependence on $\mathcal{C}$. We are interested in the *$\mathcal{C}$-optimal* sample complexity: optimal sample complexity in the worst case over the choice of per-arm priors from $\mathcal{C}$. The dependence on $\mathcal{C}$ is driven by the smallest variance $\sigma^2_{\min}(\mathcal{C}) = \inf_{\mathcal{P}\in\mathcal{C}} \text{Var}(\mathcal{P})$. The key issue is whether the dependence on K and $\sigma_{\min}(\mathcal{C})$ is polynomial or exponential; *e.g.,* the sample complexity obtained via an extension of the `RepeatedHE` can be exponential in both.

Sellke and Slivkins (2022) provide a new algorithm for sampling each arm. Compared to `RepeatedHE`, it inserts a third "branch" which combines exploration and exploitation, and allows the exploration probability to increase over time. This algorithm is *polynomially optimal* in the following sense: there is an upper bound U on its sample complexity and a lower bound L on the sample complexity of any BIC algorithm such that $U < \text{poly}\,(\,L/\sigma_{\min}(\mathcal{C})\,)$. This result achieves polynomial dependence on K and $\sigma_{\min}(\mathcal{C})$ whenever such dependence is possible, and allows for several refinements detailed below.

The dependence on K admits a very sharp separation: essentially, it is either linear or at least exponential, depending on the collection $\mathcal{C}$ of feasible per-arm priors. In particular, if $\mathcal{C}$ is finite then one compares

$$\texttt{minsupp}(\mathcal{C}) := \min_{\mathcal{P}\in\mathcal{C}} \sup(\text{support}(\mathcal{P})) \quad \text{and} \quad \Phi_{\mathcal{C}} := \max_{\mathcal{P}\in\mathcal{C}} \mathbb{E}[\mathcal{P}]. \tag{11.14}$$

The $\mathcal{C}$-optimal sample complexity is $O_{\mathcal{C}}(K)$ if $\texttt{minsupp}(\mathcal{C}) > \Phi_{\mathcal{C}}$, and $\exp\,(\,\Omega_{\mathcal{C}}(K)\,)$ if $\texttt{minsupp}(\mathcal{C}) < \Phi_{\mathcal{C}}$. The former regime is arguably quite typical, *e.g.,* it holds in the realistic scenario when all per-arm priors have full support $[0,1]$, so that $\texttt{minsupp}(\mathcal{C}) = 1 > \Phi_{\mathcal{C}}$.

The $\mathcal{C}$-optimal sample complexity is exponential in $\sigma_{\min}(\mathcal{C})$, for two canonical special cases: all per-arm priors are, resp., Beta distributions and truncated Gaussians. For the latter case, all per-arm priors are assumed to be Gaussian with the same variance $\sigma^2 \leq 1$, conditioned to lie in $[0,1]$. For Beta priors, different arms may have different variances. Given a problem instance, the optimal sample complexity is exponential in the *second*-smallest variance, but only polynomial in the smallest variance. This is important when one arm represents a well-known, default alternative, whereas all other arms are new to the agents.

The price of incentives. What is the penalty in performance incurred for the sake of the BIC property? We broadly refer to such penalties as the *price of incentives* (`PoI`). The precise definition is tricky to pin down, as the `PoI` can be multiplicative or additive, can be expressed via different performance measures, and may depend on the comparator benchmark. Here's one version for concreteness: given a BIC algorithm $\mathcal{A}$ and a bandit algorithm $\mathcal{A}^*$ that we wish to compare against, suppose $\texttt{BR}_{\mathcal{A}}(T) = c_{\texttt{mult}} \cdot \texttt{BR}_{\mathcal{A}^*}(T) + c_{\texttt{add}}$, where $\texttt{BR}_{\mathcal{A}}(T)$ is Bayesian regret of algorithm $\mathcal{A}$. Then $c_{\texttt{mult}}$, $c_{\texttt{add}}$ are, resp., multiplicative and additive `PoI`.

Let us elucidate the `PoI` for independent priors, using the results in Sellke and Slivkins (2022). Since Thompson Sampling is BIC with a warm-start (and, arguably, a reasonable benchmark to compare against), the `PoI` is only additive, arising due to collecting data for the warm-start. The `PoI` is upper-bounded by the sample complexity of collecting this data. The sufficient number of data points per arm, denote it N, is $N = O(K)$ under very mild assumptions, and even $N = O(\log K)$ for Beta priors with bounded variance. We retain all polynomial sample complexity results described above, in terms of K and $\texttt{minsupp}(\mathcal{C})$. In particular, the `PoI` is $O_{\mathcal{C}}(K)$ if $\texttt{minsupp}(\mathcal{C}) > \Phi_{\mathcal{C}}$, in the notation from (11.14).

Alternatively, the initial data points may be collected exogenously, *e.g.,* purchased at a fixed price per data point (then the `PoI` is simply the total payment). The $N = O(\log K)$ scaling is particularly appealing if each arm represents a self-interested party, *e.g.,* a restaurant, which wishes to be advertised on the platform. Then each arm can be asked to pay a small, $O(\log K)$-sized entry fee to subsidise the initial samples.

Extensions of the basic model. Several extensions generalize the exploration problem, *i.e.,* the problem faced by an algorithm (even) without the BIC constraint. `RepeatedHE` allows the algorithm to receive auxiliary feedback after each round, *e.g.,* as in combinatorial semi-bandits. This auxiliary feedback is then included in the signal $\mathcal{S}_t$. Moreover, Mansour et al. (2020) extend `RepeatedHE` to contextual bandits, under a suitable assumption on the prior which makes all context-arm pairs explorable. Immorlica et al. (2019) study an extension to contextual bandits without any assumptions, and explore all context-arm pairs that are explorable. Simchowitz and Slivkins (2023) consider incentivized exploration in reinforcement learning.

Other extensions generalize the *persuasion* problem in incentivized exploration.

- *(Not) knowing the prior:* While `RepeatedHE` requires the full knowledge of the prior in order to perform the Bayesian update, the principal is not likely to have such knowledge in practice. To mitigate this issue, one of the algorithms in Mansour et al. (2020) does not input the prior, and instead only requires its parameters (which are similar to ϵ, N_0 in `RepeatedHE`) to be consistent with it. In fact, agents can have different beliefs, as long as they are consistent with the algorithm's parameters.

- *Misaligned incentives:* The principal's incentives can be misaligned with the agents': *e.g.,* a vendor may favor more expensive products, and a hospital running a free medical trial may prefer less expensive treatments. Formally, the principal may receive its own, separate rewards for the chosen actions. `RepeatedHE` is oblivious to the principal's incentives, so `ALG` can be a bandits-with-predictions algorithm (see Section 1.5) that learns the best action for the principal. The algorithm in Mansour et al. (2022) (which explores all explorable actions) can also optimize for the principal.

- *Heterogenous agents:* Each agent has a *type* which determines her reward distributions and her prior. Extensions to contextual bandits, as discussed above, correspond to *public types* (*i.e.,* observed by the principal). Immorlica et al. (2019) also investigate *private types*, the other standard variant when the types are *not* observed. Their algorithm offers *menus* which map types to arms, incentivizes each agent to follow the offered menu, and explores all "explorable" menus.

- *Multiple agents in each round:* Multiple agents may arrive simultaneously and directly affect one another (Mansour et al., 2022). *E.g.,* drivers that choose to follow a particular route at the same time may create congestion, which affects everyone. In each round, the principal chooses a distribution D over joint actions, samples a joint action from D, and recommends it to the agents. The BIC constraint requires D to be the *Bayes Correlated Equilibrium* (Bergemann and Morris, 2013).

- *Beyond "minimal revelation":* What if the agents observe some aspects of the history, even if the principal does not wish them to? In Bahar et al. (2016), the agents observe recommendations to their friends on a social networks (but not the corresponding rewards). In Bahar et al. (2019), each agent observes the action and the reward of the previous agent. Such additional information skews the agents further towards exploitation, and makes the problem much more challenging. Both papers focus on the case of two arms, and assume, resp., deterministic rewards or one known arm.

All results in Mansour et al. (2022); Immorlica et al. (2019); Simchowitz and Slivkins (2023) follow the perspective of exploring all explorable "pieces". The "pieces" being explored range from actions to joint actions to context-arm pairs to menus to policies, depending on a particular extension.

Behavioral assumptions. All results discussed above rely heavily on standard but very idealized assumptions about agents' behavior. First, the principal announces his algorithm, the agents know and understand it, and trust the principal to faithfully implement it as announced. [5] Second, the agents either trust the BIC

[5] In theoretical economics, these assumptions are summarily called the (principal's) *power to commit.*

property of the algorithm, or can verify it independently. Third, the agents act rationally, *i.e.,* choose arms that maximize their expected utility (*e.g.,* they don't favor less risky arms, and don't occasionally choose less preferable actions). Fourth, the agents find it acceptable that they are given recommendations without any supporting information, and that they may be singled out for low-probability exploration.

One way forward is to define a particular class of algorithms and a model of agents' behavior that is (more) plausible for this class. To this end, Immorlica et al. (2020) consider algorithms which reveal some of the history to each agent, and allow a flexible model of greedy-like response. To justify such response, the sub-history revealed to each agent t consists of all rounds that precede t in some fixed and pre-determined partial order. Consequently, each agent observes the history of all agents that could possibly affect her. Put differently, each agent only interacts with a full-revelation algorithm, and the behavioral model does not need to specify how the agents interact with any algorithms that actually explore. Immorlica et al. (2020) design an algorithm in this framework which matches the state-of-art regret bounds.

The greedy algorithm. Theorem 11.7 on the Bayesian-greedy algorithm and its corollaries are from Banihashem et al. (2023).[6] A similar result holds for $K > 2$ arms, albeit with a somewhat more complex formulation. While it has been understood for decades that exploitation-only bandit algorithms fail badly for some special cases, Theorem 11.7 is the first non-trivial general result for stochastic rewards that we are aware of. Characterizing the learning performance of Bayesian-greedy more precisely is an open question, even for $K = 2$ arms and independent priors, and especially if $\mathbb{E}[\mu_1]$ is close to $\mathbb{E}[\mu_2]$. This concerns both the probability of never choosing arm 2 and Bayesian regret. The latter could be a more complex issue, because Bayesian regret can be accumulated even when arm 2 is chosen.

The frequentist version of the greedy algorithm replaces posterior mean with empirical mean: in each round, it chooses an arm with a largest empirical mean reward. Initially, each arm is tried N_0 times, for some small constant N_0 (*warm-up*). Focusing on $K = 2$ arms, we have a similar learning failure like in Theorem 11.7: the good arm is never chosen again. A trivial argument yields failure probability $e^{-\Omega(N_0)}$: consider the event when the good arm receives 0 reward in all warm-up rounds, and the bad arm receives a non-zero reward in some warm-up round. However, this trivial guarantee is rather weak because of the exponential dependence on N_0. A similar but exponentially stronger guarantee is proved in Banihashem et al. (2023), with failure probability on the order of $1/\sqrt{N_0}$. This result is extended to a broader class of agent behaviors (including, *e.g.,* mild optimism and pessimism), and to $K > 2$ arms.

Nevertheless, Bayati et al. (2020); Jedor et al. (2021) prove non-trivial (but suboptimal) regret bounds for the frequentist-greedy algorithm on problem instances with a very large number of near-optimal arms. Particularly, for Bayesian bandits with $K \geq \sqrt{T}$ arms, where the arms' mean rewards are sampled independently and uniformly.

Several papers find that the greedy algorithm (equivalently, incentivized exploration with full disclosure) performs well in theory under substantial assumptions on heterogeneity of the agents and structure of the rewards. Bastani et al. (2021); Kannan et al. (2018); Raghavan et al. (2023) consider linear contextual bandits, where the contexts come from a sufficiently "diffuse" distribution (see Section 8.7 for a more detailed discussion). Schmit and Riquelme (2018) assume additive agent-specific shift in expected reward of each action, and posit that each agent knows her shift and removes it from the reported reward.

[6] Banihashem et al. (2023) attribute Theorem 11.7 to Sellke (2019).

11.7 Literature review and discussion: other work on bandits and agents

Bandit algorithms interact with self-interested agents in a number of applications. The technical models vary widely, depending on how the agents come into the picture. We partition this literature based on what the agents actually choose: which arm to pull (in incentivized exploration), which bid to report (in an auction), how to respond to an offer (in contract design), or which bandit algorithm to interact with. While this chapter focuses on incentivized exploration, let us now survey the other three lines of work.

11.7.1 Repeated auctions: agents choose bids

Consider an idealized but fairly generic repeated auction, where in each round the auctioneer allocates one item to one of the auction participants (*agents*):

Problem protocol: Repeated auction

In each round $t = 1, 2, 3, \ldots, T$:

1. Each agent submits a message (*bid*).
2. Auctioneer's "allocation algorithm" chooses an agent and allocates one item to this agent.
3. The agent's reward is realized and observed by the algorithm and/or the agent.
4. Auctioneer assigns payments.

The auction should incentivize each agent to submit "truthful bids" representing his current knowledge/beliefs about the rewards.[7] Agents' rewards may may not be directly observable by the auctioneer, but they can be derived (in some formal sense) from the auctioneer's observations and the agents' truthful bids. The auctioneer has one of the two standard objectives: *social welfare* and *revenue* from the agents' payments. Social welfare is the total "utility" of the agents and the auctioneer, *i.e.,* since the payments cancel out, the total agents' reward. Thus, the allocation algorithm can be implemented as a bandit algorithm, where agents correspond to "arms", and algorithm's reward is either the agent's reward or the auctioneer's revenue, depending on the auctioneer's objective.

A typical motivation is *ad auctions*: auctions which allocate advertisement opportunities on the web among the competing advertisers. Hence, one ad opportunity is allocated in each round of the above model, and agents correspond to the advertisers. The auctioneer is a website or an *ad platform*: an intermediary which connects advertisers and websites with ad opportunities.

The model of *dynamic auctions* (Bergemann and Välimäki, 2010; Athey and Segal, 2013; Pavan et al., 2011; Kakade et al., 2013; Bergemann and Said, 2011) posits that the agents do not initially know much about their future rewards. Instead, each agent learns over time by observing his realized rewards when and if he is selected by the algorithm. Further, the auctioneer does not observe the rewards, and instead needs to rely on agents' bids. The auctioneer needs to create the right incentives for the agents to stay in the auction and bid their posterior mean rewards. This line of work has been an important development in theoretical economics. It is probably the first appearance of the "bandits with incentives" theme in the literature (going by the working papers). Nazerzadeh et al. (2013) consider a similar but technically different model in which the agents are incentivized to report their realized rewards. They create incentives in the "asymptotically approximate" sense: essentially, truthful bidding is at least as good as any alternative, minus a regret term.

[7] The technical definition of truthful bidding differs from one model to another. These details tend to be very lengthy, *e.g.,* compared to a typical setup in multi-armed bandits, and often require background in theoretical economics to appreciate. We keep our exposition at a more basic level.

A line of work from algorithmic economics literature (Babaioff et al., 2014; Devanur and Kakade, 2009; Babaioff et al., 2010, 2013, 2015b; Wilkens and Sivan, 2012; Gatti et al., 2012) considers a simpler model, specifically designed to showcase the interaction of bandits and auctions aspects. They consider *pay-per-click* ad auctions, where advertisers derive value only when users click on their ads, and are charged per click. Click probabilities are largely unknown, which gives rise to a bandit problem. Bids correspond to agents' per-click values, which are assumed to be fixed over time. Only the initial bids are considered, all payments are assigned after the last round, and the allocation proceeds over time as a bandit algorithm. Combination of bandit feedback and truthfulness brings about an interesting issue: while it is well-known what the payments must be to ensure truthfulness, the algorithm might not have enough information to compute them. For this reason, Explore-First is essentially the only possible deterministic algorithm Babaioff et al. (2014); Devanur and Kakade (2009). Yet, the required payments can be achieved in expectation by a randomized algorithm. Furthermore, a simple randomized transformation can turn any bandit algorithm into a truthful algorithm, with only a small loss in rewards, as long as the original algorithm satisfies a well-known necessary condition Babaioff et al. (2010, 2013, 2015b).

Another line of work (*e.g.,* Cesa-Bianchi et al., 2013; Dudík et al., 2017) concerns tuning the auction over time, *i.e.,* adjusting its parameters such as a reserve price. A fresh batch of agents is assumed to arrive in each round (and when and if the same agent arrives more than once, she only cares about the current round). This can be modeled as a contextual problem where "contexts" are bids, "arms" are the different choices for allocations, and "policies" (mappings from arms to actions) correspond to the different parameter choices. Alternatively, this can be modeled as a non-contextual problem, where the "arms" are the parameter choices.

11.7.2 Contract design: agents (only) affect rewards

In a variety of settings, the algorithm specifies "contracts" for the agents, *i.e.,* rules which map agents' performance to outcomes. Agents choose their responses to the offered contracts, and the said responses affect algorithm's rewards. The nature of the contracts and the responses depends on a particular application.

Most work in this direction posits that the contracts are adjusted over time. In each round, a new agent arrives, the algorithm chooses a "contract", the agent responds, and the reward is revealed. Agents' incentives typically impose some structure on the rewards that is useful for the algorithm. In *dynamic pricing* (Boer, 2015, a survey; see also Section 10.5.5) and *dynamic assortment* (*e.g.,* Sauré and Zeevi, 2013; Agrawal et al., 2019) the algorithm offers some items for sale, a contract specifies, resp., the price(s) and the offered assortment of items, and the agents decide whether and which products to buy. (This is a vast and active research area; a more detailed survey is beyond our scope.) In *dynamic procurement* (*e.g.,* Badanidiyuru et al., 2012, 2018; Singla and Krause, 2013) the algorithm is a buyer and the agents are sellers; alternatively, the algorithm is a contractor on a crowdsourcing market and the agents are workers. The contracts specify the payment(s) for the completed tasks, and the agents decide whether and which tasks to complete. Ho et al. (2016) study a more general model in which the agents are workers who choose their effort level (which is not directly observed by the algorithm), and the contracts specify payment for each quality level of the completed work. One round of this model is a well-known *principal-agent model* from contract theory (Laffont and Martimort, 2002).

In some other papers, the entire algorithm is a contract, from the agents' perspective. In Ghosh and Hummel (2013), the agents choose how much effort to put into writing a review, and then a bandit algorithm chooses among relevant reviews, based on user feedback such as "likes". The effort level affects the "rewards" received for the corresponding review. In Braverman et al. (2019) the agents collect rewards directly, unobserved by the algorithm, and pass some of these rewards to the algorithm. The algorithm chooses among agents based on the observed "kick-backs", and therefore incentivizes the agents.

A growing line of work studies bandit algorithms for dynamic pricing which can interact with the same agent multiple times. Agents' self-interested behavior is typically restricted. One typical assumption is that they are more myopic compared to the algorithm, placing less value on the rewards in the far future (*e.g.,* Amin et al., 2013, 2014). Alternatively, the agents also learn over time, using a low-regret online learning algorithm (*e.g.,* Heidari et al., 2016; Braverman et al., 2018).

11.7.3 Agents choose between bandit algorithms

Businesses that can deploy bandit algorithms (*e.g.,* web search engines, recommendation systems, or online retailers) often compete with one another. Users can choose which of the competitors to go to, and hence which of the bandit algorithms to interact with. Thus, we have bandit algorithms that compete for users. The said users bring not only utility (such as revenue and/or market share), but also new data to learn from. This leads to a three-way tradeoff between exploration, exploitation, and competition.

Mansour et al. (2018); Aridor et al. (2019, 2020) consider bandit algorithms that optimize a product over time and compete on product quality. They investigate whether competition incentivizes the adoption of better bandit algorithms, and how these incentives depend on the intensity of the competition. In particular, exploration may hurt algorithm's performance and reputation in the near term, with adverse competitive effects. An algorithm may even enter a "death spiral", when the short-term reputation cost decreases the number of users for the algorithm to learn from, which degrades the system's performance relative to competition and further decreases the market share. These issues are related to the relationship between competition and innovation, a well-studied topic in economics (Schumpeter, 1942; Aghion et al., 2005).

Bergemann and Välimäki (1997, 2000); Keller and Rady (2003) target a very different scenario when the competing firms experiment with *prices* rather than design alternatives. All three papers consider strategies that respond to competition and analyze Markov-perfect equilibria in the resulting game.

11.8 Exercises and hints

Exercise 11.1 (Bayesian-Greedy fails). Prove Corollary 11.8.

Hint: Consider the event $\{\mathcal{E}$ and $\mu_1 < 1 - 2\,\alpha\}$; this event is independent with μ_2.

Exercise 11.2 (performance of `RepeatedHE`). Prove Theorem 11.17(b).

Hint: The proof relies on Wald's identity. Let t_i be the i-th round $t > N_0$ in which the "exploration branch" has been chosen. Let u_i be the expected reward of `ALG` in the i-th round of its execution. Let X_i be the expected reward in the time interval $[t_i, t_{i+1})$. Use Wald's identity to prove that $\mathbb{E}[X_i] = \frac{1}{\epsilon}\, u_i$. Use Wald's identity again (in a version for non-IID random variables) to prove that $\mathbb{E}[\sum_{i=1}^{N} X_i] = \frac{1}{\epsilon}\, \mathbb{E}[\sum_{i=1}^{N} u_i]$. Observe that the right-hand side is simply $\frac{1}{\epsilon}\, \mathbb{E}[U(N)]$.

A Concentration inequalities

This appendix provides background on concentration inequalities, sufficient for this book. We use somewhat non-standard formulations that are most convenient for our applications. More background can be found in (McDiarmid, 1998) and (Dubhashi and Panconesi, 2009).

Consider random variables $X_1, X_2, \ldots$. Assume they are mutually independent, but not necessarily identically distributed. Let $\overline{X}_n = \frac{X_1+\ldots+X_n}{n}$ be the average of the first n random variables, and let $\mu_n = \mathbb{E}[\overline{X}_n]$ be its expectation. According to the Strong Law of Large Numbers,

$$\Pr\left[\, \overline{X}_n \to \mu \,\right] = 1.$$

We want to show that $\overline{X}_n$ is *concentrated* around μ_n when n is sufficiently large, in the sense that $|\overline{X}_n - \mu_n|$ is small with high probability. Thus, we are interested in statements of the following form:

$$\Pr\left[\, |\overline{X}_n - \mu_n| \leq \text{“small”} \,\right] \geq 1 - \text{“small”}.$$

Such statements are called “concentration inequalities”.

Fix n, and focus on the following high-probability event:

$$\mathcal{E}_{\alpha,\beta} := \left\{ |\overline{X}_n - \mu_n| \leq \sqrt{\alpha\beta \log(T) \,/\, n} \right\}, \qquad \alpha > 0. \tag{A.1}$$

The following statement holds, under various assumptions:

$$\Pr\left[\, \mathcal{E}_{\alpha,\beta} \,\right] \geq 1 - 2 \cdot T^{-2\alpha}, \qquad \forall \alpha > 0. \tag{A.2}$$

Here T is a fixed parameter; think of it as the time horizon in multi-armed bandits. The α controls the failure probability; taking $\alpha = 2$ suffices for most applications in this book. The additional parameter β depends on the assumptions. The $r_n = \sqrt{\frac{\alpha \log T}{n}}$ term in Eq. (A.1) is called the *confidence radius*. The interval $[\mu_n - r_n,\ \mu_n + r_n]$ is called the *confidence interval*.

Theorem A.1 (Hoeffding Inequality)**.** *Eq. (A.2) holds, with $\beta = 1$, if $X_1, \ldots, X_n \in [0,1]$.*

This is the basic result. The special case of Theorem A.1 with $X_i \in \{0,1\}$ is known as *Chernoff Bounds*.

Theorem A.2 (Extensions)**.** *Eq. (A.2) holds, for appropriate β, in the following cases:*

(a) Bounded intervals: $X_i \in [a_i, b_i]$ *for all* $i \in [n]$*, and* $\beta = \frac{1}{n}\sum_{i\in[n]}(b_i - a_i)^2$.

(b) Bounded variance: $X_i \in [0,1]$ *and* $\texttt{Variance}(X_i) \leq \beta/8$ *for all* $i \in [n]$.

(c) Gaussians: *Each* X_i*,* $i \in [n]$ *is Gaussian with variance at most* $\beta/4$.

Theorem A.3 (Beyond independence)**.** *Consider random variables $X_1, X_2, \ldots \in [0,1]$ that are not necessarily independent or identically distributed. For each $i \in [n]$, posit a number $\mu_i \in [0,1]$ such that*

$$\mathbb{E}\left[X_i \mid X_1 \in J_1, \ \ldots\ , X_{i-1} \in J_{i-1}\right] = \mu_i, \tag{A.3}$$

for any intervals $J_1, \ldots, J_{i-1} \subset [0,1]$. Then Eq. (A.2) holds, with $\beta = 4$.

This is a corollary from a well-known *Azuma-Hoeffding Inequality*. Eq. (A.3) is essentially the *martingale assumption*, often used in the literature to extend results on independent random variables.

B Properties of KL-divergence

Let us prove the properties of KL-divergence stated in Theorem 2.4. To recap the main definition, consider a finite sample space Ω, and let p, q be two probability distributions on Ω. *KL-divergence* is defined as:

$$\mathtt{KL}(p,q) = \sum_{x\in\Omega} p(x) \ln \frac{p(x)}{q(x)} = \mathbb{E}_p\left[\ln \frac{p(x)}{q(x)}\right].$$

Lemma B.1 (Gibbs' Inequality, Theorem 2.4(a))**.**
$\mathtt{KL}(p,q) \geq 0$ *for any two distributions* p, q*, with equality if and only if* $p = q$*.*

Proof. Let us define: $f(y) = y\ln(y)$. f is a convex function under the domain $y > 0$. Now, from the definition of the KL divergence we get:

$$\begin{aligned}
\mathtt{KL}(p,q) &= \sum_{x\in\Omega} q(x)\,\frac{p(x)}{q(x)} \ln \frac{p(x)}{q(x)} \\
&= \sum_{x\in\Omega} q(x) f\left(\frac{p(x)}{q(x)}\right) \\
&\geq f\left(\sum_{x\in\Omega} q(x)\frac{p(x)}{q(x)}\right) && \textit{(by Jensen's inequality)} \\
&= f\left(\sum_{x\in\Omega} p(x)\right) = f(1) = 0,
\end{aligned}$$

In the above application of Jensen's inequality, f is not a linear function, so the equality holds (*i.e.,* $\mathtt{KL}(p,q) = 0$) if and only if $p = q$. □

Lemma B.2 (Chain rule for product distributions, Theorem 2.4(b))**.**
Let the sample space be a product $\Omega = \Omega_1 \times \Omega_1 \times \cdots \times \Omega_n$*. Let* p *and* q *be two distributions on* Ω *such that* $p = p_1 \times p_2 \times \cdots \times p_n$ *and* $q = q_1 \times q_2 \times \cdots \times q_n$*, where* p_j, q_j *are distributions on* Ω_j*, for each* $j \in [n]$*. Then* $\mathtt{KL}(p,q) = \sum_{j=1}^{n} \mathtt{KL}(p_j, q_j)$*.*

Proof. Let $x = (x_1, x_2, \ldots, x_n) \in \Omega$ such that $x_i \in \Omega_i$ for all $i = 1\,,\ \ldots\ , n$. Let $h_i(x_i) = \ln \frac{p_i(x_i)}{q_i(x_i)}$. Then:

$$\begin{aligned}
\mathtt{KL}(p,q) &= \sum_{x\in\Omega} p(x) \ln \frac{p(x)}{q(x)} \\
&= \sum_{i=1}^{n} \sum_{x\in\Omega} p(x) h_i(x_i) && \left[\text{since } \ln \frac{p(x)}{q(x)} = \sum_{i=1}^{n} h_i(x_i)\right] \\
&= \sum_{i=1}^{n} \sum_{x_i^\star\in\Omega_i} h_i(x_i^\star) \sum_{\substack{x\in\Omega,\\ x_i = x_i^\star}} p(x) \\
&= \sum_{i=1}^{n} \sum_{x_i\in\Omega_i} p_i(x_i) h_i(x_i) && \left[\text{since } \sum_{x\in\Omega,\ x_i = x_i^\star} p(x) = p_i(x_i^\star)\right] \\
&= \sum_{i=1}^{n} \mathtt{KL}(p_i, q_i).
\end{aligned}$$

□

Lemma B.3 (Pinsker's inequality, Theorem 2.4(c))**.** *Fix event $A \subset \Omega$. Then*

$$2\left(p(A) - q(A)\right)^2 \le \mathtt{KL}(p, q).$$

Proof. First, we claim that

$$\sum_{x\in B} p(x) \ln \frac{p(x)}{q(x)} \ge p(B) \ln \frac{p(B)}{q(B)} \qquad \text{for each event } B \subset \Omega. \tag{B.1}$$

For each $x \in B$, define $p_B(x) = p(x)/p(B)$ and $q_B(x) = q(x)/q(B)$. Then

$$\begin{aligned}
\sum_{x\in B} p(x) \ln \frac{p(x)}{q(x)} &= p(B) \sum_{x\in B} p_B(x) \ln \frac{p(B)\cdot p_B(x)}{q(B)\cdot q_B(x)} \\
&= p(B) \left(\sum_{x\in B} p_B(x) \ln \frac{p_B(x)}{q_B(x)} \right) + p(B) \ln \frac{p(B)}{q(B)} \sum_{x\in B} p_B(x) \\
&\ge p(B) \ln \frac{p(B)}{q(B)} \qquad \left[\text{since } \sum_{x\in B} p_B(x) \ln \frac{p_B(x)}{q_B(x)} = \mathtt{KL}(p_B, q_B) \ge 0 \right].
\end{aligned}$$

Now that we've proved (B.1), let us use it twice: for $B = A$ and for $B = \bar{A}$, the complement of A:

$$\sum_{x\in A} p(x) \ln \frac{p(x)}{q(x)} \ge p(A) \ln \frac{p(A)}{q(A)},$$
$$\sum_{x\notin A} p(x) \ln \frac{p(x)}{q(x)} \ge p(\bar{A}) \ln \frac{p(\bar{A})}{q(\bar{A})}.$$

Now, let $a = p(A)$ and $b = q(A)$, and w.l.o.g. assume that $a < b$. Then:

$$\begin{aligned}
\mathtt{KL}(p, q) &\ge a \ln \frac{a}{b} + (1-a) \ln \frac{1-a}{1-b} \\
&= \int_a^b \left(-\frac{a}{x} + \frac{1-a}{1-x} \right) dx = \int_a^b \frac{x-a}{x(1-x)} dx \\
&\ge \int_a^b 4(x-a)dx = 2(b-a)^2. \qquad \textit{(since } x(1-x) \le \tfrac{1}{4}\textit{)} \qquad \square
\end{aligned}$$

Lemma B.4 (Random coins, Theorem 2.4(d))**.** *Fix $\epsilon \in (0, \frac{1}{2})$. Let $\mathtt{RC}_\epsilon$ denote a random coin with bias $\frac{\epsilon}{2}$,* i.e., *a distribution over $\{0, 1\}$ with expectation $(1+\epsilon)/2$. Then $\mathtt{KL}(\mathtt{RC}_\epsilon, \mathtt{RC}_0) \le 2\epsilon^2$ and $\mathtt{KL}(\mathtt{RC}_0, \mathtt{RC}_\epsilon) \le \epsilon^2$.*

Proof.

$$\begin{aligned}
\mathtt{KL}(\mathtt{RC}_0, \mathtt{RC}_\epsilon) &= \tfrac{1}{2} \ln(\tfrac{1}{1+\epsilon}) + \tfrac{1}{2} \ln(\tfrac{1}{1-\epsilon}) = -\tfrac{1}{2} \ln(1-\epsilon^2) \\
&\le -\tfrac{1}{2}(-2\epsilon^2) \qquad \textit{(as } \log(1-\epsilon^2) \ge -2\epsilon^2 \textit{ whenever } \epsilon^2 \le \tfrac{1}{2}\textit{)} \\
&= \epsilon^2. \\
\mathtt{KL}(\mathtt{RC}_\epsilon, \mathtt{RC}_0) &= \tfrac{1+\epsilon}{2} \ln(1+\epsilon) + \tfrac{1-\epsilon}{2} \ln(1-\epsilon) \\
&= \tfrac{1}{2}\left(\ln(1+\epsilon) + \ln(1-\epsilon)\right) + \tfrac{\epsilon}{2}\left(\ln(1+\epsilon) - \ln(1-\epsilon)\right) \\
&= \tfrac{1}{2} \ln(1-\epsilon^2) + \tfrac{\epsilon}{2} \ln \tfrac{1+\epsilon}{1-\epsilon}.
\end{aligned}$$

Now, $\ln(1-\epsilon^2) < 0$ and we can write $\ln \frac{1+\epsilon}{1-\epsilon} = \ln\left(1 + \frac{2\epsilon}{1-\epsilon}\right) \le \frac{2\epsilon}{1-\epsilon}$. Thus, we get:

$$\mathtt{KL}(\mathtt{RC}_\epsilon, \mathtt{RC}_0) < \tfrac{\epsilon}{2} \cdot \tfrac{2\epsilon}{1-\epsilon} = \tfrac{\epsilon^2}{1-\epsilon} \le 2\epsilon^2. \qquad \square$$

Bibliography

Yasin Abbasi-Yadkori, Dávid Pál, and Csaba Szepesvári. Improved algorithms for linear stochastic bandits. In *25th Advances in Neural Information Processing Systems (NIPS)*, pages 2312–2320, 2011.

Yasin Abbasi-Yadkori, Dávid Pál, and Csaba Szepesvári. Online-to-confidence-set conversions and application to sparse stochastic bandits. In *15th Intl. Conf. on Artificial Intelligence and Statistics (AISTATS)*, volume 22 of *JMLR Proceedings*, pages 1–9, 2012.

Naoki Abe, Alan W. Biermann, and Philip M. Long. Reinforcement learning with immediate rewards and linear hypotheses. *Algorithmica*, 37(4):263–293, 2003.

Jacob Abernethy, Elad Hazan, and Alexander Rakhlin. Competing in the Dark: An Efficient Algorithm for Bandit Linear Optimization. In *21th Conf. on Learning Theory (COLT)*, pages 263–274, 2008.

Jacob D. Abernethy and Alexander Rakhlin. Beating the adaptive bandit with high probability. In *22nd Conf. on Learning Theory (COLT)*, 2009.

Jacob D Abernethy and Jun-Kun Wang. On frank-wolfe and equilibrium computation. In *Advances in Neural Information Processing Systems (NIPS)*, pages 6584–6593, 2017.

Ittai Abraham and Dahlia Malkhi. Name independent routing for growth bounded networks. In *17th ACM Symp. on Parallel Algorithms and Architectures (SPAA)*, pages 49–55, 2005.

Alekh Agarwal, Miroslav Dudík, Satyen Kale, John Langford, and Robert E. Schapire. Contextual bandit learning with predictable rewards. In *15th Intl. Conf. on Artificial Intelligence and Statistics (AISTATS)*, pages 19–26, 2012.

Alekh Agarwal, Daniel Hsu, Satyen Kale, John Langford, Lihong Li, and Robert Schapire. Taming the monster: A fast and simple algorithm for contextual bandits. In *31st Intl. Conf. on Machine Learning (ICML)*, 2014.

Alekh Agarwal, Sarah Bird, Markus Cozowicz, Miro Dudik, Luong Hoang, John Langford, Lihong Li, Dan Melamed, Gal Oshri, Siddhartha Sen, and Aleksandrs Slivkins. Multiworld testing: A system for experimentation, learning, and decision-making, 2016. A white paper, available at `https://github.com/Microsoft/mwt-ds/raw/master/images/MWT-WhitePaper.pdf`.

Alekh Agarwal, Alina Beygelzimer, Miroslav Dudík, John Langford, and Hanna Wallach. A reductions approach to fair classification. *Fairness, Accountability, and Transparency in Machine Learning (FATML)*, 2017a.

Alekh Agarwal, Sarah Bird, Markus Cozowicz, Luong Hoang, John Langford, Stephen Lee, Jiaji Li, Dan Melamed, Gal Oshri, Oswaldo Ribas, Siddhartha Sen, and Alex Slivkins. Making contextual decisions with low technical debt, 2017b. Techical report at `arxiv.org/abs/1606.03966`.

Alekh Agarwal, Haipeng Luo, Behnam Neyshabur, and Robert E. Schapire. Corralling a band of bandit algorithms. In *30th Conf. on Learning Theory (COLT)*, pages 12–38, 2017c.

Alekh Agarwal, Nan Jiang, Sham M Kakade, and Wen Sun. Reinforcement learning: Theory and algorithms, 2020. Book draft, circulated since 2019. Available at `https://rltheorybook.github.io`.

Philippe Aghion, Nicholas Bloom, Richard Blundell, Rachel Griffith, and Peter Howitt. Competition and innovation: An inverted u relationship. *Quaterly J. of Economics*, 120(2):701–728, 2005.

Rajeev Agrawal. The continuum-armed bandit problem. *SIAM J. Control and Optimization*, 33(6):1926–1951, 1995.

Shipra Agrawal and Nikhil R. Devanur. Bandits with concave rewards and convex knapsacks. In *15th ACM Conf. on Economics and Computation (ACM-EC)*, 2014.

Shipra Agrawal and Nikhil R. Devanur. Linear contextual bandits with knapsacks. In *29th Advances in Neural Information Processing Systems (NIPS)*, 2016.

Shipra Agrawal and Nikhil R. Devanur. Bandits with global convex constraints and objective. *Operations Research*, 67(5):1486–1502, 2019. Preliminary version in *ACM EC 2014*.

Shipra Agrawal and Navin Goyal. Analysis of Thompson Sampling for the multi-armed bandit problem. In *25nd Conf. on Learning Theory (COLT)*, 2012.

Shipra Agrawal and Navin Goyal. Further optimal regret bounds for thompson sampling. In *16th Intl. Conf. on Artificial Intelligence and Statistics (AISTATS)*, pages 99–107, 2013.

Shipra Agrawal and Navin Goyal. Near-optimal regret bounds for thompson sampling. *J. of the ACM*, 64(5):30:1–30:24, 2017. Preliminary version in *AISTATS 2013*.

Shipra Agrawal, Zizhuo Wang, and Yinyu Ye. A dynamic near-optimal algorithm for online linear programming. *Operations Research*, 62(4):876–890, 2014.

Shipra Agrawal, Nikhil R. Devanur, and Lihong Li. An efficient algorithm for contextual bandits with knapsacks, and an extension to concave objectives. In *29th Conf. on Learning Theory (COLT)*, 2016.

Shipra Agrawal, Vashist Avadhanula, Vineet Goyal, and Assaf Zeevi. Mnl-bandit: A dynamic learning approach to assortment selection. *Operations Research*, 67(5):1453–1485, 2019. Prelinimary version in *ACM EC 2016*.

Nir Ailon, Zohar Karnin, and Thorsten Joachims. Reducing dueling bandits to cardinal bandits. In *Intl. Conf. on Machine Learning (ICML)*, pages 856–864, 2014.

Chamy Allenberg, Peter Auer, László Györfi, and György Ottucsák. Hannan consistency in on-line learning in case of unbounded losses under partial monitoring. In *17th Intl. Conf. on Algorithmic Learning Theory (ALT)*, 2006.

Noga Alon, Nicolò Cesa-Bianchi, Claudio Gentile, and Yishay Mansour. From bandits to experts: A tale of domination and independence. In *27th Advances in Neural Information Processing Systems (NIPS)*, pages 1610–1618, 2013.

Noga Alon, Nicolò Cesa-Bianchi, Ofer Dekel, and Tomer Koren. Online learning with feedback graphs: Beyond bandits. In *28th Conf. on Learning Theory (COLT)*, pages 23–35, 2015.

Kareem Amin, Michael Kearns, and Umar Syed. Bandits, query learning, and the haystack dimension. In *24th Conf. on Learning Theory (COLT)*, 2011.

Kareem Amin, Afshin Rostamizadeh, and Umar Syed. Learning prices for repeated auctions with strategic buyers. In *26th Advances in Neural Information Processing Systems (NIPS)*, pages 1169–1177, 2013.

Kareem Amin, Afshin Rostamizadeh, and Umar Syed. Repeated contextual auctions with strategic buyers. In *27th Advances in Neural Information Processing Systems (NIPS)*, pages 622–630, 2014.

Animashree Anandkumar, Nithin Michael, Ao Kevin Tang, and Ananthram Swami. Distributed algorithms for learning and cognitive medium access with logarithmic regret. *IEEE Journal on Selected Areas in Communications*, 29(4): 731–745, 2011.

András Antos, Gábor Bartók, Dávid Pál, and Csaba Szepesvári. Toward a classification of finite partial-monitoring games. *Theor. Comput. Sci.*, 473:77–99, 2013.

Guy Aridor, Kevin Liu, Aleksandrs Slivkins, and Steven Wu. The perils of exploration under competition: A computational modeling approach. In *20th ACM Conf. on Economics and Computation (ACM-EC)*, 2019.

Guy Aridor, Yishay Mansour, Aleksandrs Slivkins, and Steven Wu. Competing bandits: The perils of exploration under competition., 2020. Working paper. Subsumes conference papers in *ITCS 2018* and *ACM EC 2019*. Available at `https://arxiv.org/abs/2007.10144`.

Sanjeev Arora, Elad Hazan, and Satyen Kale. The multiplicative weights update method: a meta-algorithm and applications. *Theory of Computing*, 8(1):121–164, 2012.

Susan Athey and Ilya Segal. An efficient dynamic mechanism. *Econometrica*, 81(6):2463–2485, November 2013. A preliminary version has been available as a working paper since 2007.

J.-Y. Audibert, R. Munos, and Cs. Szepesvári. Exploration-exploitation trade-off using variance estimates in multi-armed bandits. *Theoretical Computer Science*, 410:1876–1902, 2009.

Jean-Yves Audibert, Sébastien Bubeck, and Rémi Munos. Best arm identification in multi-armed bandits. In *23rd Conf. on Learning Theory (COLT)*, pages 41–53, 2010.

Jean-Yves Audibert, Sébastien Bubeck, and Gábor Lugosi. Regret in online combinatorial optimization. *Mathematics of Operations Research*, 39(1):31–45, 2014.

J.Y. Audibert and S. Bubeck. Regret Bounds and Minimax Policies under Partial Monitoring. *J. of Machine Learning Research (JMLR)*, 11:2785–2836, 2010. Preliminary version in *COLT 2009*.

Peter Auer. Using confidence bounds for exploitation-exploration trade-offs. *J. of Machine Learning Research (JMLR)*, 3:397–422, 2002. Preliminary version in *41st IEEE FOCS*, 2000.

Peter Auer and Chao-Kai Chiang. An algorithm with nearly optimal pseudo-regret for both stochastic and adversarial bandits. In *29th Conf. on Learning Theory (COLT)*, 2016.

Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2-3):235–256, 2002a.

Peter Auer, Nicolò Cesa-Bianchi, Yoav Freund, and Robert E. Schapire. The nonstochastic multiarmed bandit problem. *SIAM J. Comput.*, 32(1):48–77, 2002b. Preliminary version in *36th IEEE FOCS*, 1995.

Peter Auer, Ronald Ortner, and Csaba Szepesvári. Improved Rates for the Stochastic Continuum-Armed Bandit Problem. In *20th Conf. on Learning Theory (COLT)*, pages 454–468, 2007.

Peter Auer, Pratik Gajane, and Ronald Ortner. Adaptively tracking the best arm with an unknown number of distribution changes. In *Conf. on Learning Theory (COLT)*, 2019.

Robert J. Aumann. Subjectivity and correlation in randomized strategies. *J. of Mathematical Economics*, 1:67–96, 1974.

Orly Avner and Shie Mannor. Concurrent bandits and cognitive radio networks. In *European Conf. on Machine Learning and Principles and Practice of Knowledge Discovery in Databases (ECML PKDD)*, pages 66–81, 2014.

Baruch Awerbuch and Robert Kleinberg. Online linear optimization and adaptive routing. *J. of Computer and System Sciences*, 74(1):97–114, February 2008. Preliminary version in *36th ACM STOC*, 2004.

Baruch Awerbuch, David Holmer, Herbert Rubens, and Robert D. Kleinberg. Provably competitive adaptive routing. In *24th Conf. of the IEEE Communications Society (INFOCOM)*, pages 631–641, 2005.

Mohammad Gheshlaghi Azar, Alessandro Lazaric, and Emma Brunskill. Online stochastic optimization under correlated bandit feedback. In *31th Intl. Conf. on Machine Learning (ICML)*, pages 1557–1565, 2014.

Abdellah Aznag, Vineet Goyal, and Noemie Perivier. Mnl-bandit with knapsacks. In *22th ACM Conf. on Economics and Computation (ACM-EC)*, 2021.

Moshe Babaioff, Robert Kleinberg, and Aleksandrs Slivkins. Truthful mechanisms with implicit payment computation. In *11th ACM Conf. on Electronic Commerce (ACM-EC)*, pages 43–52, 2010.

Moshe Babaioff, Robert Kleinberg, and Aleksandrs Slivkins. Multi-parameter mechanisms with implicit payment computation. In *13th ACM Conf. on Electronic Commerce (ACM-EC)*, pages 35–52, 2013.

Moshe Babaioff, Yogeshwer Sharma, and Aleksandrs Slivkins. Characterizing truthful multi-armed bandit mechanisms. *SIAM J. on Computing (SICOMP)*, 43(1):194–230, 2014. Preliminary version in *10th ACM EC*, 2009.

Moshe Babaioff, Shaddin Dughmi, Robert D. Kleinberg, and Aleksandrs Slivkins. Dynamic pricing with limited supply. *ACM Trans. on Economics and Computation*, 3(1):4, 2015a. Special issue for *13th ACM EC*, 2012.

Moshe Babaioff, Robert Kleinberg, and Aleksandrs Slivkins. Truthful mechanisms with implicit payment computation. *J. of the ACM*, 62(2):10:1–10:37, 2015b. Subsumes conference papers in *ACM EC 2010* and *ACM EC 2013*.

Ashwinkumar Badanidiyuru, Robert Kleinberg, and Yaron Singer. Learning on a budget: posted price mechanisms for online procurement. In *13th ACM Conf. on Electronic Commerce (ACM-EC)*, pages 128–145, 2012.

Ashwinkumar Badanidiyuru, Robert Kleinberg, and Aleksandrs Slivkins. Bandits with knapsacks. In *54th IEEE Symp. on Foundations of Computer Science (FOCS)*, 2013.

Ashwinkumar Badanidiyuru, John Langford, and Aleksandrs Slivkins. Resourceful contextual bandits. In *27th Conf. on Learning Theory (COLT)*, 2014.

Ashwinkumar Badanidiyuru, Robert Kleinberg, and Aleksandrs Slivkins. Bandits with knapsacks. *J. of the ACM*, 65 (3):13:1–13:55, 2018. Preliminary version in *FOCS 2013*.

Gal Bahar, Rann Smorodinsky, and Moshe Tennenholtz. Economic recommendation systems. In *16th ACM Conf. on Electronic Commerce (ACM-EC)*, page 757, 2016.

Gal Bahar, Rann Smorodinsky, and Moshe Tennenholtz. Social learning and the innkeeper's challenge. In *ACM Conf. on Economics and Computation (ACM-EC)*, pages 153–170, 2019.

Gal Bahar, Omer Ben-Porat, Kevin Leyton-Brown, and Moshe Tennenholtz. Fiduciary bandits. In *37th Intl. Conf. on Machine Learning (ICML)*, pages 518–527, 2020.

James P. Bailey and Georgios Piliouras. Multiplicative weights update in zero-sum games. In *ACM Conf. on Economics and Computation (ACM-EC)*, pages 321–338, 2018.

Santiago R. Balseiro and Yonatan Gur. Learning in repeated auctions with budgets: Regret minimization and equilibrium. *Manag. Sci.*, 65(9):3952–3968, 2019. Preliminary version in *ACM EC 2017*.

Kiarash Banihashem, MohammadTaghi Hajiaghayi, Suho Shin, and Aleksandrs Slivkins. Bandit social learning: Exploration under myopic behavior. In *37th Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Peter L. Bartlett, Varsha Dani, Thomas Hayes, Sham Kakade, Alexander Rakhlin, and Ambuj Tewari. High-probability regret bounds for bandit online linear optimization. In *21th Conf. on Learning Theory (COLT)*, 2008.

Gábor Bartók, Dean P. Foster, Dávid Pál, Alexander Rakhlin, and Csaba Szepesvári. Partial monitoring - classification, regret bounds, and algorithms. *Math. Oper. Res.*, 39(4):967–997, 2014.

Hamsa Bastani, Mohsen Bayati, and Khashayar Khosravi. Mostly exploration-free algorithms for contextual bandits. *Management Science*, 67(3):1329–1349, 2021. Working paper available on `arxiv.org` since 2017.

Mohsen Bayati, Nima Hamidi, Ramesh Johari, and Khashayar Khosravi. Unreasonable effectiveness of greedy algorithms in multi-armed bandit with many arms. In *33rd Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Dirk Bergemann and Stephen Morris. Robust predictions in games with incomplete information. *Econometrica*, 81 (4):1251–1308, 2013.

Dirk Bergemann and Stephen Morris. Information design: A unified perspective. *Journal of Economic Literature*, 57 (1):44–95, March 2019.

Dirk Bergemann and Maher Said. Dynamic auctions: A survey. In *Wiley Encyclopedia of Operations Research and Management Science*. John Wiley & Sons, 2011.

Dirk Bergemann and Juuso Välimäki. Market diffusion with two-sided learning. *The RAND Journal of Economics*, pages 773–795, 1997.

Dirk Bergemann and Juuso Välimäki. Experimentation in markets. *The Review of Economic Studies*, 67(2):213–234, 2000.

Dirk Bergemann and Juuso Välimäki. The dynamic pivot mechanism. *Econometrica*, 78(2):771–789, 2010. Preliminary versions have been available since 2006.

Donald A. Berry and Bert Fristedt. *Bandit problems: sequential allocation of experiments*. Springer, Heidelberg, Germany, 1985.

Omar Besbes and Assaf Zeevi. Dynamic pricing without knowing the demand function: Risk bounds and near-optimal algorithms. *Operations Research*, 57(6):1407–1420, 2009.

Omar Besbes and Assaf J. Zeevi. Blind network revenue management. *Operations Research*, 60(6):1537–1550, 2012.

Alina Beygelzimer, John Langford, Lihong Li, Lev Reyzin, and Robert E. Schapire. Contextual bandit algorithms with supervised learning guarantees. In *14th Intl. Conf. on Artificial Intelligence and Statistics (AISTATS)*, 2011.

Alberto Bietti, Alekh Agarwal, and John Langford. A contextual bandit bake-off. *J. of Machine Learning Research (JMLR)*, 22:133:1–133:49, 2021.

Kostas Bimpikis, Yiangos Papanastasiou, and Nicos Savva. Crowdsourcing exploration. *Management Science*, 64(4): 1727–1746, 2018.

Avrim Blum. Empirical support for winnow and weighted-majority based algorithms: Results on a calendar scheduling domain. *Machine Learning*, 26:5–23, 1997.

Avrim Blum and Yishay Mansour. From external to internal regret. *J. of Machine Learning Research (JMLR)*, 8(13): 1307–1324, 2007. Preliminary version in *COLT 2005*.

Avrim Blum, Vijay Kumar, Atri Rudra, and Felix Wu. Online learning in online auctions. In *14th ACM-SIAM Symp. on Discrete Algorithms (SODA)*, pages 202–204, 2003.

Avrim Blum, MohammadTaghi Hajiaghayi, Katrina Ligett, and Aaron Roth. Regret minimization and the price of total anarchy. In *40th ACM Symp. on Theory of Computing (STOC)*, pages 373–382, 2008.

Arnoud V. Den Boer. Dynamic pricing and learning: Historical origins, current research, and new directions. *Surveys in Operations Research and Management Science*, 20(1), June 2015.

Etienne Boursier and Vianney Perchet. SIC-MMAB: synchronisation involves communication in multiplayer multi-armed bandits. In *32nd Advances in Neural Information Processing Systems (NeurIPS)*, pages 12048–12057, 2019.

Mark Braverman, Jieming Mao, Jon Schneider, and Matt Weinberg. Selling to a no-regret buyer. In *ACM Conf. on Economics and Computation (ACM-EC)*, pages 523–538, 2018.

Mark Braverman, Jieming Mao, Jon Schneider, and S. Matthew Weinberg. Multi-armed bandit problems with strategic arms. In *Conf. on Learning Theory (COLT)*, pages 383–416, 2019.

Guy Bresler, George H. Chen, and Devavrat Shah. A latent source model for online collaborative filtering. In *27th Advances in Neural Information Processing Systems (NIPS)*, pages 3347–3355, 2014.

Guy Bresler, Devavrat Shah, and Luis Filipe Voloch. Collaborative filtering with low regret. In *The Intl. Conf. on Measurement and Modeling of Computer Systems (SIGMETRICS)*, pages 207–220, 2016.

George W. Brown. Some notes on computation of games solutions. Technical Report P-78, The Rand Corporation, 1949.

Sébastien Bubeck. *Bandits Games and Clustering Foundations*. PhD thesis, Univ. Lille 1, 2010.

Sébastien Bubeck and Nicolo Cesa-Bianchi. Regret Analysis of Stochastic and Nonstochastic Multi-armed Bandit Problems. *Foundations and Trends in Machine Learning*, 5(1):1–122, 2012. Published with *Now Publishers* (Boston, MA, USA). Also available at `https://arxiv.org/abs/1204.5721`.

Sébastien Bubeck and Che-Yu Liu. Prior-free and prior-dependent regret bounds for thompson sampling. In *26th Advances in Neural Information Processing Systems (NIPS)*, pages 638–646, 2013.

Sébastien Bubeck and Mark Sellke. First-order bayesian regret analysis of thompson sampling. In *31st Intl. Conf. on Algorithmic Learning Theory (ALT)*, pages 196–233, 2020.

Sébastien Bubeck and Aleksandrs Slivkins. The best of both worlds: stochastic and adversarial bandits. In *25th Conf. on Learning Theory (COLT)*, 2012.

Sébastien Bubeck, Rémi Munos, and Gilles Stoltz. Pure Exploration in Multi-Armed Bandit Problems. *Theoretical Computer Science*, 412(19):1832–1852, 2011a.

Sébastien Bubeck, Rémi Munos, Gilles Stoltz, and Csaba Szepesvari. Online Optimization in X-Armed Bandits. *J. of Machine Learning Research (JMLR)*, 12:1587–1627, 2011b. Preliminary version in *NIPS 2008*.

Sébastien Bubeck, Gilles Stoltz, and Jia Yuan Yu. Lipschitz bandits without the lipschitz constant. In *22nd Intl. Conf. on Algorithmic Learning Theory (ALT)*, pages 144–158, 2011c.

Sébastien Bubeck, Ofer Dekel, Tomer Koren, and Yuval Peres. Bandit convex optimization: \(\sqrt{T}\) regret in one dimension. In *28th Conf. on Learning Theory (COLT)*, pages 266–278, 2015.

Sébastien Bubeck, Yin Tat Lee, and Ronen Eldan. Kernel-based methods for bandit convex optimization. In *49th ACM Symp. on Theory of Computing (STOC)*, pages 72–85. ACM, 2017.

Sébastien Bubeck, Yuanzhi Li, Haipeng Luo, and Chen-Yu Wei. Improved path-length regret bounds for bandits. In *Conf. on Learning Theory (COLT)*, 2019.

Sébastien Bubeck, Yuanzhi Li, Yuval Peres, and Mark Sellke. Non-stochastic multi-player multi-armed bandits: Optimal rate with collision information, sublinear without. In *33rd Conf. on Learning Theory (COLT)*, pages 961–987, 2020.

Adam Bull. Adaptive-treed bandits. *Bernoulli J. of Statistics*, 21(4):2289–2307, 2015.

Adrian Rivera Cardoso, He Wang, and Huan Xu. Online saddle point problem with applications to constrained online convex optimization. *arXiv preprint arXiv:1806.08301*, 2018.

Felipe Caro and Jérémie Gallien. Dynamic assortment with demand learning for seasonal consumer goods. *Management Science*, 53(2):276–292, 2007.

Alexandra Carpentier and Andrea Locatelli. Tight (lower) bounds for the fixed budget best arm identification bandit problem. In *29th Conf. on Learning Theory (COLT)*, pages 590–604, 2016.

Alexandra Carpentier and Rémi Munos. Bandit theory meets compressed sensing for high dimensional stochastic linear bandit. In *15th Intl. Conf. on Artificial Intelligence and Statistics (AISTATS)*, volume 22 of *JMLR Proceedings*, pages 190–198, 2012.

Nicolò Cesa-Bianchi and Gábor Lugosi. Potential-based algorithms in on-line prediction and game theory. *Machine Learning*, 51(3):239–261, 2003.

Nicolò Cesa-Bianchi and Gábor Lugosi. *Prediction, learning, and games*. Cambridge University Press, Cambridge, UK, 2006.

Nicolò Cesa-Bianchi and Gábor Lugosi. Combinatorial bandits. *J. Comput. Syst. Sci.*, 78(5):1404–1422, 2012. Preliminary version in *COLT 2009*.

Nicolò Cesa-Bianchi, Yoav Freund, David Haussler, David P. Helmbold, Robert E. Schapire, and Manfred K. Warmuth. How to use expert advice. *J. ACM*, 44(3):427–485, 1997.

Nicoló Cesa-Bianchi, Claudio Gentile, and Yishay Mansour. Regret minimization for reserve prices in second-price auctions. In *ACM-SIAM Symp. on Discrete Algorithms (SODA)*, 2013.

Nicolò Cesa-Bianchi, Pierre Gaillard, Claudio Gentile, and Sébastien Gerchinovitz. Algorithmic chaining and the role of partial feedback in online nonparametric learning. In *30th Conf. on Learning Theory (COLT)*, pages 465–481, 2017.

Deepayan Chakrabarti, Ravi Kumar, Filip Radlinski, and Eli Upfal. Mortal multi-armed bandits. In *22nd Advances in Neural Information Processing Systems (NIPS)*, pages 273–280, 2008.

Yeon-Koo Che and Johannes Hörner. Recommender systems as mechanisms for social learning. *Quarterly Journal of Economics*, 133(2):871–925, 2018. Working paper since 2013, titled 'Optimal design for social learning'.

Bangrui Chen, Peter I. Frazier, and David Kempe. Incentivizing exploration by heterogeneous users. In *Conf. on Learning Theory (COLT)*, pages 798–818, 2018.

Tianyi Chen and Georgios B Giannakis. Bandit convex optimization for scalable and dynamic iot management. *IEEE Internet of Things Journal*, 2018.

Tianyi Chen, Qing Ling, and Georgios B Giannakis. An online convex optimization approach to proactive network resource allocation. *IEEE Transactions on Signal Processing*, 65(24):6350–6364, 2017.

Wei Chen, Yajun Wang, and Yang Yuan. Combinatorial multi-armed bandit: General framework and applications. In *20th Intl. Conf. on Machine Learning (ICML)*, pages 151–159, 2013.

Wei Chen, Wei Hu, Fu Li, Jian Li, Yu Liu, and Pinyan Lu. Combinatorial multi-armed bandit with general reward functions. In *29th Advances in Neural Information Processing Systems (NIPS)*, pages 1651–1659, 2016.

Yifang Chen, Chung-Wei Lee, Haipeng Luo, and Chen-Yu Wei. A new algorithm for non-stationary contextual bandits: Efficient, optimal, and parameter-free. In *Conf. on Learning Theory (COLT)*, 2019.

Wang Chi Cheung and David Simchi-Levi. Assortment optimization under unknown multinomial logit choice models, 2017. Technical report, available at `http://arxiv.org/abs/1704.00108`.

Yun Kuen Cheung and Georgios Piliouras. Vortices instead of equilibria in minmax optimization: Chaos and butterfly effects of online learning in zero-sum games. In *Conf. on Learning Theory (COLT)*, pages 807–834, 2019.

Shein-Chung Chow and Mark Chang. Adaptive design methods in clinical trials – a review. *Orphanet Journal of Rare Diseases*, 3(11):1750–1172, 2008.

Paul Christiano, Jonathan A. Kelner, Aleksander Madry, Daniel A. Spielman, and Shang-Hua Teng. Electrical flows, laplacian systems, and faster approximation of maximum flow in undirected graphs. In *43rd ACM Symp. on Theory of Computing (STOC)*, pages 273–282. ACM, 2011.

Wei Chu, Lihong Li, Lev Reyzin, and Robert E. Schapire. Contextual Bandits with Linear Payoff Functions. In *14th Intl. Conf. on Artificial Intelligence and Statistics (AISTATS)*, 2011.

Lee Cohen and Yishay Mansour. Optimal algorithm for bayesian incentive-compatible exploration. In *ACM Conf. on Economics and Computation (ACM-EC)*, pages 135–151, 2019.

Richard Combes and Alexandre Proutière. Unimodal bandits without smoothness, 2014a. Working paper, available at `http://arxiv.org/abs/1406.7447`.

Richard Combes and Alexandre Proutière. Unimodal bandits: Regret lower bounds and optimal algorithms. In *31st Intl. Conf. on Machine Learning (ICML)*, pages 521–529, 2014b.

Richard Combes, Chong Jiang, and Rayadurgam Srikant. Bandits with budgets: Regret lower bounds and optimal algorithms. *ACM SIGMETRICS Performance Evaluation Review*, 43(1):245–257, 2015.

Richard Combes, Stefan Magureanu, and Alexandre Proutière. Minimal exploration in structured stochastic bandits. In *30th Advances in Neural Information Processing Systems (NIPS)*, pages 1763–1771, 2017.

Eric W. Cope. Regret and convergence bounds for a class of continuum-armed bandit problems. *IEEE Trans. Autom. Control.*, 54(6):1243–1253, 2009.

Thomas Cover. Behavior of sequential predictors of binary sequences. In *Proc. of the 4th Prague Conference on Information Theory, Statistical Decision Functions and Random Processes*, pages 263–272. Publishing House of the Czechoslovak Academy of Sciences, 1965.

Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. John Wiley & Sons, New York, NY, USA, 1991.

Varsha Dani, Thomas P. Hayes, and Sham Kakade. The Price of Bandit Information for Online Optimization. In *20th Advances in Neural Information Processing Systems (NIPS)*, 2007.

Varsha Dani, Thomas P. Hayes, and Sham Kakade. Stochastic Linear Optimization under Bandit Feedback. In *21th Conf. on Learning Theory (COLT)*, pages 355–366, 2008.

Constantinos Daskalakis and Qinxuan Pan. A counter-example to karlin's strong conjecture for fictitious play. In *55th IEEE Symp. on Foundations of Computer Science (FOCS)*, pages 11–20, 2014.

Constantinos Daskalakis and Ioannis Panageas. Last-iterate convergence: Zero-sum games and constrained min-max optimization. In *10th Innovations in Theoretical Computer Science Conf. (ITCS)*, volume 124, pages 27:1–27:18, 2019.

Constantinos Daskalakis, Alan Deckelbaum, and Anthony Kim. Near-optimal no-regret algorithms for zero-sum games. *Games and Economic Behavior*, 92:327–348, 2015. Preliminary version in *ACM-SIAM SODA 2011*.

Constantinos Daskalakis, Andrew Ilyas, Vasilis Syrgkanis, and Haoyang Zeng. Training gans with optimism. In *6th International Conference on Learning Representations (ICLR)*, 2018.

Ofer Dekel, Ambuj Tewari, and Raman Arora. Online bandit learning against an adaptive adversary: from regret to policy regret. In *29th Intl. Conf. on Machine Learning (ICML)*, 2012.

Thomas Desautels, Andreas Krause, and Joel Burdick. Parallelizing exploration-exploitation tradeoffs with gaussian process bandit optimization. In *29th Intl. Conf. on Machine Learning (ICML)*, 2012.

Nikhil Devanur and Sham M. Kakade. The price of truthfulness for pay-per-click auctions. In *10th ACM Conf. on Electronic Commerce (ACM-EC)*, pages 99–106, 2009.

Nikhil R. Devanur and Thomas P. Hayes. The AdWords problem: Online keyword matching with budgeted bidders under random permutations. In *10th ACM Conf. on Electronic Commerce (ACM-EC)*, pages 71–78, 2009.

Nikhil R. Devanur, Kamal Jain, Balasubramanian Sivan, and Christopher A. Wilkens. Near optimal online algorithms and fast approximation algorithms for resource allocation problems. In *12th ACM Conf. on Electronic Commerce (ACM-EC)*, pages 29–38, 2011.

Nikhil R. Devanur, Kamal Jain, Balasubramanian Sivan, and Christopher A. Wilkens. Near optimal online algorithms and fast approximation algorithms for resource allocation problems. *J. ACM*, 66(1):7:1–7:41, 2019. Preliminary version in *ACM EC 2011*.

Wenkui Ding, Tao Qin, Xu-Dong Zhang, and Tie-Yan Liu. Multi-armed bandit with budget constraint and variable costs. In *27th AAAI Conference on Artificial Intelligence (AAAI)*, 2013.

Mo Dong, Qingxi Li, Doron Zarchy, Philip Brighten Godfrey, and Michael Schapira. PCC: re-architecting congestion control for consistent high performance. In *12th USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, pages 395–408, 2015.

Mo Dong, Tong Meng, Doron Zarchy, Engin Arslan, Yossi Gilad, Brighten Godfrey, and Michael Schapira. PCC vivace: Online-learning congestion control. In *15th USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, pages 343–356, 2018.

Devdatt P. Dubhashi and Alessandro Panconesi. *Concentration of Measure for the Analysis of Randomized Algorithms*. Cambridge University Press, 2009.

Miroslav Dudík, Daniel Hsu, Satyen Kale, Nikos Karampatziakis, John Langford, Lev Reyzin, and Tong Zhang. Efficient optimal leanring for contextual bandits. In *27th Conf. on Uncertainty in Artificial Intelligence (UAI)*, 2011.

Miroslav Dudík, Dumitru Erhan, John Langford, and Lihong Li. Sample-efficient nonstationary policy evaluation for contextual bandits. In *28th Conf. on Uncertainty in Artificial Intelligence (UAI)*, pages 247–254, 2012.

Miroslav Dudík, Dumitru Erhan, John Langford, and Lihong Li. Doubly robust policy evaluation and optimization. *Statistical Science*, 29(4):1097–1104, 2014.

Miroslav Dudík, Katja Hofmann, Robert E. Schapire, Aleksandrs Slivkins, and Masrour Zoghi. Contextual dueling bandits. In *28th Conf. on Learning Theory (COLT)*, 2015.

Miroslav Dudík, Nika Haghtalab, Haipeng Luo, Robert E. Schapire, Vasilis Syrgkanis, and Jennifer Wortman Vaughan. Oracle-efficient online learning and auction design. In *58th IEEE Symp. on Foundations of Computer Science (FOCS)*, pages 528–539, 2017.

Eyal Even-Dar, Shie Mannor, and Yishay Mansour. PAC bounds for multi-armed bandit and Markov decision processes. In *15th Conf. on Learning Theory (COLT)*, pages 255–270, 2002.

Eyal Even-Dar, Shie Mannor, and Yishay Mansour. Action elimination and stopping conditions for the multi-armed bandit and reinforcement learning problems. *J. of Machine Learning Research (JMLR)*, 7:1079–1105, 2006.

Uriel Feige, Tomer Koren, and Moshe Tennenholtz. Chasing ghosts: Competing with stateful policies. *SIAM J. on Computing (SICOMP)*, 46(1):190–223, 2017. Preliminary version in *IEEE FOCS 2014*.

Jon Feldman, Monika Henzinger, Nitish Korula, Vahab S. Mirrokni, and Clifford Stein. Online stochastic packing applied to display ad allocation. In *18th Annual European Symp. on Algorithms (ESA)*, pages 182–194, 2010.

Zhe Feng, Chara Podimata, and Vasilis Syrgkanis. Learning to bid without knowing your value. In *19th ACM Conf. on Economics and Computation (ACM-EC)*, pages 505–522, 2018.

Sarah Filippi, Olivier Cappé, Aurélien Garivier, and Csaba Szepesvári. Parametric bandits: The generalized linear case. In *24th Advances in Neural Information Processing Systems (NIPS)*, pages 586–594, 2010.

Arthur Flajolet and Patrick Jaillet. Logarithmic regret bounds for bandits with knapsacks. *arXiv preprint arXiv:1510.01800*, 2015.

Abraham Flaxman, Adam Kalai, and H. Brendan McMahan. Online Convex Optimization in the Bandit Setting: Gradient Descent without a Gradient. In *16th ACM-SIAM Symp. on Discrete Algorithms (SODA)*, pages 385–394, 2005.

Dean Foster and Rakesh Vohra. Calibrated learning and correlated equilibrium. *Games and Economic Behavior*, 21: 40–55, 1997.

Dean Foster and Rakesh Vohra. Asymptotic calibration. *Biometrika*, 85:379–390, 1998.

Dean Foster and Rakesh Vohra. Regret in the on-line decision problem. *Games and Economic Behavior*, 29:7–36, 1999.

Dylan J. Foster and Alexander Rakhlin. Beyond UCB: optimal and efficient contextual bandits with regression oracles. In *37th Intl. Conf. on Machine Learning (ICML)*, 2020.

Dylan J. Foster, Zhiyuan Li, Thodoris Lykouris, Karthik Sridharan, and Éva Tardos. Learning in games: Robustness of fast convergence. In *29th Advances in Neural Information Processing Systems (NIPS)*, pages 4727–4735, 2016a.

Dylan J. Foster, Zhiyuan Li, Thodoris Lykouris, Karthik Sridharan, and Éva Tardos. Learning in games: Robustness of fast convergence. In *29th Advances in Neural Information Processing Systems (NIPS)*, pages 4727–4735, 2016b.

Dylan J. Foster, Alekh Agarwal, Miroslav Dudík, Haipeng Luo, and Robert E. Schapire. Practical contextual bandits with regression oracles. In *35th Intl. Conf. on Machine Learning (ICML)*, pages 1534–1543, 2018.

Peter Frazier, David Kempe, Jon M. Kleinberg, and Robert Kleinberg. Incentivizing exploration. In *ACM Conf. on Economics and Computation (ACM-EC)*, pages 5–22, 2014.

Yoav Freund and Robert E Schapire. Game theory, on-line prediction and boosting. In *9th Conf. on Learning Theory (COLT)*, pages 325–332, 1996.

Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.

Yoav Freund and Robert E Schapire. Adaptive game playing using multiplicative weights. *Games and Economic Behavior*, 29(1-2):79–103, 1999.

Yoav Freund, Robert E Schapire, Yoram Singer, and Manfred K Warmuth. Using and combining predictors that specialize. In *29th ACM Symp. on Theory of Computing (STOC)*, pages 334–343, 1997.

Aurélien Garivier and Olivier Cappé. The KL-UCB Algorithm for Bounded Stochastic Bandits and Beyond. In *24th Conf. on Learning Theory (COLT)*, 2011.

Aurélien Garivier and Eric Moulines. On upper-confidence bound policies for switching bandit problems. In *22nd Intl. Conf. on Algorithmic Learning Theory (ALT)*, pages 174–188, 2011.

Nicola Gatti, Alessandro Lazaric, and Francesco Trovo. A Truthful Learning Mechanism for Contextual Multi-Slot Sponsored Search Auctions with Externalities. In *13th ACM Conf. on Electronic Commerce (ACM-EC)*, 2012.

Arpita Ghosh and Patrick Hummel. Learning and incentives in user-generated content: multi-armed bandits with endogenous arms. In *Innovations in Theoretical Computer Science Conf. (ITCS)*, pages 233–246, 2013.

J. C. Gittins. Bandit processes and dynamic allocation indices (with discussion). *J. Roy. Statist. Soc. Ser. B*, 41: 148–177, 1979.

John Gittins, Kevin Glazebrook, and Richard Weber. *Multi-Armed Bandit Allocation Indices*. John Wiley & Sons, Hoboken, NJ, USA, 2nd edition, 2011.

Daniel Golovin, Andreas Krause, and Matthew Streeter. Online learning of assignments. In *Advances in Neural Information Processing Systems (NIPS)*, 2009.

Noah Golowich, Sarath Pattathil, and Constantinos Daskalakis. Tight last-iterate convergence rates for no-regret learning in multi-player games. In *33rd Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Benjamin Golub and Evan D. Sadler. Learning in social networks. In Yann Bramoullé, Andrea Galeotti, and Brian Rogers, editors, *The Oxford Handbook of the Economics of Networks*. Oxford University Press, 2016.

Nick Gravin, Yuval Peres, and Balasubramanian Sivan. Towards optimal algorithms for prediction with expert advice. In *27th ACM-SIAM Symp. on Discrete Algorithms (SODA)*, pages 528–547, 2016.

Nick Gravin, Yuval Peres, and Balasubramanian Sivan. Tight lower bounds for multiplicative weights algorithmic families. In *44th Intl. Colloquium on Automata, Languages and Programming (ICALP)*, pages 48:1–48:14, 2017.

Jean-Bastien Grill, Michal Valko, and Rémi Munos. Black-box optimization of noisy functions with unknown smoothness. In *28th Advances in Neural Information Processing Systems (NIPS)*, 2015.

Anupam Gupta, Robert Krauthgamer, and James R. Lee. Bounded geometries, fractals, and low–distortion embeddings. In *44th IEEE Symp. on Foundations of Computer Science (FOCS)*, pages 534–543, 2003.

Anupam Gupta, Tomer Koren, and Kunal Talwar. Better algorithms for stochastic bandits with adversarial corruptions. In *Conf. on Learning Theory (COLT)*, pages 1562–1578, 2019.

András György, Levente Kocsis, Ivett Szabó, and Csaba Szepesvári. Continuous time associative bandit problems. In *20th Intl. Joint Conf. on Artificial Intelligence (IJCAI)*, pages 830–835, 2007.

András György, Tamás Linder, Gábor Lugosi, and György Ottucsák. The on-line shortest path problem under partial monitoring. *J. of Machine Learning Research (JMLR)*, 8:2369–2403, 2007.

Yuxuan Han, Jialin Zeng, Yang Wang, Yang Xiang, and Jiheng Zhang. Optimal contextual bandits with knapsacks under realizibility via regression oracles. In *26th Intl. Conf. on Artificial Intelligence and Statistics (AISTATS)*, 2023. Available at `arxiv.org/abs/2210.11834` since October 2022.

James Hannan. Approximation to bayes risk in repeated play. *Contributions to the Theory of Games*, 3:97–139, 1957.

Sergiu Hart and Andreu Mas-Colell. A simple adaptive procedure leading to correlated equilibrium. *Econometrica*, 68:1127–1150, 2000.

Elad Hazan. Introduction to Online Convex Optimization. *Foundations and Trends in Optimization*, 2(3-4):157–325, 2015. Published with *Now Publishers* (Boston, MA, USA). Also available at https://arxiv.org/abs/1909.05207.

Elad Hazan and Satyen Kale. Better algorithms for benign bandits. *Journal of Machine Learning Research*, 12: 1287–1311, 2011. Preliminary version published in *ACM-SIAM SODA 2009*.

Elad Hazan and Kfir Y. Levy. Bandit convex optimization: Towards tight bounds. In *27th Advances in Neural Information Processing Systems (NIPS)*, pages 784–792, 2014.

Elad Hazan and Nimrod Megiddo. Online Learning with Prior Information. In *20th Conf. on Learning Theory (COLT)*, pages 499–513, 2007.

Hoda Heidari, Mohammad Mahdian, Umar Syed, Sergei Vassilvitskii, and Sadra Yazdanbod. Pricing a low-regret seller. In *33rd Intl. Conf. on Machine Learning (ICML)*, pages 2559–2567, 2016.

Chien-Ju Ho, Aleksandrs Slivkins, and Jennifer Wortman Vaughan. Adaptive contract design for crowdsourcing markets: Bandit algorithms for repeated principal-agent problems. *J. of Artificial Intelligence Research*, 55:317–359, 2016. Preliminary version appeared in *ACM EC 2014*.

Katja Hofmann, Lihong Li, and Filip Radlinski. Online evaluation for information retrieval. *Foundations and Trends® in Information Retrieval*, 10(1):1–117, 2016. Published with *Now Publishers* (Boston, MA, USA).

Junya Honda and Akimichi Takemura. An asymptotically optimal bandit algorithm for bounded support models. In *23rd Conf. on Learning Theory (COLT)*, 2010.

Johannes Hörner and Andrzej Skrzypacz. Learning, experimentation, and information design. In Bo Honoré, Ariel Pakes, Monika Piazzesi, and Larry Samuelson, editors, *Advances in Economics and Econometrics: 11th World Congress*, volume 1, page 63–98. Cambridge University Press, 2017.

Justin Hsu, Zhiyi Huang, Aaron Roth, and Zhiwei Steven Wu. Jointly private convex programming. In *27th ACM-SIAM Symp. on Discrete Algorithms (SODA)*, pages 580–599, 2016.

Nicole Immorlica, Jieming Mao, Aleksandrs Slivkins, and Steven Wu. Bayesian exploration with heterogenous agents. In *The Web Conference (formerly known as* WWW*)*, 2019.

Nicole Immorlica, Jieming Mao, Aleksandrs Slivkins, and Steven Wu. Incentivizing exploration with selective data disclosure. In *ACM Conf. on Economics and Computation (ACM-EC)*, pages 647–648, 2020. Working paper available at `https://arxiv.org/abs/1811.06026`.

Nicole Immorlica, Karthik Abinav Sankararaman, Robert Schapire, and Aleksandrs Slivkins. Adversarial bandits with knapsacks. *J. of the ACM*, August 2022. Preliminary version in *60th IEEE FOCS*, 2019.

Matthieu Jedor, Jonathan Louëdec, and Vianney Perchet. Be greedy in multi-armed bandits, 2021. Working paper, available on `https://arxiv.org/abs/2101.01086`.

Junchen Jiang, Rajdeep Das, Ganesh Ananthanarayanan, Philip A. Chou, Venkat N. Padmanabhan, Vyas Sekar, Esbjorn Dominique, Marcin Goliszewski, Dalibor Kukoleca, Renat Vafin, and Hui Zhang. Via: Improving internet telephony call quality using predictive relay selection. In *ACM SIGCOMM (ACM SIGCOMM Conf. on Applications, Technologies, Architectures, and Protocols for Computer Communications)*, pages 286–299, 2016.

Junchen Jiang, Shijie Sun, Vyas Sekar, and Hui Zhang. Pytheas: Enabling data-driven quality of experience optimization using group-based exploration-exploitation. In *14th USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, pages 393–406, 2017.

Sham M. Kakade, Ilan Lobel, and Hamid Nazerzadeh. Optimal dynamic mechanism design and the virtual-pivot mechanism. *Operations Research*, 61(4):837–854, 2013.

Adam Tauman Kalai and Santosh Vempala. Efficient algorithms for online decision problems. *J. of Computer and Systems Sciences*, 71(3):291–307, 2005. Preliminary version in *COLT 2003*.

Satyen Kale, Lev Reyzin, and Robert E. Schapire. Non-Stochastic Bandit Slate Problems. In *24th Advances in Neural Information Processing Systems (NIPS)*, pages 1054–1062, 2010.

Nathan Kallus. Instrument-armed bandits. In *29th Intl. Conf. on Algorithmic Learning Theory (ALT)*, pages 529–546, 2018.

Emir Kamenica. Bayesian persuasion and information design. *Annual Review of Economics*, 11(1):249–272, 2019.

Emir Kamenica and Matthew Gentzkow. Bayesian Persuasion. *American Economic Review*, 101(6):2590–2615, 2011.

Sampath Kannan, Jamie Morgenstern, Aaron Roth, Bo Waggoner, and Zhiwei Steven Wu. A smoothed analysis of the greedy algorithm for the linear contextual bandit problem. In *Advances in Neural Information Processing Systems (NIPS)*, pages 2231–2241, 2018.

D.R. Karger and M. Ruhl. Finding Nearest Neighbors in Growth-restricted Metrics. In *34th ACM Symp. on Theory of Computing (STOC)*, pages 63–66, 2002.

Emilie Kaufmann, Nathaniel Korda, and Rémi Munos. Thompson sampling: An asymptotically optimal finite-time analysis. In *23rd Intl. Conf. on Algorithmic Learning Theory (ALT)*, pages 199–213, 2012.

Emilie Kaufmann, Olivier Cappé, and Aurélien Garivier. On the complexity of best-arm identification in multi-armed bandit models. *J. of Machine Learning Research (JMLR)*, 17:1:1–1:42, 2016.

Michael Kearns, Seth Neel, Aaron Roth, and Zhiwei Steven Wu. Preventing fairness gerrymandering: Auditing and learning for subgroup fairness. In *35th Intl. Conf. on Machine Learning (ICML)*, pages 2564–2572, 2018.

Godfrey Keller and Sven Rady. Price dispersion and learning in a dynamic differentiated-goods duopoly. *RAND Journal of Economics*, pages 138–165, 2003.

Thomas Kesselheim and Sahil Singla. Online learning with vector costs and bandits with knapsacks. In *33rd Conf. on Learning Theory (COLT)*, pages 2286–2305, 2020.

Jon Kleinberg and Eva Tardos. *Algorithm Design*. Addison Wesley, 2005.

Jon Kleinberg, Aleksandrs Slivkins, and Tom Wexler. Triangulation and embedding using small sets of beacons. *J. of the ACM*, 56(6), September 2009a. Subsumes conference papers in *IEEE FOCS 2004* and *ACM-SIAM SODA 2005*.

Robert Kleinberg. Nearly tight bounds for the continuum-armed bandit problem. In *18th Advances in Neural Information Processing Systems (NIPS)*, 2004.

Robert Kleinberg. Anytime algorithms for multi-armed bandit problems. In *17th ACM-SIAM Symp. on Discrete Algorithms (SODA)*, pages 928–936, 2006.

Robert Kleinberg. *CS683: Learning, Games, and Electronic Markets*, a class at Cornell University. Lecture notes, available at `http://www.cs.cornell.edu/courses/cs683/2007sp/`, Spring 2007.

Robert Kleinberg and Aleksandrs Slivkins. Sharp dichotomies for regret minimization in metric spaces. In *21st ACM-SIAM Symp. on Discrete Algorithms (SODA)*, 2010.

Robert Kleinberg, Alexandru Niculescu-Mizil, and Yogeshwer Sharma. Regret bounds for sleeping experts and bandits. In *21st Conf. on Learning Theory (COLT)*, pages 425–436, 2008a.

Robert Kleinberg, Aleksandrs Slivkins, and Eli Upfal. Multi-armed bandits in metric spaces. In *40th ACM Symp. on Theory of Computing (STOC)*, pages 681–690, 2008b.

Robert Kleinberg, Georgios Piliouras, and Éva Tardos. Multiplicative updates outperform generic no-regret learning in congestion games: extended abstract. In *41st ACM Symp. on Theory of Computing (STOC)*, pages 533–542, 2009b.

Robert Kleinberg, Aleksandrs Slivkins, and Eli Upfal. Bandits and experts in metric spaces. *J. of the ACM*, 66(4): 30:1–30:77, May 2019. Merged and revised version of conference papers in *ACM STOC 2008* and *ACM-SIAM SODA 2010*. Also available at `http://arxiv.org/abs/1312.1277`.

Robert D. Kleinberg and Frank T. Leighton. The value of knowing a demand curve: Bounds on regret for online posted-price auctions. In *IEEE Symp. on Foundations of Computer Science (FOCS)*, pages 594–605, 2003.

Levente Kocsis and Csaba Szepesvari. Bandit Based Monte-Carlo Planning. In *17th European Conf. on Machine Learning (ECML)*, pages 282–293, 2006.

Wouter M. Koolen, Manfred K. Warmuth, and Jyrki Kivinen. Hedging structured concepts. In *23rd Conf. on Learning Theory (COLT)*, 2010.

Andreas Krause and Cheng Soon Ong. Contextual gaussian process bandit optimization. In *25th Advances in Neural Information Processing Systems (NIPS)*, pages 2447–2455, 2011.

Ilan Kremer, Yishay Mansour, and Motty Perry. Implementing the "wisdom of the crowd". *J. of Political Economy*, 122(5):988–1012, 2014. Preliminary version in *ACM EC 2013*.

Akshay Krishnamurthy, Alekh Agarwal, and Miroslav Dudík. Contextual semibandits via supervised learning oracles. In *29th Advances in Neural Information Processing Systems (NIPS)*, 2016.

Akshay Krishnamurthy, John Langford, Aleksandrs Slivkins, and Chicheng Zhang. Contextual bandits with continuous actions: Smoothing, zooming, and adapting. *J. of Machine Learning Research (JMLR)*, 27(137):1–45, 2020. Preliminary version at *COLT 2019*.

Branislav Kveton, Zheng Wen, Azin Ashkan, Hoda Eydgahi, and Brian Eriksson. Matroid bandits: Fast combinatorial optimization with learning. In *13th Conf. on Uncertainty in Artificial Intelligence (UAI)*, pages 420–429, 2014.

Branislav Kveton, Csaba Szepesvári, Zheng Wen, and Azin Ashkan. Cascading bandits: Learning to rank in the cascade model. In *32nd Intl. Conf. on Machine Learning (ICML)*, pages 767–776, 2015a.

Branislav Kveton, Zheng Wen, Azin Ashkan, and Csaba Szepesvári. Combinatorial cascading bandits. In *28th Advances in Neural Information Processing Systems (NIPS)*, pages 1450–1458, 2015b.

Branislav Kveton, Zheng Wen, Azin Ashkan, and Csaba Szepesvári. Tight regret bounds for stochastic combinatorial semi-bandits. In *18th Intl. Conf. on Artificial Intelligence and Statistics (AISTATS)*, 2015c.

Jean-Jacques Laffont and David Martimort. *The Theory of Incentives: The Principal-Agent Model*. Princeton University Press, 2002.

Lifeng Lai, Hai Jiang, and H. Vincent Poor. Medium access in cognitive radio networks: A competitive multi-armed bandit framework. In *42nd Asilomar Conference on Signals, Systems and Computers*, 2008.

Tze Leung Lai and Herbert Robbins. Asymptotically efficient Adaptive Allocation Rules. *Advances in Applied Mathematics*, 6:4–22, 1985.

John Langford and Tong Zhang. The Epoch-Greedy Algorithm for Contextual Multi-armed Bandits. In *21st Advances in Neural Information Processing Systems (NIPS)*, 2007.

Tor Lattimore and Csaba Szepesvári. *Bandit Algorithms*. Cambridge University Press, Cambridge, UK, 2020.

Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. A contextual-bandit approach to personalized news article recommendation. In *19th Intl. World Wide Web Conf. (WWW)*, 2010.

Lihong Li, Wei Chu, John Langford, and Xuanhui Wang. Unbiased offline evaluation of contextual-bandit-based news article recommendation algorithms. In *4th ACM Intl. Conf. on Web Search and Data Mining (WSDM)*, 2011.

Lihong Li, Shunbao Chen, Jim Kleban, and Ankur Gupta. Counterfactual estimation and optimization of click metrics in search engines: A case study. In *24th Intl. World Wide Web Conf. (WWW)*, pages 929–934, 2015.

Lihong Li, Yu Lu, and Dengyong Zhou. Provably optimal algorithms for generalized linear contextual bandits. In *34th Intl. Conf. on Machine Learning (ICML)*, volume 70, pages 2071–2080, 2017.

Shuai Li, Alexandros Karatzoglou, and Claudio Gentile. Collaborative filtering bandits. In *16th ACM Intl. Conf. on Research and Development in Information Retrieval (SIGIR)*, pages 539–548, 2016.

Xiaocheng Li, Chunlin Sun, and Yinyu Ye. The symmetry between arms and knapsacks: A primal-dual approach for bandits with knapsacks. In *38th Intl. Conf. on Machine Learning (ICML)*, 2021.

Nick Littlestone and Manfred K. Warmuth. The weighted majority algorithm. *Information and Computation*, 108(2): 212–260, 1994.

Keqin Liu and Qing Zhao. Distributed learning in multi-armed bandit with multiple players. *IEEE Trans. Signal Processing*, 58(11):5667–5681, 2010.

Tyler Lu, Dávid Pál, and Martin Pál. Showing Relevant Ads via Lipschitz Context Multi-Armed Bandits. In *14th Intl. Conf. on Artificial Intelligence and Statistics (AISTATS)*, 2010.

Haipeng Luo, Chen-Yu Wei, Alekh Agarwal, and John Langford. Efficient contextual bandits in non-stationary worlds. In *Conf. on Learning Theory (COLT)*, pages 1739–1776, 2018.

Thodoris Lykouris, Vasilis Syrgkanis, and Éva Tardos. Learning and efficiency in games with dynamic population. In *27th ACM-SIAM Symp. on Discrete Algorithms (SODA)*, pages 120–129, 2016.

Thodoris Lykouris, Vahab Mirrokni, and Renato Paes-Leme. Stochastic bandits robust to adversarial corruptions. In *50th ACM Symp. on Theory of Computing (STOC)*, 2018a.

Thodoris Lykouris, Karthik Sridharan, and Éva Tardos. Small-loss bounds for online learning with partial information. In *31st Conf. on Learning Theory (COLT)*, pages 979–986, 2018b.

Stefan Magureanu, Richard Combes, and Alexandre Proutiere. Lipschitz bandits: Regret lower bound and optimal algorithms. In *27th Conf. on Learning Theory (COLT)*, pages 975–999, 2014.

Mehrdad Mahdavi, Rong Jin, and Tianbao Yang. Trading regret for efficiency: online convex optimization with long term constraints. *J. of Machine Learning Research (JMLR)*, 13(Sep):2503–2528, 2012.

Mehrdad Mahdavi, Tianbao Yang, and Rong Jin. Stochastic convex optimization with multiple objectives. In *Advances in Neural Information Processing Systems (NIPS)*, pages 1115–1123, 2013.

Odalric-Ambrym Maillard and Rémi Munos. Online Learning in Adversarial Lipschitz Environments. In *European Conf. on Machine Learning and Principles and Practice of Knowledge Discovery in Databases (ECML PKDD)*, pages 305–320, 2010.

Odalric-Ambrym Maillard, Rémi Munos, and Gilles Stoltz. A finite-time analysis of multi-armed bandits problems with kullback-leibler divergences. In *24th Conf. on Learning Theory (COLT)*, 2011.

Maryam Majzoubi, Chicheng Zhang, Rajan Chari, Akshay Krishnamurthy, John Langford, and Aleksandrs Slivkins. Efficient contextual bandits with continuous actions. In *33rd Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Shie Mannor and John N. Tsitsiklis. The sample complexity of exploration in the multi-armed bandit problem. *J. of Machine Learning Research (JMLR)*, 5:623–648, 2004.

Yishay Mansour, Aleksandrs Slivkins, and Steven Wu. Competing bandits: Learning under competition. In *9th Innovations in Theoretical Computer Science Conf. (ITCS)*, 2018.

Yishay Mansour, Aleksandrs Slivkins, and Vasilis Syrgkanis. Bayesian incentive-compatible bandit exploration. *Operations Research*, 68(4):1132–1161, 2020. Preliminary version in *ACM EC 2015*.

Yishay Mansour, Aleksandrs Slivkins, Vasilis Syrgkanis, and Steven Wu. Bayesian exploration: Incentivizing exploration in Bayesian games. *Operations Research*, 70(2):1105–1127, 2022. Preliminary version in *ACM EC 2016*.

Colin McDiarmid. Concentration. In M. Habib. C. McDiarmid. J. Ramirez and B. Reed, editors, *Probabilistic Methods for Discrete Mathematics*, pages 195–248. Springer-Verlag, Berlin, 1998.

H. Brendan McMahan. A survey of algorithms and analysis for adaptive online learning. *J. of Machine Learning Research (JMLR)*, 18:90:1–90:50, 2017.

H. Brendan McMahan and Avrim Blum. Online Geometric Optimization in the Bandit Setting Against an Adaptive Adversary. In *17th Conf. on Learning Theory (COLT)*, pages 109–123, 2004.

Neri Merhav, Erik Ordentlich, Gadiel Seroussi, and Marcelo J. Weinberger. On sequential strategies for loss functions with memory. *IEEE Trans. on Information Theory*, 48(7):1947–1958, 2002.

Panayotis Mertikopoulos, Christos H. Papadimitriou, and Georgios Piliouras. Cycles in adversarial regularized learning. In *29th ACM-SIAM Symp. on Discrete Algorithms (SODA)*, pages 2703–2717, 2018.

Stanislav Minsker. Estimation of extreme values and associated level sets of a regression function via selective sampling. In *26th Conf. on Learning Theory (COLT)*, pages 105–121, 2013.

Marco Molinaro and R. Ravi. Geometry of online packing linear programs. In *39th Intl. Colloquium on Automata, Languages and Programming (ICALP)*, pages 701–713, 2012.

Herve Moulin and Jean-Paul Vial. Strategically zero-sum games: the class of games whose completely mixed equilibria cannot be improved upon. *Intl. J. of Game Theory*, 7(3):201–221, 1978.

Rémi Munos. Optimistic optimization of a deterministic function without the knowledge of its smoothness. In *25th Advances in Neural Information Processing Systems (NIPS)*, pages 783–791, 2011.

Rémi Munos. From bandits to monte-carlo tree search: The optimistic principle applied to optimization and planning. *Foundations and Trends in Machine Learning*, 7(1):1–129, 2014.

Rémi Munos and Pierre-Arnaud Coquelin. Bandit algorithms for tree search. In *23rd Conf. on Uncertainty in Artificial Intelligence (UAI)*, 2007.

Hamid Nazerzadeh, Amin Saberi, and Rakesh Vohra. Dynamic pay-per-action mechanisms and applications to online advertising. *Operations Research*, 61(1):98–111, 2013. Preliminary version in *WWW 2008*.

Michael J Neely and Hao Yu. Online convex optimization with time-varying constraints. *arXiv preprint arXiv:1702.04783*, 2017.

Denis Nekipelov, Vasilis Syrgkanis, and Éva Tardos. Econometrics for learning agents. In *16th ACM Conf. on Electronic Commerce (ACM-EC)*, pages 1–18, 2015.

Gergely Neu. First-order regret bounds for combinatorial semi-bandits. In *28th Conf. on Learning Theory (COLT)*, 2015.

Daniel Ngo, Logan Stapleton, Vasilis Syrgkanis, and Steven Wu. Incentivizing compliance with algorithmic instrumentsincentivizing compliance with algorithmic instruments. In *38th Intl. Conf. on Machine Learning (ICML)*, 2021.

Noam Nisan and Gali Noti. An experimental evaluation of regret-based econometrics. In *26th Intl. World Wide Web Conf. (WWW)*, pages 73–81, 2017.

Sandeep Pandey, Deepak Agarwal, Deepayan Chakrabarti, and Vanja Josifovski. Bandits for Taxonomies: A Model-based Approach. In *SIAM Intl. Conf. on Data Mining (SDM)*, 2007a.

Sandeep Pandey, Deepayan Chakrabarti, and Deepak Agarwal. Multi-armed Bandit Problems with Dependent Arms. In *24th Intl. Conf. on Machine Learning (ICML)*, 2007b.

Alessandro Pavan, Ilya Segal, and Juuso Toikka. Dynamic Mechanism Design: Revenue Equivalence, Profit Maximization, and Information Disclosure. Working paper, 2011.

Chara Podimata and Aleksandrs Slivkins. Adaptive discretization for adversarial lipschitz bandits. In *34th Conf. on Learning Theory (COLT)*, 2021.

Filip Radlinski, Robert Kleinberg, and Thorsten Joachims. Learning diverse rankings with multi-armed bandits. In *25th Intl. Conf. on Machine Learning (ICML)*, pages 784–791, 2008.

Manish Raghavan, Aleksandrs Slivkins, Jennifer Wortman Vaughan, and Zhiwei Steven Wu. Greedy algorithm almost dominates in smoothed contextual bandits. *SIAM J. on Computing (SICOMP)*, 52(2):487–524, 2023. Preliminary version at *COLT 2018*. Working paper available at `arxiv.org/abs/2005.10624`.

Alexander Rakhlin and Karthik Sridharan. Online learning with predictable sequences. In *26th Conf. on Learning Theory (COLT)*, volume 30, pages 993–1019, 2013a.

Alexander Rakhlin and Karthik Sridharan. Optimization, learning, and games with predictable sequences. In *27th Advances in Neural Information Processing Systems (NIPS)*, pages 3066–3074, 2013b.

Alexander Rakhlin and Karthik Sridharan. BISTRO: an efficient relaxation-based method for contextual bandits. In *33nd Intl. Conf. on Machine Learning (ICML)*, 2016.

Alexander Rakhlin, Karthik Sridharan, and Ambuj Tewari. Online learning via sequential complexities. *J. of Machine Learning Research (JMLR)*, 16:155–186, 2015.

Anshuka Rangi, Massimo Franceschetti, and Long Tran-Thanh. Unifying the stochastic and the adversarial bandits with knapsack. In *28th Intl. Joint Conf. on Artificial Intelligence (IJCAI)*, pages 3311–3317, 2019.

Julia Robinson. An iterative method of solving a game. *Annals of Mathematics, Second Series*, 54(2):296–301, 1951.

Ryan Rogers, Aaron Roth, Jonathan Ullman, and Zhiwei Steven Wu. Inducing approximately optimal flow using truthful mediators. In *16th ACM Conf. on Electronic Commerce (ACM-EC)*, pages 471–488, 2015.

Jonathan Rosenski, Ohad Shamir, and Liran Szlak. Multi-player bandits - a musical chairs approach. In *33nd Intl. Conf. on Machine Learning (ICML)*, pages 155–163, 2016.

Aaron Roth, Jonathan Ullman, and Zhiwei Steven Wu. Watch and learn: Optimizing from revealed preferences feedback. In *48th ACM Symp. on Theory of Computing (STOC)*, pages 949–962, 2016.

Aaron Roth, Aleksandrs Slivkins, Jonathan Ullman, and Zhiwei Steven Wu. Multidimensional dynamic pricing for welfare maximization. In *18th ACM Conf. on Electronic Commerce (ACM-EC)*, pages 519–536, 2017.

Tim Roughgarden. Intrinsic robustness of the price of anarchy. In *41st ACM Symp. on Theory of Computing (STOC)*, pages 513–522, 2009.

Tim Roughgarden. *Twenty Lectures on Algorithmic Game Theory*. Cambridge University Press, 2016.

Paat Rusmevichientong and John N. Tsitsiklis. Linearly parameterized bandits. *Mathematics of Operations Research*, 35(2):395–411, 2010.

Paat Rusmevichientong, Zuo-Jun Max Shen, and David B Shmoys. Dynamic assortment optimization with a multinomial logit choice model and capacity constraint. *Operations research*, 58(6):1666–1680, 2010.

Daniel Russo and Benjamin Van Roy. Learning to optimize via posterior sampling. *Mathematics of Operations Research*, 39(4):1221–1243, 2014.

Daniel Russo and Benjamin Van Roy. An information-theoretic analysis of thompson sampling. *J. of Machine Learning Research (JMLR)*, 17:68:1–68:30, 2016.

Daniel Russo, Benjamin Van Roy, Abbas Kazerouni, Ian Osband, and Zheng Wen. A tutorial on thompson sampling. *Foundations and Trends in Machine Learning*, 11(1):1–96, 2018. Published with *Now Publishers* (Boston, MA, USA). Also available at `https://arxiv.org/abs/1707.02038`.

Karthik Abinav Sankararaman and Aleksandrs Slivkins. Combinatorial semi-bandits with knapsacks. In *Intl. Conf. on Artificial Intelligence and Statistics (AISTATS)*, pages 1760–1770, 2018.

Karthik Abinav Sankararaman and Aleksandrs Slivkins. Bandits with knapsacks beyond the worst-case, 2021. Working paper. Available at `https://arxiv.org/abs/2002.00253` since 2020.

Denis Sauré and Assaf Zeevi. Optimal dynamic assortment planning with demand learning. *Manufacturing & Service Operations Management*, 15(3):387–404, 2013.

Denis Sauré and Assaf Zeevi. Optimal dynamic assortment planning with demand learning. *Manufacturing & Service Operations Management*, 15(3):387–404, 2013.

Sven Schmit and Carlos Riquelme. Human interaction with recommendation systems. In *Intl. Conf. on Artificial Intelligence and Statistics (AISTATS)*, pages 862–870, 2018.

Manfred Schroeder. *Fractal, Chaos and Power Laws: Minutes from an Infinite Paradise*. W. H. Freeman and Co., 1991.

Joseph Schumpeter. *Capitalism, Socialism and Democracy*. Harper & Brothers, 1942.

Yevgeny Seldin and Gabor Lugosi. A lower bound for multi-armed bandits with expert advice. In *13th European Workshop on Reinforcement Learning (EWRL)*, 2016.

Yevgeny Seldin and Gábor Lugosi. An improved parametrization and analysis of the EXP3++ algorithm for stochastic and adversarial bandits. In *30th Conf. on Learning Theory (COLT)*, 2017.

Yevgeny Seldin and Aleksandrs Slivkins. One practical algorithm for both stochastic and adversarial bandits. In *31th Intl. Conf. on Machine Learning (ICML)*, 2014.

Mark Sellke, 2019. Personal communication.

Mark Sellke and Aleksandrs Slivkins. The price of incentivizing exploration: A characterization via thompson sampling and sample complexity. *Operations Research*, 71(5), 2022. Preliminary version in *ACM EC 2021*.

Shai Shalev-Shwartz. Online learning and online convex optimization. *Foundations and Trends in Machine Learning*, 4(2):107–194, 2012.

Ohad Shamir. On the complexity of bandit linear optimization. In *28th Conf. on Learning Theory (COLT)*, pages 1523–1551, 2015.

David Simchi-Levi and Yunzong Xu. Bypassing the monster: A faster and simpler optimal algorithm for contextual bandits under realizability, 2020. Working paper, available at `https://arxiv.org/abs/2003.12699`.

Max Simchowitz and Aleksandrs Slivkins. Incentives and exploration in reinforcement learning. *Operations Research*, 2023. Ahead of Print. Working paper at `arxiv.org` since 2021.

Max Simchowitz, Kevin G. Jamieson, and Benjamin Recht. Best-of-k-bandits. In *29th Conf. on Learning Theory (COLT)*, volume 49, pages 1440–1489, 2016.

Adish Singla and Andreas Krause. Truthful incentives in crowdsourcing tasks using regret minimization mechanisms. In *22nd Intl. World Wide Web Conf. (WWW)*, pages 1167–1178, 2013.

Aleksandrs Slivkins. Towards fast decentralized construction of locality-aware overlay networks. In *26th Annual ACM Symp. on Principles Of Distributed Computing (PODC)*, pages 89–98, 2007.

Aleksandrs Slivkins. Multi-armed bandits on implicit metric spaces. In *25th Advances in Neural Information Processing Systems (NIPS)*, 2011.

Aleksandrs Slivkins. Dynamic ad allocation: Bandits with budgets. A technical report on `arxiv.org/abs/1306.0155`, June 2013.

Aleksandrs Slivkins. Contextual bandits with similarity information. *J. of Machine Learning Research (JMLR)*, 15 (1):2533–2568, 2014. Preliminary version in *COLT 2011*.

Aleksandrs Slivkins. Exploration and persuasion. In Federico Echenique, Nicole Immorlica, and Vijay Vazirani, editors, *Online and Matching-Based Market Design*. Cambridge University Press, 2023. Also available at `http://slivkins.com/work/ExplPers.pdf`.

Aleksandrs Slivkins and Eli Upfal. Adapting to a changing environment: the Brownian restless bandits. In *21st Conf. on Learning Theory (COLT)*, pages 343–354, 2008.

Aleksandrs Slivkins and Jennifer Wortman Vaughan. Online decision making in crowdsourcing markets: Theoretical challenges. *SIGecom Exchanges*, 12(2):4–23, December 2013.

Aleksandrs Slivkins, Filip Radlinski, and Sreenivas Gollapudi. Ranked bandits in metric spaces: Learning optimally diverse rankings over large document collections. *J. of Machine Learning Research (JMLR)*, 14(Feb):399–436, 2013. Preliminary version in *27th ICML*, 2010.

Aleksandrs Slivkins, Karthik Abinav Sankararaman, and Dylan J. Foster. Contextual bandits with packing and covering constraints: A modular lagrangian approach via regression. In *36th Conf. on Learning Theory (COLT)*, 2023.

Niranjan Srinivas, Andreas Krause, Sham Kakade, and Matthias Seeger. Gaussian Process Optimization in the Bandit Setting: No Regret and Experimental Design. In *27th Intl. Conf. on Machine Learning (ICML)*, pages 1015–1022, 2010.

Gilles Stoltz. *Incomplete Information and Internal Regret in Prediction of Individual Sequences*. PhD thesis, University Paris XI, ORSAY, 2005.

Matthew Streeter and Daniel Golovin. An online algorithm for maximizing submodular functions. In *Advances in Neural Information Processing Systems (NIPS)*, pages 1577–1584, 2008.

Wen Sun, Debadeepta Dey, and Ashish Kapoor. Safety-aware algorithms for adversarial contextual bandit. In *34th Intl. Conf. on Machine Learning (ICML)*, pages 3280–3288, 2017.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 1998.

Adith Swaminathan and Thorsten Joachims. Batch learning from logged bandit feedback through counterfactual risk minimization. *J. of Machine Learning Research (JMLR)*, 16:1731–1755, 2015.

Adith Swaminathan, Akshay Krishnamurthy, Alekh Agarwal, Miroslav Dudík, John Langford, Damien Jose, and Imed Zitouni. Off-policy evaluation for slate recommendation. In *30th Advances in Neural Information Processing Systems (NIPS)*, pages 3635–3645, 2017.

Vasilis Syrgkanis and Éva Tardos. Composable and efficient mechanisms. In *45th ACM Symp. on Theory of Computing (STOC)*, pages 211–220, 2013.

Vasilis Syrgkanis, Alekh Agarwal, Haipeng Luo, and Robert E. Schapire. Fast convergence of regularized learning in games. In *28th Advances in Neural Information Processing Systems (NIPS)*, pages 2989–2997, 2015.

Vasilis Syrgkanis, Akshay Krishnamurthy, and Robert E. Schapire. Efficient algorithms for adversarial contextual learning. In *33nd Intl. Conf. on Machine Learning (ICML)*, 2016a.

Vasilis Syrgkanis, Haipeng Luo, Akshay Krishnamurthy, and Robert E. Schapire. Improved regret bounds for oracle-based adversarial contextual bandits. In *29th Advances in Neural Information Processing Systems (NIPS)*, 2016b.

Csaba Szepesvári. *Algorithms for Reinforcement Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers, 2010.

Kunal Talwar. Bypassing the embedding: Algorithms for low-dimensional metrics. In *36th ACM Symp. on Theory of Computing (STOC)*, pages 281–290, 2004.

William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3-4):285–294, 1933.

Long Tran-Thanh, Archie Chapman, Enrique Munoz de Cote, Alex Rogers, and Nicholas R. Jennings. ϵ-first policies for budget-limited multi-armed bandits. In *24th AAAI Conference on Artificial Intelligence (AAAI)*, pages 1211–1216, 2010.

Long Tran-Thanh, Archie Chapman, Alex Rogers, and Nicholas R. Jennings. Knapsack based optimal policies for budget-limited multi-armed bandits. In *26th AAAI Conference on Artificial Intelligence (AAAI)*, pages 1134–1140, 2012.

Michal Valko, Alexandra Carpentier, and Rémi Munos. Stochastic simultaneous optimistic optimization. In *30th Intl. Conf. on Machine Learning (ICML)*, pages 19–27, 2013.

Alberto Vera, Siddhartha Banerjee, and Itai Gurvich. Online allocation and pricing: Constant regret via bellman inequalities. *Operations Research*, 2020.

Jun-Kun Wang and Jacob D. Abernethy. Acceleration through optimistic no-regret dynamics. In *31st Advances in Neural Information Processing Systems (NIPS)*, pages 3828–3838, 2018.

Zizhuo Wang, Shiming Deng, and Yinyu Ye. Close the gaps: A learning-while-doing algorithm for single-product revenue management problems. *Operations Research*, 62(2):318–331, 2014.

Jonathan Weed, Vianney Perchet, and Philippe Rigollet. Online learning in repeated auctions. In *29th Conf. on Learning Theory (COLT)*, volume 49, pages 1562–1583, 2016.

Chen-Yu Wei and Haipeng Luo. More adaptive algorithms for adversarial bandits. In *31st Conf. on Learning Theory (COLT)*, 2018.

Chen-Yu Wei, Chung-Wei Lee, Mengxiao Zhang, and Haipeng Luo. Linear last-iterate convergence in constrained saddle-point optimization. In *9th International Conference on Learning Representations (ICLR)*, 2021.

Chris Wilkens and Balasubramanian Sivan. Single-call mechanisms. In *13th ACM Conf. on Electronic Commerce (ACM-EC)*, 2012.

Huasen Wu, R. Srikant, Xin Liu, and Chong Jiang. Algorithms with logarithmic or sublinear regret for constrained contextual bandits. In *28th Advances in Neural Information Processing Systems (NIPS)*, 2015.

Qiqi Yan. Mechanism design via correlation gap. In *22nd ACM-SIAM Symp. on Discrete Algorithms (SODA)*, 2011.

Jia Yuan Yu and Shie Mannor. Unimodal bandits. In *28th Intl. Conf. on Machine Learning (ICML)*, pages 41–48, 2011.

Yisong Yue and Thorsten Joachims. Interactively optimizing information retrieval systems as a dueling bandits problem. In *26th Intl. Conf. on Machine Learning (ICML)*, pages 1201–1208, 2009.

Yisong Yue, Josef Broder, Robert Kleinberg, and Thorsten Joachims. The k-armed dueling bandits problem. *J. Comput. Syst. Sci.*, 78(5):1538–1556, 2012. Preliminary version in COLT 2009.

Julian Zimmert and Tor Lattimore. Connections between mirror descent, thompson sampling and the information ratio. In *33rd Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Julian Zimmert and Yevgeny Seldin. An optimal algorithm for stochastic and adversarial bandits. In *Intl. Conf. on Artificial Intelligence and Statistics (AISTATS)*, 2019.

Julian Zimmert and Yevgeny Seldin. Tsallis-inf: An optimal algorithm for stochastic and adversarial bandits. *J. of Machine Learning Research (JMLR)*, 22:28:1–28:49, 2021.

Julian Zimmert, Haipeng Luo, and Chen-Yu Wei. Beating stochastic and adversarial semi-bandits optimally and simultaneously. In *36th Intl. Conf. on Machine Learning (ICML)*, pages 7683–7692, 2019.

Masrour Zoghi, Shimon Whiteson, Rémi Munos, and Maarten de Rijke. Relative upper confidence bound for the k-armed dueling bandits problem. In *Intl. Conf. on Machine Learning (ICML)*, pages 10–18, 2014.

Shi Zong, Hao Ni, Kenny Sung, Nan Rosemary Ke, Zheng Wen, and Branislav Kveton. Cascading bandits for large-scale recommendation problems. In *32nd Conf. on Uncertainty in Artificial Intelligence (UAI)*, 2016.